
Algebra User Guide and Reference Manual

Manuel Bronstein and Marc Moreno Maza

Version 1.1.0 – April 19, 2005

Algebra Contributors

Algebra is the result of extracting the central subset of the Σ^{it} library¹ and adding new data structures and polynomial types written by Marc Moreno Maza, in order to provide a general-purpose computer algebra library.

Besides the principal authors, the following Σ^{it} project members have contributed code that is now part of the **Algebra** library:

Laurent Bernardin
Marco Codutti
Niklaus Mannhart
Thom Mulders
Julien Ohler
Hélène Prieto

¹ Σ^{it} , which was until 1997 a project of Manuel Bronstein's computer algebra group at the ETH Zurich, and whose development continues within the CAFÉ project at INRIA Sophia-Antipolis, is a special-purpose computer algebra library, see <http://www-sop.inria.fr/cafe/Manuel.Bronstein/sumit>.

Contents

1	Introduction	13
2	User Guide	14
	Basic algebraic categories	15
	Arithmetic	15
	Data structures	16
	Input/Output	16
	Linear algebra	17
	Univariate polynomials and series	17
	Multivariate polynomials	19
	Compatibility with C types	20
	Using GMP	21
	Exceptions	21
	Profiling and debugging	21
3	Reference Manual	24
	Basic Categories	
	AbelianGroup	25
	AbelianMonoid	26
	Algebra	32
	CharacteristicZero	33
	CommutativeRing	34
	DecomposableRing	40
	DifferentialExtension	43
	DifferentialRing	45
	EuclideanDomain	48
	Field	57
	FiniteCharacteristic	58
	FiniteSet	60
	FreeAlgebra	66
	FreeLinearArithmeticType	67
	FreeLinearCombinationType	68
	FreeModule	70
	FreeRRing	78
	GcdDomain	80
	Group	83
	IndexedFreeAlgebra	86
	IndexedFreeLinearArithmeticType	87
	IndexedFreeLinearCombinationType	89
	IndexedFreeModule	93

IndexedFreeRRing	97
IntegerCategory	98
IntegralDomain	102
LinearArithmeticType	109
Module	112
Monoid	113
NonCommutativeIntegralDomain	119
Ring	122
RittRing	127
RRing	130
Specializable	131
ExpressionType	133

Commutative Algebra

Automorphism	136
CanonicalSimplification	140
ChineseRemaindering	141
Complex	144
Derivation	145
Fraction	149
FractionalRoot	150
FractionBy	157
FractionByCategory	158
FractionByCategory0	159
FractionCategory	162
FractionFieldCategory	165
FractionFieldCategory0	166
LinearCombinationFraction	168
ModularComputation	174
Product	175
ResidueClassRing	184
ReducibleModulusException	185
ReducibleModulusExceptionType	186
SimpleAlgebraicExtension	187
SimpleAlgebraicExtensionCategory	188
SourceOfPrimes	189
UnivariatePolynomialMod	190
UnivariatePolynomialModSqfr	191
UnivariatePolynomialQuotient	192
UnivariatePolynomialQuotientSqfr	200

Finite Fields

SmallPrimes	203
LazyHalfWordSizePrimes	204
HalfWordSizePrimes	205

WordSizePrimes	206
PrimeCollection	207
PrimeField2	216
PrimeFieldCategory	217
PrimeFieldCategory0	219
PrimitiveRoots	221
PthPowering	224
SmallPrimeField	226
SmallPrimeField0	227
SmallPrimeFieldCategory	228
SmallPrimeFieldCategory0	229
ZechPrimeField	232

Linear Algebra

Backsolve	233
DenseMatrix	236
DivisionFreeGaussElimination	237
FractionFreeGaussElimination	238
HermiteGaussElimination	239
LinearAlgebra	240
LinearAlgebraRing	255
LinearEliminationCategory	269
MatrixCategory	282
MatrixCategory2	304
MatrixCategoryOverFraction	306
ModulopGaussElimination	310
OrdinaryGaussElimination	323
OverdeterminedLinearSystemSolver	324
SpecializationLinearAlgebra	326
TwoStepFractionFreeGaussElimination	328
UnivariatePolynomialCRTLinearAlgebra	329
UnivariatePolynomialPopovLinearAlgebra	332
Vector	334
VectorOverFraction	339

Univariate Polynomials and Series

DenseUnivariatePolynomial	340
DenseUnivariateTaylorSeries	341
FactorizationRing	342
FFTRing	345
GenericModularPolynomialGcdPackage	349
HeuristicGcd	350
ModularUnivariateGcd	354
ModulopUnivariateGcd	356
PrimeFieldUnivariateFactorizer	358

RationalRootRing	363
Resultant	365
SparseUnivariatePolynomial0	371
SparseUnivariatePolynomial1	372
SparseUnivariatePolynomial	373
UnivariateFactorialPolynomial	374
UnivariateFreeLinearArithmeticType	377
UnivariateFreeRing	383
UnivariateFreeRing2	390
UnivariateFreeRingOverFraction	392
UnivariateGcdRing	393
UnivariateIntegralFactorizer	396
UnivariateMonomial	399
UnivariatePolynomialAlgebra	408
UnivariatePolynomialKaratsuba	431
UnivariatePolynomialRing	432
UnivariatePolynomialSquareFree	435
UnivariateTaylorSeriesType	438
UnivariateTaylorSeriesType2Poly	442
UnivariateTaylorSeriesNewtonSolver	449

Multivariate Polynomials

BivariateUtilitiesPackage	452
DirectProduct	453
DirectProductCategory	454
DistributedMultivariatePolynomial0	455
DistributedMultivariatePolynomial1	456
ExponentCategory	457
FiniteAbelianMonoidRing0	458
FiniteVariableType	459
GeneralExponentCategory	461
IntegerExponentVectorCategory	464
IntegerPolynomial	465
MachineIntegerDegreeLexicographicalExponent	466
MachineIntegerDegreeReverseLexicographicalExponent	467
MachineIntegerExponentVectorCategory	468
MachineIntegerLexicographicalExponent	469
OrderedSymbol	470
OrderedVariableList	471
OrderedVariableTuple	472
PolynomialRing	473
PolynomialRing0	474
PolynomialTypeRing	475
RecursiveMultivariatePolynomial0	476
RecursiveMultivariatePolynomialCategory0	477

ResultantOfBivariatePolynomialsOverSmallPrimeField	478
SparseIntegerMultivariatePolynomial	479
SparseMultivariatePolynomial	480
StandardFilteredRing	481
VariableType	482

Expression Trees

ExpressionTree	483
ExpressionTreeAnd	494
ExpressionTreeAssign	495
ExpressionTreeBigO	496
ExpressionTreeCase	497
ExpressionTreeComplex	498
ExpressionTreeEqual	499
ExpressionTreeExpt	500
ExpressionTreeFactorial	501
ExpressionTreeGreaterEqual	502
ExpressionTreeGreaterThan	503
ExpressionTreeIf	504
ExpressionTreeLeaf	505
ExpressionTreeLessEqual	520
ExpressionTreeLessThan	521
ExpressionTreeLispList	522
ExpressionTreeList	523
ExpressionTreeMatrix	524
ExpressionTreeMinus	525
ExpressionTreeNotEqual	526
ExpressionTreeOperator	527
ExpressionTreeOperatorTools	533
ExpressionTreePlus	537
ExpressionTreePrefix	538
ExpressionTreeQuotient	539
ExpressionTreeSubscript	540
ExpressionTreeTimes	541
ExpressionTreeVector	542

Shells

Evaluator	543
InfixExpressionParser	544
LispExpressionParser	546
MakePartialRing	547
Maple	548
Parsable	552
Parser	554
ParserReader	558

ParsingTools	560
PartialRing	563
Scanner	564
Shell	566
Token	569

Utilities

AlgebraLibraryInformation	579
IndexedVariable	580
Permutation	583
Sequence	588

4 libaldor Reference Manual 598

Arithmetic

AldorInteger	599
AdditiveType	600
ArithmeticType	605
BinaryPowering	611
Boolean	613
BooleanArithmeticType	615
Complex	617
DoubleFloat	624
FloatType	628
GMPFloat	632
GMPInteger	638
IntegerSegment	642
IntegerType	648
LinearCombinationType	660
MachineInteger	664
OrderedArithmeticType	667
PartiallyOrderedType	670
PackableType	672
PrimitiveType	675
RandomNumberGenerator	677
SingleFloat	685
TotallyOrderedType	688

Input/Output

BinaryReader	691
BinaryWriter	696
Byte	702
Character	706
File	711
FileException	720

FileExceptionType	721
InputType	722
OutputType	724
SerializableType	726
SyntaxException	728
SyntaxExceptionType	729
TextReader	730
TextWriter	737
WriterManipulator	743

Data Structures

Array	747
ArrayException	748
ArrayExceptionType	749
ArrayType	750
BoundedFiniteDataStructureType	757
BoundedFiniteLinearStructureType	762
CheckingArray	765
CheckingList	766
CheckingMemoryBlock	767
DataStructureType	768
DynamicDataStructureType	771
FiniteLinearStructureType	774
Flags	778
HashTable	780
KeyEntry	783
HashType	787
LinearStructureType	789
List	795
ListException	796
ListExceptionType	797
ListType	798
MemoryBlock	812
PackedPrimitiveArray	813
PrimitiveArray	814
PrimitiveArrayType	815
PrimitiveMemoryBlock	821
Queue	823
Set	825
SortedAssociationSet	830
SortedList	831
SortedSet	832
Stream	833
String	842
StringBuffer	851

SymbolTable	855
TableException	859
TableExceptionType	860
TableType	861

Utilities

AldorLibraryInformation	869
VersionInformationType	870
BinarySearch	875
CommandLine	877
CopyableType	881
Generator	883
GeneratorException	887
GeneratorExceptionType	888
Partial	889
PartialFunction	893
Pointer	901
Symbol	903
Timer	907
Trace	914

1 Introduction

What is Algebra?

Algebra is a general-purpose computer algebra library designed to provide reusable and efficient algorithms for manipulating the standard objects of algebra, namely polynomials, series and matrices. Built as an extension of the `libaldor` library, it provides ALDOR programmers with an extensible computer algebra layer with a rich data type hierarchy.

How do I get and install Algebra?

You can download Algebra by anonymous ftp from the CAFÉ server at `ftp-sop.inria.fr` in `cafe/software/algebra`, or from the URL:

`http://www.inria.fr/cafe/Manuel.Bronstein/algebra/`

After downloading the file `algebra.tar.gz`, issue “`tar -xzvf algebra.tar.gz`” in order to unpack it. This will create the following directories:

- `algebra/doc`: this user guide,
- `algebra/lib`: the library,
- `algebra/include`: the required include files,
- `algebra/test`: some test files,
- `algebra/samples`: Algebra programming samples,

Once the above file is unpacked, do the following:

- add the option `-csys=XXX` to your `ALDORARGS` environment variable, where `XXX` depends on your hardware and operating system. Common values for `XXX` are `axposf1v4` for OSF1 V4.0 on a DEC Alpha, `linux-486` for linux on a 486 or above PC, and `sun4os55g-v8` for SunOS 5.5 on a SPARC v8 machine. See the file `$ALDORROOT/include/aldor.conf` for other values;
- go to `algebra/lib` and execute `sh makealgebra; shell;`
- if you want to build the GMP version of the library, execute `sh makealgebra-gmp`. See the subsection on **using GMP** for more information about using the GMP version of Algebra;
- if you want to build the debug version of the library, execute `sh makealgebrad`. See the subsection on **debugging** for more information on using the debug library.

How do I use Algebra in my programs?

Once **Algebra** is properly built, you need to set the following environment variables before using it:

- the environment variable `ALGEBRAROOT` should be set to the main **algebra** directory;
- `$ALGEBRAROOT/include` should be appended to your `INCPATH` environment variable;
- `$ALGEBRAROOT/lib` should be appended to your `LIBPATH` environment variable;

In your **ALDOR** programs, use `#include "algebra"` instead of `#include "aldor"`. When building your final executable, add the options

```
-lalgebra -laldor -y$ALGEBRAROOT/lib
```

to your compiler command line, or

```
-lalgebrad -laldord -dDEBUG -y$ALGEBRAROOT/lib
```

to link to the debug version of **Algebra**. Check the subsection on **using GMP** for the options required if you want to use the GMP library and the GMP version of **Algebra**.

If you are running **Algebra** inside the compiler interactive loop, then type the line

```
#include "aldorinterp"
```

immediately after `#include "algebra"`, which will import various things for interactive use and make the interpreter loop print values automatically. As with any **ALDOR** program, do not forget the `-q` option in order to optimize your programs, specially if performance is an issue.

Before using **Algebra** for the first time, please check your installation by running `make` in the `algebra/test` directory, followed by running `testall`.

Please report any installation problem or bugs you encounter to sumit@sophia.inria.fr.

2 User Guide

This guide introduces the common types and categories provided by **Algebra**, and presupposes some familiarity with programming in **ALDOR** and **libaldor**. If you are unfamiliar with **ALDOR** or **libaldor**, we suggest that you first go through the tutorial in `aldorlib/tutorial/`, which will familiarize you both with **ALDOR** and **libaldor**.

Algebra provides over 180 categories and domains, and over 400 different exported functions, which can look daunting at first. Remember however that a large part of those are low-level functionalities that make it possible to programmers to write efficient applications. Most of the high-level functionalities are found in a small subset of types and those are the ones described in this short guide. As you become more familiar with **Algebra**, you will find the reference guide to be more useful when browsed on line.

As you learn programming with **Algebra**, make sure to check the `algebra/samples` directory for various programming samples.

Basic algebraic categories

The basic algebraic categories provided by **Algebra** are shown in Figure 1. While it follows quite closely the usual algebraic structures, a few points need to be mentioned.

Figure 1: The **Algebra** basic algebraic category hierarchy

- The multiplication `*` is in general not assumed to be commutative, except in the subtree starting at **CommutativeRing**, which appears in blue in Figure 1. This means that code written for the other categories should be careful and not assume commutativity of `*`. Addition is on the other hand always assumed commutative.
- The **Algebra** category tree has a unique root **ExpressionType** to which almost all the types provided by **Algebra** belong. The category **ExpressionType** inherits **PrimitiveType** from **libaldor**, which means that types in **Algebra** must export an equality. That equality however does not need to be complete, as described in the `=` page of the **libaldor** reference manual. **ExpressionType** plays also an important role in input/output as explained below.
- The **libaldor** categories **AdditiveType**, **ArithmeticType** and **LinearCombinationType** already export the basic arithmetic operations provided respectively by abelian groups, rings and modules. In fact **AbelianGroup**, **Ring** and **Module** do inherit their exports from them. There is however a fundamental difference: while the **libaldor** categories export the usual operations `=`, `+`, `-`, `*`, `...`, they make no assumption about their algebraic properties whatsoever. On the other hand, the corresponding algebraic categories in **Algebra** do assume that `=` is a complete equality test, and that the arithmetic operations satisfy the algebraic axioms of their mathematical structure. So for example, while **Integer** is extended by **Algebra** to be a **Ring** (among other things), **SingleFloat** remains an **ArithmeticType** but is not a **Ring**. There is a similar relationship between **LinearArithmeticType** and **Algebra**, both provided by **Algebra**. Mathematical types such as **DenseMatrix**(*R*) or **DenseUnivariatePolynomial**(*R*) allow their argument *R* to be an **ArithmeticType** rather than a **Ring**, but those among their functions that require a complete equality on *R* are not exported when *R* is not a **Ring**. Similarly, while **DenseUnivariatePolynomial**(*R*) is always an **ArithmeticType**, it is a **Ring** only when *R* itself is a **Ring**. For example,

```
DenseMatrix DenseUnivariatePolynomial SingleFloat
```

is a valid type in **Algebra**, but Gaussian elimination is not available for such matrices, and the Euclidean algorithm is not available for their polynomial entries. Basic arithmetic, as provided by **ArithmeticType** is however available.

Arithmetic

All of the arithmetic types provided by **libaldor** (machine and software integers, floats, as well as **Complex**) remain available in **Algebra**, and most of them are extended to various algebraic

categories. As in `libaldor`, `Integer` is actually a macro that defaults to `AldorInteger`. If integer efficiency is important for your application, we strongly recommend that you link with the GMP version of `Algebra` instead, see the section on `using GMP` for more details. Regardless of the integer implementation that you choose, we also recommended that you use `MachineInteger` whenever appropriate, in particular for loop or data structure indices. Conversions between `MachineInteger` and `Integer` are provided by `coerce` and `machine`.

In addition, `Algebra` provides two different implementations of the finite field $\mathbb{Z}/p\mathbb{Z}$ when p is a machine prime: `SmallPrimeField` provides a standard implementation, while `ZechPrimeField` provides a significantly faster implementation based on a logarithmic representation of its elements. However, `ZechPrimeField` precomputes a table of size $\mathcal{O}(p)$ so it should only be used for reasonably small values of p and when a significant amount of calculations in $\mathbb{Z}/p\mathbb{Z}$ are made following the creation of the type (we have found the precomputation time to be under one second for p around 10^6 on recent workstations).

The type `Fraction( $R$ )` implements the fraction field of the `GcdDomain` R . Typical arguments are `Integer` (to get the rational numbers) or polynomial types. Note that fractions are automatically normalized after each arithmetic operation.

Data structures

All the data structure of `Algebra` are provided by `libaldor`. Some of them are extended by mathematical operations and take on a new name: `Sequence` corresponds to `Stream` from `libaldor` and `Vector` corresponds to `Array` from `libaldor`, in both cases with pointwise arithmetic operations added. Note however that `Array` is 0-indexed while `Vector` is 1-indexed.

Input/Output

`Algebra` inherits the stream I/O model provided by `libaldor`. In particular `ExpressionType` inherits `OutputType` from `libaldor`, which means that elements of a `ExpressionType` can be written in text format to any `TextWriter` (in particular I/O streams, strings or files). Because it is desirable to output mathematical objects such as matrices and polynomials in more than one format depending on the context, `Algebra` types do not in general define their own `<<` function. Rather, they implement the `extree` function, that converts their elements into elements of `ExpressionTree`. Expression trees, which are similar to Lisp's S-expressions, are the unique layer between all the types in `Algebra` and the outside world. When writing your own types, once you provide or inherit an implementation of `extree`, your elements can then be written to any `TextWriter` in any of the many formats that `ExpressionTree` understands, for example using `lisp` to produce Lisp output. In that case, the default behavior of the `<<` is to convert your objects to expression trees and then to use `tex` to produce $\text{\TeX}$ output. You can override this default behaviour by providing your own implementation of `<<` if you choose. As with `libaldor`, you can use `#include "aldorio" after #include "algebra"` in order to import the types commonly used for input and output.

Linear algebra

The basic types to use for doing linear algebra are `Vector` and `DenseMatrix`. The basic linear algebra computations are provided by the type `LinearAlgebra`. Although `Algebra` provides many specialized types for performing various sorts of triangulations and solving linear systems, `LinearAlgebra` knows about all of them and contains procedures that decide which algorithm is best suited for your particular input. So do not call directly the more specialized packages unless you have a particular reason to use a specific algorithm rather than let `Algebra` decide for you.

The type of the entries of vectors and matrices does not need to be a `Ring`, it can be an `AdditiveType` or `ArithmeticType` respectively. This allows types that do not have a full equality such as `SingleFloat` or `DenseUnivariateTaylorSeries` to be entries of vectors and matrices. However, `LinearAlgebra` requires the entries to be from a `CommutativeRing` so its functionalities are not available for matrices of series or floating point numbers.

`Algebra` contains many fraction-free algorithms that allow you to perform some computations, such as matrix inverses or solving systems, over an `IntegralDomain` rather than over a `Field` as is usually done. Those fraction-free algorithms are significantly faster over integral domains than over their fraction fields, so consider using domains such as $\mathbb{Z}$ or $R[x]$ as matrix entries rather than fractions. See for example the description of the `inverse` function to see the effect of working over domains rather than fields on the signatures of the functions.

To write generic linear algebra code that does not depend on the implementation of matrices, use a type parameter of category `MatrixCategory`. For example, a normal form package could look like:

```
NormalForms(R: IntegralDomain, M: MatrixCategory R): with {
    frobenius: M -> M;      -- Returns the Frobenius form of its argument
    ...
} == add { ... }
```

Univariate polynomials and series

The basic types to use for computing with polynomials and series are `DenseUnivariatePolynomial` and `DenseUnivariateTaylorSeries` respectively. Both types are univariate but can be nested if needed to produce dense multivariate polynomials and series with a fixed number of variables. When the number of variables is too large for a dense representation, you can also use `SparseUnivariatePolynomial` but be aware that for univariate or bivariate use, its arithmetic is much less efficient than the one of `DenseUnivariatePolynomial`.

As for matrices, the type of the coefficients of polynomials or series does not need to be a `Ring`, it can be an `ArithmeticType` instead. This allows types that do not have a full equality such as `SingleFloat` to be used as coefficients, but some polynomial functionalities are only available when the coefficient type is a `Ring` or something stronger. The polynomial and series types take a `Symbol` as second argument. That symbol is used only for output when converting the polynomial or series to an `ExpressionTree`, so it is not necessary to give one when you use polynomials or series inside a calculation. If you insist on naming the variable, use `-` with any string as argument to create a symbol. For example, `DenseUnivariatePolynomial(Integer)` and `SparseUnivariatePolynomial(Fraction Integer, -"x")` are both valid polynomial types. Whether you give a name for a variable or not, you can override that choice using `apply` with a

Symbol or ExpressionTree as argument. For example, if p is a polynomial or series, `stdout(p, "-z")` writes p to `stdout` using “z” as variable name, and `p(extree leaf("-y"))` returns the ExpressionTree corresponding to p with “y” as variable name.

To write generic code for manipulating polynomials or series use a type parameter usually of category `UnivariatePolynomialCategory` or `UnivariateTaylorSeriesCategory` respectively. Because polynomials, skew-polynomials and series share many common operations, `Algebra` provides in fact a complete category hierarchy for them, shown in Figure 2. Those categories make it

Figure 2: The `Algebra` univariate polynomial category hierarchy

possible to write functions that work for polynomials, skew-polynomials, series or any combination of them. `UnivariatePolynomialCategory(R)` is the category for types whose elements are usual polynomials of the form $\sum_n r_n x^n$ with finite support. While R is not assumed to be commutative, the generator x is assumed to commute with coefficients in R , *i.e.* $rx = xr$. Similarly, `UnivariateTaylorSeriesCategory(R)` is the category for types whose elements are series of the form $\sum_n r_n x^n$. Here also, R is not assumed to be commutative, but the generator x commutes with coefficients in R . Those two categories are shown in blue in Figure 2, as they are the only ones assuming that x commutes with R . The more general category `UnivariatePolynomialAlgebra(R)` is for types whose elements are sums of the form $\sum_n r_n x^n$ with finite support, but where x does not necessarily commute with R . Polynomials and skew-polynomials are both in that category. Even more general, `MonogenicAlgebra(R)` is for types whose elements are sums of the form $\sum_n r_n P_n$ with finite support, but where the family P_n is not necessarily the power basis x^n . An example of such a type is `UnivariateFactorialPolynomial` for which $P_n = x(x-1)\dots(x-n+1)$. Finally, `MonogenicLinearArithmeticType(R)` is for types whose elements are potentially infinite sums of the form $\sum_n r_n P_n$. Code written at that level works for polynomials, series and skew-polynomials as well.

The elements of `DenseUnivariateTaylorSeries` are lazy series represented by their coefficient sequences, themselves of type `Sequence`. Those coefficient sequences are in turn represented as `Stream` from `libaldor`. Therefore, the usual way to write a function producing a series is to produce first its coefficient stream s , then call `sequence` on s to produce the coefficient sequence, and finally call `series` on the coefficient sequence to produce the series. Since streams are lazy, constructing a series does not compute any of its coefficients until they are specifically requested by another operation. There are several ways to create streams, all documented in the `libaldor` reference manual, and you should become familiar with them before programming with series. For example, the following function takes constants a_0 and c and produces the hypergeometric series $\sum_{n \geq 0} a_n x^n$ where $a_{n+1}/a_n = c$.

```
SeriesSample(R:CommutativeRing, Rx:UnivariateTaylorSeriesCategory R): with {
  hypergeometricSeries: (R, R) -> Rx;
} == add {
  hypergeometricSeries(a0:R, c:R):Rx == {
    import from Sequence F;
    zero? a0 => 0;
    -- the following creates the stream [a0, c a0, c^2 a0, ... ]
```

```

        coeffs:Stream R := orbit(a0, (x:R):R +-> c * x);
        series sequence coeffs;
    }
}

```

Finally, the type `UnivariateTaylorSeriesCategory2Poly` provides conversions between univariate polynomials and series.

Multivariate polynomials

The basic types for computing with multivariate polynomials are `SparseMultivariatePolynomial` and `DistributedMultivariatePolynomial1`. Both types use a sparse representation. Elements of the former one are regarded as univariate polynomials with polynomial coefficients. Elements of the latter one are regarded as lists of terms where a term is the product of a coefficient by a power product of variables.

To construct a multivariate polynomial type with `SparseMultivariatePolynomial` one needs a `Ring` as a coefficient type and a `VariableType` as a variable type.

```

#include "algebra"
import from String, Symbol;
macro V == OrderedVariableTuple("-x","-y","-z");
import from MachineInteger, V;
macro P == SparseMultivariatePolynomial(Integer,V);
x: P := variable(1)$V :: P;
y: P := variable(2)$V :: P;
z: P := variable(3)$V :: P;
p := (y + z)*x^2 + (y^3 + z)*x + z^4;

```

To construct a multivariate polynomial type with `DistributedMultivariatePolynomial1` one needs a `Ring` as a coefficient type, a `VariableType` as a variable type, and `ExponentCategory(V)` as an exponent type. So we can continue the above sample program as follows

```

macro E1 == MachineIntegerDegreeLexicographicalExponent(V);
macro E2 == MachineIntegerLexicographicalExponent(V);
macro Q1 == DistributedMultivariatePolynomial1(Integer, V, E1);
macro Q2 == DistributedMultivariatePolynomial1(Integer, V, E2);
q1 := expand(E1)(Q1)(p);
q2 := expand(E2)(Q2)(p);

```

The above polynomials appear respectively as follows in an interpreter session.

```

%20 >> q1 := expand(E1)(Q1)(p)
x*y^3 + z^4 + x^2*y + x^2*z + x*z @ Q1

%21 >> q2 := expand(E2)(Q2)(p);
x^2*y + x^2*z + x*y^3 + x*z + z^4 @ Q2

```

Do not forget to enter

```
#include "aldorinterp"
```

after

```
#include "algebra"
```

in every interpreter session.

There are other multivariate polynomial types like `RecursiveMultivariatePolynomial0`. It is similar to `SparseMultivariatePolynomial` but takes a third argument which is a univariate polynomial type constructor:

```
UP: (R: Join(ArithmeticType, ExpressionType), avar: Symbol == new()
      ) -> UnivariatePolynomialCategory(R),
```

This third argument is used for the internal representation.

Another useful type is `IntegerPolynomial` which provides multivariate polynomial with integer coefficient and variables from `OrderedSymbol`. This multivariate polynomial type is easy to use as shown by the following interpreter session.

```
%1 >> #include "algebra"
                                         Comp: 130 msec, Interp: 0 msec
%2 >> #include "aldorinterp"
                                         Comp: 70 msec, Interp: 0 msec
%3 >> macro P == IntegerPolynomial;
                                         Comp: 0 msec, Interp: 0 msec
%4 >> import from String, P, Integer;
                                         Comp: 210 msec, Interp: 0 msec
%5 >> x: P := "x" :: P
x @ IntegerPolynomial
                                         Comp: 0 msec, Interp: 520 msec
%6 >> y: P := "y" :: P
y @ IntegerPolynomial
                                         Comp: 0 msec, Interp: 0 msec
%7 >> z: P := "z" :: P
z @ IntegerPolynomial
                                         Comp: 0 msec, Interp: 0 msec
%8 >> p := (y + z)*x^2 + (y^3 + z)*x + z^4 + 3::P;
z^4+(x^2+x)*z+x*y^3+x^2*y+3 @ IntegerPolynomial
```

Observe the `3::P`. Indeed, it is necessary to convert the integer 3 into a polynomial. Observe also the ordering on symbols is reversed w.r.t. the dictionary one.

Compatibility with C types

All the types of `libaldor` that are compatible with their C counterparts remain so inside `Algebra`. In addition, if you are using the GMP version of the library, then `Integer` and `Float` are compatible with `mpz_t` and `mpf_t` respectively.

Using GMP

As in `libaldor`, the type `Integer` is actually a macro, which defaults to `AldorInteger`, the software integers provided by the ALDOR runtime. For efficiency or other reasons, you may prefer to use the GMP library, which is supported by `Algebra`. The easiest way to use GMP is to compile all your source files with the option `-dGMP` and then to use the options

```
-lalgebra-gmp -laldor -cruntime=foam-gmp,gmp -y$ALGEBRAROOT/lib
```

when linking your final executable. All you need is GMP 3.0 or later installed in a file called `libgmp.a` to produce executables. Using GMP generally produces more efficient programs, but programs calling GMP cannot be interpreted by the ALDOR compiler, nor can they run inside its interactive loop.

Using the `-dGMP` option allows you to compile the same sources either with or without GMP, which can be appreciable, but you must ensure that you do not mix files compiled with and without `-dGMP` since the macro `Integer` would then be expanded into two different types.

An additional advantage of using GMP, is that `GMPInteger` exports and uses internally several of the in-place or higher-level operations of GMP, which are not available with `AldorInteger`. In addition, variables of type `GMPInteger` are compatible with the C type `mpz_t` from GMP, so you can directly call C programs that use GMP from your ALDOR code.

Exceptions

In addition to the exceptions thrown by `libaldor`, `Algebra` throws a `ReducibleModulusException` when a divisor of zero is discovered in a `SimpleAlgebraicExtension`. This can be used to implement algorithms based on lazy factorisation, since such an exception contains a non-trivial factor of the polynomial defining the extension.

Profiling and debugging

The macros `TIME`, `TRACE` and `AGAT` provided by `libaldor` remain available in `Algebra`. In addition, `Algebra` also comes with a debug version, which makes many assertions about the arguments of the functions called as well as their results. While those assertions slow down the code considerably they tend to be quite useful when chasing bugs since the release version of `Algebra` does not make validity checks to most of its inputs. The debug version of `Algebra` must be used jointly with the debug version of `libaldor`. To use them, just compile your application with the

```
-lalgebrad -laldord -dDEBUG
```

options rather than `-lalgebra -laldor`. It is also preferable when debugging to add the `-q1` option in order to prevent inlining.

3 Reference Manual

AbelianGroup

Usage

AbelianGroup: Category

Description

AbelianGroup is the category of commutative groups.

Exports

AdditiveType

AbelianMonoid

AbelianMonoid

Usage

AbelianMonoid: Category

Description

AbelianMonoid is the category of commutative monoids.

Exports

	ExpressionType	
0:	%	zero
+:	(%, %) → %	sum
*:	(Integer, %) → %	product by an integer
add!:	(%, %) → %	In-place sum
zero?:	% → Boolean	test for 0

Usage

0

Signature

0: %

Returns

Returns the constant 0.

Usage

$$x + y$$

Signature

$$+:\ (\%,\%) \rightarrow \%$$

Parameter	Type	Description
x,y	$\%$	elements of the monoid

Returns

Returns the sum $x + y$.

Usage $n * x$ **Signature** $*: (\text{Integer}, \%) \rightarrow \%$

Parameter	Type	Description
n	Integer	An integer
x	%	An element of the monoid to be multiplied by n

ReturnsReturns the product nx .

Usage

add!(x, y)

Signature

add!: ($\%$, $\%$) $\rightarrow$ $\%$

Parameter	Type	Description
x	$\%$	An element of the monoid (to be destroyed)
y	$\%$	An element of the monoid to be added to x

Returns

Returns the sum $x + y$, where the storage used by x is allowed to be destroyed or reused, so x is lost after this call.

Remarks

This function may cause x to be destroyed, so do not use it unless x has been locally allocated, and is guaranteed not to share space with other elements. Some functions are not necessarily copying their arguments and can thus create memory aliases.

Usage

zero? x

Signature

zero?: 'a → Boolean

Parameter	Type	Description
x	'a	an element of the monoid

Returns

Returns the result of $x = 0$ using the semantics of $=$ of the monoid.

Algebra

Usage

Algebra R:Category

Parameter	Type	Description
R	Ring	The coefficient ring

Description

Algebra R is the category of algebras over R , *i.e.* R -rings such that $R \cdot 1$ is included in the center (in other word, multiplication by R is bilinear).

Exports

`RRing R`

CharacteristicZero

Usage

CharacteristicZero: Category

Description

CharacteristicZero is the category of rings of characteristic 0.

Exports

Ring

CommutativeRing

Usage

CommutativeRing: Category

Description

CommutativeRing is the category of commutative rings.

Exports

Ring

canonicalUnitNormal?:	$\rightarrow \text{Boolean}$	Check if <code>unitNormal</code> is canonical
cutoff:	$\text{Z} \rightarrow \text{Z}$	Cutoffs for various fast algorithms
reciprocal:	$\% \rightarrow \text{Partial } \%$	Inverse
unitNormal:	$\% \rightarrow (\%, \%, \%)$ $(\%, \%) \rightarrow (\%, \%)$	Representative of the associates
unit?:	$\% \rightarrow \text{Boolean}$	Test whether an element is a unit

where

`Z == MachineInteger`

Usage

canonicalUnitNormal?

Signature

canonicalUnitNormal?: Boolean

Returns

Returns *true* if `unitNormal` is canonical in the following sense: if $x = vx'$ for some unit v and `unitNormal( $x$ )` returns (y, u, u^{-1}) and `unitNormal( $x'$ )` returns (y', u', u'^{-1}) , then $y = y'$.

See also

`unitNormal`, `unit?`

Usage

cutoff t

Signature

cutoff: `MachineInteger` $\rightarrow$ `MachineInteger`

Returns

Returns various cutoffs for asymptotically fast algorithms, which are then used for structures (*e.g.* polynomials or matrices) over this ring when the input size is greater than the corresponding cutoff. The parameter t denotes the algorithm in question and must be one of the `CUTOFF_XXX` values defined in `include/algebrauid.as`

Remarks

If a cutoff is -1 , then the corresponding algorithm is not used at all over this ring. The default value is always -1 so you only need to define other values if you want particular algorithms to be used over your rings.

Usage

reciprocal x

Signature

reciprocal: $\% \rightarrow \text{Partial } \%$

Parameter	Type	Description
x	$\%$	An element of the ring

Returns

Returns the unique y such that $x y = 1$ if such a y exists, *failed* otherwise.

Usage

```
(y, u, u1) := unitNormalx
(y, z1) := unitNormal(x, z)
```

Signatures

```
unitNormal: % → (% , % . %)
unitNormal: (% , %) → (% . %)
```

Parameter	Type	Description
x, y	$\%$	Elements of the ring

Returns

unitNormal(x) returns (y, u, u^{-1}) , while unitNormal(x,z) returns $(y, u^{-1}z)$. In both cases, $x = uy$ and u is a unit.

See also

canonicalUnitNormal?, unit?

Usage

unit? x

Signature

unit?: 'a → Boolean

Parameter	Type	Description
x	'a	An element of the ring

Returns

Returns *true* if x is a unit, *i.e.* $xy = 1$ for some y , *false* otherwise.

See also

canonicalUnitNormal?, unitNormal

DecomposableRing

Usage

DecomposableRing: Category

Description

DecomposableRing is the category of commutative rings whose elements can sometimes be decomposed into products (not to be confused with true factorization).

Exports

CommutativeRing

provablyIrreducible?: % $\rightarrow$ Boolean

someFactors: % $\rightarrow$ List % Get some factors

Usage

provablyIrreducible? x

Signature

provablyIrreducible?: % $\rightarrow$ Boolean

Parameter	Type	Description
x	%	A ring element

Returns

Returns *true* if x can be proven to be irreducible, *false* if either x is reducible or the proof of irreducibility cannot be obtained quickly enough.

Remarks

This function is not meant to use factorization or catch all irreducible elements, even when those functionalities are available. It is however meant to be efficient.

Usage

someFactors x

Signature

someFactors: % $\rightarrow$ List %

Parameter	Type	Description
x	%	A ring element

Returns

Returns $[x_1, \dots, x_n]$ such that each x_i divides x exactly.

Remarks

This function is not meant to use factorization or return a complete decomposition, even when those functionalities are available. It is however meant to be efficient.

DifferentialExtension

Usage

DifferentialExtension R: Category

Parameter	Type	Description
R	CommutativeRing	The base ring

Description

DifferentialExtension(R) is the category of differential extensions of R.

Exports

CommutativeRing
lift: Derivation R $\rightarrow$ Derivation % Extension of a derivation

if R has DifferentialRing then
DifferentialRing

Usage

lift D

Signature

lift: Derivation R $\rightarrow$ Derivation %

Parameter	Type	Description
D	Derivation R	A derivation on R

Returns

Returns the derivation D extended to the ring extension.

DifferentialRing

Usage

DifferentialRing: Category

Description

DifferentialRing is the category of commutative differential rings.

Exports

CommutativeRing

derivation: $\rightarrow$ Derivation % The derivation

differentiate: $(\%, \text{Integer}) \rightarrow \%$ Differentiate an element

Usage

derivation

Signature

derivation: Derivation %

Returns

Returns the derivation of the ring.

See also

differentiate(DifferentialRing)

Usage

differentiate x
differentiate(x , n)

Signature

differentiate: $(\%, \text{Integer}) \rightarrow \%$

Parameter	Type	Description
x	$\%$	The element to differentiate
n	Integer	The order of differentiation (optional)

Returns

differentiate x returns x' , the derivative of x .
differentiate(x , n) returns $x^{(n)}$, the n^{th} derivative of x .

See also

derivation(DifferentialRing)

EuclideanDomain

Usage

EuclideanDomain: Category

Description

EuclideanDomain is the category of commutative Euclidean domains.

Exports

GcdDomain		
diophantine:	$(\%, \%, \%) \rightarrow \partial \%$	Linear diophantine solver
divide:	$(\%, \%) \rightarrow (\%, \%)$	Euclidean division
divide!:	$(\%, \%, \%) \rightarrow (\%, \%)$	In-place Euclidean division
euclid:	$(\%, \%) \rightarrow \%$	Euclidean gcd
euclid!:	$(\%, \%) \rightarrow \%$	In-place Euclidean gcd
euclideanSize:	$\% \rightarrow \text{Integer}$	Size function of the domain
extendedEuclidean:	$(\%, \%) \rightarrow (\%, \%, \%)$	Extended Euclidean Algorithm
	$(\%, \%, \%) \rightarrow \partial \text{Cross}(\%, \%)$	
quo:	$(\%, \%) \rightarrow \%$	Quotient
rem:	$(\%, \%) \rightarrow \%$	Remainder
remainder!:	$(\%, \%) \rightarrow \%$	In-place remainder
rationalReconstruction:	$(\%, \%, \text{Z}, \text{Z}) \rightarrow \partial \text{Cross}(\%, \%)$	Rational reconstruction

where

∂ == Partial
Z == Integer

Usage

diophantine(a, b, m)

Signature

diophantine: (% , % , %) $\rightarrow$ Partial %

Parameter	Type	Description
a	%	An element of the ring
b	%	The right hand side of the equation
m	%	The nonzero modulus

Returns

If the diophantine equation $ax \equiv b \pmod{m}$ has solutions in the ring, returns a solution x such that either $x = 0$ or $|x| < |m|$. Returns *failed* if the equation has no solution.

Usage

divide(a, b)
 a quo b
 a rem b

Signatures

divide: $(\%, \%) \rightarrow (\%, \%)$
 quo,rem: $(\%, \%) \rightarrow \%$

Parameter	Type	Description
a, b	$\%$	Element of the ring, $y \neq 0$

Returns

$a \text{ rem } b$ returns r such that either $r = 0$ or $0 \leq |r| < |b|$ and $a \equiv r \pmod{b}$, $a \text{ quo } b$ returns q such that $a - bq = 0$ or $0 \leq |a - bq| < |b|$, and $\text{divide}(a, b)$ returns the pair $(a \text{ quo } b, a \text{ rem } b)$.

Usage

divide!(x, y, z)

Signature

divide!: ($\%$, $\%$, $\%$) $\rightarrow$ ($\%$, $\%$)

Parameter	Type	Description
x	$\%$	An element of the ring (to be destroyed)
y	$\%$	An element of the ring
z	$\%$	A placeholder for the quotient (to be destroyed)

Returns

Returns (q, r) such that $x = qy + r$ and either $r = 0$ or $0 \leq |r| < |x|$, where the storage used by x and z is allowed to be destroyed or reused, so x and z is lost after this call.

Remarks

This function may cause x and z to be destroyed, so do not use it unless x and z have been locally allocated, and are guaranteed not to share space with other elements. Some functions are not necessarily copying their arguments and can thus create memory aliases.

See also

remainder!

Usage

euclid(x, y)
 euclid!(x, y)

Signature

euclid: (% , %) $\rightarrow$ %

Parameter	Type	Description
x, y	%	Elements of the ring

Returns

Returns $\text{gcd}(x, y)$ computed by the Euclidean algorithm. When using euclid!(x, y), the storage used by x and y is allowed to be destroyed or reused, so x and y are lost after this call.

Remarks

The call euclid!(x, y) may cause x and y to be destroyed, so do not use it unless x and y have been locally allocated, and are guaranteed not to share space with other elements. Some functions are not necessarily copying their arguments and can thus create memory aliases.

See also

gcd, gcd!

Usage

euclideanSize x

SignatureeuclideanSize: $\% \rightarrow \text{Integer}$

Parameter	Type	Description
x	$\%$	A nonzero element of the ring

Returns

Returns $|x|$, the euclidean size of x. It is connected to the Euclidean remainder, in that the remainder r of a by b is either 0, or satisfies $0 \leq |r| < |b|$.

Usage

extendedEuclidean(a, b)
 extendedEuclidean(a, b, c)

Signatures

extendedEuclidean: $(\%, \%) \rightarrow (\%, \%, \%)$
 extendedEuclidean: $(\%, \%, \%) \rightarrow \mathbf{Partial}(\%, \%)$

Parameter	Type	Description
a, b, c	$\%$	Elements of the ring

Returns

extendedEuclidean(a, b) returns (g, x, y) such that $g = \gcd(a, b) = ax + by$.
 extendedEuclidean(a, b, c) returns either a solution (x, y) of the diophantine equation $ax + by = c$, or *failed* if it has no solution in the ring.
 For the values returned by both calls, either $x = 0$ or $|x| < |b|$.

Usage

rationalReconstruction(u, m, n, d)

Signature

rationalReconstruction: (% , % , Integer, Integer) $\rightarrow$ Partial Cross(% , %)

Parameter	Type	Description
u	%	An element of the ring
m	%	A nonzero modulus
n, d	Integer	Bounds for the size of the result

Returns

Returns either (a, b) such that $a/b = u \pmod{m}$, $|a| \leq n$ and $|b| \leq d$, or *failed* if no such a, b exist.

Remarks

The resulting a and b might not be unique, depending on the values of the bounds n and d .

Usage

remainder!(x, y)

Signature

remainder!: (% , %) → %

Parameter	Type	Description
x	%	An element of the ring (to be destroyed)
y	%	An element of the ring

Returns

Returns the remainder of x by y , where the storage used by x is allowed to be destroyed or reused, so x is lost after this call.

Remarks

This function may cause x to be destroyed, so do not use it unless x has been locally allocated, and is guaranteed not to share space with other elements. Some functions are not necessarily copying their arguments and can thus create memory aliases.

Field

Usage

Field: Category

Description

Field is the category of commutative fields.

Exports

EuclideanDomain

Group

FiniteCharacteristic

Usage

FiniteCharacteristic: Category

Description

FiniteCharacteristic is the category of finite characteristic rings.

Exports

Ring

pthPower: % → % Exponentiation to the characteristic

pthPower!: % → % In-place exponentiation to the characteristic

Usage

pthPower x
 pthPower! x

Signature

pthPower: $\% \rightarrow \%$

Parameter	Type	Description
x	$\%$	An element of the ring

Returns

Return x^p where p is the characteristic of the ring.

Remarks

pthPower! does not make a copy of x , which is therefore modified after the call. It is unsafe to use the variable x after the call, unless it has been assigned to the result of the call, as in `x := pthPower! x`.

FiniteSet

Usage

FiniteSet: Category

Description

FiniteSet is the category of finite sets.

Exports

ExpressionType

TotallyOrderedType

#: Integer

number of elements

apply: (% , ExpressionTree) → ExpressionTree

Conversion to an expression tree

index: % → Integer

Index of an element

lookup: Integer → %

Element with a given index

random: () → %

Get a random element

Usage

#

Signatures

#: Integer

Returns

Returns the number of elements of the type.

Usage

apply(p, x)
p x

Signature

apply: (% , ExpressionTree) → ExpressionTree

Parameter	Type	Description
p	%	An element
x	ExpressionTree	A name for the variables

Returns

Returns p as an expression tree, using x as root variable name.

Usage

index *p*

Signature

index: $\% \rightarrow \text{Integer}$

Parameter	Type	Description
<i>p</i>	$\%$	An element

Returns

Returns the index of *p*.

See also

lookup(FiniteSet)

Usage

lookup *j*

Signature

lookup: Integer $\rightarrow$ %

Parameter	Type	Description
<i>j</i>	Integer	An index

Returns

Returns the element with index *j*.

See also

index(FiniteSet)

Usage

random()

Signature

random: $() \rightarrow \%$

Returns

Returns a random element.

FreeAlgebra

Usage

FreeAlgebra R: Category

Parameter	Type	Description
R	ExpressionType ArithmeticType	The coefficient domain

Description

FreeAlgebra R is a category for free algebras over an arbitrary arithmetic system R and with respect to an arbitrary basis. Its elements are assumed to have finite support.

Exports

FreeRRing R

If R has Ring then
Algebra R

FreeLinearArithmeticType

Usage

FreeLinearArithmeticType R: Category

Parameter	Type	Description
R	ExpressionType ArithmeticType	The coefficient domain

Description

FreeLinearArithmeticType R is the category of arithmetic types containing linear combinations of their elements with coefficients in R with respect to a linearly independent generating set. Its elements are not assumed to have finite support, so this type cannot be asserted to be a Algebra R even when R is a Ring.

Exports

FreeLinearCombinationType R
LinearArithmeticType R

FreeLinearCombinationType

Usage

FreeLinearCombinationType R: Category

Parameter	Type	Description
R	ExpressionType ArithmeticType	The coefficient domain

Description

FreeLinearCombinationType R is the category of types containing linear combinations of their elements with coefficients in R with respect to a linearly independent generating set. Its elements are not assumed to have finite support, so this type cannot be asserted to be a **Module** R even when R is a **Ring**.

Exports

ExpressionType

LinearCombinationType R

map: $(R \rightarrow R) \rightarrow \% \rightarrow \%$ Lift a mapping

map!: $(R \rightarrow R) \rightarrow \% \rightarrow \%$ Lift a mapping

Usage

```
map f
map! f
map(f)(m)
map!(f)(m)
```

Signature

```
map: (R → R) → % → %
```

Parameter	Type	Description
f	$R \rightarrow R$	A map
m	$\%$	An element of the module

Description

map(f)(m) returns

$$f(m) = \sum_i f(r_i)e_i$$

where $m = \sum_i r_i e_i$, while map(f) returns the mapping $m \rightarrow f(m)$. In both cases, map! does not make a copy of m but modifies it in place.

FreeModule

Usage

FreeModule R: Category

Parameter	Type	Description
R	ExpressionType ArithmeticType	The coefficient domain

Description

FreeModule is a category for free modules over an arbitrary arithmetic system R and with respect to an arbitrary basis. Its elements are assumed to have finite support.

Exports

FreeLinearCombinationType R		
leadingCoefficient:	$\% \rightarrow R$	The leading coefficient
nonZeroCoefficients:	$\% \rightarrow \text{Generator } R$	Iterate over the coefficients
reductum:	$\% \rightarrow \%$	All terms except the leading one
reductum!:	$\% \rightarrow \%$	All terms except the leading one
support:	$\% \rightarrow \text{Generator Cross}(R, \%)$	Make an iterator
term?:	$\% \rightarrow \text{Boolean}$	Test for a monomial
trailingCoefficient:	$\% \rightarrow R$	The trailing coefficient

If R has Ring then

Module R

If R has GcdDomain then

content:	$\% \rightarrow R$	Content
primitive:	$\% \rightarrow (R, \%)$	Content and primitive part
primitive!:	$\% \rightarrow (R, \%)$	Content and primitive part
primitivePart:	$\% \rightarrow \%$	Primitive part
primitivePart!:	$\% \rightarrow \%$	Primitive part

If R has Field then

monic:	$\% \rightarrow \%$	Make monic
monic!:	$\% \rightarrow \%$	Make monic

Usage

```

content m
content! m
primitive m
primitive! m
primitivePart m
primitivePart! m

```

Signatures

```

content,content!:      % → R
primitive,primitive!:  % → (R, %)
primitivePart,primitivePart!: % → %

```

Parameter	Type	Description
m	$\%$	An element of the module

Description

`content(m)` returns the gcd of the coefficients of m , while `primitive(m)` and `primitivePart(m)` return respectively $(c, c^{-1}m)$ and $c^{-1}m$ where $c = \text{content}(p)$.

Remarks

The storage used by m is allowed to be destroyed or reused if `content!`, `primitive!` or `primitivePart!` is used, so m is lost after those calls. This may cause m to be destroyed, so do not use this unless m has been locally allocated, and is guaranteed not to share space with other elements. Some functions are not necessarily copying their arguments and can thus create memory aliases.

Usage

term? m

Signatureterm?: % $\rightarrow$ Boolean

Parameter	Type	Description
m	%	An element of the module

ReturnsReturns *true* if $m = re$ for $r \in R$ and e an element of the basis, *false* otherwise.**Remarks**term?(0) returns *true*.

Usage

leadingCoefficient m
trailingCoefficient p

Signature

leadingCoefficient, trailingCoefficient: $\% \rightarrow R$

Parameter	Type	Description
m	$\%$	An element of the module

Returns

leadingCoefficient(m) and trailingCoefficient(m) return respectively the leading and trailing coefficient of m . Both return 0 when $p = 0$.

See also

coefficients, nonZeroCoefficients

Usagemonic *m*monic! *m***Signature**monic: % $\rightarrow$ %

Parameter	Type	Description
<i>m</i>	%	An element of the module

ReturnsReturns $r^{-1}m$ where r is the leading coefficient of m , returns 0 if $m = 0$.**Remarks**

The storage used by m is allowed to be destroyed or reused if `monic!` is used, so m is lost after this call. This may cause m to be destroyed, so do not use this unless m has been locally allocated, and is guaranteed not to share space with other elements. Some functions are not necessarily copying their arguments and can thus create memory aliases.

Usage

for c in nonZeroCoefficients m repeat { ... }

Signature

nonZeroCoefficients: % $\rightarrow$ Generator R

Parameter	Type	Description
m	%	An element of the module

Returns

Returns a generator that produces all the nonzero coefficients of m .

Usage

reductum m
 reductum! m

Signature

reductum: $\% \rightarrow \%$

Parameter	Type	Description
m	$\%$	An element of the module

Returns

Returns the reductum of m , *i.e.* $p - re$ where r is the leadingCoefficient of p and e the corresponding element. Returns 0 if $m = 0$.

Remarks

The storage used by m is allowed to be destroyed or reused if reductum! is used, so m is lost after this call. This may cause m to be destroyed, so do not use this unless m has been locally allocated, and is guaranteed not to share space with other elements. Some functions are not necessarily copying their arguments and can thus create memory aliases.

Usage

support m

Signature

support: % $\rightarrow$ Generator Cross(R, %)

Parameter	Type	Description
m	%	An element of the module

Description

support(m) generates the terms of m, *i.e.* the smallest number of pairs (r, e) where e lies in the basis and whose sum is m .

FreeRRing

Usage

FreeRRing R: Category

Parameter	Type	Description
R	ExpressionType ArithmeticType	The coefficient domain

Description

FreeRRing R is a category for free R-rings over an arbitrary arithmetic system R and with respect to an arbitrary basis. Its elements are assumed to have finite support.

Exports

FreeLinearArithmeticType R

FreeModule R

ground?: % $\rightarrow$ Boolean Test for a ground element

If R has CharacteristicZero then

CharacteristicZero

If R has FiniteCharacteristic then

FiniteCharacteristic

If R has Ring then

RRing R

if R has RittRing then

RittRing

Usage

ground? m

Signatureground?: % $\rightarrow$ Boolean

Parameter	Type	Description
m	%	An element of the module

ReturnsReturns *true* if $m = r \cdot 1$ for $r \in R$, *false* otherwise.

GcdDomain

Usage

GcdDomain: Category

Description

GcdDomain is the category of commutative Gcd domains.

Exports

IntegralDomain

gcd:	(%, %) → %	Gcd of 2 elements
gcd:	Generator % → %	Gcd of several elements
gcd!:	(%, %) → %	In-place gcd of 2 elements
gcdquo:	(%, %) → (%, %, %)	Gcd with quotients
gcdquo:	List % → (%, List %)	Gcd with quotients
lcm:	(%, %) → %	Lcm of 2 elements
lcm:	List % → %	Lcm of several elements

Usage

```

gcd( $x_1, x_2$ )
gcd!( $x_1, x_2$ )
gcd g
lcm( $x_1, x_2$ )
lcm [ $x_1, \dots, x_n$ ]

```

Signatures

```

gcd,lcm:  (% , %) → %
gcd!:     (% , %) → %
gcd:      Generator % → %
lcm:      List % → %

```

Parameter	Type	Description
x_i	%	Elements of the ring
g	Generator %	Generates elements of the ring

Returns

gcd(x_1, x_2) and lcm(x_1, x_2) return respectively a greatest common divisor and least common multiple of x_1 and x_2 , while gcd(g) return a greatest common divisor of all the elements generated by g and lcm($[x_1, \dots, x_n]$) return a least common multiple of the x_i .

Remarks

With certain types, for example polynomials, the generator version can be more efficient than iterating the binary version. The function gcd! may cause x_1 and x_2 to be destroyed, so do not use it unless x_1 and x_2 have been locally allocated, and are guaranteed not to share space with other elements. Some functions are not necessarily copying their arguments and can thus create memory aliases.

See also

gcdquo

Usage

gcdquo(x_1, x_2)
gcdquo [$x_1, \dots, x_n$]

Signatures

gcdquo: $(\%, \%) \rightarrow (\%, \%, \%)$
gcdquo: **List** $\% \rightarrow (\%, \text{List } \%)$

Parameter	Type	Description
x_i	$\%$	Elements of the ring

Returns

gcdquo(x_1, x_2) returns (g, y_1, y_2) where $g = \gcd(x_1, x_2)$, $x_1 = gy_1$ and $x_2 = gy_2$,
gcdquo($[x_1, \dots, x_n]$) returns $(g, [y_1, \dots, y_n])$ where $g = \gcd(x_1, \dots, x_n)$ and $x_i = gy_i$.

Remarks

With certain types, for example polynomials, the n-ary version can be more efficient than iterating the binary version.

See also

gcd

Group

Usage

Group: Category

Description

Group is the category of groups, not necessarily commutative.

Exports

Monoid

$/:$ $(\%, \%) \rightarrow \%$ quotient

$\text{inv}:$ $\% \rightarrow \%$ inverse

Usage
 x/y

Signature
 $/: \% \rightarrow \%$

Parameter	Type	Description
x,y	$\%$	Elements of the group

Returns
Returns xy^{-1} .

Usage

inv x

Signature

inv: % → %

Parameter	Type	Description
x	%	An element of the group

Returns

Returns x^{-1} .

IndexedFreeAlgebra

Usage

IndexedFreeAlgebra(R,E): Category

Parameter	Type	Description
R	ExpressionType ArithmeticType	The coefficient domain
E	ExpressionType TotallyOrderedType	The index domain

Description

IndexedFreeAlgebra(R , E) is a category for free algebras over an arbitrary arithmetic system R , with respect to a linearly independent generating set E . Its elements are assumed to have finite support.

Exports

IndexedFreeRRing(R , E)

IndexedFreeLinearArithmeticType

Usage

`IndexedFreeLinearArithmeticType(R, E): Category`

Parameter	Type	Description
R	<code>ExpressionType</code> <code>ArithmeticType</code>	The coefficient domain
E	<code>ExpressionType</code>	The index domain

Description

`IndexedFreeLinearArithmeticType R` is the category of arithmetic types containing linear combinations of their elements with coefficients in `R` with respect to a linearly independent generating set. Its elements are not assumed to have finite support, so this type cannot be asserted to be a `Algebra R` even when `R` is a `Ring`.

Exports

`IndexedFreeLinearCombinationType(R, E)`

`FreeLinearArithmeticType R`

`add!:` $(\%, R, E, \%) \rightarrow \%$ Add a shifted element

Usage

add!(p, c, e, q)

Signature

add!: ($\%$, R, E, $\%$) $\rightarrow$ $\%$

Parameter	Type	Description
p	$\%$	An element of the module (to be destroyed)
c	R	A scalar
e	E	The degree of the term to add
q	$\%$	An element of the module

Returns

add!(p, c, e, q) computes the sum $p + ceq$.

Remarks

The storage used by p is allowed to be destroyed or reused, so p is lost after this call. This may cause p to be destroyed, so do not use this unless p has been locally allocated, and is thus guaranteed not to share space with other polynomials.

IndexedFreeLinearCombinationType

Usage

IndexedFreeLinearCombinationType(R , E): Category

Parameter	Type	Description
R	ExpressionType ArithmeticType	The coefficient domain
E	ExpressionType	The index domain

Description

IndexedFreeLinearCombinationType(R , E) is the category of types containing linear combinations of their elements with coefficients in R with respect to a linearly independent generating set. Its elements are not assumed to have finite support, so this type cannot be asserted to be a Module R even when R is a Ring.

Exports

FreeLinearCombinationType R
add!: ($\%$, R , E) $\rightarrow$ $\%$ In-place addition of a term
coefficient: ($\%$, E) $\rightarrow R$ Extraction of a coefficient
monomial: $E \rightarrow \%$ Creation of a monic term
term: (R , E) $\rightarrow \%$ Creation of a term

Usage

add!(p, c, e)

Signature

add!: ($\%$, R, E) $\rightarrow$ $\%$

Parameter	Type	Description
p	$\%$	An element of the module (to be destroyed)
c	R	A scalar
e	E	The degree of the term to add

Returns

add!(p, c, e) computes the sum $p + ce$.

Remarks

The storage used by p is allowed to be destroyed or reused, so p is lost after this call. This may cause p to be destroyed, so do not use this unless p has been locally allocated, and is thus guaranteed not to share space with other polynomials. Some functions, like `reductum` are not necessarily copying their arguments and can thus create memory aliases.

Usage

coefficient(p , e)

Signature

coefficient: $(\%, E) \rightarrow R$

Parameter	Type	Description
p	$\%$	An element of the module
e	E	An exponent

Returns

Returns the coefficient of e in p .

Usage

monomial(*e*)
term(*r*, *e*)

Signatures

monomial: $E \rightarrow \%$
term: $(R, E) \rightarrow \%$

Parameter	Type	Description
<i>r</i>	R	A scalar
<i>e</i>	E	An exponent

Returns

monomial(*e*) and term(*r*, *e*) return respectively the monomial *e* and the term *re*.

IndexedFreeModule

Usage

IndexedFreeModule(R,E): Category

Parameter	Type	Description
R	ExpressionType ArithmeticType	The coefficient domain
E	ExpressionType TotallyOrderedType	The index domain

Description

IndexedFreeModule(R , E) is a category for free modules over an arbitrary arithmetic system R , with respect to a linearly independent generating set E . Its elements are assumed to have finite support.

Exports

FreeModule R

IndexedFreeLinearCombinationType(R , E)

degree:	% $\rightarrow$ E	degree
generator:	% $\rightarrow$ Generator Cross(R , E)	iterate through all the terms
leadingMonomial:	% $\rightarrow$ %	leading monomial
leadingTerm:	% $\rightarrow$ (R , E)	leading term
terms:	% $\rightarrow$ Generator Cross(R , E)	iterate through all the terms
trailingDegree:	% $\rightarrow$ E	trailing degree
trailingMonomial:	% $\rightarrow$ %	trailing monomial
trailingTerm:	% $\rightarrow$ (R , E)	trailing term

if R has HashType and E has HashType then

HashType

if R has SerializableType and E has SerializableType then

SerializableType

Usage

degree p
trailingDegree p

Signature

degree,trailingDegree: % $\rightarrow$ E

Parameter	Type	Description
p	%	A nonzero element of the module

Returns

degree(p) and trailingDegree(p) return respectively the degree and trailing degree of p , *i.e.* e_n and e_m where $p = \sum_{i=m}^n a_i e_i$ and $a_n \neq 0 \neq a_m$. When $p = 0$, both functions return an arbitrary value in E .

See also

leadingCoefficient,leadingMonomial, trailingCoefficient,trailingMonomial

Usage

```

for term in p repeat { (c, n) := term; ... }
for term in generator p repeat { (c, n) := term; ... }
for term in terms p repeat { (c, n) := term; ... }

```

Signature

```

generator, terms:  % → Generator Cross(R, E)

```

Parameter	Type	Description
p	%	An element of the module

Description

Both functions allow an element of the module to be iterated independently of its representation. Both generators yield pairs of the form (a, e) , with $a \neq 0$. The difference between the two is that `generator(p)` yields the terms in decreasing exponents, while `terms(p)` yields the terms in increasing exponents.

Example

```

import from Integer, DenseUnivariatePolynomial Integer;

x := monom;
p := x^3 + 2*x - 1;
for term in p repeat { (c, n) := term; print << c << ", " << n << newline }

```

writes

```

    1,3
    2,1
   -1,0

```

to the standard stream print.

See also

```

coefficients, revert

```

Usage

```
leadingMonomial p
(r, e) := leadingTerm p
trailingMonomial p
(r, e) := trailingTerm p
```

Signatures

```
leadingMonomial, trailingMonomial:  % → %
leadingTerm, trailingTerm:          % → (R, E)
```

Parameter	Type	Description
p	%	An element of the module

Returns

When $p = \sum_{i=m}^n a_i e_i$ and $a_n \neq 0 \neq a_m$, then `leadingMonomial(p)` and `trailingMonomial(p)` return respectively e_n and e_m , while then `leadingTerm(p)` and `trailingTerm(p)` return respectively (a_n, e_n) and (a_m, e_m) . When $p = 0$, `leadingMonomial(0)` and `trailingMonomial(0)` both return 0, while `leadingTerm(p)` and `trailingTerm(p)` are undefined.

See also

```
degree,leadingCoefficient, trailingCoefficient,trailingDegree
```


IndexedFreeRRing

Usage

IndexedFreeRRing(R,E): Category

Parameter	Type	Description
R	ExpressionType ArithmeticType	The coefficient domain
E	ExpressionType TotallyOrderedType	The index domain

Description

IndexedFreeRRing(R , E) is a category for free R -rings over an arbitrary arithmetic system R , with respect to a linearly independent generating set E . Its elements are assumed to have finite support.

Exports

FreeRRing R

IndexedFreeModule(R , E)

IndexedFreeLinearArithmeticType(R , E)

IntegerCategory

Usage

IntegerCategory: Category

Description

IntegerCategory is the category of integer-like rings.

Exports

CharacteristicZero

EuclideanDomain

IntegerType

Parsable

Specializable

integer: $\% \rightarrow \text{Integer}$ Conversion to an integer

rationalReconstruction: $\% \rightarrow \% \rightarrow \text{Partial Cross}(\%, \%)$ Rational reconstruction

Usage

binomial(n, m)

Signaturebinomial: (%,%) $\rightarrow$ %

Parameter	Type	Description
n, m	%	Integers

Returns

Returns

$$\binom{n}{m} = \frac{n!}{m!(n-m)!}.$$

Returns 0 if either $m < 0$ or $n < m$.

Usage

integer n

Signature

integer: % $\rightarrow$ Integer

Parameter	Type	Description
n	%	An integer

Returns

Returns n as an integer.

Usage

rationalReconstruction m
 rationalReconstruction(m)(u)

Signature

rationalReconstruction: $\% \rightarrow \% \rightarrow \text{Partial Cross}(\%, \%)$

Parameter	Type	Description
m	$\%$	A nonzero modulus
u	$\%$	An Integer

Returns

rationalReconstruction(m)(u) returns either (a, b) such that $a/b = u \pmod{m}$, $|a| \leq \sqrt{(m-1)/2}$ and $|b| \leq \sqrt{(m-1)/2}$, or *failed* if no such a, b exist.

Remarks

The resulting a and b are guaranteed to be unique.

See also

rationalReconstruction

IntegralDomain

Usage

IntegralDomain: Category

Description

IntegralDomain is the category of commutative integral domains.

Exports

CommutativeRing

divDiffProd:	(%, %, %, %, %) → %	Combined product and quotient
divSumProd:	(%, %, %, %, %, %, %) → %	Combined product and quotient
exactQuotient:	(%, %) → Partial %	Exact quotient
order:	% → % → Integer	Order of divisibility
orderquo:	% → % → (Integer, %)	Order of divisibility and quotient
quotient:	(%, %) → %	Exact quotient
quotient!:	(%, %) → %	In-place exact quotient
quotientBy:	% → (% → %)	Exact quotient procedure
quotientBy!:	% → (% → %)	In-place exact quotient procedure

Usage

divDiffProd(a1, a2, b1, b2, q)

Signature

divDiffProd: (% , % , % , % , %) $\rightarrow$ %

Parameter	Type	Description
<i>a1</i> , <i>a2</i> , <i>b1</i> , <i>b2</i> , <i>q</i>	%	Elements of the ring

Returns

divDiffProd(a1, a2, b1, b2, q) returns $(a1\ a2 - b1\ b2)/q$.

Usage

divSumProd(a1, a2, b1, b2, c1, c2, q)

Signature

divSumProd: (% , % , % , % , % , % , %) $\rightarrow$ %

Parameter	Type	Description
<i>a1, a2, b1, b2, c1, c2, q</i>	%	Elements of the ring

Returns

divSumProd(a1, a2, b1, b2, c1, c2, q) returns $(a1\ a2 + b1\ b2 + c1\ c2)/q$.

Usage

exactQuotient(x, y)

Signature

exactQuotient: (% , %) $\rightarrow$ Partial %

Parameter	Type	Description
x	%	The numerator
y	%	The denominator

Returns

Returns the unique q such that $x = q y$ if such a q exists, *failed* otherwise.

See also

quotient

Usage

order a
 order(a)(b)

Signature

order: % $\rightarrow$ % $\rightarrow$ Integer

Parameter	Type	Description
a	%	A nonunit element of the domain
b	%	A nonzero element of the domain

Returns

order(a)(b) returns the unique nonnegative integer $n = \nu_a(b)$ such that $a^n \mid b$ and $a^{n+1} \nmid b$, while order(a) returns the map $b \rightarrow \nu_a(b)$.

Remarks

order(a) makes some precalculations based on a, so if you need to use order(a)(b) several times with the same a and different b's, it is more efficient to compute order(a) once and assign it, as in the example below. In addition, if you need to compute $b/a^{\nu_a(b)}$, then it is more efficient to use the orderquo function.

Example

The following function computes the orders at $x^2 + 1$ of a list of polynomials $l := [p_1, \dots, p_n]$:

```
orders(l:List DenseUnivariatePolynomial Integer):List Integer == {
  import from DenseUnivariatePolynomial Integer;
  nu := order(monom * monom + 1);    -- order function at x^2 + 1
  [nu(p) for p in l];
}
```

See also

orderquo

Usage

```
orderquo a
(n, c) := orderquo(a)(b);
```

Signature

```
orderquo:  % → % → (Integer, %)
```

Parameter	Type	Description
a	%	A nonunit element of the domain
b	%	A nonzero element of the domain

Returns

orderquo(a)(b) returns (n, c) such that $b = ca^n$ and $a \nmid c$, while orderquo(a) returns the map $b \rightarrow \text{orderquo}(a)(b)$.

Remarks

orderquo(a) makes some precalculations based on a, so if you need to use orderquo(a)(b) several times with the same a and different b's, it is more efficient to compute orderquo(a) once and assign it.

See also

```
order
```

Usage

```

quotient(x, y)
quotient!(x, y)
quotientBy(y)
quotientBy(y)(x)
quotientBy!(y)
quotientBy!(y)(x)

```

Signatures

```

quotient, quotient!:      (% , %) → %
quotientBy, quotientBy!: % → % → %

```

Parameter	Type	Description
x	%	The numerator
y	%	The denominator

Returns

`quotient(x,y)` and `quotient!(x,y)` return the unique q such that $x = qy$ if such a q exists, while `quotientBy(y)` returns the map $x \rightarrow \text{quotient}(x, y)$ and `quotientBy!(y)` returns the map $x \rightarrow \text{quotient!}(x, y)$. When using `quotient!` or `quotientBy!`, the storage used by x is allowed to be destroyed or reused, so x is lost after this call.

Remarks

Use those functions only when it is guaranteed that y divides x exactly in the ring. If there is a nonzero remainder, this function may produce a wrong quotient instead of an error, so use `exactQuotient` when there is the possibility of a nonzero remainder. When however y is known to divide x exactly, then `quotient` can have better efficiency than the other divisions.

See also

```
exactQuotient
```

LinearArithmeticType

Usage

LinearArithmeticType R:Category

Parameter	Type	Description
R	ExpressionType AdditiveType	The coefficient domain

Description

LinearArithmeticType R is the category of arithmetic types containing linear combinations of their elements with coefficients in R.

Remarks

Use `Algebra` instead if R is always meant to be a `Ring`.

Exports

ArithmeticType
ExpressionType
LinearCombinationType R
 $\wedge$: (`%`, `Integer`) $\rightarrow$ `%` exponentiation
`coerce`: `R` $\rightarrow$ `%` Natural embedding

Usage $x \wedge n$ **Signatures** $\wedge: (\%, \text{Integer}) \rightarrow \%$

Parameter	Type	Description
x	$\%$	an element of the type
n	Integer	an exponent

ReturnsReturns x to the power n .

Usage

coerce r

Signature

coerce: $R \rightarrow \%$

Parameter	Type	Description
r	R	An element of the base type

Returns

Returns $r \cdot 1$.

Module

Usage

Module R :Category

Parameter	Type	Description
R	Ring	The coefficient ring

Description

Module R is the category of modules over R .

Exports

AbelianGroup

ExpressionType

LinearCombinationType R

Monoid

Usage

Monoid: Category

Description

Monoid is the category of monoids, not necessarily commutative.

Exports

ExpressionType

1:	%	one
*	(%, %) → %	product
^:	(%, Integer) → %	exponentiation
one?:	% → Boolean	test for 1
times!:	(%, %) → %	In-place product

Usage

1

Signature

1: %

Returns

Returns the constant 1.

Usage $x * y$ **Signature** $*: (\%,\%) \rightarrow \%$

Parameter	Type	Description
x, y	$\%$	elements of the monoid

ReturnsReturns the product xy .

Usage $x \wedge n$ **Signatures** $\wedge: (\%, \text{Integer}) \rightarrow \%$

Parameter	Type	Description
x	$\%$	an element of the monoid
n	Integer	an exponent

ReturnsReturns x^n .

Usage

one? x

Signature

one?: % $\rightarrow$ Boolean

Parameter	Type	Description
x	%	An element of the monoid

Returns

Returns *true* if $x = 1$, *false* otherwise.

Usage

times!(x, y)

Signature

times!: (% , %) → %

Parameter	Type	Description
x	%	An element of the monoid (to be destroyed)
y	%	An element of the monoid to be multiplied by x

Returns

Returns the product xy , where the storage used by x is allowed to be destroyed or reused, so x can be lost after this call.

Remarks

This function may cause x to be destroyed, so do not use it unless x has been locally allocated, and is guaranteed not to share space with other elements. Some functions are not necessarily copying their arguments and can thus create memory aliases.

NonCommutativeIntegralDomain

Usage

NonCommutativeIntegralDomain: Category

Description

NonCommutativeIntegralDomain is the category of non-commutative integral domains.

Exports

Ring

leftExactQuotient: ($\%$, $\%$) $\rightarrow$ Partial $\%$ Left exact quotient

rightExactQuotient: ($\%$, $\%$) $\rightarrow$ Partial $\%$ Right exact quotient

Usage

leftExactQuotient(x, y)

Signature

leftExactQuotient: (% , %) → Partial %

Parameter	Type	Description
x	%	The numerator
y	%	The denominator

Returns

Returns either q such that $x = y q$ if such a q exists, *failed* otherwise.

See also

rightExactQuotient(NonCommutativeIntegralDomain)

Usage

rightExactQuotient(x, y)

Signature

rightExactQuotient: ($\%$, $\%$) $\rightarrow$ **Partial** $\%$

Parameter	Type	Description
x	$\%$	The numerator
y	$\%$	The denominator

Returns

Returns either q such that $x = q y$ if such a q exists, *failed* otherwise.

See also

leftExactQuotient(NonCommutativeIntegralDomain)

Ring

Usage

Ring: Category

Description

Ring is the category of rings, not necessarily commutative.

Exports

AbelianGroup

ArithmeticType

Monoid

characteristic: Integer

characteristic

coerce: Integer $\rightarrow$ %

embedding of the integers

factorial: (% , % , MachineInteger) $\rightarrow$ %

Generalized factorial

random: () $\rightarrow$ %

Get a random element

Usage

characteristic

Signature

characteristic: `Integer`

Returns

Returns the characteristic of the ring.

Usage

coerce n
n::%

Signature

coerce: Integer $\rightarrow$ %

Parameter	Type	Description
<i>n</i>	Integer	an integer

Returns

Returns *n* seen as an element of the ring.

Usage

factorial(a, s, n)

Signature

factorial: (% , % , MachineInteger) $\rightarrow$ %

Parameter	Type	Description
a, s	%	Elements of the ring
n	MachineInteger	A nonnegative integer

Returns

Returns the generalized factorial $\prod_{i=0}^{n-1}(a + is)$.

Usage

random()

Signature

random: $() \rightarrow \%$

Returns

Returns a random element.

RittRing

Usage

RittRing: Category

Description

RittRing is the category of rings of characteristic θ in which all nonzero integers are invertible. The center of such a ring contains an isomorphic image of the rational numbers.

Exports

CharacteristicZero

`/:` `(%, Integer) → %` Division by a nonzero integer

`inv:` `Integer → %` Inversion of a nonzero integer

Usage x / n **Signature** $/: (\%, \text{Integer}) \rightarrow \%$

Parameter	Type	Description
x	$\%$	An element of the ring
n	Integer	A nonzero integer

ReturnsReturns x/n as an element of the ring.

Usage

inv n

Signature

inv: Integer $\rightarrow$ %

Parameter	Type	Description
n	Integer	A nonzero integer

Returns

Returns $1/n$ as an element of the ring.

RRing

Usage

RRing R:Category

Parameter	Type	Description
R	Ring	The coefficient ring

Description

RRing R is the category of R-rings, *i.e.* rings that are also R-modules.

Exports

LinearArithmeticType R

Module R

Ring

Specializable

Usage

Specializable: Category

Description

Specializable is the category of specializable types.

Exports

specialization: (Image:CommutativeRing) $\rightarrow$ PartialFunction(%, Image) Morphism

Usage

specialization R

Signature

specialization: (Image:CommutativeRing) $\rightarrow$ PartialFunction(% ,Image)

Parameter	Type	Description
<i>R</i>	CommutativeRing	A ring

Returns

Returns a partial map from % into R.

ExpressionType

Usage

ExpressionType: Category

Description

ExpressionType is the category of types whose elements can be converted to `ExpressionTree`.

Exports

OutputType

PrimitiveType

extree: % → ExpressionTree Conversion to an expression tree

relativeSize: % → MachineInteger Complexity measure

Usage

extree x

Signature

extree: % $\rightarrow$ ExpressionTree

Parameter	Type	Description
x	%	The element to convert

Description

Converts x to an expression tree.

Usage

relativeSize x

SignaturerelativeSize: % $\rightarrow$ MachineInteger

Parameter	Type	Description
x	%	An element of the type

Returns

Returns some measure of the complexity of x.

Remarks

This measure does not have to be absolute, but it should be usable to compare 2 elements of the type and decide which one is the “cheapest” for calculations. This measure has no mathematical meaning in general, but can be used for selection strategies, for example in Gaussian elimination.

Automorphism

Usage

import from Automorphism R

Parameter	Type	Description
R	Ring	The ring on which the automorphisms operate

Description

Automorphism R provides automorphisms on R .

Exports

Group

apply:	$(\%, R, \text{Integer}) \rightarrow R$	Apply a morphism to an element of R
function:	$\% \rightarrow (R \rightarrow R)$	Action of a morphism
morphism:	$(R \rightarrow R) \rightarrow \%$	Create a morphism
	$(R \rightarrow R, R \rightarrow R) \rightarrow \%$	
	$((R, \text{Integer}) \rightarrow R) \rightarrow \%$	

Usage σx $\sigma(x, n)$ `apply(σ , x)``apply(σ , x, n)`**Signature**`apply: (% , R, n:Integer == 1) → R`

Parameter	Type	Description
σ	<code>%</code>	An automorphism of R
x	<code>R</code>	An element of R
n	<code>Integer</code>	The number of times to apply (optional)

ReturnsReturns $\sigma^n x$.

Usage

function σ

Signature

function: $\% \rightarrow (R \rightarrow R)$

Parameter	Type	Description
σ	$\%$	An automorphism of R

Returns

Returns the map corresponding to the action of σ on the ring.

Usage

morphism f
 morphism(f, f^{-1})
 morphism g

Signatures

morphism: $(R \rightarrow R) \rightarrow \%$
 morphism: $(R \rightarrow R, R \rightarrow R) \rightarrow \%$
 morphism: $((R, \text{Integer}) \rightarrow R) \rightarrow \%$

Parameter	Type	Description
f	$R \rightarrow R$	A function
f^{-1}	$R \rightarrow R$	The inverse function of f
g	$(R, \text{Integer}) \rightarrow R$	A function

Description

morphism f creates the morphism σ on R given by

$$\sigma x = f(x)$$

for any $x \in R$. The morphism is not necessarily invertible, so any attempt to use its inverse causes an error.

morphism(f, f^{-1}) creates the invertible morphism σ on R given by

$$\sigma x = f(x) \quad \sigma^{-1} x = f^{-1}(x)$$

for any $x \in R$.

morphism g creates the morphism σ on R given by

$$\sigma^n x = g(x, n)$$

for any $x \in R$. This morphism is considered invertible, so g must also be defined for negative integers.

Remarks

The maps passed as arguments must be ring morphisms, and the maps f and f^{-1} must be inverses of each other. When an efficient algorithm for computing σ^n is known, for example for $\sigma = 1_R$, then the form morphism g with $g: (R, \text{Integer}) \rightarrow R$ should be used to avoid repeated iterations of σ , which is the default behavior.

Usage

CanonicalSimplification: Category

Description

CanonicalSimplification is the category of domains supporting a canonical simplifier. This means that for any p such that p equals its canonical associate and for any a , the element $a \bmod p$ is a canonical representative of the residue class of a by p . That is, for any b the relation $a \bmod p = b \bmod p$ is equivalent to p divides $b - a$. In addition $a \bmod p$ equals its canonical associate. If the domain is Euclidean, then it must support also a symmetric canonical simplifier. See the paper *On the genericity of the Modular Gcd Algorithm* by Kaltofen and Monagan in the proc. of ISSAC 1999.

Exports

CommutativeRing

mod:	(%, %) → %	residue class representative
mod_+:	(%, %) → %	modular sum
mod_-:	(%, %) → %	modular difference
mod_*:	(%, %) → %	modular product
recipMod	(%, %) → Partial %	modular reciprocal

if % has EuclideanDomain then

symmetricMod: (%, %) → % symmetric residue class representative

ChineseRemaindering

Usage

import from ChineseRemainderingR

Parameter	Type	Description
R	EuclideanDomain	an Euclidean Domain.

Description

ChineseRemaindering provides Garner's Chinese Remaindering Algorithm implemented over an arbitrary EuclideanDomain.

Exports

combine: $(R,R) \rightarrow (R,R) \rightarrow R$ combine interpolated result with new modulus.
interpolate: $(\text{List } R, \text{List } R) \rightarrow R$ interpolate given residues and moduli.

if R has IntegerType then

combine: $(R, \text{MachineInteger}) \rightarrow (R, \text{MachineInteger}) \rightarrow R$ combine with new modulus.

Usage

combine(M,m)
 combine(M,m)(A,a)

Signatures

combine: $(R,R) \rightarrow (R,R) \rightarrow R$
 combine: $(R, \text{MachineInteger}) \rightarrow (R, \text{MachineInteger}) \rightarrow R$

Parameter	Type	Description
M	R	A product of primes.
m	R	A new modulus.
	MachineInteger	
A	R	The interpolated value modulo M .
a	R	The residue modulo m .
	MachineInteger	

Returns

Returns the unique X in $R/(mM)$ such that $X = A \pmod{M}$ and $X = a \pmod{m}$.

Usage

interpolate(*p*,*m*)

Signature

interpolate: (List R,List R) $\rightarrow$ R

Parameter	Type	Description
<i>p</i>	List R	A list of residues.
<i>m</i>	List R	A list of moduli.

Returns

Returns the interpolated value from the residues and the corresponding moduli.

Complex

Usage

import from Complex R

Parameter	Type	Description
R	ArithmeticType ExpressionType	Type to be extended

Description

Complex R implements the algebraic extension of R generated by a root of $X^2 + 1 = 0$. This type, already provided by `libaldor`, is extended by `Algebra`. Only the additional exports are documented here, see the `libaldor` reference manual for the basic exports and assumptions on the parameter R .

Exports

LinearArithmeticType R

if R has CharacteristicZero then
CharacteristicZero

if R has CommutativeRing then
CommutativeRing

if R has IntegralDomain then
IntegralDomain

if R has Field then
Field

if R has FiniteCharacteristic then
FiniteCharacteristic

if R has FiniteField then
FiniteField

if R has Parsable then
Parsable

if R has Ring then
Algebra R

if R has RittRing then
RittRing

Derivation

Usage

import from Derivation R

Parameter	Type	Description
R	Ring	The ring on which the derivations operate

Description

Derivation R provides derivations on R .

Exports

Module R

apply:	$(\%, R, \text{Integer}) \rightarrow R$	Differentiate an element of R
derivation:	$(R \rightarrow R) \rightarrow \%$	Create a derivation
function:	$\% \rightarrow (R \rightarrow R)$	Action of a derivation

Usage

apply(D , x)
apply(D , x , n)
 D x
 $D(x,n)$

Signature

apply: ($\%$, R , $.\text{Integer} == 1$) $\rightarrow R$

Parameter	Type	Description
D	$\%$	A derivation
x	R	An element to differentiate
n	Integer	The number of times to differentiate (optional)

Returns

Returns $D^n x$, *i.e.* the result of applying D to x n times.

Usage

derivation f

Signature

derivation: $(R \rightarrow R) \rightarrow \%$

Parameter	Type	Description
f	$R \rightarrow R$	A map

Returns

Returns the derivation D on R given by

$$Dx = f(x)$$

for any $x \in R$.

Remarks

f must satisfy the rules of a derivation, namely:

$$f(x + y) = f(x) + f(y), \quad \text{and} \quad f(xy) = xf(y) + f(x)y$$

for any $x, y \in R$.

Usage

function D

Signature

function: $\% \rightarrow (R \rightarrow R)$

Parameter	Type	Description
D	$\%$	A derivation

Returns

Returns the map corresponding to the action of D on the ring.

Fraction

Usage

import from Fraction R

Parameter	Type	Description
R	GcdDomain	a gcd domain

Description

Fraction R forms the fraction field of the gcd domain R . Fractions are automatically normalized in this field.

Exports

FractionFieldCategory R

FractionalRoot

Usage

import from FractionalRoot

Description

FractionalRoot(R) provides fractions of R with multiplicities.

Parameter	Type	Description
R	CommutativeRing	A ring

Exports

ExpressionType		
fractionalRoot:	$(R, R, \text{Integer}) \rightarrow \%$	Create a root
integral?:	$\% \rightarrow \text{Boolean}$	Test whether root is integral
integralRoot:	$(R, \text{Integer}) \rightarrow \%$	Create a root
integralValue:	$\% \rightarrow R$	Value of an integral root
multiplicity:	$\% \rightarrow \text{Integer}$	Multiplicity of a root
setMultiplicity!:	$(\%, \text{Integer}) \rightarrow \%$	Change a multiplicity
value:	$\% \rightarrow (R, R)$	Value of a root

Signature

integral?: % $\rightarrow$ Boolean

Usage

integral? r

Parameter	Type	Description
<i>r</i>	%	A root

Returns

Return *true* if r is in R, *false* otherwise.

Usage

fractionalRoot(*a*, *b*, *n*)
 integralRoot(*a*, *n*)

Signatures

fractionalRoot: (R, R, Integer) → %
 integralRoot: (R, Integer) → %

Parameter	Type	Description
<i>a</i>	R	A numerator
<i>b</i>	R	A denominator
<i>n</i>	Integer	A multiplicity

Returns

Return the root *a* or *a/b* with multiplicity *n*.

Usage

integralValue r

Signature

integralValue: % $\rightarrow$ Integer

Parameter	Type	Description
r	%	A root

Returns

Returns the value of the integral root r , ignoring its multiplicity.

See also

value

Usage

multiplicity *r*

Signature

multiplicity: $\% \rightarrow \text{Integer}$

Parameter	Type	Description
<i>r</i>	$\%$	A root

Returns

Return the multiplicity of *r*.

Usage

setMultiplicity!(*r*, *m*)

Signature

setMultiplicity!: (*%*, Integer) → Integer

Parameter	Type	Description
<i>r</i>	<i>%</i>	A root
<i>m</i>	Integer	Its new multiplicity

Description

Sets the multiplicity of *r* to *m* and returns *r*.

Usage

(n, d) := value r

Signature

value: % $\rightarrow$ (Integer, Integer)

Parameter	Type	Description
<i>r</i>	%	A root

Returns

Return (n, d) such that the value of r is n/d .

See also

integralValue

FractionBy

Usage

```
import from FractionBy(R, p, irr?)
```

Parameter	Type	Description
R	IntegralDomain	an integral domain
p	R	a nonzero nonunit of R
$irr?$	Boolean	Indicates whether p is known to be irreducible in R

Description

FractionBy(R, p, irr?) forms the fractions of the integral domain R by the nonzero nonunit p , *i.e.* the set of all fractions whose denominator is a power of p . Fractions are normalized in the sense that p does not divide the numerators. Indicating whether p is irreducible if for efficiency purposes only, always use *false* when it is unknown.

Exports

FractionByCategory R

FractionByCategory

Usage

FractionByCategory R: Category

Parameter	Type	Description
R	IntegralDomain	an integral domain

Description

FractionByCategory(R) is the category of fractions of the integral domain R by some nonzero nonunit $p \in R$, *i.e.* the set of all fractions whose denominator is a power of p .

Exports

FractionByCategory0 R

FractionByCategory0

Usage

FractionByCategory0 R: Category

Parameter	Type	Description
R	IntegralDomain	an integral domain

Description

FractionByCategory0(R) is the category of fractions of the integral domain R by some nonzero nonunit $p \in R$, *i.e.* the set of all fractions whose denominator is a power of p .

Exports

FractionCategory R
order: $\% \rightarrow \text{Integer}$ Valuation at p
shift: $(\%, \text{Integer}) \rightarrow \%$ Multiplication by a power of p

Usage

order x

Signature

order: % $\rightarrow$ Integer

Parameter	Type	Description
x	%	A fraction whose denominator is a power of p

Returns

Returns n such that $x = ap^n$ and $a \in R, p \nmid a$.

Usage

shift(x, n)

Signature

shift: ($\%$, Integer) $\rightarrow$ $\%$

Parameter	Type	Description
x	$\%$	A fraction whose denominator is a power of p
n	Integer	An exponent

Returns

Returns xp^n .

FractionCategory

Usage

FractionCategory R: Category

Parameter	Type	Description
<i>R</i>	IntegralDomain	an integral domain

Description

FractionCategory R is the category of subrings of the fraction field of R.

Exports

Algebra R

DifferentialExtension R

LinearAlgebraRing

denominator: % $\rightarrow$ R Denominator of a fraction

normalize: % $\rightarrow$ % Normalize a fraction

numerator: % $\rightarrow$ R Numerator of a fraction

if R has CharacteristicZero then

CharacteristicZero

if R has FactorizationRing then

FactorizationRing

if R has FiniteCharacteristic then

FiniteCharacteristic

if R has RationalRootRing then

RationalRootRing

if R has Specializable then

Specializable

if R has UnivariateGcdRing then

UnivariateGcdRing

Usage

denominator x
 numerator x

Signature

denominator,numerator: % $\rightarrow$ R

Parameter	Type	Description
x	%	A fraction

Returns

Returns respectively the denominator and the numerator of a fraction.

Usage

normalize x

Signature

normalize: % $\rightarrow$ %

Parameter	Type	Description
x	%	A fraction

Description

Normalize x as much as possible given the category of R .

FractionFieldCategory

Usage

FractionFieldCategory R: Category

Parameter	Type	Description
R	IntegralDomain	an integral domain

Description

FractionFieldCategory R is the category of the fraction fields of R.

Exports

FractionFieldCategory0 R

FractionFieldCategory0

Usage

FractionFieldCategory0 R: Category

Parameter	Type	Description
<i>R</i>	IntegralDomain	an integral domain

Description

FractionFieldCategory0 R is the category of the fraction fields of R.

Exports

Field
FractionCategory R
/: (R,R) → % Quotient of two ring elements

if *R* has CharacteristicZero then
 RittRing

if *R* has Parsable then
 Parsable

if *R* has SerializableType then
 SerializableType

if *R* has TotallyOrderedType then
 TotallyOrderedType

Usage

n / d

Signature

/: (R,R) → %

Parameter	Type	Description
<i>n</i>	R	An element of the ring.
<i>d</i>	R	A nonzero element of the ring.

Returns

Returns the quotient *n* over *d*.

LinearCombinationFraction

Usage

LinearCombinationFraction(R, LR, Q, LQ): Category

Parameter	Type	Description
R	IntegralDomain	an integral domain
LR	LinearCombinationType R	a type over R
Q	FractionCategory R	a fraction domain of R
LQ	LinearCombinationType Q	a type over Q

Description

LinearCombinationFraction(R, LR, Q, LQ) is a category for domains providing conversions between types integral and rational coefficients.

Exports

$*$: $(Q, LR) \rightarrow LQ$ Product of a fraction by an integral element
makeIntegral: $LQ \rightarrow (R, LR)$ Conversion to an integral element
makeRational: $LR \rightarrow LQ$ Conversion to a rational element

if Q has FractionByCategory0 R

makeIntegralBy: $LQ \rightarrow (Integer, LR)$ Conversion to an integral element

if Q has FractionFieldCategory0 R

normalize: $LR \rightarrow (R, LQ)$ Conversion to a monic rational element

Usage $c * A$ **Signature** $*: (Q, LR) \rightarrow LQ$

Parameter	Type	Description
c	Q	A fraction
A	LR	An integral element

Returns

Returns cA as a rational element.

Usage

$(a, A) := \text{makeIntegral } B$

Signature

$\text{makeIntegral}: \text{LQ} \rightarrow (\text{R}, \text{LR})$

Parameter	Type	Description
B	LQ	A rational element

Returns

Returns (a, A) such that $A = aB$ is integral. If R is a `GcdDomain`, then A is primitive.

Usage

$(\mu, A) := \text{makeIntegralBy } B$

Signature

$\text{makeIntegralBy}: \text{LQ} \rightarrow (\text{Integer}, \text{LR})$

Parameter	Type	Description
B	LQ	A rational element

Returns

Returns (μ, A) such that $A = \text{shift}(B, \mu)$ is integral.

Usage

makeRational A

Signature

makeRational: $LR \rightarrow LQ$

Parameter	Type	Description
A	LR	An integral element

Returns

Returns A as a rational element.

Usage

$(a, B) := \text{normalize } A$

Signature

normalize: $LR \rightarrow (R, LQ)$

Parameter	Type	Description
A	LR	An integral element

Returns

Returns (a, B) such that $A = aB$, and B is a normalized rational element.

Remarks

The normalization depends on the actual type LQ. For polynomial, it means that the leading coefficient is 1, for vectors that the first coordinate is 1, etc.

ModularComputation

Usage

ModularComputation: Category

Description

ModularComputation is the category of domains that support modular algorithm such as the modular gcd algorithm. More precisely, for every element p of $\%$ the operation `residueClassRing` returns a domain implementing the residue class ring R/p . In addition, if $\%$ is an Euclidean domain, the operation `combine` implements the Chinese Remaindering algorithm.

Exports

`CanonicalSimplification`

`residueClassRing: (p: %) → ResidueClassRing (% ,p)` Residue class ring

if $\%$ has `EuclideanDomain` then

`combine: (% , %) → (% , %) → %` Chinese remaindering algorithm

Product

Usage

import from Product R

Parameter	Type	Description
R	CommutativeRing	A commutative ring

Description

Product R provides finite products of elements of R, *i.e.* elements of the type $\prod_{i=1}^n r_i^{e_i}$ where the r_i 's are in R and the e_i 's are integers.

Exports

CopyableType

Monoid

#:	$\% \rightarrow \text{MachineInteger}$	Number of terms
divisors:	$\% \rightarrow \text{Generator R}$	Iterate through all the divisors
expand:	$\% \rightarrow R$	Multiply-out a product
expandFraction:	$\% \rightarrow (R, R)$	Multiply-out a product
generator:	$\% \rightarrow \text{Generator Cross}(R, \text{Integer})$	Make an iterator
log:	$(M:\text{AbelianMonoid}, R \rightarrow M) \rightarrow (\% \rightarrow M)$	Lift a logarithm
term:	$(R, \text{Integer}) \rightarrow \%$	Create a single term r^e
times!:	$(\%, R, \text{Integer}) \rightarrow \%$	In-place multiplication

Usage

p

Signatures

#: % → MachineInteger

Parameter	Type	Description
<i>p</i>	%	A product

Returns

Returns the number of terms in the product p.

Usage

for d in divisors p repeat { ... }

Signature

divisors: % $\rightarrow$ Generator R

Parameter	Type	Description
p	%	A product

Description

This generator yields all the products of the form $\prod_{i=1}^n r_i^{f_i}$ where $p = \prod_{i=1}^n r_i^{e_i}$ and $0 \leq f_i \leq e_i$.

Example

```
import from Integer, Product Integer, List Integer;

p := term(3, 1) * term(2, 2) * term(5, 2)      -- p = 3^1 2^2 5^2 = 300
l := sort! [divisors p];
creates the list
      [1,2,3,4,5,6,10,12,15,20,25,30,50,60,75,100,150,300]
```

of all the divisors of 300.

Usage

expand p

Signature

expand: % $\rightarrow$ R

Parameter	Type	Description
p	%	A product

Returns

Returns the product of all the terms in p.

Remarks

When R is not a field, expand(p) only works when p has only nonnegative exponents. Use `expandFraction` when p can have negative exponents and R is not a field.

Usage

expandFraction p

Signature

expandFraction: $\% \rightarrow (\mathbf{R}, \mathbf{R})$

Parameter	Type	Description
p	$\%$	A product

Returns

Returns (n, d) where n is the product of all the terms in p having positive exponents and d is the product of all terms in p having negative exponents.

See also

expand

Usage

for term in p repeat { (c, n) := term; ... }
 for term in generator p repeat { (c, n) := term; ... }

Signature

generator: % $\rightarrow$ **Generator Cross**(R, Integer)

Parameter	Type	Description
p	%	A product

Description

This function allows a product to be iterated independently of its representation. The generator yields pairs of the form (a, n) where a^n is a term in p .

Example

```
import from Integer, Product Integer;

p := term(3, 1) * term(2, 11) * term(5, 2)      -- p = 3^1 2^11 5^2
for term in p repeat { (c, n) := term; stdout << c << ", " << n << newline; }
writes
      3,1
      2,11
      5,2
to the standard stream stdout.
```

Usage

log(M,f)
 log(M,f)(p)

Signature

log: (M:AbelianMonoid, R → M) → % → M

Parameter	Type	Description
M	AbelianMonoid	the image monoid
f	$R \rightarrow M$	a logarithmic function on R
p	%	A product

Description

log(M,f)(p) returns $\sum_n n f(a_n)$ where $p = \prod_n a_n^n$, while log(M,f) returns the map $\prod_n a_n^n \rightarrow \sum_n n f(a_n)$.

Usage

term(*r*, *n*)

Signature

term: (R, Integer) → %

Parameter	Type	Description
<i>r</i>	R	A ring element
<i>n</i>	Integer	An exponent

Returns

Returns r^e as a product.

Usage

times!(p, r, n)

Signature

times!: (% , R, Integer) $\rightarrow$ %

Parameter	Type	Description
p	%	A product
r	R	A ring element
n	Integer	An exponent

Returns

Returns $p r^e$ as a product, where the storage used by p is allowed to be destroyed or reused, so p is lost after this call.

Remarks

The storage used by p is allowed to be destroyed or reused, so p is lost after this call. This may cause p to be destroyed, so do not use this unless p has been locally allocated, and is thus guaranteed not to share space with other polynomials. Some functions are not necessarily copying their arguments and can thus create memory aliases.

ResidueClassRing

Usage

ResidueClassRing (R,p): Category

Parameter	Type	Description
R	CommutativeRing	The ring
p	R	The modulo

Description

ResidueClassRing (R,p) specifies an implementation of the residue class ring R/p . The canonical pre-image of an element x of % is the element r of R such that x is a canonical representative of r and r equals its canonical associate.

Exports

CommutativeRing

modularRep: $R \rightarrow \%$ Residue class

canonicalPreImage: $\% \rightarrow R$ Canonical pre-image

if R has EuclideanDomain then

symmetricPreImage: $\% \rightarrow R$ Symmetric pre-image

ReducibleModulusException

Usage

```
throw ReducibleModulusException(R, m, a)
try ...catch E in { E has ReducibleModulusExceptionType R => ...}
```

Parameter	Type	Description
R	<code>CommutativeRing</code>	A commutative ring
m	<code>R</code>	The modulus
a	<code>R</code>	A proper factor of m

Description

`ReducibleModulusException(R, m, a)` is an exception type thrown by inversion modulo a reducible element of `R`.

ReducibleModulusExceptionType

Usage

ReducibleModulusExceptionType R: Category

Parameter	Type	Description
R	CommutativeRing	A commutative ring

Description

ReducibleModulusExceptionType R is the category of exceptions thrown by inversion modulo a reducible element of R. The constants **modulus** and **factor** contain the modulus and a proper factor respectively.

Usage

```
import from SimpleAlgebraicExtension(R, Rx, p)
import from SimpleAlgebraicExtension(R, Rx, p, x)
```

Parameter	Type	Description
R	IntegralDomain	The coefficient ring of the polynomials
Rx	UnivariatePolynomialAlgebra R	The type of the modulus
p	Rx	The modulus
x	Symbol	The generator name (optional)

Description

`SimpleAlgebraicExtension(R, Rx, p)` implements the algebraic extension $Rx/(p)$, where p is expected to be irreducible. It is possible to use this type with reducible p , but then divisions can throw the exception `ReducibleModulusException`. Use `UnivariatePolynomialMod` if you want to prevent divisions from being available.

Exports

```
SimpleAlgebraicExtensionCategory(R, Rx)
```

SimpleAlgebraicExtensionCategory

Usage

`SimpleAlgebraicExtensionCategory(R, Rx):Category`

Parameter	Type	Description
R	<code>IntegralDomain</code>	The coefficient ring of the polynomials
Rx	<code>UnivariatePolynomialAlgebra R</code>	The type of the modulus

Description

`SimpleAlgebraicExtensionCategory(R, Rx)` is the category of extensions of R of the form $Rx/(p)$ for some irreducible $p \in Rx$. Use `UnivariatePolynomialQuotientSqfr` when p is known to be reducible but squarefree, and `UnivariatePolynomialQuotient` when p is known to be reducible and not squarefree.

Remarks

It is possible to use this category with a reducible modulus, but divisions can then throw the exception `ReducibleModulusException`.

Exports

`IntegralDomain`

`UnivariatePolynomialQuotientSqfr(R, Rx)`

if R has `Field` then

`Field`

if R has `Field` and R has `CharacteristicZero` and R has `FactorizationRing` then

`FactorizationRing`

if R has `FiniteField` then

`FiniteField`

SourceOfPrimes

Usage

SourceOfPrimes: Category

Description

SourceOfPrimes is the category of domains supporting a (partial) primality test and a source of primes (which may finite or infinite).

Exports

CommutativeRing

prime?: % → Partial Boolean Primality test

prime?: % → Boolean Primality test

getPrime: () → Partial % Prime source seed

nextPrime: () → Partial % Prime source next

if % has EuclideanDomain then

getPrimeOfSize: () → Partial % Prime source seed

Usage

```
import from UnivariatePolynomialMod(R, Rx, p)
import from UnivariatePolynomialMod(R, Rx, p, x)
```

Parameter	Type	Description
R	<code>CommutativeRing</code>	The coefficient ring of the polynomials
Rx	<code>UnivariatePolynomialAlgebra R</code>	The type of the modulus
p	<code>Rx</code>	The modulus
x	<code>Symbol</code>	The generator name (optional)

Description

`UnivariatePolynomialMod(R, Rx, p)` implements the univariate polynomials modulo p , *i.e.* the ring $Rx/(p)$, where p is assumed to be monic, but with no other restrictions. Use instead the types `UnivariatePolynomialModSqfr` when p is also known to be squarefree, and `SimpleAlgebraicExtension` when p is also known to be irreducible.

Exports

```
UnivariatePolynomialQuotient(R, Rx)
```

Usage

```
import from UnivariatePolynomialModSqfr(R, Rx, p)
import from UnivariatePolynomialModSqfr(R, Rx, p, x)
```

Parameter	Type	Description
R	<code>CommutativeRing</code>	The coefficient ring of the polynomials
Rx	<code>UnivariatePolynomialAlgebra R</code>	The type of the modulus
p	<code>Rx</code>	The modulus
x	<code>Symbol</code>	The generator name (optional)

Description

`UnivariatePolynomialModSqfr(R, Rx, p)` implements the univariate polynomials modulo p , *i.e.* the ring $Rx/(p)$, where p is assumed to be monic and squarefree, but not necessarily irreducible. Use instead the types `UnivariatePolynomialMod` when p is not squarefree, and `SimpleAlgebraicExtension` when p is known to be irreducible.

Exports

```
UnivariatePolynomialQuotientSqfr(R, Rx)
```

Usage

UnivariatePolynomialQuotient(R, Rx):Category

Parameter	Type	Description
R	CommutativeRing	The coefficient ring of the polynomials
Rx	UnivariatePolynomialAlgebra R	A polynomial type over R

Description

UnivariatePolynomialQuotient(R, Rx) is the category of extensions of R of the form $Rx/(p)$ for some $p \in Rx$, where p is assumed to be monic, but with no other restrictions. Use instead the categories `UnivariatePolynomialQuotientSqfr` when p is also known to be squarefree, and `SimpleAlgebraicExtensionCategory` when p is also known to be irreducible.

Exports

Algebra R		
CommutativeRing		
coefficient:	$(\%, Z) \rightarrow R$	Extraction of a coefficient
compose:	$\% \rightarrow \% \rightarrow \%$	Modular composition
definingPolynomial:	Rx	Defining polynomial
generator:	$\% \rightarrow \text{Generator Cross}(R, Z)$	iterate through all the terms
knownIrreducible?:	Boolean	Irreducible modulus?
lift:	$\% \rightarrow Rx$	Conversion to a polynomial
map:	$(R \rightarrow R) \rightarrow \% \rightarrow \%$	Lift a mapping
map!:	$(R \rightarrow R) \rightarrow \% \rightarrow \%$	Lift a mapping
monom:	$\%$	Generator of the algebra
reduce:	$Rx \rightarrow \%$	Reduction of a polynomial
value:	$(P:\text{POLY } \%) \rightarrow (P, Z, Z) \rightarrow \%$	Evaluation at a rational number

where

$Z == \text{Integer}$
 $\text{POLY} == \text{UnivariatePolynomialAlgebra}$

if R has `CharacteristicZero` then

`CharacteristicZero`

if R has `FiniteCharacteristic` then

`FiniteCharacteristic`

if R has `FiniteSet` then

`FiniteSet`

Usage

compose(p)(q)

Parameter	Type	Description
p, q	%	Polynomials

Returns

Returns

$$q(p) = \sum_{i=0}^n a_i p^i$$

where $q = \sum_{i=0}^n a_i x^i$.

Remarks

If you want to compute $q_1(p), \dots, q_k(p)$ for several q_i 's, use the curried version as follows: `f := compose p; for i in 1..k repeat r.i := f(q.i);` , since the various calls to `f` will share a table of powers of p .

Usage

definingPolynomial

Signature

definingPolynomial: Rx

Returns

Returns the polynomial p such that the extension is $Rx/(p)$.

Usage`knownIrreducible?`**Signature**`knownIrreducible?: Boolean`**Returns**

Returns *true* if the modulus is known to be irreducible, *false* otherwise. Note that the modulus could be irreducible, even if `knownIrreducible?` is *false*.

Usage

lift q

Signaturelift: % $\rightarrow$ Rx

Parameter	Type	Description
q	%	An element of the algebraic extension

ReturnsReturns q as an element of Rx .

Usage

monom

Signature

monom: %

Returns

Returns the image in the quotient of the term x from Rx . That element generates this type as an algebra over R .

See also

monom

Usage

reduce q

Signature

reduce: Rx $\rightarrow$ %

Parameter	Type	Description
q	Rx	A polynomial

Returns

Returns the remainder of q modulo p as an element of $Rx/(p)$.

Usage

value(P)(p, n, d)

Signature

value: (P:UnivariatePolynomialAlgebra %) → (P, Integer, Integer) → %

Parameter	Type	Description
P	UnivariatePolynomialAlgebra %	A polynomial type
p	P	A polynomial
n	Integer	A numerator
d	Integer	A nonzero denominator

Returns

Returns $d^e p(n/d)$ where e is the smallest nonnegative exponent such that $d^e p(n/d)$ is an element of the extension.

Usage

UnivariatePolynomialQuotientSqfr(R, Rx):Category

Parameter	Type	Description
R	CommutativeRing	The coefficient ring of the polynomials
Rx	UnivariatePolynomialAlgebra R	The type of the modulus

Description

UnivariatePolynomialQuotientSqfr(R , Rx) is the category of extensions of R of the form $Rx/(p)$ for some $p \in Rx$, where p is assumed to be monic and squarefree, but not necessarily irreducible. Use instead the categories **UnivariatePolynomialQuotient** when p is not squarefree, and **SimpleAlgebraicExtensionCategory** when p is known to be irreducible.

Exports

```

UnivariatePolynomialQuotient(R, Rx)
newtonSeries:  Rxx                Newton series of the generator
norm:         % → R              Norm
              (P:POLY %) → P → Rx
trace:        % → R              Trace
              (P:POLY %) → P → Rx

```

where

```

POLY == UnivariatePolynomialAlgebra
Rxx  == DenseUnivariateTaylorSeries R

```

if R has **RationalRootRing** then

```
RationalRootRing
```

if R has **CharacteristicZero** and R has **Field** then

```
DifferentialExtension R
```


Usage

newtonSeries

Signature

newtonSeries: DenseUnivariateTaylorSeries R

Returns

Returns the series

$$\sum_{n \geq 0} Tr(\alpha^n) x^n$$

where α is the image in the quotient of the term x from Rx , and Tr is the trace from the quotient into R .

See also

monom,trace

Usage

```

norm q
trace q
norm P
trace P
norm(P)(p)
trace(P)(p)

```

Signatures

```

norm,trace:  % → R
norm,trace:  (P:UnivariatePolynomialAlgebra %) → P → Rx

```

Parameter	Type	Description
q	<code>%</code>	An element of the algebraic extension
P	<code>UnivariatePolynomialAlgebra %</code>	A polynomial type
p	<code>P</code>	A polynomial

Description

`norm( $q(\alpha)$ )` and `trace( $q(\alpha)$ )` return respectively the product and sum of the $q(\alpha)$ over all the roots of the polynomial defining the extension, while `norm(P)( $p(\alpha, x)$ )` and `trace(P)( $p(\alpha, x)$ )` return respectively the product and sum of the $p(\alpha, x)$ over all the roots of the polynomial defining the extension.

SmallPrimes

Usage

```
import from SmallPrimes
```

Description

SmallPrimes implements functionalities to obtain and manipulate small odd primes. Only a specific set $\mathcal{P}$ of small primes is available.

Exports

```
PrimeCollection
```

Usage

```
import from LazyHalfWordSizePrimes
```

Description

LazyHalfWordSizePrimes implements functionalities to obtain and manipulate half-word-size odd primes for lazy algorithms. Those primes are a few bits less than half-word-size, which allows for accumulation before reduction. Only a specific set $\mathcal{P}$ of half-word-size primes is available.

Exports

PrimeCollection

HalfWordSizePrimes

Usage

import from HalfWordSizePrimes

Description

HalfWordSizePrimes implements functionalities to obtain and manipulate half-word-size odd primes. Only a specific set $\mathcal{P}$ of half-word-size primes is available.

Exports

PrimeCollection

WordSizePrimes

Usage

```
import from WordSizePrimes
```

Description

WordSizePrimes implements functionalities to obtain and manipulate word-size primes. Only a specific set $\mathcal{P}$ of word-size primes is available.

Exports

```
PrimeCollection
```

PrimeCollection

Usage

PrimeCollection: Category

Description

PrimeCollection is the category of collections of primes with various properties and various sizes.

Exports

allPrimes:	$() \rightarrow \text{Generator } Z$	Generate all the primes
fourierPrime:	$Z \rightarrow (Z, Z)$	Fourier prime
maxPrime:	$\rightarrow Z$	Largest prime
nextPrime:	$Z \rightarrow Z$	First prime above a given number
previousPrime:	$Z \rightarrow Z$	First prime below a given number
primeInCollection?:	$Z \rightarrow \text{Boolean}$	Check a prime
primRoot:	$Z \rightarrow Z$	Modular primitive root
randomPrime:	$() \rightarrow Z$	Random prime

where

$Z == \text{MachineInteger}$

Usage

for p in allPrimes() repeat { ... }

Signature

allPrimes: () → Generator MachineInteger

Description

This function allows a loop to iterate over all the primes provided by the collection.

Usage

fourierPrime n

Signature

fourierPrime: `MachineInteger` $\rightarrow$ (`MachineInteger`,`MachineInteger`)

Returns

Returns (p, ω) such that p is a prime of the form $p = 2^n k + 1$ with k odd, and ω is a primitive $2^{n^{\text{th}}}$ root of unity in $\mathbb{F}_p$. Returns $(0, 0)$ if n is too large.

Usage

maxPrime

Signature

maxPrime: MachineInteger

Returns

Returns the largest prime in the collection.

Usage

nextPrime n

Signature

nextPrime: MachineInteger $\rightarrow$ MachineInteger

Parameter	Type	Description
n	MachineInteger	An integer

Returns

Returns the smallest prime p in the collection with $n < p$, 0 if there are none.

See also

previousPrime(PrimeCollection), randomPrime(PrimeCollection)

Usage

previousPrime n

Signature

previousPrime: MachineInteger $\rightarrow$ MachineInteger

Parameter	Type	Description
n	MachineInteger	An integer

Returns

Returns the largest prime p in the collection with $n > p$, 0 if there are none.

See also

nextPrime(PrimeCollection), randomPrime(PrimeCollection)

Usage

primeInCollection? n

Signature

primeInCollection?: MachineInteger $\rightarrow$ Boolean

Parameter	Type	Description
n	MachineInteger	An integer

Returns

Returns *true* if n is a prime in the collection, *false* otherwise (n could still be prime in that case).

Usage

primRoot p

Signature

primRoot: MachineInteger $\rightarrow$ MachineInteger

Parameter	Type	Description
p	MachineInteger	A prime

Returns

Returns a generator of the multiplicative group $(\mathbb{Z}/p\mathbb{Z})^*$ or 0 if p is not a prime in the table, or is no primitive root is stored.

See also

primitiveRoot(PrimitiveRoots)

Usage

randomPrime()

Signature

randomPrime: () → MachineInteger

Returns

Returns a random prime in the collection.

See also

nextPrime(PrimeCollection), previousPrime(PrimeCollection)

PrimeField2

Usage

```
import from PrimeField2
```

Description

PrimeField2 implements the finite field with two elements.

Exports

```
SmallPrimeFieldCategory
```


PrimeFieldCategory

Usage

PrimeFieldCategory: Category

Description

PrimeFieldCategory is the category for prime fields, *i.e.* fields of the form $\mathbb{Z}/p\mathbb{Z}$ where $p \in \mathbb{Z}$ is a prime.

Exports

PrimeFieldCategory0

FactorizationRing

roots: (P:POLY %) → P → List Cross(%, Integer) In-field roots

rootsSqfr: (P:POLY %) → P → List % In-field roots

where

POLY == UnivariatePolynomialAlgebra0

Usage

rootsSqfr P
rootsSqfr(P)(p)

Signature

rootsSqfr: (P:UnivariatePolynomialAlgebra0 %) → P → Generator %

Parameter	Type	Description
P	UnivariatePolynomialAlgebra0 %	a polynomial type
p	P	a squarefree polynomial

Returns

Returns a generator that produces all the roots of p , which must be squarefree, in its coefficient field.

PrimeFieldCategory0

Usage

PrimeFieldCategory0: Category

Description

PrimeFieldCategory0 is the category for prime fields, *i.e.* fields of the form $\mathbb{Z}/p\mathbb{Z}$ where $p \in \mathbb{Z}$ is a prime.

Exports

FiniteField

SerializableType

lift: $\% \rightarrow \text{Integer}$ Conversion to an integer

Usage

lift x

Signature

lift: % $\rightarrow$ Integer

Parameter	Type	Description
x	%	an element of the field

Returns

Return x seen as an integer.

PrimitiveRoots

Usage

import from PrimitiveRoots

Description

PrimitiveRoots implements functionalities needed to compute generators of small cyclic groups.

Exports

`factors:` $Z \rightarrow \text{List } Z$ List of prime factors
`primitiveRoot:` $Z \rightarrow Z$ Modular primitive root

where

$Z == \text{MachineInteger}$

Usage

factors n

Signature

factors: `MachineInteger` $\rightarrow$ `List MachineInteger`

Parameter	Type	Description
n	<code>MachineInteger</code>	An integer

Returns

Returns all the prime factors p of n with $1 < d < |n|$.

Usage

primitiveRoot p

Signature

primitiveRoot: MachineInteger $\rightarrow$ MachineInteger

Parameter	Type	Description
p	MachineInteger	A prime

Returns

Returns a generator of the multiplicative group $(\mathbb{Z}/p\mathbb{Z})^*$.

Remarks

The argument p must be a prime number, otherwise this function returns any integer with no significance.

PthPowering

Usage

import from PthPowering R

Parameter	Type	Description
R	FiniteCharacteristic	A finite characteristic ring

Description

PthPowering provides efficient exponentiation of elements of R .

Exports

pExponentiation: $(T, \text{Integer}) \rightarrow T$ p^{th} -powering
pExponentiation!: $(T, \text{Integer}) \rightarrow T$ In-place p^{th} -powering

Usage

pExponentiation(*a*, *n*)
 pExponentiation!(*a*, *n*)

Signature

pExponentiation: $(R, \text{Integer}) \rightarrow R$

Parameter	Type	Description
<i>a</i>	<i>R</i>	The element to exponentiate
<i>n</i>	Integer	The exponent

Returns

Returns a^n . The exponent n must be nonnegative. When using pExponentiation!(a, n), the storage used by a is allowed to be destroyed or reused, so a is lost after this call.

Remarks

A call to pExponentiation!(a, n) may cause a to be destroyed, so do not use it unless a has been locally allocated, and is guaranteed not to share space with other elements. Some functions are not necessarily copying their arguments and can thus create memory aliases.

SmallPrimeField

Usage

import from SmallPrimeField p

Parameter	Type	Description
p	MachineInteger	The characteristic

Description

SmallPrimeField p implements the finite field $\mathbb{Z}/p\mathbb{Z}$ where $p \in \mathbb{Z}$ is a word-size prime.

Exports

SmallPrimeFieldCategory

SmallPrimeField0

Usage

import from SmallPrimeField0 p

Parameter	Type	Description
<i>p</i>	MachineInteger	The characteristic

Description

SmallPrimeField0 p is an internal implementation of the finite field $\mathbb{Z}/p\mathbb{Z}$ where $p \in \mathbb{Z}$ is a word-size prime. Use SmallPrimeField instead.

Exports

SmallPrimeFieldCategory0

SmallPrimeFieldCategory

Usage

SmallPrimeFieldCategory: Category

Description

SmallPrimeFieldCategory is the category for prime fields of the form $\mathbb{Z}/p\mathbb{Z}$ where $p \in \mathbb{Z}$ is a machine prime.

Exports

PrimeFieldCategory
SmallPrimeFieldCategory0
UnivariateGcdRing

SmallPrimeFieldCategory0

Usage

SmallPrimeFieldCategory0: Category

Description

SmallPrimeFieldCategory0 is the category for prime fields of the form $\mathbb{Z}/p\mathbb{Z}$ where $p \in \mathbb{Z}$ is a machine prime.

Exports

PrimeFieldCategory0

SerializableType

coerce: MachineInteger $\rightarrow$ % conversion to a field element

discreteLogTable: Cross(A, A, Boolean) discrete log table if available

machine: % $\rightarrow$ MachineInteger conversion to a machine integer

where

A == PrimitiveArray MachineInteger

Usage

n::%
machine m

Signatures

coerce: MachineInteger → %
machine: % → MachineInteger

Parameter	Type	Description
<i>m</i>	%	an element of the field
<i>n</i>	MachineInteger	a machine integer

Returns

n::% returns n as an element of the field and machine(m) returns m as a MachineInteger.

Usage

`((log, exp, ok?) := discreteLogTable`

Signature

`discreteLogTable: Cross(A, A, Boolean)`

where

`A == PrimitiveArray MachineInteger`

Returns

Returns `(log, exp, ok?)` such that if `ok?` is *true*, then `exp.i` is g^i and `log.i` is $\log_g(i)$ where g is a generator of the multiplicative of the group (g is stored in `exp.1`).

ZechPrimeField

Usage

import from ZechPrimeField p

Parameter	Type	Description
p	MachineInteger	The characteristic

Description

ZechPrimeField p implements the finite field $\mathbb{Z}/p\mathbb{Z}$, using discrete logarithm and exponential tables for multiplication, where $p \in \mathbb{Z}$ is a word-size prime.

Exports

SmallPrimeFieldCategory

Backsolve

Usage

import from Backsolve(R,M)

Parameter	Type	Description
R	IntegralDomain	A coefficient ring
M	MatrixCategory R	A matrix type over R

Description

(R,M) provides operations for backsolving triangular systems

Exports

backsolve: (M,Z $\rightarrow$ Z,ARR Z,Z,Z,R) $\rightarrow$ V R Backsolve a triangular system
backsolve: (M,Z $\rightarrow$ Z,ARR Z,Z,M,R) $\rightarrow$ (M, V R) Backsolve a triangular system

if R has GcdDomain then

backsolve: (M,Z $\rightarrow$ Z,ARR Z,Z,Z) $\rightarrow$ V R Backsolve a triangular system
backsolve: (M,Z $\rightarrow$ Z,ARR Z,Z,M) $\rightarrow$ (M, V R) Backsolve a triangular system
backsolve2: (M,Z $\rightarrow$ Z,ARR Z,Z,Z,R) $\rightarrow$ V R Backsolve a triangular system
backsolve2: (M,Z $\rightarrow$ Z,ARR Z,Z,M,R) $\rightarrow$ (M, V R) Backsolve a triangular system

where

Z == MachineInteger
ARR == PrimitiveArray
V == Vector

Usage

```

backsolve(a,p,st,r,c)
backsolve(a,p,st,r,b)
backsolve(a,p,st,r,c,d)
backsolve(a,p,st,r,b,d)

```

Signatures

```

backsolve: (M,Z → Z,PrimitiveArray Z,Z,Z) → Vector R
backsolve: (M,Z → Z,PrimitiveArray Z,Z,M) → (M,Vector R)
backsolve: (M,Z → Z,PrimitiveArray Z,Z,Z,R) → Vector R
backsolve: (M,Z → Z,PrimitiveArray Z,Z,M,R)→(M,Vector R)

```

Parameter	Type	Description
a	M	A matrix representing a Row Echelon Form (REF)
p	$Z \rightarrow Z$	A permutation of the rows of a
st	PrimitiveArray Z	The stairs of the REF
r	Z	The number of leading columns (before the c^{th} column)
c	Z	The column to be backsolved
b	M	A matrix whose columns have to be backsolved
d	R	A maximal denominator needed for a dependence relation

where

$Z == \text{MachineInteger}$

Description

Backsolves a triangular system. The triple (a, p, st) represents a matrix in REF: for $j \geq st(i)$ entry (i,j) of the REF is stored in $a(p(i), j)$. The parameter d may be omitted if R is has `GcdDomain`, in which case the system is solved in a minimal way. Otherwise, d must be such that d times the c -th column of the REF (resp. d times the columns of b) is a linear combination of the first r leading columns of the REF (the j^{th} column is called leading if $j = st(i)$ for some i).

Returns

`backsolve(a,p,st,r,c)` returns a primitive vector v such that $av = 0$.
`backsolve(a,p,st,r,c,d)` returns a vector v such that $av = 0$, the c^{th} coordinate of v is d and otherwise $v(j) \neq 0$ only if $j \leq r$ and the j -th column is leading.
`backsolve(a,p,st,r,b)` returns a matrix s and a vector t such that when $t(l) \neq 0$, then a times the l^{th} column of s equals $t(l)$ times the l^{th} column of b , and when $t(l) = 0$, then the l^{th} column of b is not a linear combination of the columns of a . $s(j, l) \neq 0$ only if the j^{th} column is leading. Furthermore the gcd of all the entries in the l^{th} column of s and $t(l)$ is 1.
`backsolve(a,p,st,r,b,d)` returns a matrix s and a vector t such that when $t(l) \neq 0$, then a times the l^{th} column of s equals d times the l^{th} column of b , and when $t(l) = 0$, then the l^{th} column of b is not a linear combination of the columns of a . $s(j, l) \neq 0$ only if the j^{th} column is leading.

Usage

```
backsolve2(a,p,st,r,c,d)
backsolve2(a,p,st,r,b,d)
```

Signatures

```
backsolve2: (M,Z → Z,PrimitiveArray Z,Z,Z,R) → Vector R
backsolve2: (M,Z → Z,PrimitiveArray Z,Z,M,R) → (M,Vector R)
```

Parameter	Type	Description
<i>a</i>	M	A matrix representing a Row Echelon Form (REF)
<i>p</i>	$Z \rightarrow Z$	A permutation of the rows of <i>a</i>
<i>st</i>	PrimitiveArray Z	The stairs of the REF
<i>r</i>	Z	The number of leading columns (before the c^{th} column)
<i>c</i>	Z	The column to be backsolved
<i>b</i>	M	A matrix whose columns have to be backsolved
<i>d</i>	R	A maximal denominator needed for a dependence relation

where

$Z == \text{MachineInteger}$

Description

Backsolves a triangular system in a minimal way. The triple (a, p, st) represents a matrix in REF: for $j \geq st(i)$ entry (i,j) of the REF is stored in $a(p(i), j)$. The parameter d must be such that d times the c -th column of the REF (resp. d times the columns of b) is a linear combination of the first r leading columns of the REF (the j^{th} column is called leading if $j = st(i)$ for some i).

Returns

`backsolve2(a,p,st,r,c,d)` returns a primitive vector v such that $av = 0$, and $v(j) \neq 0$ only if $j = c$ or $j \leq r$ and the j^{th} column is leading.

`backsolve2(a,p,st,r,b,d)` returns a matrix s and a vector t such that when $t(l) \neq 0$, then a times the l^{th} column of s equals $t(l)$ times the l^{th} column of b , and when $t(l) = 0$, then the l^{th} column of b is not a linear combination of the columns of a . $s(j, l) \neq 0$ only if the j^{th} column is leading.

DenseMatrix

Usage

import from DenseMatrix R

Parameter	Type	Description
R	ExpressionType ArithmeticType	The coefficient domain

Description

DenseMatrix R provides dense mutable matrices with entries in R. They are 1-indexed and do not bound check.

Exports

MatrixCategory R

DivisionFreeGaussElimination

Usage

import from DivisionFreeGaussElimination(R,M)

Parameter	Type	Description
R	CommutativeRing	A coefficient ring
M	MatrixCategory R	A matrix type over R

Exports

LinearEliminationCategory(R,M)

Description

This domain implements division-free Gaussian elimination on matrices.

FractionFreeGaussElimination

Usage

import from FractionFreeGaussElimination(R,M)

Parameter	Type	Description
R	IntegralDomain	A coefficient ring
M	MatrixCategory R	A matrix type over R

Exports

LinearEliminationCategory(R,M)

Description

This domain implements fraction-free Gaussian elimination on matrices.

HermiteGaussElimination

Usage

import from HermiteGaussElimination(R,M)

Parameter	Type	Description
R	EuclideanDomain	A coefficient ring
M	MatrixCategory R	A matrix type over R

Exports

LinearEliminationCategory(R,M)

Description

This domain implements Hermite reduction on matrices.

Usage

```
import from LinearAlgebra(R, M)
```

Parameter	Type	Description
R	<code>CommutativeRing</code>	The coefficient domain
M	<code>MatrixCategory R</code>	A matrix type

Description

`LinearAlgebra(R, M)` provides basic linear algebra functionalities for matrices over R .

Exports

<code>invertibleSubmatrix:</code>	$M \rightarrow (\text{Boolean}, \text{AZ}, \text{AZ})$	Probable maximal minor
<code>maxInvertibleSubmatrix:</code>	$M \rightarrow (\text{AZ}, \text{AZ})$	Maximal minor
<code>span:</code>	$M \rightarrow \text{AZ}$	Span

if R has `IntegralDomain` then

<code>cycle:</code>	$(V \rightarrow V, V) \rightarrow (V, M)$	First dependence relation
<code>cycle:</code>	$(M, V) \rightarrow (V, M)$	First dependence relation
<code>determinant:</code>	$M \rightarrow R$	Determinant
<code>factorOfDeterminant:</code>	$M \rightarrow (\text{Boolean}, R)$	Probable determinant
<code>firstDependence:</code>	$(\text{Generator } V, \text{MachineInteger}) \rightarrow V$	First dependence relation
<code>inverse:</code>	$M \rightarrow (M, V)$	Inverse
<code>kernel:</code>	$M \rightarrow M$	Kernel
<code>particularSolution:</code>	$(M, M) \rightarrow (M, V)$	A solution
<code>rank:</code>	$M \rightarrow \text{MachineInteger}$	Rank
<code>rankLowerBound:</code>	$M \rightarrow (\text{Boolean}, \text{MachineInteger})$	Probable rank
<code>solve:</code>	$(M, M) \rightarrow (M, M, V)$	All solutions
<code>subKernel:</code>	$M \rightarrow (\text{Boolean}, M)$	Subspace of the kernel

where

```
AZ == Array MachineInteger
V   == Vector R
```


Usage

cycle(f,v)
cycle(A,v)

Signatures

cycle: (Vector R → Vector R, Vector R) → (Vector R, M)
cycle: (M, Vector R) → (Vector R, M)

Parameter	Type	Description
f	Vector R → Vector R	A map
A	M	A matrix
v	Vector R	A vector whose cycle is wanted

Returns

Returns $([a_0, \dots, a_n], m)$ where

$$\sum_{i=0}^n a_i A^i v = 0 \quad \left(\text{resp. } \sum_{i=0}^n a_i f^i(v) = 0 \right),$$

and the columns of m are $v, f(v), \dots, f^n(v)$ (resp. $v, Av, \dots, A^n v$).

Remarks

The relation is as small as possible, meaning that $v, f(v), \dots, f^{n-1}(v)$ (resp. $v, Av, \dots, A^{n-1}v$) are linearly independent over R. The iterates of v under f must all have the same dimension.

See also

firstDependence

Usage

determinant *a*

Signature

determinant: $M \rightarrow R$

Parameter	Type	Description
<i>a</i>	M	A matrix

Returns

Returns the determinant of *a*.

See also

factorOfDeterminant

Usage

factorOfDeterminant a

Signature

factorOfDeterminant: $M \rightarrow (\text{Boolean}, R)$

Parameter	Type	Description
<i>a</i>	M	A matrix

Returns

Returns $(det?, d)$ such that d is always a factor of the determinant of a , and d is exactly the determinant of a if $det?$ is *true*.

Remarks

d can also happen to be the determinant of a when $det?$ is *false*, but the algorithm was unable to prove it.

See also

determinant

Usage

firstDependence(*gen*,*n*)

Signature

firstDependence: (Generator Vector R, MachineInteger) $\rightarrow$ Vector R

Parameter	Type	Description
<i>gen</i>	Generator Vector R	A generator of vectors
<i>n</i>	MachineInteger	The dimension of the vectors generated

Description

Returns a vector v which contains the coefficients of a dependence relation among the vectors generated by *gen*. The relation is as small as possible, meaning that if v has dimension m then the first $m - 1$ vectors generated are linearly independent over R. The dimension of the vectors generated by *gen* must be n . There must be a relation between the vectors generated.

See also

cycle

Usage

inverse a

Signatureinverse: $M \rightarrow (M, \text{Vector } R)$

Parameter	Type	Description
a	M	A matrix

ReturnsReturns $(b, [d_1, \dots, d_n])$ such that

$$ab = \begin{pmatrix} d_1 & & & \\ & d_2 & & \\ & & \ddots & \\ & & & d_n \end{pmatrix}.$$

Remarks

$\prod_{i=1}^n d_i \neq 0$ if and only if a is invertible. In that case, $a^{-1} = bd^{-1}$ where d is the diagonal matrix with diagonal $d_1, \dots, d_n$. To compute the inverse of a as a product of a diagonal matrix on the left rather than the right, let (b, d) be the result of calling inverse on $\text{transpose}(a)$. Then, $(a^t)^{-1} = bd^{-1}$, so $a^{-1} = d^{-1}b^t$. When R is a **Field**, then $d_i \in \{0, 1\}$ for $1 \leq i \leq n$, so $b = a^{-1}$ when R is a **Field** and a is invertible.

Usage

invertibleSubmatrix a

Signature

invertibleSubmatrix: $M \rightarrow (\text{Boolean}, \text{Array MachineInteger}, \text{Array MachineInteger})$

Parameter	Type	Description
<i>a</i>	M	A matrix

Returns

Returns $(\text{max?}, [r_1, \dots, r_r], [c_1, \dots, c_r])$ where $r \leq \text{rank}(a)$ and the submatrix of a formed by the intersections of the rows r_i and c_i is always invertible. If *max?* is *true*, then r is exactly the rank of a and the given minor is of maximal size.

Remarks

r can also happen to be the rank of a when *max?* is *false*, but the algorithm was unable to prove it.

See also

maxInvertibleSubmatrix

Usage

kernel *a*

Signature

kernel: $M \rightarrow M$

Parameter	Type	Description
<i>a</i>	M	A matrix

Returns

Returns a matrix whose columns form a basis of the kernel of *a*.

See also

solve, subKernel

Usage

maxInvertibleSubmatrix *a*

Signature

maxInvertibleSubmatrix: $M \rightarrow (\text{Array MachineInteger}, \text{Array MachineInteger})$

Parameter	Type	Description
<i>a</i>	M	A matrix

Returns

Returns $([r_1, \dots, r_r], [c_1, \dots, c_r])$ where r is the rank of a and the submatrix of a formed by the intersections of the rows r_i and c_i is invertible.

See also

invertibleSubmatrix

Usage

particularSolution(a, b)

Signature

particularSolution: (M, M) → (M, Vector R)

Parameter	Type	Description
a, b	M	Matrices

Returns

Returns $(m, [d_1, \dots, d_n])$ such that

$$am = b \begin{pmatrix} d_1 & & & \\ & d_2 & & \\ & & \ddots & \\ & & & d_n \end{pmatrix}.$$

Remarks

For each i , $d_i \neq 0$ if and only if the system $ax = i^{\text{th}}$ column of b has a solution, which is then d_i^{-1} times the i^{th} column of m . When R is a **Field**, then $d_i \in \{0, 1\}$ for $1 \leq i \leq n$, so m is a solution of $ax = b$ when R is a **Field** and all the d_i 's are nonzero.

See also

solve

Usage

rank *a*

Signature

rank: $M \rightarrow \text{MachineInteger}$

Parameter	Type	Description
<i>a</i>	M	A matrix

Returns

Returns the rank of *a*.

See also

rankLowerBound, span

Usage

rankLowerBound a

SignaturerankLowerBound: $M \rightarrow (\text{Boolean}, \text{MachineInteger})$

Parameter	Type	Description
<i>a</i>	M	A matrix

ReturnsReturns $(rank?, r)$ such that $r \leq \text{rank}(a)$, and r is exactly the rank of a if $rank?$ is *true*.**Remarks** r can also happen to be the rank of a when $rank?$ is *false*, but the algorithm was unable to prove it.**See also**

rank, span

Usage

span a

Signature

span: M → Array MachineInteger

Parameter	Type	Description
<i>a</i>	M	A matrix

Returns

Returns $[c_1, \dots, c_r]$ where r is the rank of a and the span of a is generated by its columns $c_1, \dots, c_r$.

See also

rank

Usage

solve(a, b)

Signature

solve: (M, M) $\rightarrow$ (M, M, Vector R)

Parameter	Type	Description
a, b	M	Matrices

Returns

Returns $(w, m, [d_1, \dots, d_n])$ such that the columns of w form a basis of the kernel of a and

$$am = b \begin{pmatrix} d_1 & & & \\ & d_2 & & \\ & & \ddots & \\ & & & d_n \end{pmatrix}.$$

Remarks

For each i , $d_i \neq 0$ if and only if the system $ax = i^{\text{th}}$ column of b has a solution, which is then d_i^{-1} times the i^{th} column of m . When R is a **Field**, then $d_i \in \{0, 1\}$ for $1 \leq i \leq n$, so the general solution of $ax = b$ when R is a **Field** and all the d_i 's are nonzero is $x = m + \sum_j r_j w_j$ where w_j is the j^{th} column of w .

See also

kernel, particularSolution

Usage

subKernel a

Signature

subKernel: M → (Boolean, M)

Parameter	Type	Description
<i>a</i>	M	A matrix

Returns

Returns $(ker?, m)$ such that the columns of m , which are always linearly independent over R , generate a subspace of the kernel of a , and generate the full kernel if $ker?$ is *true*.

Remarks

m can also happen to generate the full kernel of a when $ker?$ is *false*, but the algorithm was unable to prove it.

See also

kernel

Usage

LinearAlgebraRing: Category

Description

LinearAlgebraRing is the category of rings that export algorithms for linear algebra for matrices over themselves.

Exports

IntegralDomain

determinant:	(M:MC) → M → %	Determinant
factorOfDeterminant:	(M:MC) → M → (B, %)	Probable determinant
invertibleSubmatrix:	(M:MC) → M → (B, AZ, AZ)	Probable maximal minor
inverse:	(M:MC) → M → (M, V)	Inverse
kernel:	(M:MC) → M → M	Kernel
linearDependence:	(Generator V, Z) → V	First dependence relation
maxInvertibleSubmatrix:	(M:MC) → M → (AZ, AZ)	Maximal minor
particularSolution:	(M:MC) → (M, M) → (M, V)	A solution
rank:	(M:MC) → M → Z	Rank
rankLowerBound:	(M:MC) → M → (B, Z)	Probable rank
solve:	(M:MC) → (M, M) → (M, M, V)	All solutions
span:	(M:MC) → M → AZ	Span
subKernel:	(M:MC) → M → (B, M)	Subspace of the kernel

where

AZ == Array MachineInteger
B == Boolean
MC == MatrixCategory %
V == Vector %
Z == MachineInteger

Usage

determinant(M)(a)

Signature

determinant: (M:MatrixCategory %) → M → R

Parameter	Type	Description
M	MatrixCategory %	A matrix type
a	M	A matrix

Returns

Returns the determinant of a .

See also

factorOfDeterminant

Usage

factorOfDeterminant(M)(a)

Signature

factorOfDeterminant: (M:MatrixCategory %) → M → (Boolean, R)

Parameter	Type	Description
M	MatrixCategory %	A matrix type
a	M	A matrix

Returns

Returns $(det?, d)$ such that d is always a factor of the determinant of a , and d is exactly the determinant of a if $det?$ is *true*.

Remarks

d can also happen to be the determinant of a when $det?$ is *false*, but the algorithm was unable to prove it.

See also

determinant

Usage

inverse(M)(a)

Signature

inverse: (M:MatrixCategory %) → M → (M, Vector R)

Parameter	Type	Description
M	MatrixCategory %	A matrix type
a	M	A matrix

Returns

Returns $(b, [d_1, \dots, d_n])$ such that

$$ab = \begin{pmatrix} d_1 & & & \\ & d_2 & & \\ & & \ddots & \\ & & & d_n \end{pmatrix}.$$

Remarks

$\prod_{i=1}^n d_i \neq 0$ if and only if a is invertible. When R is a `Field`, then $d_i \in \{0, 1\}$ for $1 \leq i \leq n$, so $b = a^{-1}$ when R is a `Field` and a is invertible.

Usage

invertibleSubmatrix(M)(a)

Signature

invertibleSubmatrix: (M:MatrixCategory %) → M → (Boolean, AZ, AZ)

where

AZ == Array MachineInteger

Parameter	Type	Description
M	MatrixCategory %	A matrix type
a	M	A matrix

Returns

Returns $(max?, [r_1, \dots, r_r], [c_1, \dots, c_r])$ where $r \leq \text{rank}(a)$ and the submatrix of a formed by the intersections of the rows r_i and c_i is always invertible. If $max?$ is *true*, then r is exactly the rank of a and the given minor is of maximal size.

Remarks

r can also happen to be the rank of a when $max?$ is *false*, but the algorithm was unable to prove it.

See also

maxInvertibleSubmatrix

Usage

kernel(M)(a)

Signature

kernel: (M:MatrixCategory %) → M → M

Parameter	Type	Description
M	MatrixCategory %	A matrix type
a	M	A matrix

Returns

Returns a matrix whose columns form a basis of the kernel of a .

See also

solve,subKernel

Usage

linearDependence(gen,n)

Signature

linearDependence: (Generator Vector R, MachineInteger) $\rightarrow$ Vector R

Parameter	Type	Description
<i>gen</i>	Generator Vector R	A generator of vectors
<i>n</i>	MachineInteger	The dimension of the vectors generated

Description

Returns a vector v which contains the coefficients of a dependence relation among the vectors generated by gen . The relation is as small as possible, meaning that if v has dimension m then the first $m - 1$ vectors generated are independent. The dimension of the vectors generated by gen must be n . There must be a relation between the vectors generated.

Usage

maxInvertibleSubmatrix(M)(a)

Signature

maxInvertibleSubmatrix: (M:MatrixCategory %) → M → (AZ, AZ)

where

AZ == Array MachineInteger

Parameter	Type	Description
M	MatrixCategory %	A matrix type
a	M	A matrix

Returns

Returns $([r_1, \dots, r_r], [c_1, \dots, c_r])$ where r is the rank of a and the submatrix of a formed by the intersections of the rows r_i and c_i is invertible.

See also

invertibleSubmatrix

Usage

particularSolution(M)(a, b)

Signature

particularSolution: (M:MatrixCategory %) → (M, M) → (M, Vector R)

Parameter	Type	Description
M	MatrixCategory %	A matrix type
a, b	M	Matrices

Returns

Returns $(m, [d_1, \dots, d_n])$ such that

$$am = b \begin{pmatrix} d_1 & & & \\ & d_2 & & \\ & & \ddots & \\ & & & d_n \end{pmatrix}.$$

Remarks

For each i , $d_i \neq 0$ if and only if the system $ax = i^{\text{th}}$ column of b has a solution, which is then d_i^{-1} times the i^{th} column of m . When R is a **Field**, then $d_i \in \{0, 1\}$ for $1 \leq i \leq n$, so m is a solution of $ax = b$ when R is a **Field** and all the d_i 's are nonzero.

See also

solve

Usage

rank(M)(a)

Signature

rank: (M:MatrixCategory %) → M → MachineInteger

Parameter	Type	Description
M	MatrixCategory %	A matrix type
a	M	A matrix

ReturnsReturns the rank of a .**See also**

rankLowerBound,span

Usage

rankLowerBound(M)(a)

Signature

rankLowerBound: (M:MatrixCategory %) → M → (Boolean, MachineInteger)

Parameter	Type	Description
M	MatrixCategory %	A matrix type
a	M	A matrix

Returns

Returns $(rank?, r)$ such that $r \leq \text{rank}(a)$, and r is exactly the rank of a if $rank?$ is *true*.

Remarks

r can also happen to be the rank of a when $rank?$ is *false*, but the algorithm was unable to prove it.

See also

rank, span

Usage

solve(M)(a, b)

Signature

solve: (M:MatrixCategory %) → (M, M) → (M, M, Vector R)

Parameter	Type	Description
M	MatrixCategory %	A matrix type
a, b	M	Matrices

Returns

Returns $(w, m, [d_1, \dots, d_n])$ such that the columns of w form a basis of the kernel of a and

$$am = b \begin{pmatrix} d_1 & & & \\ & d_2 & & \\ & & \ddots & \\ & & & d_n \end{pmatrix}.$$

Remarks

For each i , $d_i \neq 0$ if and only if the system $ax = i^{\text{th}}$ column of b has a solution, which is then d_i^{-1} times the i^{th} column of m . When R is a **Field**, then $d_i \in \{0, 1\}$ for $1 \leq i \leq n$, so the general solution of $ax = b$ when R is a **Field** and all the d_i 's are nonzero is $x = m + \sum_j r_j w_j$ where w_j is the j^{th} column of w .

See also

kernel, particularSolution

Usage

span(M)(a)

Signature

span: (M:MatrixCategory %) → M → Array MachineInteger

Parameter	Type	Description
M	MatrixCategory %	A matrix type
a	M	A matrix

Returns

Returns $[c_1, \dots, c_r]$ where r is the rank of a and the span of a is generated by its columns $c_1, \dots, c_r$.

See also

rank

Usage

subKernel(M)(a)

Signature

subKernel: (M:MatrixCategory %) → M → (Boolean, M)

Parameter	Type	Description
M	MatrixCategory %	A matrix type
a	M	A matrix

Returns

Returns $(ker?, m)$ such that the columns of m , which are always linearly independent over R , generate a subspace of the kernel of a , and generate the full kernel if $ker?$ is *true*.

Remarks

m can also happen to generate the full kernel of a when $ker?$ is *false*, but the algorithm was unable to prove it.

See also

kernel

Usage

LinearEliminationCategory(R,M): Category

Parameter	Type	Description
R	CommutativeRing	A coefficient ring
M	MatrixCategory R	A matrix type over R

Description

LinearEliminationCategory is a common category for linear elimination computations. The category provides operations for computing a row echelon form (REF) of a matrix and the first dependence relation among vectors.

Exports

extendedRowEchelon:	$M \rightarrow (M, Z \rightarrow Z, Z, \text{ARR } Z, Z, M)$	REF of a matrix
extendedRowEchelon!:	$M \rightarrow (Z \rightarrow Z, Z, \text{ARR } Z, Z, M)$	REF of a matrix
extendedRowEchelonForm:	$M \rightarrow (M, M)$	REF of a matrix
maxInvertibleSubmatrix:	$M \rightarrow (\text{Array } Z, \text{Array } Z)$	Maximal minor
maxInvertibleSubmatrix!:	$M \rightarrow (\text{Array } Z, \text{Array } Z)$	Maximal minor
pivot:	$(M, Z \rightarrow Z, Z, Z) \rightarrow Z$	Select a pivot
rowEchelon:	$M \rightarrow (M, Z \rightarrow Z, Z, \text{ARR } Z, Z)$	REF of a matrix
rowEchelon!:	$M \rightarrow (Z \rightarrow Z, Z, \text{ARR } Z, Z)$	REF of a matrix
	$(M, M) \rightarrow (Z \rightarrow Z, Z, \text{ARR } Z, Z)$	
rowEchelonForm:	$M \rightarrow M$	REF of a matrix
span:	$M \rightarrow \text{Array } Z$	Span of a matrix
span!:	$M \rightarrow \text{Array } Z$	Span of a matrix

if R has IntegralDomain then

denominators:	$(M, Z \rightarrow Z, Z, \text{ARR } Z) \rightarrow \text{ARR } R$	Maximal denominators
dependence:	$(\text{Generator } V, Z) \rightarrow (M, Z \rightarrow Z, Z, R)$	First linear dependence
deter:	$(M, Z \rightarrow Z, Z, Z) \rightarrow R$	Determinant
determinant:	$M \rightarrow R$	Determinant
determinant!:	$M \rightarrow R$	Determinant
rank!:	$M \rightarrow Z$	Rank of a matrix
rank:	$M \rightarrow Z$	Rank of a matrix

where

Z	==	MachineInteger
ARR	==	PrimitiveArray
V	==	Vector R

Usage

denominators(a,p,r,st)

Signature

denominators: (M,Z $\rightarrow$ Z,Z, PrimitiveArray Z) $\rightarrow$ PrimitiveArray R

where

Z == MachineInteger

Parameter	Type	Description
<i>a</i>	M	A matrix in REF form
<i>p</i>	MachineInteger $\rightarrow$ MachineInteger	A permutation of the rows of <i>a</i>
<i>r</i>	MachineInteger	The number of stairs of the REF
<i>st</i>	PrimitiveArray MachineInteger	The stairs of the REF

Description

(*a,p,r,st*) must be the representation of a REF computed by `rowEchelon`, `extendedRowEchelon` or their bang-versions. When *d* is returned then for $st(i) < j < st(i + 1)$ we have that *d*(*i*) times the *j*-th column of the REF is a linear combination of the first *i* leading columns of the REF. (The *i*-th column is called leading if *i* is a stair)

Usage

dependence(gen,n)

Signature

dependence: (Generator Vector R,Z) $\rightarrow$ (M,Z $\rightarrow$ Z,Z,R)

where

Z == MachineInteger

Parameter	Type	Description
<i>gen</i>	Generator Vector R	A generator of vectors
<i>n</i>	MachineInteger	The dimension of the vectors generated

Description

dependence(gen,n) computes the first dependence among the vectors generated. First means that dependence(gen,n) stops as soon as a dependence relation exists among the vectors generated so far. dependence(gen,n) returns a matrix a , a permutation p , the length r of a relation and the maximal denominator d needed for a dependence relation. After applying p to the rows of a one gets a matrix whose first $r - 1$ columns form an upper-triangular matrix. d times the last column of a is a linear combination of the first $r - 1$ columns of a . A dependence relation between the columns of a is also a dependence relation between the vectors generated.

Usage

deter(a,p,r,d)

Signature

deter: (M,Z $\rightarrow$ Z,Z,Z,Z) $\rightarrow$ R

where

Z == MachineInteger

Parameter	Type	Description
<i>a</i>	M	A matrix in REF form
<i>p</i>	MachineInteger $\rightarrow$ MachineInteger	A permutation of the rows of <i>a</i>
<i>r</i>	MachineInteger	The number of stairs of the REF
<i>d</i>	MachineInteger	The sign of p

Description

(*a,p,r,d*) must be the representation of a REF computed by rowEchelon, extendedRowEchelon or their bang-versions. The determinant of the original matrix is returned.

Usage

determinant a
determinant! a

Signature

determinant: $M \rightarrow R$

Parameter	Type	Description
a	M	A matrix whose determinant has to be computed

Returns

Returns the determinant of a .

Remarks

determinant! does not make a copy of a , but performs all the computations in-place, modifying the entries of a .

Usage

extendedRowEchelon a
 extendedRowEchelon! a

Signatures

extendedRowEchelon: $M \rightarrow (M, Z \rightarrow Z, Z, \text{PrimitiveArray } Z, Z, M)$
 extendedRowEchelon!: $M \rightarrow (Z \rightarrow Z, Z, \text{PrimitiveArray } Z, Z, M)$

where

$Z == \text{MachineInteger}$

Parameter	Type	Description
a	M	A matrix whose REF and corresponding transformation matrix have to be computed

Description

We say that a matrix a is in REF if there are r (the rank) and $j_1 < j_2 < \dots < j_r$ (the stairs) such that $a(i, j_i) \neq 0$, $a(i, j) = 0$ for $j < j_i$ and $a(i, j) = 0$ for $i > r$.

We say that a matrix b is a REF of the matrix a if b is in REF and there exists a non-singular matrix u such that $ua = b$.

extendedRowEchelon a computes a REF of a in a . It returns (c, p, r, st, d, w) where c is a matrix, p is a permutation, r is the number of stairs, st are the stairs, d is the sign of p and w is a matrix. For $i > r$, $st(i)$ is set to $m + 1$ where m is the number of columns of a . For $j \geq j_i$ the entry (i, j) of the REF is stored as entry $(p(i), j)$ in c . The other entries of c may have random values. The entry (i, j) of the transformation matrix u is stored as entry $(p(i), j)$ in w .

Remarks

extendedRowEchelon! does not make a copy of a , but performs all the computations in-place, storing the final result in a .

See also

extendedRowEchelonForm

Usage

extendedRowEchelonForm *a*

Signature

extendedRowEchelonForm: $M \rightarrow (M, M)$

Parameter	Type	Description
<i>a</i>	<i>M</i>	A matrix whose REF has to be computed

Description

We say that a matrix *a* is in REF if there are *r* (the rank) and $j_1 < j_2 < \dots < j_r$ (the stairs) such that $a(i, j_i) \neq 0$, $a(i, j) = 0$ for $j < j_i$ and $a(i, j) = 0$ for $i > r$.

We say that a matrix *b* is a REF of the matrix *a* if *b* is in REF and there exists a non-singular matrix *u* such that $ua = b$.

Returns

Returns a REF *b* of *a* and the transformation matrix *u* such that $ua = b$.

See also

extendedRowEchelon

Usage

maxInvertibleSubmatrix a
 maxInvertibleSubmatrix! a

Signature

maxInvertibleSubmatrix: $M \rightarrow (\text{Array MachineInteger}, \text{Array MachineInteger})$

Parameter	Type	Description
a	M	A matrix

Returns

Returns $([r_1, \dots, r_r], [c_1, \dots, c_r])$ where r is the rank of a and the submatrix of a formed by the intersections of the rows r_i and c_i is invertible.

Remarks

maxInvertibleSubmatrix! does not make a copy of a , but performs all the computations in-place, modifying the entries of a .

See also

rank,span

Usage

`pivot(a, p, c, r)`

Signature

`pivot: (M,Z → Z, Z, Z) → Z`

where

`Z == MachineInteger`

Parameter	Type	Description
<i>a</i>	M	A matrix
<i>p</i>	MachineInteger → MachineInteger	A permutation
<i>c</i>	MachineInteger	A column index
<i>r</i>	MachineInteger	A row index

Returns

Returns the row index of the an appropriate pivot for column *c* at row *r* or below. The matrix considered is *a* with its rows permuted by *p*.

Usage

rank *a*
rank! *a*

Signature

rank: M $\rightarrow$ MachineInteger

Parameter	Type	Description
<i>a</i>	M	A matrix whose rank has to be computed

Returns

Returns the rank of *a*.

Remarks

rank! does not make a copy of *a*, but performs all the computations in-place, modifying the entries of *a*.

See also

span

Usage

```
rowEchelon a
rowEchelon! a
rowEchelon!(a, b)
```

Signatures

```
rowEchelon:  M → (M, Z → Z, Z, PrimitiveArray Z, Z)
rowEchelon!: M → (Z → Z, Z, PrimitiveArray Z, Z)
rowEchelon!: (M, M) → (Z → Z, Z, PrimitiveArray Z, Z)
```

where

```
Z == MachineInteger
```

Parameter	Type	Description
a	M	A matrix whose REF has to be computed
b	M	A matrix to transform in the same way than a

Description

We say that a matrix a is in REF if there are r (the rank) and $j_1 < j_2 < \dots < j_r$ (the stairs) such that $a(i, j_i) \neq 0$, $a(i, j) = 0$ for $j < j_i$ and $a(i, j) = 0$ for $i > r$.

We say that a matrix b is a REF of the matrix a if b is in REF and there exists a non-singular matrix u such that $ua = b$.

`rowEchelon a` computes a REF of a . It returns (c, p, r, st, d) where c is a matrix, p is a permutation, r is the number of stairs, st are the stairs and d is the sign of p . For $i > r$, $st(i)$ is set to $m + 1$ where m is the number of columns of a . For $j \geq j_i$ the entry (i, j) of the REF is stored as entry $(p(i), j)$ in c . The other entries of c may have random values.

`rowEchelon!(a, b)` does the same than `rowEchelon!(a)` but applies all the elementary transformations applied to a also to b .

Remarks

`rowEchelon!` does not make a copy of a , but performs all the computations in-place, storing the final result in a . In addition, `rowEchelon!(a, b)` performs all the computations relative to b in b .

See also

```
rowEchelonForm
```

Usage

rowEchelonForm *a*

Signature

rowEchelonForm: $M \rightarrow M$

Parameter	Type	Description
<i>a</i>	M	A matrix whose REF has to be computed

Description

We say that a matrix a is in REF if there are r (the rank) and $j_1 < j_2 < \dots < j_r$ (the stairs) such that $a(i, j_i) \neq 0$, $a(i, j) = 0$ for $j < j_i$ and $a(i, j) = 0$ for $i > r$.

We say that a matrix b is a REF of the matrix a if b is in REF and there exists a non-singular matrix u such that $ua = b$.

Returns

Returns a REF of a .

See also

rowEchelon!

Usage

span a
span! a

Signature

span: M → Array MachineInteger

Parameter	Type	Description
a	M	A matrix whose span has to be computed

Returns

Returns $[c_1, \dots, c_r]$ where r is the rank of a and the span of a is generated by its columns $c_1, \dots, c_r$.

Remarks

span! does not make a copy of a , but performs all the computations in-place, modifying the entries of a .

See also

maxInvertibleSubmatrix,rank

MatrixCategory

Usage

MatrixCategory R: Category

Parameter	Type	Description
R	ExpressionType ArithmeticType	The coefficient domain

Description

MatrixCategory R is a common category for matrices of arbitrary sizes with coefficients in R. They are 1-indexed and whether they do bound checking depends on each particular matrix type.

Exports

CopyableType

LinearArithmeticType R

$*$:	$(\%, V) \rightarrow V$	multiplication by a vector
$[]$:	$\text{Tuple } V \rightarrow \%$	create a matrix
	$\text{Generator } V \rightarrow \%$	
apply:	$(\%, I, I) \rightarrow R$	extract an entry
	$(\%, I, I, I, I) \rightarrow \%$	extract a submatrix
	$(\%, \text{Array } I, \text{Array } I) \rightarrow \%$	
colCombine!:	$(\%, R, I, R, I) \rightarrow \%$	In-place linear combination of columns
	$(\%, R, I, R, I, I, I) \rightarrow \%$	
	$(\%, (R, R) \rightarrow R, I, I) \rightarrow \%$	
	$(\%, (R, R) \rightarrow R, I, I, I, I) \rightarrow \%$	
colSwap!:	$(\%, I, I) \text{ to } \%$	Swap columns in-place
	$(\%, I, I, I, I) \text{ to } \%$	
column:	$(\%, I) \rightarrow V$	extraction of a column
columns:	$\% \rightarrow \text{Generator } V$	iteration over the columns
companion:	$V \rightarrow \%$	creates a companion matrix
	$(V, R) \rightarrow \%$	
diagonal:	$V \rightarrow \%$	creates a diagonal matrix
	$(R, I) \rightarrow \%$	
diagonal?:	$\% \rightarrow \text{Boolean}$	test for a diagonal matrix
dimensions:	$\% \rightarrow (I, I)$	get row and column dimensions
map:	$(R \rightarrow R) \rightarrow V \rightarrow \%$	lift a mapping
map:	$(R \rightarrow R) \rightarrow \% \rightarrow \%$	lift a mapping
map!:	$(R \rightarrow R) \rightarrow \% \rightarrow \%$	lift a mapping
numberOfColumns:	$\% \rightarrow I$	number of columns of the matrix
numberOfRows:	$\% \rightarrow I$	number of rows of the matrix
one:	$I \rightarrow \%$	identity matrix
one?:	$\% \rightarrow \text{Boolean}$	test for an identity matrix

<code>row:</code>	$(\%, I) \rightarrow V$	extraction of a row
<code>rowCombine!:</code>	$(\%, R, I, R, I) \rightarrow \%$ $(\%, R, I, R, I, I) \rightarrow \%$ $(\%, (R, R) \rightarrow R, I, I) \rightarrow \%$ $(\%, (R, R) \rightarrow R, I, I, I, I) \rightarrow \%$	In-place linear combination of rows
<code>rowSwap!:</code>	$(\%, I, I) \text{ to } \%$ $(\%, I, I, I, I) \text{ to } \%$	Swap rows in-place
<code>rows:</code>	$\% \rightarrow \text{Generator } V$	iteration over the rows
<code>scalar?:</code>	$\% \rightarrow \text{Boolean}$	test for a scalar matrix
<code>set!:</code>	$(\%, I, I, R) \rightarrow R$	set an entry in the matrix
<code>setMatrix!:</code>	$(\%, I, I, \%) \rightarrow \%$	modify a submatrix of a matrix
<code>square?:</code>	$\% \rightarrow \text{Boolean}$	test for a square matrix
<code>tensor:</code>	$(\%, \%) \rightarrow \%$	tensor product
<code>transpose:</code>	$V \rightarrow \%$	transpose a vector
<code>transpose:</code>	$\% \rightarrow \%$	transpose a matrix
<code>transpose!:</code>	$\% \rightarrow \%$	transpose a matrix in-place
<code>zero:</code>	$(I, I) \rightarrow \%$	create a zero matrix
<code>zero!:</code>	$\% \rightarrow ()$	make all the entries zero
<code>zero?:</code>	$\% \rightarrow \text{Boolean}$	test if all entries are zero

if R has Ring then

`random:` $() \rightarrow \%$ create a random matrix
 $(I, I) \rightarrow \%$

if R has DifferentialRing then

`wronskian:` $V \rightarrow \%$ Wronskian matrix

where

`I` == MachineInteger

`V` == Vector R

if R has SerializableType then

SerializableType

Usage

A * v

Signature

*: (% , Vector R) → Vector R

Parameter	Type	Description
<i>A</i>	%	a matrix
<i>v</i>	Vector R	a vector

Returns

Returns the vector *Av*.

Usage

$[v_1, \dots, v_n]$
[v for v in g]

Signatures

`[]`: Tuple Vector R $\rightarrow$ %
`[]`: Generator Vector R $\rightarrow$ %

Parameter	Type	Description
$v_1, \dots, v_n$	Vector R	vectors
g	Generator Vector R	an iterator producing vectors

Returns

Returns the matrix whose i^{th} column is v_i , respectively the i^{th} vector generated by g .

Usage

$A(n, m)$
 $\text{apply}(A, n, m)$
 $A(n, m, r, c)$
 $\text{apply}(A, n, m, r, c)$
 $A(a, b)$
 $\text{apply}(A, a, b)$

Signatures

$\text{apply}: (\%, \text{MachineInteger}, \text{MachineInteger}) \rightarrow R$
 $\text{apply}: (\%, \text{MachineInteger}, \text{MachineInteger}, \text{MachineInteger}, \text{MachineInteger}) \rightarrow \%$
 $\text{apply}: (\%, \text{Array MachineInteger}, \text{Array MachineInteger}) \rightarrow \%$

Parameter	Type	Description
A	$\%$	A matrix
n, m, r, c	<code>MachineInteger</code>	indices
a, b	<code>Array MachineInteger</code>	indices

Returns

$A(n, m)$ returns the entry of A at its n^{th} row and m^{th} column, while $A(n, m, r, c)$ returns the submatrix of A having $A(m, n)$ in its top-left corner and r rows and c columns. For more general submatrices, $A([a_1, \dots, a_r], [b_1, \dots, b_s])$ returns the submatrix of A consisting of the intersection of the rows $a_1, \dots, a_r$ and columns $b_1, \dots, b_r$ of A .

Usage

```

colCombine!(A, c1, j1, c2, j2)
colCombine!(A, c1, j1, c2, j2, i1, i2)
colCombine!(A, f, j1, j2)
colCombine!(A, f, j1, j2, i1, i2)
rowCombine!(A, c1, i1, c2, i2)
rowCombine!(A, c1, i1, c2, i2, j1, j2)
rowCombine!(A, f, i1, i2)
rowCombine!(A, f, i1, i2, j1, j2)

```

Signatures

```

colCombine!,rowCombine!:  (% , R, I, R, I) → %
colCombine!,rowCombine!:  (% , R, I, R, I, I, I) → %
colCombine!,rowCombine!:  (%,(R,R) → R, I, I) → %
colCombine!,rowCombine!:  (% , (R,R) → R, I, I, I, I) → %

```

where

```
I == MachineInteger
```

Parameter	Type	Description
A	$\%$	A matrix
f	$(R,R) \rightarrow R$	An binary operation on R
$c1, c2$	R	coefficients from R
$j1, j2$	MachineInteger	column indices
$i1, i2$	MachineInteger	row indices

Description

The j_1^{th} column (resp. i_1^{th} row) of A is replaced by the result of applying f pointwise to its j_1^{th} and j_2^{th} columns (resp. i_1^{th} and i_2^{th} rows). If the last 2 arguments i_1, i_2 (resp. j_1, j_2) are present, then this operation is applied only for rows i_1 to i_2 (resp. columns j_1 to j_2) inclusive. The form with c_1 and c_2 is equivalent to the first one with the function f defined by $f(x_1, x_2) = c_1x_1 + c_2x_2$.

Usage

```
colSwap!(A, j1, j2)
colSwap!(A, j1, j2, i1, i2)
rowSwap!(A, i1, i2)
rowSwap!(A, i1, i2, j1, j2)
```

Signatures

```
colSwap!,rowSwap!: (% , I, I) → %
colSwap!,rowSwap!: (% , I, I, I, I) → %
```

where

```
I == MachineInteger
```

Parameter	Type	Description
A	%	A matrix
$j1, j2$	MachineInteger	column indices
$i1, i2$	MachineInteger	row indices

Description

The j_1^{th} and j_2^{th} columns (resp. i_1^{th} and i_2^{th} rows) of A are exchanged in-place. If the last 2 arguments i_1, i_2 (resp. j_1, j_2) are present, then this operation is applied only for rows i_1 to i_2 (resp. columns j_1 to j_2) inclusive.

Usage

column(A,n)
row(A,n)

Signature

colSwap!,rowSwap!: (%MachineInteger) → Vector R

Parameter	Type	Description
A	%	A matrix
n	MachineInteger	An index

Returns

Returns the n^{th} column (resp. row) of A as a vector.

Usage

for v in columns A repeat { ... }
for v in rows A repeat { ... }

Signature

columns,rows: % $\rightarrow$ Generator Vector R

Parameter	Type	Description
<i>A</i>	%	A matrix

Description

This generator yields the columns (resp. rows) of A in succession.

Usage

```
companion [r1,...,rn]
companion([r1,...,rn],a)
```

Signature

```
companion: (Vector R, R) -> %
```

Parameter	Type	Description
$r_1, \dots, r_n$	R	Entries
a	R	A subdiagonal entry (optional, default is 1)

Returns

Returns the companion matrix

$$\begin{pmatrix} 0 & & & r_1 \\ a & \ddots & & r_2 \\ & \ddots & & r_3 \\ & & & \vdots \\ & & a & r_n \end{pmatrix}$$

See also

diagonal

Usage

`diagonal [r1, ..., rn]`
`diagonal(r, n)`
`diagonal? A`
`scalar? A`
`square? A`

Signatures

`diagonal:` `Vector R → %`
`diagonal:` `(R, MachineInteger) → %`
`diagonal?, scalar?, square?:` `% → Boolean`

Parameter	Type	Description
$r, r_1, \dots, r_n$	<code>R</code>	Entries
n	<code>MachineInteger</code>	<code>A</code> size
A	<code>%</code>	<code>A</code> matrix

Description

`diagonal([r1, ..., rn])` returns a diagonal matrix whose diagonal elements are $r_1, \dots, r_n$, while `diagonal(r, n)` returns an $n \times n$ diagonal matrix with r on its diagonal, and `square?(A)` (resp. `diagonal A` and `scalar? A`) return *true* if A is a square (resp. diagonal and diagonal with the same entry on its diagonal) matrix, *false* otherwise.

See also

`companion, one?`

Usage

dimensions A

Signature

dimensions: % $\rightarrow$ (MachineInteger,MachineInteger)

Parameter	Type	Description
<i>A</i>	%	A matrix

Returns

The number of rows and columns in *A*

See also

numberOfColumns,numberOfRows

Usage

```
map f
map! f
map(f)([v1, ..., vn])
map(f)(A)
map!(f)(A)
```

Signatures

```
map: (R → R) → Vector R → %
map: (R → R) → % → %
map!: (R → R) → % → %
```

Parameter	Type	Description
f	$R \rightarrow R$	a map
v_i	R	Entries of a vector
A	$\%$	A matrix

Description

$\text{map}(f)([v_1, \dots, v_n])$ returns the square matrix

$$\begin{pmatrix} v_1 & \dots & v_n \\ f(v_1) & \dots & f(v_n) \\ \vdots & & \vdots \\ f^{n-1}(v_1) & \dots & f^{n-1}(v_n) \end{pmatrix}$$

while $\text{map}(f)(A)$ returns $f(A)$, *i.e.* f applied to A pointwise, and $\text{map}(f)$ returns either the mapping $v \rightarrow f(v)$ or $A \rightarrow f(A)$. For matrices, $\text{map}!$ does not make a copy of A but modifies it in place.

Usage

numberOfColumns A
numberOfRows A

Signature

numberOfColumns,numberOfRows: % $\rightarrow$ MachineInteger

Parameter	Type	Description
<i>A</i>	%	A matrix

Returns

The number of columns (resp. rows) in *A*.

See also

dimensions

Usage

one n
one? A

Signatures

one: MachineInteger $\rightarrow$ %
one?: % $\rightarrow$ Boolean

Parameter	Type	Description
n	MachineInteger	an integer
A	%	A matrix

Description

one(n) returns an $n \times n$ identity matrix, while one?(A) returns *true* if A is an identity matrix, *false* otherwise.

See also

diagonal?,zero,zero?

Usage

random()
random(n,m)

Signatures

random: () → %
random: (MachineInteger, MachineInteger) → %

Parameter	Type	Description
<i>n,m</i>	MachineInteger	The dimensions of the new matrix.

Returns

random() returns a random matrix with random size, while random(n, m) returns a random matrix with n rows and m columns.

Usage

```
set!(A, n, m, x)
A(n,m) := x
```

Signature

```
set!:  (%MachineInteger,MachineInteger,R) → R
```

Parameter	Type	Description
<i>A</i>	%	A matrix
<i>n, m</i>	MachineInteger	Indices
<i>c</i>	R	An entry

Description

Sets $A(n,m)$ to c and returns c .

Usage

setMatrix!(A, n, m, B)

Signature

setMatrix!: (*%*,MachineInteger,MachineInteger,*%*) → *%*

Parameter	Type	Description
<i>A</i> , <i>B</i>	<i>%</i>	Matrices
<i>n</i> , <i>m</i>	MachineInteger	Indices

Description

Inserts *B* as a submatrix of *A* starting at *A*(*n*,*m*) and returns *B*.

Usage

tensor(A,B)

Signature

tensor: (%,%) → %

Parameter	Type	Description
A, B	%	Matrices

Returns

Returns $A \otimes B$, *i.e.* the matrix satisfying $(A \otimes B)(u \otimes v) = Au \otimes Bv$ for all vectors u, v .

Usage

transpose v
transpose A
transpose! A

Signatures

transpose: **Vector** R $\rightarrow$ %
transpose,transpose!: % $\rightarrow$ %

Parameter	Type	Description
v	Vector R	A vector
A	%	A matrix

Returns

Return the transpose of v (resp. A).

Remarks

transpose! does not make a copy of A , which is therefore replaced by its transpose. It is only applicable to square matrices.

Usage

wronskian $[v_1, \dots, v_n]$

Signature

wronskian: **Vector** R $\rightarrow$ %

Parameter	Type	Description
v_i	R	Entries of a vector

Description

wronskian($[v_1, \dots, v_n]$) returns the square matrix

$$\begin{pmatrix} v_1 & \dots & v_n \\ v_1' & \dots & v_n' \\ \vdots & & \vdots \\ v_1^{(n-1)} & \dots & v_n^{(n-1)} \end{pmatrix}$$

Usage

zero(*n*,*m*)
 zero! *A*
 zero? *A*

Signatures

zero: (MachineInteger, MachineInteger) → %
 zero!: % → ()
 zero?: % → Boolean

Parameter	Type	Description
<i>n, m</i>	MachineInteger	integers
<i>A</i>	%	A matrix

Description

zero(*n*,*m*) returns an *n* by *m* zero matrix, while zero!(*A*) fills *A* with 0's and zero?(*A*) returns *true* if all the entries of *A* are 0, *false* otherwise.

See also

one, one?

MatrixCategory2

Usage

```
import from MatrixCategory2(R, MR, S, MS)
```

Parameter	Type	Description
R, S	ExpressionType ArithmeticType	Coefficient domains
MR	MatrixCategory R	a matrix type over R
MS	MatrixCategory S	a matrix type over S

Description

MatrixCategory2(R,MR,S,MS) provides tools for lifting maps $R \rightarrow S$ to maps $MR \rightarrow MS$.

Exports

```
map: (R → S) → MR → MS  Lift a mapping
```


Usage

map f
map(f)(m)

Signature

map: $(R \rightarrow S) \rightarrow MR \rightarrow MS$

Parameter	Type	Description
f	$R \rightarrow S$	A map
m	MR	A matrix with entries in R

Description

map(f)(m) returns a matrix whose $(i, j)^{\text{th}}$ entry is $f(m_{ij})$, while map(f) returns the mapping $m \rightarrow f(m)$.

Usage

```
import from MatrixCategoryOverFraction(R, MR, Q, MQ)
```

Parameter	Type	Description
R	IntegralDomain	an integral domain
MR	MatrixCategory R	a matrix type over R
Q	FractionCategory R	a fraction domain of R
MQ	MatrixCategory Q	a matrix type over Q

Description

MatrixCategoryOverFraction(R, MR, Q, MQ) provides useful conversions between matrices with integral and rational coefficients.

Exports

```
LinearCombinationFraction(R, MR, Q, MQ)
```

```
makeColIntegral: (MQ, I) → (R, V R)  Convert to an integral column
                  MQ → (V R, MR)      Convert to an integral matrix column by column
makeRowIntegral: (MQ, I) → (R, V R)  Convert to an integral row
                  MQ → (V R, MR)      Convert to an integral matrix row by row
```

if Q has FractionByCategory0 R

```
makeRowIntegralBy: MQ → (V Z, MR)  Convert to an integral matrix row by row
```

where

```
I == MachineInteger
Z == Integer
V == Vector
```

Usage

$(v, A) := \text{makeColIntegral } B$
 $(a, v) := \text{makeColIntegral}(B, i)$

Signatures

$\text{makeColIntegral}: MQ \rightarrow (\text{Vector } R, MR)$
 $\text{makeColIntegral}: (MQ, \text{MachineInteger}) \rightarrow (R, \text{Vector } R)$

Parameter	Type	Description
B	<code>MQ</code>	A matrix with rational coefficients
i	<code>MachineInteger</code>	A column index

Returns

$\text{makeColIntegral}(B)$ returns (v, A) such that A has integral coefficients and the j^{th} column of A is equal to $v \cdot j$ times the j^{th} column of B for each j , *i.e.*

$$A = B \begin{pmatrix} v_1 & & \\ & \ddots & \\ & & v_n \end{pmatrix}$$

$\text{makeColIntegral}(B, i)$ returns (a, v) such that v has integral coefficients and v equals a times the i^{th} column of B . If R is a `GcdDomain`, then each column of A (resp. v) is primitive.

See also

`makeRowIntegral`

Usage

$(v, A) := \text{makeRowIntegral } B$
 $(a, v) := \text{makeRowIntegral}(B, i)$

Signatures

$\text{makeRowIntegral}: \text{MQ} \rightarrow (\text{Vector } R, \text{MR})$
 $\text{makeRowIntegral}: (\text{MQ}, \text{MachineInteger}) \rightarrow (R, \text{Vector } R)$

Parameter	Type	Description
B	MQ	A matrix with rational coefficients
i	MachineInteger	A row index

Returns

$\text{makeRowIntegral}(B)$ returns (v, A) such that A has integral coefficients and the j^{th} row of A is equal to $v \cdot j$ times the j^{th} row of B for each j , *i.e.*

$$A = \begin{pmatrix} v_1 & & \\ & \ddots & \\ & & v_n \end{pmatrix} B$$

$\text{makeRowIntegral}(B, i)$ returns (a, v) such that v has integral coefficients and v equals a times the i^{th} row of B . If R is a `GcdDomain`, then each row of A (resp. v) is primitive.

See also

`makeColIntegral`

Usage

$(v, A) := \text{makeRowIntegralBy } B$

Parameter	Type	Description
B	MQ	A matrix with rational coefficients

Returns

Returns (v, A) such that A has integral coefficients and the j^{th} row of A is equal to the j^{th} row of B shifted by $v.j$.

Usage

```
import from ModulopGaussElimination
```

Exports

deter:	$(M, Z, V, Z, Z, Z) \rightarrow Z$	Determinant
determinant!:	$(M, Z, Z) \rightarrow Z$	Determinant
extendedRowEchelon!:	$(M, Z, Z, Z) \rightarrow (V, Z, V, M)$	REF of a matrix
firstDependence!:	$(\text{Generator } V, Z, Z, M, M) \rightarrow Z$	First dependence relation
inverse!:	$(M, Z, Z, M, V) \rightarrow ()$	Inverse
kernel!:	$(M, Z, Z, M) \rightarrow Z$	Kernel
maxInvertibleSubmatrix!:	$(M, Z, Z) \rightarrow (V, V)$	Maximal minor
particularSolution!:	$(M, Z, Z, M, Z, Z, M, V) \rightarrow ()$	A solution
rank!:	$(M, Z, Z, Z) \rightarrow Z$	Rank
rowEchelon!:	$(M, Z, Z, Z) \rightarrow (V, Z, V)$	REF of a matrix
solve!:	$(M, Z, Z, M, Z, Z, M, V, M) \rightarrow Z$	All solutions
span!:	$(M, Z, Z) \rightarrow V$	Span

where

```
Z  ==  MachineInteger
V  ==  PrimitiveArray MachineInteger
M  ==  PrimitiveArray PrimitiveArray MachineInteger
```

Description

This domain implements ordinary Gaussian elimination on dense matrices over the integers modulo a machine prime.

Usage

deter(*a*,*n*,*σ*,*r*,*d*,*p*)

Signature

deter: (PrimitiveArray PrimitiveArray $\mathbb{Z}, \mathbb{Z}$, PrimitiveArray $\mathbb{Z}, \mathbb{Z}, \mathbb{Z}$) $\rightarrow \mathbb{Z}$

where

$\mathbb{Z} == \text{MachineInteger}$

Parameter	Type	Description
<i>a</i>	A A MachineInteger	A square matrix in REF form
<i>n</i>	MachineInteger	The size of <i>a</i>
<i>σ</i>	A MachineInteger	A permutation of the rows of <i>a</i>
<i>r</i>	MachineInteger	The number of stairs of the REF
<i>d</i>	MachineInteger	The sign of <i>σ</i>
<i>p</i>	MachineInteger	a prime

where

$\mathbb{A} == \text{PrimitiveArray}$

Description

(*a*,*σ*,*r*,*d*) must be the representation of a REF computed by either `extendedRowEchelon!` or `rowEchelon!`. The determinant of the original matrix over $\mathbb{Z}/p\mathbb{Z}$ is returned.

Usage

determinant!(a,n,p)

Signature

determinant!: (PrimitiveArray PrimitiveArray Z,Z,Z) → Z

where

Z == MachineInteger

Parameter	Type	Description
<i>a</i>	PrimitiveArray PrimitiveArray MachineInteger	A square matrix
<i>n</i>	MachineInteger	The size of <i>a</i>
<i>p</i>	MachineInteger	a prime

Returns

Returns the determinant of *a* over $\mathbb{Z}/p\mathbb{Z}$.

Remarks

Does not make a copy of *a*, but performs all the computations in-place, modifying the entries of *a*.

Usage

extendedRowEchelon!(a, r, c, p)

Signature

extendedRowEchelon!: (A A Z,Z,Z) $\rightarrow$ (A Z, Z, A Z, Z, A A Z)

where

Z == MachineInteger

A == PrimitiveArray

Parameter	Type	Description
<i>a</i>	PrimitiveArray PrimitiveArray MachineInteger	A matrix
<i>r</i>	MachineInteger	Number of rows of <i>a</i>
<i>c</i>	MachineInteger	Number of columns of <i>a</i>
<i>p</i>	MachineInteger	A prime

Description

We say that a matrix *a* is in REF if there are *r* (the rank) and $j_1 < j_2 < \dots < j_r$ (the stairs) such that $a(i, j_i) \neq 0$, $a(i, j) = 0$ for $j < j_i$ and $a(i, j) = 0$ for $i > r$.

We say that a matrix *b* is a REF of the matrix *a* if *b* is in REF and there exists a non-singular matrix *u* such that $ua = b$.

extendedRowEchelon!(a,r,c,p) computes a REF of *a* over $\mathbb{Z}/p\mathbb{Z}$. It returns (σ, r, st, d, w) where σ is a permutation, *r* is the number of stairs, *st* are the stairs, *d* is the sign of σ and *w* is a matrix. For $i > r$, *st*(*i*) is set to $m + 1$ where *m* is the number of columns of *a*. For $j \geq j_i$ the entry (*i*, *j*) of the REF is stored as entry (*p*(*i*), *j*) in *a*. The other entries of *a* may have random values. The entry (*i*, *j*) of the transformation matrix *u* is stored as entry (*p*(*i*), *j*) in *w*.

Remarks

Does not make a copy of *a*, but performs all the computations in-place, modifying the entries of *a*.

See also

rowEchelon!

Usage

firstDependence!(gen, n, p, a, d)

Signature

firstDependence!: (Generator A Z, Z, Z, A A Z, A A Z) → Z

where

A == PrimitiveArray

Z == MachineInteger

Parameter	Type	Description
<i>gen</i>	Generator A MachineInteger	A generator of vectors
<i>n</i>	MachineInteger	The dimension of the vectors generated
<i>p</i>	MachineInteger	a prime
<i>a</i>	A A MachineInteger	A work matrix
<i>d</i>	A A MachineInteger	A matrix for the dependence

where

A == PrimitiveArray

Description

firstDependence!(gen,n,p,a,d) computes the first dependence over $\mathbb{Z}/p\mathbb{Z}$ among the vectors generated by *gen*. It returns the number r of vectors generated. This number is as small as possible, meaning that the first $r - 1$ vectors are linearly independent over $\mathbb{Z}/p\mathbb{Z}$. The coefficients of the dependence are stored in the first column of *d*, which must have at least r rows, upon return. The work matrix *a* must have n rows and at least r columns (note that $r \leq n + 1$). The dimension of the vectors generated by *gen* must be n . There must be a relation between the vectors generated.

Usage

inverse!(a,n,p,b,d)

Signature

inverse!: (A A Z, Z, Z, A A Z, A Z) → Z

where

Z == MachineInteger
A == PrimitiveArray

Parameter	Type	Description
<i>a</i> , <i>b</i>	PrimitiveArray PrimitiveArray MachineInteger	Square matrices
<i>n</i>	MachineInteger	The size of <i>a</i> , <i>b</i> and <i>d</i>
<i>p</i>	MachineInteger	a prime
<i>d</i>	PrimitiveArray MachineInteger	a vector

Returns

Fills *b* and *d* such that $d_i \in \{0, 1\}$ for each *i* and

$$ab = \begin{pmatrix} d_0 & & & \\ & d_1 & & \\ & & \ddots & \\ & & & d_{n-1} \end{pmatrix}.$$

Remarks

$\prod_{i=0}^{n-1} d_i = 1$ if and only if *a* is invertible, in which case $b = a^{-1}$. Does not make a copy of *a*, but performs all the computations in-place, modifying the entries of *a*.

Usage

kernel(a,r,c,p,w)

Signature

kernel: (A A Z,Z,Z,A A Z) → Z

where

Z == MachineInteger
A == PrimitiveArray

Parameter	Type	Description
<i>a</i> , <i>w</i>	PrimitiveArray PrimitiveArray MachineInteger	Matrices
<i>r</i>	MachineInteger	Number of rows of <i>a</i>
<i>c</i>	MachineInteger	Number of columns of <i>a</i>
<i>p</i>	MachineInteger	a prime

Returns

Stores a basis of the kernel of *a* in the columns of *w*, which must be large enough, and returns the dimension of the kernel.

Remarks

Does not make a copy of *a*, but performs all the computations in-place, modifying the entries of *a*.

Usage

maxInvertibleSubmatrix!(a,r,c,p)

Signature

maxInvertibleSubmatrix!: (A A Z,Z,Z) $\rightarrow$ (Array Z, Array Z)

where

Z == MachineInteger

A == PrimitiveArray

Parameter	Type	Description
a	PrimitiveArray PrimitiveArray MachineInteger	A matrix
r	MachineInteger	Number of rows of a
c	MachineInteger	Number of columns of a
p	MachineInteger	a prime

Returns

Returns $([r_1, \dots, r_r], [c_1, \dots, c_r])$ where r is the rank of a over $\mathbb{Z}/p\mathbb{Z}$ and the submatrix of a formed by the intersections of the rows r_i and c_i is invertible over $\mathbb{Z}/p\mathbb{Z}$.

Remarks

Does not make a copy of a , but performs all the computations in-place, modifying the entries of a .

Usage

particularSolution!(a,ra,ca,b,cb,p,w,d)

Signature

particularSolution!: (A A Z,Z,Z,A A Z,Z,Z,A A Z,A Z) $\rightarrow$ ()

where

Z == MachineInteger

A == PrimitiveArray

Parameter	Type	Description
a, b, w	PrimitiveArray PrimitiveArray MachineInteger	Matrices
ra	MachineInteger	Number of rows of a
ca	MachineInteger	Number of columns of a
cb	MachineInteger	Number of columns of b
p	MachineInteger	a prime
d	PrimitiveArray MachineInteger	a vector

Returns

Fills w and d such that $d_i \in \{0, 1\}$ for each i and

$$aw = b \begin{pmatrix} d_0 & & & \\ & d_1 & & \\ & & \ddots & \\ & & & d_{n-1} \end{pmatrix}.$$

Remarks

For each i , $d_i = 1$ if and only if the system $ax = (i + 1)^{\text{th}}$ column of b has a solution, which is then the $(i + 1)^{\text{th}}$ column of w , which must have the same dimensions than b , which must have the same number of rows that a . Does not make a copy of a , but performs all the computations in-place, modifying the entries of a , while b is not modified.

Usage

rank!(a,r,c,p)

Signature

rank!: (PrimitiveArray PrimitiveArray Z,Z,Z) → Z

where

Z == MachineInteger

Parameter	Type	Description
<i>a</i>	PrimitiveArray PrimitiveArray MachineInteger	A matrix
<i>r</i>	MachineInteger	Number of rows of <i>a</i>
<i>c</i>	MachineInteger	Number of columns of <i>a</i>
<i>p</i>	MachineInteger	a prime

Returns

Returns the rank of *a* over $\mathbb{Z}/p\mathbb{Z}$.

Remarks

Does not make a copy of *a*, but performs all the computations in-place, modifying the entries of *a*.

Usage

rowEchelon!(a, r, c, p)

Signature

rowEchelon!: (A A Z,Z,Z) $\rightarrow$ (A Z, Z, A Z, Z)

where

Z == MachineInteger

A == PrimitiveArray

Parameter	Type	Description
<i>a</i>	PrimitiveArray PrimitiveArray MachineInteger	A matrix
<i>r</i>	MachineInteger	Number of rows of <i>a</i>
<i>c</i>	MachineInteger	Number of columns of <i>a</i>
<i>p</i>	MachineInteger	A prime

Description

We say that a matrix *a* is in REF if there are *r* (the rank) and $j_1 < j_2 < \dots < j_r$ (the stairs) such that $a(i, j_i) \neq 0$, $a(i, j) = 0$ for $j < j_i$ and $a(i, j) = 0$ for $i > r$.

We say that a matrix *b* is a REF of the matrix *a* if *b* is in REF and there exists a non-singular matrix *u* such that $ua = b$.

rowEchelon!(a,r,c,p) computes a REF of *a* over $\mathbb{Z}/p\mathbb{Z}$. It returns (σ, r, st, d) where σ is a permutation, *r* is the number of stairs, *st* are the stairs and *d* is the sign of *p*. For $i > r$, *st*(*i*) is set to $m + 1$ where *m* is the number of columns of *a*. For $j \geq j_i$ the entry (*i*, *j*) of the REF is stored as entry (*p*(*i*), *j*) in *a*. The other entries of *a* may have random values.

Remarks

Does not make a copy of *a*, but performs all the computations in-place, modifying the entries of *a*.

See also

extendedRowEchelon!

Usage

solve!(a,ra,ca,b,cb,p,w,d,k)

Signature

solve!: (A A Z,Z,Z,A A Z,Z,Z,A A Z,A Z,A A Z) $\rightarrow$ Z

where

Z == MachineInteger

A == PrimitiveArray

Parameter	Type	Description
a, b, w, k	PrimitiveArray PrimitiveArray MachineInteger	Matrices
ra	MachineInteger	Number of rows of a
ca	MachineInteger	Number of columns of a
cb	MachineInteger	Number of columns of b
p	MachineInteger	a prime
d	PrimitiveArray MachineInteger	a vector

Returns

Fills w and d such that $d_i \in \{0, 1\}$ for each i and

$$aw = b \begin{pmatrix} d_0 & & & \\ & d_1 & & \\ & & \ddots & \\ & & & d_{n-1} \end{pmatrix}.$$

Furthermore, solve! stores a basis of the kernel of a in the columns of k , which must be large enough, and returns the dimension of the kernel.

Remarks

For each i , $d_i = 1$ if and only if the system $ax = (i + 1)^{\text{th}}$ column of b has a solution, which is then the $(i + 1)^{\text{th}}$ column of w , which must have the same dimensions than b , which must have the same number of rows that a . Does not make a copy of a , but performs all the computations in-place, modifying the entries of a , while b is not modified.

Usage

span!(a,r,c,p)

Signature

span!: (PrimitiveArray PrimitiveArray $\mathbb{Z}, \mathbb{Z}, \mathbb{Z}$) $\rightarrow$ Array $\mathbb{Z}$

where

$\mathbb{Z} == \text{MachineInteger}$

Parameter	Type	Description
a	PrimitiveArray PrimitiveArray MachineInteger	A matrix
r	MachineInteger	Number of rows of a
c	MachineInteger	Number of columns of a
p	MachineInteger	a prime

Returns

Returns $[c_1, \dots, c_r]$ where r is the rank of a over $\mathbb{Z}/p\mathbb{Z}$ and the span of a is generated by its columns $c_1, \dots, c_r$.

Remarks

Does not make a copy of a , but performs all the computations in-place, modifying the entries of a .

OrdinaryGaussElimination

Usage

import from OrdinaryGaussElimination(F,M)

Parameter	Type	Description
F	Field	A coefficient field
M	MatrixCategory F	A matrix type over F

Exports

LinearEliminationCategory(R,M)

Description

This domain implements ordinary Gaussian elimination on matrices.

OverdeterminedLinearSystemSolver

Usage

`import from OverdeterminedLinearSystemSolver(R, M)`

Parameter	Type	Description
R	<code>IntegralDomain</code>	A domain
M	<code>MatrixCategory R</code>	A matrix type over R

Description

`OverdeterminedLinearSystemSolver(R, M)` provides a solver for overdetermined linear algebraic equations with coefficients in R .

Exports

`kernel!` $(M, M \rightarrow M) \rightarrow M$ Solve an overdetermined system

Usage

kernel!(m, f)

Signature

kernel!: (M, M → M) → M

Parameter	Type	Description
<i>m</i>	M	A matrix
<i>f</i>	M → M	Computes a kernel

Returns

Returns a basis for the kernel of m, uses a specialized algorithm if m is overdetermined, uses f when the system is no longer overdetermined.

Remarks

Can destroy m.

Usage

```
import from SpecializationLinearAlgebra(R, M)
```

Parameter	Type	Description
R	CommutativeRing Specializable	The coefficient domain
M	MatrixCategory R	A matrix type

Description

SpecializationLinearAlgebra(R, M) provides basic linear algebra functionalities using specializations for matrices over R .

Exports

```
rankLowerBound: M → Partial Cross (Boolean, MachineInteger) Probable rank
```

Usage

rankLowerBound a

Signature

rankLowerBound: $M \rightarrow \text{Partial Cross}(\text{Boolean}, \text{MachineInteger})$

Parameter	Type	Description
<i>a</i>	M	A matrix

Returns

Returns *failed* if specialization failed, otherwise $(rank?, r)$ such that $r \leq \text{rank}(a)$, and r is exactly the rank of a if $rank?$ is *true*.

Remarks

r can also happen to be the rank of a when $rank?$ is *false*, but the algorithm was unable to prove it.

TwoStepFractionFreeGaussElimination

Usage

```
import from TwoStepFractionFreeGaussElimination(R,M)
```

Parameter	Type	Description
R	IntegralDomain	A coefficient ring
M	MatrixCategory R	A matrix type over R

Exports

```
LinearEliminationCategory(R,M)
```

Description

This domain implements two-step fraction-free Gaussian elimination on matrices.

Usage

```
import from UnivariatePolynomialCRTLinearAlgebra(R, RX, M)
```

Parameter	Type	Description
R	IntegralDomain	The coefficient ring
RX	UnivariatePolynomialAlgebra R	Polynomials over R
M	MatrixCategory RX	A matrix type over RX

Description

UnivariatePolynomialCRTLinearAlgebra(F , FX , M) provides basic linear algebra functionalities using the Chinese Remainder Theorem from RX to R for matrices over RX .

Exports

degreeBound:	$M \rightarrow \text{Integer}$	Degree bound for the determinant
determinant:	$M \rightarrow RX$	Determinant
	$(M, RX) \rightarrow RX$	
	$(M, RX, \text{Integer}) \rightarrow RX$	

Usage

degreeBound a

Signature

degreeBound: M → Integer

Parameter	Type	Description
<i>a</i>	M	A matrix

Returns

Returns *n* such that $\deg |a| \leq n$.

Usage

determinant a
determinant(a, d)
determinant(a, d, n)

Signatures

determinant: $M \rightarrow RX$
determinant: $(M, RX) \rightarrow RX$
determinant: $(M, RX, Integer) \rightarrow RX$

Parameter	Type	Description
a	M	A matrix
d	RX	A known factor of $ a $ (optional)
n	$Integer$	A known degree bound on $ a $ (optional)

Returns

All calls to `determinant` return the determinant of a .

Remarks

The extra parameters d and n are optional. If they are provided, then d must divide $|a|$ exactly, and n must be such that $\deg |a| \leq n$.

Usage

```
import from UnivariatePolynomialPopovLinearAlgebra(F, FX, M)
```

Parameter	Type	Description
F	Field	The coefficient field
FX	UnivariatePolynomialAlgebra F	Polynomials over F
M	MatrixCategory FX	A matrix type over FX

Description

UnivariatePolynomialPopovLinearAlgebra(F , FX , M) provides basic linear algebra functionalities using weak Popov forms for matrices over FX .

Exports

determinant:	$M \rightarrow FX$	Determinant
hermite:	$M \rightarrow M$	Hermite form
kernel:	$M \rightarrow M$	Kernel
maxInvertibleSubmatrix:	$M \rightarrow (AZ, AZ)$	Maximal minor
popov:	$M \rightarrow M$	weak Popov form
rank:	$M \rightarrow \text{MachineInteger}$	Rank
span:	$M \rightarrow AZ$	Span

where

$AZ == \text{Array MachineInteger}$

Usage

hermite a

Signature

hermite: M → M

Parameter	Type	Description
<i>a</i>	M	A matrix

Returns

Returns the Hermite form of *a*.

Vector

Usage

import from Vector R

Parameter	Type	Description
R	ExpressionType AdditiveType	The coefficient domain

Description

Vector R provides vectors of arbitrary size with entries in R. They are 1-indexed and without bound checking.

Exports

AdditiveType

BoundedFiniteLinearStructureType R

ExpressionType

zero: MachineInteger $\rightarrow$ % zero vector

zero!: % $\rightarrow$ () make all the entries zero

zero?: % $\rightarrow$ Boolean test if all entries are zero

if R has ArithmeticType then

LinearCombinationType R

dot: (% , %) $\rightarrow$ R dot product

tensor: (% , %) $\rightarrow$ % tensor product

if R has Ring then

random: () $\rightarrow$ % random vector

MachineInteger $\rightarrow$ %

Usage`dot(u,v)`**Signature**`dot: (%,%) $\rightarrow$  R`

Parameter	Type	Description
u, v	<code>%</code>	Vectors of the same size

ReturnsReturns $u \cdot v = \sum_i u_i v_i$.

Usage

random()
random n

Signatures

random: () → %
random: MachineInteger → %

Parameter	Type	Description
n	MachineInteger	The dimension of the new vector.

Returns

random() returns a random vector of size at most 100, while random(n) returns a random vector of size n.

Usage

tensor(u,v)

Signature

tensor: (%,%) $\rightarrow$ %

Parameter	Type	Description
u, v	%	Vectors with coefficients from R.

Returns

Returns $u \otimes v = (u_1v_1, u_1v_2, \dots, u_1v_m, u_2v_1, \dots, u_nv_m)$.

Usage

zero n
 zero! v
 zero? v

Signatures

zero: `MachineInteger` $\rightarrow$ %
 zero!: % $\rightarrow$ ()
 zero?: % $\rightarrow$ `Boolean`

Parameter	Type	Description
n	<code>MachineInteger</code>	The dimension of the new vector
v	%	A vector with coefficients from $\mathbb{R}$

Description

zero(n) returns a zero vector of size n , while zero!(v) fills v with 0's and zero?(v) returns *true* if all the entries of v are 0, *false* otherwise.

VectorOverFraction

Usage

import from VectorOverFraction(R , Q)

Parameter	Type	Description
R	IntegralDomain	an integral domain
Q	FractionCategory R	a fraction domain over R

Description

VectorOverFraction(R , Q) provides useful conversions between vectors with integral and rational coefficients.

Exports

LinearCombinationFraction(R , Vector R , Q , Vector Q)

Usage

```
import from DenseUnivariatePolynomial R
import from DenseUnivariatePolynomial(R, x)
```

Parameter	Type	Description
R	ExpressionType ArithmeticType	The coefficient domain
x	Symbol	The variable name (optional)

Description

DenseUnivariatePolynomial(R, x) implements dense univariate polynomials with coefficients in R.

Exports

```
UnivariatePolynomialAlgebra R
```

Usage

```
import from DenseUnivariateTaylorSeries R
import from DenseUnivariateTaylorSeries(R, x)
```

Parameter	Type	Description
R	ExpressionType ArithmeticType	The coefficient domain
x	Symbol	The variable name (optional)

Description

DenseUnivariateTaylorSeries(R, x) implements univariate Taylor series with coefficients in R.

Exports

```
UnivariateTaylorSeriesType R
```

FactorizationRing

Usage

FactorizationRing: Category

Description

FactorizationRing is the category of rings that export an algorithm for factoring univariate polynomials over themselves into irreducibles.

Exports

GcdDomain

RationalRootRing

factor: (P:POL %) → P → (%, Product P) Factorization into irreducibles

fractionalRoots: (P:POL %) → P → Generator FR % Roots in the fraction field

roots: (P:POL %) → P → Generator FR % Roots in the coefficient ring

where

FR == FractionalRoot

POL == UnivariatePolynomialAlgebra0

Usage

factor(P)(p)

Signature

factor: (P: UnivariatePolynomialAlgebra0 %) → P → (% , Product P)

Parameter	Type	Description
P	UnivariatePolynomialAlgebra0 %	A polynomial type
p	P	A polynomial

Returns

Returns $(c, p_1^{e_1} \cdots p_n^{e_n})$ such that each p_i is irreducible, the p_i 's have no common factors, and

$$p = c \prod_{i=1}^n p_i^{e_i} .$$

Usage

fractionalRoots(P)(p)
roots(P)(p)

Signature

fractionalRoots,roots: (P: POL %) → P → Generator FractionalRoot %
where
POL == UnivariatePolynomialAlgebra0

Parameter	Type	Description
P	UnivariatePolynomialAlgebra0 %	A polynomial type
p	P	A polynomial

Returns

Returns a generator that produces all the roots p either in the ring or in its fraction field.

FFTRing

Usage

FFTRing: Category

Description

FFTRing is the category of rings that export an algorithm for the FFT product of univariate polynomials over themselves.

Exports

CommutativeRing

fft: (P:POL %) → (P, P) → Partial P FFT product

fft!: (P:POL %) → (P, P, P) → Boolean FFT product

fftCutoff: → MachineInteger Cutoff for FFT in $R[x]$

where

POL == UnivariatePolynomialAlgebra0

Usage

fft(P)(p,q)

Signature

fft: (P: UnivariatePolynomialAlgebra0 %) → (P, P) → Partial P

Parameter	Type	Description
P	UnivariatePolynomialAlgebra0 %	A polynomial type
p,q	P	Polynomials

Returns

Returns the product pq computed using FFT.

See also

fft!

Usage

fft!(P)(r,p,q)

Signature

fft!: (P: UnivariatePolynomialAlgebra0 %) → (P, P, P) → Boolean

Parameter	Type	Description
P	UnivariatePolynomialAlgebra0 %	A polynomial type
r,p,q	P	Polynomials

Returns

Replaces r by $r + pq$ where the product is computed using FFT. The space occupied by the first argument r is allowed to be reused. Returns *true* if the product could not be computed by the FFT method, *false* otherwise.

See also

fft

Usage

fftCutoff

Signature

fftCutoff: MachineInteger

Returns

Returns n such that the FFT multiplication is used in $R[x]$ for polynomials of degree greater than or equal to n .

Remarks

If this constant is 0, then FFT multiplication is not used at all in $R[x]$.

GenericModularPolynomialGcdPackage

Usage

import from GenericModularPolynomialGcdPackage (R, U)

Parameter	Type	Description
R	EuclideanDomain SourceOfPrimes ModularComputation	
U	UnivariatePolynomialAlgebra (R)	

Description

GenericModularPolynomialGcdPackage (R, U) provides a generic modular gcd algorithm. See the paper *On the genericity of the Modular Gcd Algorithm* by Kaltofen and Monagan in the proc. of ISSAC 1999.

Exports

modularGcd: $(U, U) \rightarrow \text{Partial } (U)$ gcd (failure if not enough primes)

HeuristicGcd

Usage

```
import from HeuristicGcd(Z, P)
```

Parameter	Type	Description
Z	IntegerCategory	An integer-like ring
P	UnivariatePolynomialAlgebra0 Z	Polynomials over Z

Description

HeuristicGcd provides an implementation of the HEUGCD algorithm for univariate polynomials over the integers.

Exports

balancedRemainder:	$(Z, Z) \rightarrow Z$	Symmetric Remainder
heuristicGcd:	$(P, P) \rightarrow (\text{Partial } P, P, P)$	the Heuristic gcd algorithm
radixInterpolate:	$(Z, Z) \rightarrow P$	Radix interpolation

Usage

balancedRemainder(*n*, *m*)

Signature

balancedRemainder: $(\mathbb{Z}, \mathbb{Z}) \rightarrow \mathbb{Z}$

Parameter	Type	Description
<i>n</i>	$\mathbb{Z}$	An integer
<i>m</i>	$\mathbb{Z}$	An integer

Returns

Returns $-m/2 \leq r < m/2$ such that $n \equiv r \pmod{m}$.

Usage

heuristicGcd(p_1, p_2)

Signature

heuristicGcd: $(P, P) \rightarrow (\text{Partial } P, P, P)$

Parameter	Type	Description
p_1, p_2	P	Polynomials

Returns

Returns (g, q_1, q_2) such that $g = \gcd(p_1, p_2)$, $p_1 = gq_1$ and $p_2 = gq_2$.

Remarks

This heuristic can fail in theory, in which case *(failed, p_1, p_2)* is returned, although this has never been reported.

Usage

radixInterpolate(*n*, *m*)

Signature

radixInterpolate: $(\mathbb{Z}, \mathbb{Z}) \rightarrow \mathbb{P}$

Parameter	Type	Description
<i>n</i>	$\mathbb{Z}$	A point
<i>m</i>	$\mathbb{R}$	The value of the desired polynomial at <i>n</i>

Returns

Returns the unique polynomial p such that $p(n) = m$ and

$$\|p\|_{\infty} \leq \frac{n-1}{2}.$$

Remarks

The above bound is useful when an upper bound M on $\|p\|_{\infty}$ is known a priori, since it is then sufficient to take $n \geq 2M + 1$.

Usage

```
import from ModularUnivariateGcd(Z, P)
```

Parameter	Type	Description
Z	IntegerCategory	An integer-like ring
P	UnivariatePolynomialAlgebra Z	Polynomials over Z

Description

ModularUnivariateGcd provides an implementation of a modular GCD algorithm for univariate polynomials over the integer, using the Chinese Remainder Theorem.

Exports

```
modularGcd: (P, P) → (Partial P, P, P)  The Modular Gcd algorithm
```

Usage

`modularGcd( $p_1, p_2$ )`

Signatures

`modularGcd`: $(P, P) \rightarrow (\text{Partial } P, P, P)$

Parameter	Type	Description
p_1, p_2	P	Polynomials over Z

Returns

Returns (g, y, z) such that $g = \gcd(p_1, p_2)$ or *failed*, and if g is not *failed*, then $p = yg$ and $q = zg$.

Remarks

This algorithm can fail because it runs out of primes. This will happen if the gcd has coefficients with more than around 3000 digits.

ModulopUnivariateGcd

Usage

```
import from ModulopUnivariateGcd
```

Description

ModulopUnivariateGcd provides an implementation of an inplace gcd for polynomials modulo a machine prime.

Exports

```
gcd!: (ARR Z, Z, ARR Z, Z, Z, Z) → (ARR Z, Z, Z)  in-place gcd
```

where

```
Z      ==  MachineInteger
```

```
ARR    ==  PrimitiveArray
```

Usage

```
gcd!(a, n, b, m, α, p)
gcd!(a, n, b, m, α, p, [l1, ..., lp-1], [e1, ..., ep-1])
```

Signatures

```
gcd!: (ARR Z, Z, ARR Z, Z, Z) → (ARR Z, Z, Z)
gcd!: (ARR Z, Z, ARR Z, Z, Z, ARR Z, ARR Z) → (ARR Z, Z, Z)
```

where

```
Z      == MachineInteger
ARR    == Array
```

Parameter	Type	Description
a, b	PrimitiveArray MachineInteger	polynomials modulo p
n, m	MachineInteger	their degrees
α	MachineInteger	a leading coefficient
p	MachineInteger	a prime
$[l_0, \dots, l_{p-1}]$	PrimitiveArray MachineInteger	log table
$[e_0, \dots, e_{p-1}]$	PrimitiveArray MachineInteger	exp table

Description

Given 2 polynomials stored in a and b of degrees n and m respectively, computes a $\gcd(a, b)$ in $F_p[x]$ with leading coefficient α . Requires $n \geq m$ and that the polynomials are stored leading coefficient first. Returns the degree of the gcd and its starting index in the array.

Remarks

The result can be stored in either a or b , so the function also returns the appropriate array. Note that both a and b are destroyed. The last 2 optional arrays are such that $g^{l_i} = i$ and $e_i = g^i$ where g is a primitive root for the multiplicative group modulo p . They are used for fast multiplication if provided.

Usage

```
import from PrimeFieldUnivariateFactorizer(F, P)
```

Parameter	Type	Description
F	PrimeFieldCategory0	Coefficient ring of the polynomials
P	UnivariatePolynomialAlgebra F	A polynomial ring

Description

PrimeFieldUnivariateFactorizer provides implementations of various univariate factorization algorithms over a prime field.

Exports

berlekamp:	$P \rightarrow \text{List } P$	Berlekamp's algorithm
cantorZassenhaus:	$P \rightarrow \text{List } P$	Cantor-Zassenhaus algorithm
factor:	$P \rightarrow (F, \text{PROD})$	Factor (default algorithm)
factor:	$(P \rightarrow \text{List } P) \rightarrow (P \rightarrow (F, \text{PROD}))$	Factor (given algorithm)
roots:	$(P, \text{Boolean}) \rightarrow \text{Generator FR } F$	Roots of a polynomial

where

```
FR    == FractionalRoot
PROD  == Product P
```

Usage

berlekamp p

Signature

berlekamp: $P \rightarrow \text{List } P$

Parameter	Type	Description
p	P	The square free and monic polynomial to factor.

Returns

Returns the list of irreducible factors of p .

Usage

cantorZassenhaus p

Signature

cantorZassenhaus: P $\rightarrow$ List P

Parameter	Type	Description
p	P	The square free and monic polynomial to factor.

Returns

Returns the list of irreducible factors of p .

Usage

roots(*p*, *sqrfree?*)

Signature

roots: (P, Boolean $\rightarrow$ Generator FractionalRoot F

Parameter	Type	Description
<i>p</i>	P	A polynomial.
<i>sqrfree?</i>	Boolean	Indicates whether <i>p</i> is squarefree

Returns

Returns the roots of *p* in F with their multiplicities. Assumes that *p* is squarefree if *sqrfree?* is *true*.

Usage

factor p
 factor(e)(p)

Signatures

factor: P → (F, Product P)
 factor: (P → List P) → (P → (F, Product P))

Parameter	Type	Description
p	P	The polynomial to factor.
e	P → List P	The factorization engine to use.

Returns

Returns $(c, p_1^{e_1} \cdots p_n^{e_n})$ such that each p_i is irreducible, the p_i 's have no common factors, and

$$p = c \prod_{i=1}^n p_i^{e_i} .$$

Remarks

Uses the factorizer e for factoring the monic squarefree factors of p .

RationalRootRing

Usage

RationalRootRing: Category

Description

RationalRootRing is the category of gcd domains that export an algorithm for finding the rational roots of univariate polynomials over themselves.

Exports

Ring

integerRoots: (P:POL %) → P → Generator FR Integer Integer roots

rationalRoots: (P:POL %) → P → Generator FR Integer Rational roots

where

FR == FractionalRoot

POL == UnivariatePolynomialAlgebra0

Usage

integerRoots(P)(p)
rationalRoots(P)(p)

Signature

integerRoots,rationalRoots: (P: POL %) → P → Generator RationalRoot
where
POL == UnivariatePolynomialAlgebra0

Parameter	Type	Description
P	UnivariatePolynomialAlgebra0 %	A polynomial type
p	P	A polynomial

Returns

Return $[(r_1, e_1), \dots, (r_n, e_n)]$ where the r_i 's are the integer or rational roots of p and have multiplicity e_i .

Resultant

Usage

```
import from Resultant(R,P)
```

Parameter	Type	Description
R	IntegralDomain	Coefficient ring for the polynomials
P	UnivariatePolynomialAlgebra0 R	A polynomial ring

Exports

lastSPRS:	$(P,P) \rightarrow P$	last non-zero remainder in the SPRS
extendedLastSPRS:	$(P,P) \rightarrow (P,P,P)$	extended last non-zero remainder in the SPRS
resultant:	$(P,P) \rightarrow R$	polynomial resultant
SPRS:	$(P,P) \rightarrow \text{List } P$	Subresultant Polynomial Remainder Sequence
subResultantGcd:	$(P,P) \rightarrow P$	GCD

Description

Resultant(R,P) implements polynomial resultant computations.

UsagelastSPRS(a, b)**Signature**lastSPRS: $(P, P) \rightarrow P$

Parameter	Type	Description
a, b	P	Two polynomials

Description

lastSPRS(a, b) returns the last non-zero remainder in the subresultant polynomial remainder sequence of a and b . a and b both should be non-zero and $\deg(a)$ should be at least $\deg(b)$.

Usage

extendedLastSPRS(a,b)

Signature

extendedLastSPRS: $(P,P) \rightarrow (P,P,P)$

Parameter	Type	Description
a,b	P	Two polynomials

Description

extendedLastSPRS(a,b) returns (r, s, t) , where r is the last non-zero remainder in the subresultant polynomial remainder sequence of a and b and s, t are polynomials such that $r = sa + tb$. a and b both should be non-zero and $\deg(a)$ should be at least $\deg(b)$.

Usage

resultant(a,b)

Signature

resultant: (P,P) $\rightarrow$ R

Parameter	Type	Description
<i>a,b</i>	P	Two polynomials

Description

resultant(a,b) returns the resultant of *a* and *b*. *a* and *b* both should be non-zero.

Usage

SPRS(a,b)

Signature

SPRS: (P,P) → List P

Parameter	Type	Description
<i>a,b</i>	P	Two polynomials

Description

SPRS(a,b) returns the list of remainders in the subresultant polynomial remainder sequence of *a* and *b*. *a* and *b* both should be non-zero and $\deg(a)$ should be at least $\deg(b)$. The list has increasing degree, i.e. the first element in the list is the last remainder in the remainder sequence.

Usage

subResultantGcd(a,b)

Signature

subResultantGcd: $(P,P) \rightarrow P$

Parameter	Type	Description
a,b	P	Two polynomials

Description

subResultantGcd(a,b) returns the last non-zero remainder in the subresultant polynomial remainder sequence of either a and b or b and a . If R is a Gcd domain, this is then $\gcd(a,b)$. Also returns $\gcd(a,b)$ if either $a = 0$ or $b = 0$.

Usage

```
import from SparseUnivariatePolynomial0 R
import from SparseUnivariatePolynomial0(R, x)
```

Parameter	Type	Description
R	ExpressionType ArithmeticType	The coefficient domain
x	Symbol	The variable name (optional)

Description

`SparseUnivariatePolynomial0(R, x)` implements free modules over an arbitrary arithmetic system R , with respect to free monoid generated by x . Its elements are assumed to have finite support. The representation is sparse.

Exports

```
IndexedFreeModule (R,Integer)
```

SparseUnivariatePolynomial1

Usage

```
import from SparseUnivariatePolynomial1 R
import from SparseUnivariatePolynomial1(R, x)
```

Parameter	Type	Description
R	ExpressionType ArithmeticType	The coefficient domain
x	Symbol	The variable name (optional)

Description

SparseUnivariatePolynomial1(R, x) implements sparse univariate polynomials with coefficients in R.

Exports

```
UnivariatePolynomialRing R
```

SparseUnivariatePolynomial

Usage

```
import from SparseUnivariatePolynomial R
import from SparseUnivariatePolynomial(R, x)
```

Parameter	Type	Description
R	ExpressionType ArithmeticType	The coefficient domain
x	Symbol	The variable name (optional)

Description

SparseUnivariatePolynomial(R , x) implements sparse univariate polynomials with coefficients in R .

Exports

```
UnivariatePolynomialAlgebra R
```

UnivariateFactorialPolynomial

Usage

```
import from UnivariateFactorialPolynomial(R, Rx)
```

Parameter	Type	Description
R	Ring	The coefficient domain
Rx	UnivariatePolynomialAlgebra R	A polynomial type over R

Description

UnivariateFactorialPolynomial(R , Rx) implements univariate factorial polynomials with coefficients in R . Those are polynomials with respect to the basis of the descending factorials $(x^n)_{n \geq 0}$, where $x^n = x(x-1)\dots(x-n+1)$. Rx is used for representing the factorial polynomials, so you can choose between sparse and dense representations.

Exports

```
UnivariateFreeRing  $R$ 
```

```
coerce:       $Rx \rightarrow \%$       Conversion to a factorial polynomial
```

```
expand:       $\% \rightarrow Rx$           Conversion from a factorial polynomial
```

```
trailExpand:  $\% \rightarrow (\text{Integer}, Rx)$  Conversion from a factorial polynomial
```

```
if  $R$  has CommutativeRing then  
  CommutativeRing
```

```
if  $R$  has IntegralDomain then  
  IntegralDomain
```

```
if  $R$  has RationalRootRing then  
  integerRoots:   $\% \rightarrow \text{List FractionalRoot Integer}$  Integer roots  
  rationalRoots:  $\% \rightarrow \text{List FractionalRoot Integer}$  Rational roots
```

Usage

```

p::%
coerce p
expand q
(n, h) := trailExpand q

```

Signatures

```

coerce:      Rx → %
expand:      % → Rx
trailExpand: % → (Integer, Rx)

```

Parameter	Type	Description
p	Rx	A polynomial
q	%	A factorial polynomial

Description

$p::\%$ converts p from the power basis $(x^n)_{n \geq 0}$ to the factorial basis $(x^n)_{n \geq 0}$, while $\text{expand}(q)$ performs the reverse conversion and $\text{trailExpand}(q)$ returns (n, h) such that $q = x^n h$.

Usage

integerRoots p
rationalRoots p

Signature

integerRoots,rationalRoots: % → List FractionalRoot Integer

Parameter	Type	Description
p	%	A factorial polynomial

Returns

Return $[(r_1, e_1), \dots, (r_n, e_n)]$ where the r_i 's are the integer or rational roots of p and have multiplicity e_i .

Usage

UnivariateFreeLinearArithmeticType R: Category

Parameter	Type	Description
R	ExpressionType ArithmeticType	The coefficient domain

Description

UnivariateFreeLinearArithmeticType is a common category for commutative and noncommutative univariate polynomials and power series with coefficients in an arbitrary arithmetic system R and with respect to an arbitrary basis $(P_n)_{n \geq 0}$. Its elements are not assumed to have finite support, so this type cannot be asserted to be an `RRing` R even when R is a `Ring`.

Exports

<code>IndexedFreeLinearArithmeticType(R, Z)</code>		
<code>add!:</code>	<code>(%, R, Z) → %</code>	In-place addition of a term
<code>add!:</code>	<code>(%, R, Z, %) → %</code>	In-place product and sum
<code>apply:</code>	<code>(%, TREE) → TREE</code>	Conversion to an expression tree
<code>apply:</code>	<code>(OUT, %, Symbol) → OUT</code>	Write an element to a port
<code>coefficients:</code>	<code>% → Generator R</code>	Iterate over all the coefficients
<code>monom:</code>	<code>→ %</code>	Term with degree 1 and coefficient 1
<code>setCoefficient!:</code>	<code>(%, Z, R) → %</code>	In-place replacement of a coefficient
<code>shift:</code>	<code>(%, Z) → %</code>	Exponent translation
<code>shift!:</code>	<code>(%, Z) → %</code>	In-place exponent translation
<code>truncate:</code>	<code>(%, Z) → %</code>	Truncation
<code>truncate!:</code>	<code>(%, Z) → %</code>	In-place truncation

where

<code>OUT</code>	<code>==</code>	<code>TextWriter</code>
<code>TREE</code>	<code>==</code>	<code>ExpressionTree</code>
<code>Z</code>	<code>==</code>	<code>Integer</code>

Usage

```

apply(p, t)
p t
apply(port, p, x)
port(p, x)

```

Signatures

```

apply:  (% , ExpressionTree) → ExpressionTree
apply:  (TextWriter, % , Symbol) → TextWriter

```

Parameter	Type	Description
p	%	A polynomial or series
t	ExpressionTree	An expression tree
x	Symbol	A name for the variables
$port$	TextWriter	An output port

Description

`apply(p, t)` returns `p` as an expression tree, using `t` as root variable name, while `apply(port, p, x)` sends `p` to `port` using `x` as root variable name, and returns the output port afterwards.

Example

```

import from Integer, DenseUnivariatePolynomial Integer;

p := term(1, 3) + term(2, 1) - term(1, 0);  -- p = x^3 + 2 x - 1
stdout(map((n:Integer):Integer +-> n + n)(p), -"x");
writes

      2*x^3+4*x-2
to the standard stream stdout.

```

Usage

for c in coefficients m repeat { ... }

Signature

coefficients: % $\rightarrow$ Generator R

Parameter	Type	Description
m	%	An element of the module

Returns

Returns a generator that produces all the coefficients of m , including the ones which are 0.

See also

generator, terms

Usage

monom

Signature

monom: %

Returns

Returns P_1 , *i.e.* the term with degree 1 and coefficient 1.

See also

monomial

Usage

shift(*p*, *m*)
 shift!(*p*, *m*)

Signature

shift: (*%*, Integer) → *%*

Parameter	Type	Description
<i>p</i>	<i>%</i>	A polynomial or series
<i>m</i>	Integer	The amount to shift

Returns

Returns

$$\sum_{i \geq \max(0, -m)} a_i P_{i+m}$$

where $p = \sum_{i \geq 0} a_i P_i$.

Remarks

When using shift!, the storage used by *p* is allowed to be destroyed or reused, so *p* is lost after this call. This may cause *p* to be destroyed, so do not use this unless *p* has been locally allocated, and is thus guaranteed not to share space with other series or polynomials.

Usage

truncate(*p*, *m*)
 truncate!(*p*, *m*)

Signature

truncate: (%,Integer) → %

Parameter	Type	Description
<i>p</i>	%	A polynomial or series
<i>m</i>	Integer	The truncation order

Returns

Returns the truncation of *p* at order *m*, *i.e.*

$$\sum_{i=0}^{m-1} a_i P_i$$

where $p = \sum_{i \geq 0} a_i P_i$.

Remarks

When using truncate!, the storage used by *p* is allowed to be destroyed or reused, so *p* is lost after this call. This may cause *p* to be destroyed, so do not use this unless *p* has been locally allocated, and is thus guaranteed not to share space with other series or polynomials.

Usage

UnivariateFreeRing R: Category

Parameter	Type	Description
R	ExpressionType ArithmeticType	The coefficient domain

Description

UnivariateFreeRing is a common category for commutative and noncommutative univariate polynomials with coefficients in an arbitrary arithmetic system R and with respect to an arbitrary basis $(P_n)_{n \geq 0}$.

Exports

```
CopyableType
IndexedFreeRRing(R, Integer)
UnivariateFreeLinearArithmeticType R
coerce:      Vector R → %           Create a polynomial
companion:   % → DenseMatrix R     Companion matrix
monomial!:   (% , R, Z) → %         In-place monomial
random:      (Integer, () → R, Integer) → %  Creation of a random polynomial
revert:      % → %                 Left-Right reversion
revert!:     % → %                 In-place left-right reversion
vectorize!:  Vector R → % → Vector R  Conversion to a vector
```

where

```
Z == Integer
```

```
if R has HashType then
```

```
  HashType
```

```
if R has Ring then
```

```
  random: (Integer, Integer) → %  Creation of a random polynomial
```

```
if R has SerializableType then
```

```
  SerializableType
```

Usage

```

v::%
vectorize! v
vectorize!(v)(p)

```

Signatures

```

coerce:      Vector R → %
vectorize!:  Vector R → % → Vector R

```

Parameter	Type	Description
p	$\%$	A polynomial
v	$\text{Vector } R$	A coefficient vector

Description

$[v_1, \dots, v_n]::\%$ returns the polynomial $\sum_{i=0}^{n-1} v_{i+1}P_i$, while $\text{vectorize!}(v)(\sum_{j=0}^d a_jP_j)$ fills v with $[a_0, \dots, a_d, 0, \dots]$ and returns v .

Remarks

If $d > \#v - 1$, then the high coefficients of p are simply ignored.

Usage

companion p

Signature

companion: % $\rightarrow$ DenseMatrix R

Parameter	Type	Description
p	%	A polynomial

Returns

Returns the companion matrix

$$\begin{pmatrix} 0 & & & -a_0 \\ a_n & \ddots & & -a_1 \\ & \ddots & & -a_2 \\ & & & \vdots \\ & & a_n & -a_{n-1} \end{pmatrix}$$

where $p = \sum_{i=0}^n a_i P_i$.

Usage

monomial!(p, c, n)

Signature

monomial!: ($\%$, R, Integer) $\rightarrow$ $\%$

Parameter	Type	Description
p	$\%$	A polynomial (to be destroyed)
c	R	A scalar
n	Integer	An exponent

Returns

Returns the monomial cP_n .

Remarks

The storage used by p is allowed to be destroyed or reused so p is lost after this call. This may cause p to be destroyed, so do not use this unless p has been locally allocated, and is guaranteed not to share space with other elements. Some functions are not necessarily copying their arguments and can thus create memory aliases.

Usage

```
random(n[, m])
random(n, f[, m])
```

Signatures

```
random: (Integer, Integer) → %
random: (Integer, () → R, Integer) → %
```

Parameter	Type	Description
n	Integer	The desired degree
f	$() \rightarrow R$	A random generator for R
m	Integer	The desired number of terms (optional)

Returns

Returns a monic random polynomial of degree n with at most m terms, ($n + 1$ terms if m is not present). Uses $f()$ to generate the coefficients if the parameter f is present, the **random** function otherwise.

Usage

revert p
 revert! p

Signature

revert: % $\rightarrow$ %

Parameter	Type	Description
p	%	A polynomial

Returns

Returns $\sum_{i=0}^n a_i P_{n-i}$ where $p = \sum_{i=0}^n a_i P_i$ and $a_n \neq 0$. Returns 0 if $p = 0$.

Remarks

The storage used by p is allowed to be destroyed or reused when `revert!` is used, so p is lost after those calls. This may cause p to be destroyed, so do not use this unless p has been locally allocated, and is guaranteed not to share space with other elements. Some functions are not necessarily copying their arguments and can thus create memory aliases.

See also

`terms`

Usage

times(p, q, l, h)

Signature

times: (% , % , Integer, Integer) $\rightarrow$ %

Parameter	Type	Description
p, q	%	Polynomials
l, h	Integer	Lower and upper bound of coefficients to be computed

Returns

times(p, q, l, h) returns a polynomial s containing some coefficients of the product pq . The i th coefficient of s equals the $l + i$ th coefficient of pq . Here $l \leq h$ should hold.

Usage

```
import from UnivariateFreeRing2(R,Rx,S,Sy)
```

Parameter	Type	Description
R, S	ExpressionType ArithmeticType	Coefficient domains
Rx	UnivariateFreeRing R	A monogenic algebra over R
Sy	UnivariateFreeRing S	A monogenic algebra over R

Description

UnivariateFreeRing2(R,Rx,S,Sy) provides tools for lifting maps $R \rightarrow S$ to maps $Rx \rightarrow Sy$.

Exports

```
map: ( $R \rightarrow S$ )  $\rightarrow Rx \rightarrow Sy$  Lift a mapping
```

Usage

```
map f
map(f)(p)
```

Signature

```
map:  (R → S) → Rx → Sy
```

Parameter	Type	Description
f	$R \rightarrow S$	A map
p	%	A polynomial with coefficient in R

Description

map(f)(p) returns

$$f(p) = \sum_i f(a_i)y^i$$

where $p = \sum_i a_i x^i$, while map(f) returns the mapping $p \rightarrow f(p)$.

UnivariateFreeRingOverFraction

Usage

```
import from UnivariateFreeRingOverFraction(R, PR, Q, PQ)
```

Parameter	Type	Description
R	IntegralDomain	an integral domain
PR	UnivariateFreeRing R	a univariate free finite algebra type over R
Q	FractionCategory R	a fraction domain of R
PQ	UnivariateFreeRing R	a univariate free finite algebra type over Q

Description

UnivariateFreeRingOverFraction(R , PR , Q , PQ) provides useful conversions between polynomials with integral and rational coefficients.

Exports

```
LinearCombinationFraction( $R$ ,  $PR$ ,  $Q$ ,  $PQ$ )
```


UnivariateGcdRing

Usage

UnivariateGcdRing: Category

Description

UnivariateGcdRing is the category of rings which export a gcd algorithm for univariate polynomials over themselves.

Exports

GcdDomain

gcdUP: (P:UnivariatePolynomialAlgebra0) → (P, P) → P Gcd

gcdUP!: (P:UnivariatePolynomialAlgebra0) → (P, P) → P Gcd

gcdquoUP: (P:UnivariatePolynomialAlgebra0) → (P, P) → (P,P,P) Gcd

Usage

gcdUP(P)(p, q)
gcdUP!(P)(p, q)

Signature

gcdUP: (P: UnivariatePolynomialAlgebra0 %) → (P, P) → P

Parameter	Type	Description
P	UnivariatePolynomialAlgebra0 %	A polynomial type
p, q	P	Polynomials

Returns

Both function return $\gcd(p, q)$. When gcdUP! is used, the storage used by x_1 and x_2 is allowed to be destroyed or reused, so p and q are lost after this call.

Usage

gcdquoUP(P)(p, q)

Signature

gcdquoUP: (P: UnivariatePolynomialAlgebra0 %) → (P,P) → (P,P,P)

Parameter	Type	Description
P	UnivariatePolynomialAlgebra0 %	A polynomial type
p,q	P	Polynomials

Returns

Returns (g, y, z) such that $g = \gcd(p, q)$, $p = gy$ and $q = gz$.

UnivariateIntegralFactorizer

Usage

```
import from UnivariateIntegralFactorizer(Z, P)
```

Parameter	Type	Description
Z	IntegerCategory	An integer-like ring
P	UnivariatePolynomialAlgebra0 Z	A polynomial type over Z

Description

UnivariateIntegralFactorizer(Z , P) implements a factorizer for polynomials with integer coefficients.

Exports

factor:	$P \rightarrow (Z, \text{Product } P)$	Factor
integerRoots:	$P \rightarrow \text{Generator FractionalRoot Integer}$	Integer roots
rationalRoots:	$P \rightarrow \text{Generator FractionalRoot Integer}$	Rational roots

Usage

factor p

Signature

factor: P → (Z,Product P)

Parameter	Type	Description
p	P	A polynomial with integer coefficients

Returns

Returns $(c, p_1^{e_1} \cdots p_n^{e_n})$ such that each p_i is irreducible, the p_i 's have no common factors, and

$$p = c \prod_{i=1}^n p_i^{e_i}.$$

See also

squareFree

Usage

integerRoots p
 rationalRoots p

Signature

integerRoots,rationalRoots: $P \rightarrow \text{Generator FractionalRoot Integer}$

Parameter	Type	Description
p	P	A polynomial with integer coefficients

Returns

integerRoots(p) (resp. rationalRoots(p)) return $[(r_1, e_1), \dots, (r_n, e_n)]$ where the r_i 's are the integer (resp. rational) roots of p and have multiplicity e_i .

Usage

```
import from UnivariateMonomial R
import from UnivariateMonomial(R, x)
```

Parameter	Type	Description
R	ExpressionType ArithmeticType	Coefficients of the monomials
x	Symbol	The variable name (optional)

Description

This type implements univariate monomials with coefficients in R .

Exports

```
ExpressionType
apply:      (% , TREE) → TREE  Applies a monomial to a tree
coefficient: % → R             Extraction of the coefficient
degree:     % → Integer        The degree of a monomial
map:        (R → R) → % → %    Lift a map
map!:       (R → R) → % → %    Lift a map
monomial:   (R, Integer) → %    Creation of a monomial
setCoefficient!: (% , R) → R    In-place coefficient modification
setDegree!:  (% , Z) → Z        In-place degree modification
```

if R has `FiniteCharacteristic` then

```
pthPower:   % → %    Exponentiation to the characteristic
pthPower!:  % → %    In-place exponentiation to the characteristic
```

where

```
TREE == ExpressionTree
```

Usage

apply(p, t)

Signature

apply: (% , ExpressionTree) → ExpressionTree

Parameter	Type	Description
<i>p</i>	%	A monomial
<i>t</i>	ExpressionTree	An expression tree

Returns

Returns p as an expression tree using t as root variable name.

Usage

coefficient p

Signature

coefficient: % $\rightarrow$ R

Parameter	Type	Description
p	%	A monomial

Returns

Returns the coefficient of p , *i.e.* c where $p = c x^n$.

See also

degree, setCoefficient!

Usage

degree p

Signature

degree: % $\rightarrow$ Integer

Parameter	Type	Description
p	%	A monomial

Returns

Returns the degree of p , *i.e.* n where $p = c x^n$.

See also

coefficient

Usage

```
map f
map! f
map(f)(p)
map!(f)(p)
```

Signature

```
map:  (R → R) → % → %
```

Parameter	Type	Description
f	$R \rightarrow R$	A map
p	$\%$	A monomial

Description

`map(f)(p)` returns $f(a)x^n$ where $p = ax^n$, while `map(f)` returns the mapping $p \rightarrow f(p)$. In both cases, `map!` does not make a copy of p but modifies it in place.

Usage

monomial(*c*, *n*)

Signature

monomial: (R, Integer) → %

Parameter	Type	Description
<i>c</i>	R	A scalar
<i>n</i>	Integer	An exponent

Returns

Returns the monomial $c x^n$.

Usage

pthPower p
pthPower! p

Signature

pthPower: % $\rightarrow$ %

Parameter	Type	Description
p	%	A monomial

Returns

Returns $p^{\text{characteristic}}$.

Remarks

pthPower! does not make a copy of p , which is therefore modified after the call. It is unsafe to use the variable p after the call, unless it has been assigned to the result of the call, as in `p := pthPower! p`.

Usage`setCoefficient!(p, c)`**Signature**`setCoefficient!:(%, R) → R`

Parameter	Type	Description
p	%	A monomial
c	R	A scalar

Description

Sets the coefficient of p to c , *i.e.* changes $p = d x^n$ into $c x^n$

Returns

Returns the new coefficient c .

See also`coefficient`

Usage

setDegree!(p, c)

Signature

setDegree!: ($\%$, Integer) $\rightarrow$ Integer

Parameter	Type	Description
p	$\%$	A monomial
n	Integer	An exponent

Description

Sets the degree of p to n , *i.e.* changes $p = c x^m$ into $c x^n$

Returns

Returns the new degree n .

See also

degree

Usage

UnivariatePolynomialAlgebra R: Category

Parameter	Type	Description
R	ExpressionType ArithmeticType	The coefficient domain

Description

UnivariatePolynomialAlgebra is the category of univariate polynomials with coefficients in an arbitrary domain R and with respect to the power basis $(x^n)_{n \geq 0}$.

Exports

UnivariatePolynomialRing R

IndexedFreeAlgebra(R, Integer)

apply: $(\%, R) \rightarrow R$ Evaluate a polynomial

apply: $(\%, \%) \rightarrow \%$ Evaluate a polynomial

equal?: $(\%, \%, \%, \text{Integer}) \rightarrow \text{Boolean}$ Truncated equality

Horner: $(\%, R) \rightarrow (\%, R)$ Horner division by $x - a$

if R has CharacteristicZero then

ordinaryPoint: $\% \rightarrow \text{Integer}$ Point where a polynomial is nonzero

if R has CommutativeRing then

DifferentialRing

lift: $(\text{Derivation } R, \%) \rightarrow \text{Derivation } \%$ Extend a derivation

monicDivide: $(\%, \%) \rightarrow (\%, \%)$ Polynomial division

monicDivide!: $(\%, \%) \rightarrow (\%, \%)$ Polynomial division

monicDivideBy: $\% \rightarrow \% \rightarrow (\%, \%)$ Polynomial division

monicDivideBy!: $\% \rightarrow \% \rightarrow (\%, \%)$ Polynomial division

monicQuotient: $(\%, \%) \rightarrow \%$ Quotient

monicQuotient!: $(\%, \%) \rightarrow \%$ Quotient

monicQuotientBy: $\% \rightarrow \% \rightarrow \%$ Quotient

monicQuotientBy!: $\% \rightarrow \% \rightarrow \%$ Quotient

monicRemainder: $(\%, \%) \rightarrow \%$ Remainder

monicRemainder!: $(\%, \%) \rightarrow \%$ Remainder

monicRemainderBy: $\% \rightarrow \% \rightarrow \%$ Remainder

monicRemainderBy!: $\% \rightarrow \% \rightarrow \%$ Remainder

if R has CommutativeRing and R has RittRing then

integrate: $\% \rightarrow \%$ Integration

$(\%, \text{Integer}) \rightarrow \%$

if R has FactorizationRing then

factor: $\% \rightarrow (R, \text{Product } \%)$ Factorisation into irreducibles

fractionalRoots: $\% \rightarrow \text{Generator FractionalRoot } R$ Roots in the fraction field


```

    roots: % → Generator FractionalRoot R  Roots in the coefficient ring

if R has Field then
    EuclideanDomain
    rationalReconstruction: % → % → Partial Cross(%, %)  Rational reconstruction
    sparseMultiple:         (% , Integer) → %              Multiple in  $k[x^n]$ 

if R has FiniteField then
    LinearAlgebraRing

if R has GcdDomain then
    DecomposableRing
    GcdDomain
    squareFree: % → (R, Product %)  Squarefree factorisation
    squareFreePart: % → %           Squarefree part

if R has GcdDomain and R has RationalRootRing then
    dispersion: % → Integer          Dispersion
                (% , %) → Integer
    integerDistances: % → List Integer  Integer spread
                (% , %) → List Integer
    universalBound: (% , %) → List Cross(%, Integer)  Universal bound

if R has IntegralDomain then
    IntegralDomain
    pseudoDivide: (% , %) → (% , %)  Polynomial pseudo-division
    pseudoRemainder: (% , %) → %      Pseudo-remainder
    pseudoRemainder!: (% , %) → %      Pseudo-remainder
    resultant: (% , %) → R            Resultant of 2 polynomials

if R has OrderedArithmeticType then
    height: % → R  Max norm over all the coefficients

if R has RationalRootRing then
    RationalRootRing
    integerRoots: % → Generator FractionalRoot Integer  Integer roots
    rationalRoots: % → Generator FractionalRoot Integer  Rational roots

if R has Ring then
    values: (% , R) → Generator R  Generate values of a polynomial

if R has Specializable then
    Specializable

```

Usage

```
apply(p, a)
apply(p, q)
p a
p q
```

Signatures

```
apply:  (% , R) → R
apply:  (% , %) → %
```

Parameter	Type	Description
<i>p</i>	%	A polynomial
<i>q</i>	%	A polynomial
<i>a</i>	R	A scalar

Returns

Returns

$$p(a) = \sum_{i=0}^n a_i a^i$$

or

$$p(q) = \sum_{i=0}^n a_i q^i$$

where $p = \sum_{i=0}^n a_i x^i$.

Usage

equal?(a, b, c, n)

Signature

equal?: (% , % , % , Integer) → Boolean

Parameter	Type	Description
a, b, c	%	Polynomials
n	Integer	The order of truncation

Returns

Returns *true* if $a = bc \pmod{x^n}$, *false* otherwise.

Usage

height p

Signature

height: % → R

Parameter	Type	Description
p	%	A polynomial

Returns

Returns

$$||p||_{\infty} = \max_{i=0}^n (|a_i|)$$

where $p = \sum_{i=0}^n a_i x^i$.

Usage

Horner(p, a)

Signature

Horner: ($\%$, R) $\rightarrow$ ($\%$, R)

Parameter	Type	Description
p	$\%$	A polynomial
a	R	A point

Returns

Returns $(q, p(a))$ such that $p = q(x - a) + p(a)$.

Usage

integrate p
 integrate(p, n)

Signatures

integrate: $\% \rightarrow \%$
 integrate: $(\%, \text{Integer}) \rightarrow \%$

Parameter	Type	Description
p	$\%$	A polynomial
n	Integer	The order of integration

Returns

integrate(p) returns $\int p(x)dx$, while integrate(s, n) returns $\int \dots \int s(x)dx^n$.

Usage

lift(D, x')

Signature

lift: (Derivation R, %) → Derivation %

Parameter	Type	Description
D	Derivation R	A derivation on R
x'	%	The desired derivative of x

Returns

Returns the unique extension of the derivation D such that $Dx = x'$.

Usage

```

monicXXX(a, b)
monicXXX!(a, b)
monicXXXBy(b)(a)
monicXXXBy!(b)(a)

```

Signatures

```

monicDivide, monicDivide!:      (% , %) → (% , %)
monicDivideBy, monicDivideBy!:  % → % → (% , %)
monicQuotient, monicQuotient!:  (% , %) → %
monicRemainder, monicRemainder!: (% , %) → %
monicQuotientBy, monicQuotientBy!: % → % → %
monicRemainderBy, monicRemainderBy!: % → % → %

```

Parameter	Type	Description
a	%	A polynomial
b	%	A polynomial whose leading coefficient is a unit in R

Returns

monicRemainder(a , b) returns r such that either $r = 0$ or $\deg(r) < \deg(b)$ or $a \equiv r \pmod{b}$, monicQuotient(a , b) returns q such that $a - bq = 0$ or $\deg(a - bq) < \deg(b)$, and monicDivide(a , b) returns (q, r) such that $a = bq + r$ and either $r = 0$ or $\deg(r) < \deg(b)$. The functions monicDivide!, monicQuotient! and monicRemainder! return the same results but allow the storage used by a to be destroyed or reused. Finally, monicXXXBy(b) returns the map $a \rightarrow \text{monicXXX}(a, b)$, while monicXXXBy!(b) returns the map $a \rightarrow \text{monicXXX!}(a, b)$.

Remarks

When using monicXXX!(a, b) or monicXXXBy!(b)(a), the storage used by a is allowed to be destroyed or reused, so a is lost after this call. This may cause a to be destroyed, so do not use this unless a has been locally allocated, and is thus guaranteed not to share space with other polynomials.

See also

```
pseudoRemainder
```


Usage

ordinaryPoint p

Signature

ordinaryPoint: % $\rightarrow$ Integer

Parameter	Type	Description
p	%	A nonzero polynomial

Returns

Returns an integer n such that $p(n) \neq 0$.

Usage

pseudoDivide(a, b)

Signature

pseudoDivide: $(\%, \%) \rightarrow (\%, \%)$

Parameter	Type	Description
a	$\%$	A polynomial
b	$\%$	A nonzero polynomial

Returns

Returns (q, r) such that $c^n a = bq + r$ and either $r = 0$ or $\deg(r) < \deg(b)$, where c is the leading coefficient of b and $n = \deg(a) - \deg(b) + 1$.

See also

pseudoRemainder

Usage

pseudoRemainder(a, b)
 pseudoRemainder!(a, b)

Signature

pseudoRemainder: $(\%, \%) \rightarrow \%$

Parameter	Type	Description
a	$\%$	A polynomial
b	$\%$	A nonzero polynomial

Returns

Returns r such that $c^n a = bq + r$ and either $r = 0$ or $\deg(r) < \deg(b)$, where c is the leading coefficient of b and $n = \deg(a) - \deg(b) + 1$.

Remarks

When using `pseudoRemainder!(a, b)`, the storage used by a is allowed to be destroyed or reused, so a is lost after this call. This may cause a to be destroyed, so do not use this unless a has been locally allocated, and is thus guaranteed not to share space with other polynomials.

See also

`pseudoDivide`

Usage

rationalReconstruction m
 rationalReconstruction(m)(u)

Signature

rationalReconstruction: $\% \rightarrow \% \rightarrow \text{Partial Cross}(\%, \%)$

Parameter	Type	Description
m	$\%$	A modulus of positive degree
u	$\%$	A polynomial

Returns

rationalReconstruction(m)(u) returns either (a, b) such that $a/b = u \pmod{m}$, $\deg(a) \leq (\deg(m) - 1)/2$ and $\deg(b) \leq (\deg(m) - 1)/2$, or *failed* if no such a, b exist.

Remarks

The resulting a and b are guaranteed to be unique.

See also

rationalReconstruction

Usage

resultant(p, q)

Signature

resultant: (% , %) $\rightarrow$ R

Parameter	Type	Description
p	%	A polynomial
q	%	A polynomial

Returns

Returns the resultant of p and q.

Usage

sparseMultiple(p, n)

Signature

sparseMultiple: (% , Integer) → %

Parameter	Type	Description
p	%	A polynomial
n	Integer	A positive integer

Returns

Returns a nonzero polynomial $q = \sum_{i=0}^m a_i x^i$ of minimal degree such that $q(x^n)$ is a multiple of $p(x)$.

Usage

```
squareFree p
squareFreePart p
```

Signatures

```
squareFree:      % → (R, Product %)
squareFreePart:  % → %
```

Parameter	Type	Description
p	%	A polynomial

Description

squareFree(p) returns $(c, p_1^{e_1} \cdots p_n^{e_n})$ such that each p_i is squarefree, the p_i 's have no common factors, and

$$p = c \prod_{i=1}^n p_i^{e_i},$$

while squareFreePart(p) returns p^* such that p^* is squarefree, $p^* \mid p$ and every irreducible factor of p divides p^* .

Usage

values(p, a)

Signature

values: (%R) → Generator R

Parameter	Type	Description
p	%	A polynomial
a	R	A scalar

Returns

Returns a generator generating the sequence $p(a), p(a + 1), p(a + 2), \dots$

Remarks

values uses arrays of differences and can be more efficient than repeated Horner evaluation.

Usage

factor p

Signature

factor: % → (R, Product %)

Parameter	Type	Description
p	%	A polynomial

Returns

Returns $(c, p_1^{e_1} \cdots p_n^{e_n})$ such that each p_i is irreducible, the p_i 's have no common factors, and

$$p = c \prod_{i=1}^n p_i^{e_i}.$$

Usage

fractionalRoots p
roots p

Signature

fractionalRoots,roots: % $\rightarrow$ Generator FractionalRoot %

Parameter	Type	Description
p	%	A polynomial

Returns

Returns a generator that produces all the roots p either in the ring or in its fraction field.

Usage

```

dispersion p
dispersion(p, q)
integerDistances p
integerDistances(p, q)

```

Signatures

```

dispersion:      % → Integer
dispersion:      (%,% ) → Integer
integerDistances: % → List Integer
integerDistances: (%,% ) → List Integer

```

Parameter	Type	Description
p	%	A nonzero polynomial
q	%	A nonzero polynomial (optional)

Returns

`integerDistances(p, q)` returns all the integers $e \in \mathbb{Z}$ such that for each such e there exists α in an algebraic closure of the fraction field of R such that $p(\alpha) = q(\alpha + e) = 0$, while `dispersion(p, q)` returns -1 if `integerDistances(p, q)` contains only elements strictly smaller than 0, its maximal nonnegative element otherwise.

Remarks

The parameter q is optional for both functions, its default value being p .

Usage

integerRoots p
 rationalRoots p

Signature

integerRoots,rationalRoots: % $\rightarrow$ Generator FractionalRoot Integer

Parameter	Type	Description
p	%	A polynomial

Returns

Return $[(r_1, e_1), \dots, (r_n, e_n)]$ where the r_i 's are the integer or rational roots of p and have multiplicity e_i .

Usage

minIntegerRoot p
maxIntegerRoot p

Signature

minIntegerRoot,maxIntegerRoot: % $\rightarrow$ Partial Integer

Parameter	Type	Description
p	%	A polynomial

Returns

Return *failed* if p has no integer root, its smallest (resp. largest) one otherwise.

Usage

universalBound(a, b)

Signature

universalBound: (%,%) $\rightarrow$ List Cross(% , Integer)

Parameter	Type	Description
a, b	%	Nonzero polynomials

Returns

Return $[(p_1, e_1), \dots, (p_n, e_n)]$ such that any polynomial bounded by a and b (in the sense of S.A. Abramov, *Rational solutions of linear difference and q -difference equations with polynomial coefficients*, Proceedings of ISSAC'95) must be a factor of $u = \prod_{i=1}^n \prod_{j=0}^{e_i} p_i(x - j)$.

UnivariatePolynomialKaratsuba

Usage

```
import from UnivariatePolynomialKaratsuba R
```

Parameter	Type	Description
R	<code>CommutativeRing</code>	The coefficient ring of the polynomials

Description

`UnivariatePolynomialKaratsuba R` implements Karatsuba multiplication for dense univariate polynomials with coefficients in `R`.

Exports

```
karatsuba!: (A, A, Z, A, Z, Z, (A, A, Z, A, Z) → ()) → ()  Karatsuba multiplication
```

where

```
A == PrimitiveArray R
```

```
Z == MachineInteger
```

Usage

UnivariatePolynomialRing R: Category

Parameter	Type	Description
R	ExpressionType ArithmeticType	The coefficient domain

Description

UnivariatePolynomialRing is a common category for commutative and noncommutative univariate polynomials with coefficients in an arbitrary arithmetic system R and with respect to the power basis $(x^n)_{n \geq 0}$.

Exports

UnivariateFreeRing R
add!: (% ,R,Z,% , Z, Z) $\rightarrow$ % In-place partial product and sum
compose: (% , %) $\rightarrow$ % Compose polynomials
translate: (% , R) $\rightarrow$ % Translate a polynomial

if R has Parsable then
Parsable

where
Z == Integer

Usage

add!(p, c, m, q, n, N)

Signature

add!: (%₀, R, Integer, %₀, Integer, Integer) → %₀

Parameter	Type	Description
p	% ₀	A polynomial (to be destroyed)
c	R	A scalar
m	Integer	The degree of the monomial to add
q	% ₀	A polynomial to be multiplied by cx^m and added to p
n	Integer	A lower threshold
N	Integer	An upper treshold

Description

add!(p, c, m, q, n, N) computes all the terms of degree at least n and at most N of

$$p + cx^mq = \sum_{i=0}^{d+m} (a_i + cb_{i-m})x^i,$$

where $p = \sum_{i=0}^d a_i x^i$ and $q = \sum_{i=0}^d b_i x^i$. Note that m is allowed to be negative. For efficiency reasons it is sometimes sufficient to compute some terms of that sum only. All other coefficients of p are not changed.

Remarks

The storage used by p is allowed to be destroyed or reused, so p is lost after this call. This may cause p to be destroyed, so do not use this unless p has been locally allocated, and is thus guaranteed not to share space with other polynomials. Some functions, like `reductum` are not necessarily copying their arguments and can thus create memory aliases.

Usage

compose(p, q)
 translate(p, r)

Parameter	Type	Description
p, q	%	Polynomials
r	R	Amount to translate

Returns

compose(p, q) returns

$$p(q) = \sum_{i=0}^n a_i q^i$$

where $p = \sum_{i=0}^n a_i x^i$, while translate(p, r) returns $p(x - r)$.

Usage

```
import from UnivariatePolynomialSquareFree(R, P)
```

Parameter	Type	Description
R	GcdDomain	Coefficient ring of the polynomials
P	UnivariatePolynomialAlgebra0 R	A polynomial ring

Description

UnivariatePolynomialSquareFree provides implementations of various squarefree factorization algorithms.

Exports

```
musser: P → (R, Product P)  Musser's algorithm
yun:    P → (R, Product P)  Yun's algorithm
```

Usage

musser p

Signature

musser: P → (R, Product P)

Parameter	Type	Description
p	P	The polynomial to factor

Returns

Returns $(c, p_1^{e_1} \cdots p_n^{e_n})$ such that each p_i is squarefree, the p_i 's have no common factors, and

$$p = c \prod_{i=1}^n p_i^{e_i}.$$

See also

yun

Usage

yun p

Signature

yun: P → (R, Product P)

Parameter	Type	Description
p	P	The polynomial to factor

Returns

Returns $(c, p_1^{e_1} \cdots p_n^{e_n})$ such that each p_i is squarefree, the p_i 's have no common factors, and

$$p = c \prod_{i=1}^n p_i^{e_i}.$$

See also

musser

UnivariateTaylorSeriesType

Usage

UnivariateTaylorSeriesType R: Category

Parameter	Type	Description
R	ExpressionType ArithmeticType	The coefficient domain

Description

UnivariateTaylorSeriesType R is the category of univariate Taylor series with coefficients in R.

Exports

UnivariateFreeLinearArithmeticType R

degree: % $\rightarrow$ Partial Integer upper bound on the degree
dot: Vector R $\rightarrow$ Vector % $\rightarrow$ % linear combination
finite?: % $\rightarrow$ Boolean check whether the support is finite
series: Sequence R $\rightarrow$ % creation of a series

where

UPC == UnivariatePolynomialAlgebra

if R has CommutativeRing then

differentiate: % $\rightarrow$ % Differentiation
 (% , Integer) $\rightarrow$ %
reciprocal: % $\rightarrow$ Partial % Inverse

if R has CommutativeRing and R has RittRing then

integrate: % $\rightarrow$ % Integration
 (% , Integer) $\rightarrow$ %

if R has FloatType then

reciprocal: % $\rightarrow$ Partial % Inverse

Usage

degree s
finite? s

Signatures

degree: % $\rightarrow$ Partial Integer
finite?: % $\rightarrow$ Boolean

Parameter	Type	Description
<i>s</i>	%	a series

Returns

finite?(s) returns *true* if s is known to have finite support and *false* otherwise, while degree(s) returns [*n*] if s is known to have finite support and $\deg(s) \leq n = 0$, *failed* otherwise.

Usage

differentiate s
differentiate(s, n)
integrate s

Signatures

differentiate,integrate: $\% \rightarrow \%$
differentiate,integrate: $(\%, \text{Integer}) \rightarrow \%$

Parameter	Type	Description
s	$\%$	a series
n	Integer	The order of differentiation or integration

Returns

differentiate(s), differentiate(s, n), integrate(s) and integrate(s, n) return respectively ds/dx , $d^n s/dx^n$, $\int s(x)dx$ and $\int \dots \int s(x)dx^n$.

Usage

series s

Signature

series: Sequence R $\rightarrow$ %

Parameter	Type	Description
<i>s</i>	Sequence R	a coefficient sequence

Returns

Returns *s* viewed as a series.

Usage

```
import from UnivariateTaylorSeriesType2Poly(R, RXX, RX)
```

Parameter	Type	Description
R	ExpressionType ArithmeticType	The coefficient domain
RXX	UnivariateTaylorSeriesType R	A series type over R
RX	UnivariatePolynomialAlgebra R	A polynomial type over R

Description

UnivariateTaylorSeriesType2Poly(R , RXX , RX) provides conversion tools between polynomials and series.

Exports

```
expand:    R → RX → RXX      series expansion at a point
truncate:  (RXX, Integer) → RX  truncation of a series
```

if R has Field then

```
expandFraction:  R → Fraction RX → (Integer, RXX)  series expansion at a point
```

if R has FloatType then

```
expandFraction:  R → Fraction RX → (Integer, RXX)  series expansion at a point
```

if R has CommutativeRing then

```
monicNewtonSeries:  RX → RXX                      Newton series
tryExpandFraction:  R → (RX, RX) → (Integer, Partial RXX)  try series expansion
```

if R has IntegralDomain then

```
polynomials:  (V RXX, Integer) → (V RX, M R)      interpolation
              (V RXX, Integer, Integer) → (V RX, M R)
```

where

```
V  == Vector
M  == DenseMatrix
```

Usage

expand a
expand(a)(p)

Signature

expand: $R \rightarrow RX \rightarrow RXX$

Parameter	Type	Description
a	R	the expansion point
p	RX	a polynomial

Returns

expand(a)(p) returns the series expansion of p around a .

See also

expandFraction

Usage

```

expandFraction a
expandFraction(a)(f)

```

Signature

```

expandFraction:  R → Fraction RX → (Integer, RXX)

```

Parameter	Type	Description
a	R	the expansion point
f	Fraction RX	a rational function

Returns

expandFraction(a)(f) returns (n, s) where $n \leq 0$ and the series expansion of f around a is $(x - a)^n s(x - a)$. In addition, $s(a) \neq 0$ whenever $n < 0$.

See also

```

expand, tryExpandFraction

```

Usage

polynomials($[s_1, \dots, s_n]$, N)
 polynomials($[s_1, \dots, s_n]$, N, M)

Signatures

polynomials: (Vector RXX, Integer) $\rightarrow$ (Vector RX, DenseMatrix R)
 polynomials: (Vector RXX, Integer, Integer) $\rightarrow$ (Vector RX, DenseMatrix R)

Parameter	Type	Description
s_i	RXX	series
N	Integer	a degree bound
M	Integer	an upper bound

Description

polynomials($[s_1, \dots, s_n]$, N , M) returns $([p_1, \dots, p_s], A)$ such that the series $[s_1, \dots, s_n]A$ all have coefficients 0 from x^{N+1} to x^M , and $[p_1, \dots, p_s]$ are the truncations to order N of $[s_1, \dots, s_n]A$. If the upper bound M is not given, then the series returned have coefficients 0 from x^{N+1} up to an order that is determined heuristically.

Usage

monicNewtonSeries p

Signature

monicNewtonSeries: $RX \rightarrow RXX$

Parameter	Type	Description
p	RX	A monic polynomial

Returns

Returns the series

$$\sum_{n \geq 0} (y_1 + \dots + y_d)^n x^n$$

where $y_1, \dots, y_d$ are all the roots of p .

Usage

tryExpandFraction a
 tryExpandFraction(a)(p,q)

Signature

tryExpandFraction: $R \rightarrow (RX, RX) \rightarrow (\text{Integer}, \text{Partial } RXX)$

Parameter	Type	Description
a	R	the expansion point
p, q	RX	numerator and denominator of a rational function

Returns

If $q(a)$ is a unit in R , then $\text{tryExpandFraction}(a)(p,q)$ returns (n, s) where $n \leq 0$ and the series expansion of p/q around a is $(x-a)^n s(x-a)$. In addition, $s(a) \neq 0$ whenever $n < 0$. Otherwise, it returns (n, failed) where n is the order of p/q at $x = a$.

See also

expand, expandFraction

Usage

truncate(s, m)

Signature

truncate: (RXX,Integer) → RX

Parameter	Type	Description
s	%	a series
m	Integer	the truncation order

Returns

Returns the truncation of s at order m , *i.e.*

$$\sum_{n=0}^{m-1} a_n x^n$$

where $s = \sum_{n \geq 0} a_n x^n$.

Usage

```
import from UnivariateTaylorSeriesNewtonSolver(R, Rx, RxY)
import from UnivariateTaylorSeriesNewtonSolver(R, Rx, RX, RXY)
```

Parameter	Type	Description
R	ExpressionType ArithmeticType	The coefficient domain
Rx	UnivariateTaylorSeriesType R	Series over R
RxY	UnivariatePolynomialAlgebra Rx	Polynomials over Rx
RX	UnivariatePolynomialAlgebra R	Polynomials over R
RXY	UnivariatePolynomialAlgebra RX	Polynomials over RX

Description

UnivariateTaylorSeriesNewtonSolver provides a Newton solver for computing roots in $R[[x]]$ of polynomials in either $R[x, y]$ or $R[[x]][y]$.

Exports

```
if R has CommutativeRing then
  differentiate: RxY → RxY      Differentiation
  root:         (RXY, R) → Rx    Root of a polynomial
               (RxY, R) → Rx

if R has FloatType then
  root: (RXY, RXY, R) → Rx      Root of a polynomial
       (RxY, RxY, R) → Rx
```

Usage

differentiate p

Signature

differentiate: RxY $\rightarrow$ RxY

Parameter	Type	Description
<i>p</i>	RxY	a polynomial

Returns

Returns $\frac{dp}{dy}$.

Usage

$\text{root}(p, s_0)$
 $\text{root}(p, p', s_0)$

Signatures

$\text{root}: (\text{RXY}, \text{R}) \rightarrow \text{Rx}$
 $\text{root}: (\text{RxY}, \text{R}) \rightarrow \text{Rx}$
 $\text{root}: (\text{RXY}, \text{RXY}, \text{R}) \rightarrow \text{Rx}$
 $\text{root}: (\text{RxY}, \text{RxY}, \text{R}) \rightarrow \text{Rx}$

Parameter	Type	Description
p	RXY RxY	a nonzero polynomial
p'	RXY RxY	the derivative of p with respect to y
s_0	R	a simple root of $p(0, y)$

Description

Returns a series $s(x)$ such that $p(x, s(x)) = 0$ and $s(0) = s_0$. The initial value s_0 must satisfy $p(0, s_0) = 0$, and in addition, $\frac{dp}{dy}(0, s_0)$ must be a unit in R .

Remarks

The parameter p' must be given only when R has `FloatType`, since differentiation is not available for polynomials over such rings.

Usage

```
import from BivariateUtilitiesPackage (U,V)
```

Parameter	Type	Description
U	UnivariatePolynomialAlgebra Integer	
V	UnivariatePolynomialAlgebra U	

Description

BivariateUtilitiesPackage (U,V) provides basic operations for bivariate polynomials over the integer numbers. In particular, it provides support for modular methods. In the above description, the smallest variable of a polynomial from V refers to the variable of U and its degree refers to the its degree as a univariate polynomial w.r.t. the variable of V.

Exports

maxNorm:	$U \rightarrow Z$	maximum absolute value of a coefficient
maxNorm:	$V \rightarrow Z$	maximum absolute value of a coefficient
degree_U:	$V \rightarrow Z$	degree w.r.t. the smallest variable
resultantCoefficientBound:	$(V, V) \rightarrow Z$	upper bound for the resultant maxNorm
resultantDegreeBound:	$(V, V) \rightarrow Z$	degree bound for the resultant
primeBad?:	$(V, V, I) \rightarrow B$	true iff the degree of one of the two polynomials drops modulo the number

where

```
I == MachineInteger
Z == Integer
B == Boolean
```

DirectProduct

Usage

import from DirectProduct (n,T)

Parameter	Type	Description
n	MachineInteger	The dimension of the direct product
T	ExpressionType	The type of the factors

Description

DirectProduct (n,T) provides n -ary direct products of elements from T . Such a product is represented by a primitive array of elements from T with size n . The indices of its components are in the range $0 \cdots n - 1$.

Exports

DirectProductCategory (n,T)

DirectProductCategory

Usage

DirectProductCategory (dim, T): Category

Parameter	Type	Description
<i>dim</i>	MachineInteger	The length of a direct product
<i>T</i>	ExpressionType	The type of each factor

Description

DirectProductCategory (dim, T) is the category of cartesian products of *dim* copies of *T*. Hence an elements of a domain of this category is a (direct) product (or tuple) of elements from *T* with length *dim*. The components of such a tuple are indexed from **firstIndex** to **lastIndex**. Thus *dim* is *lastIndex* – *firstIndex* + 1. This category is essentially designed to support the implementation of multivariate monomials involving at most *dim* – 1 (if one component is used for storing the total degree) or *dim* (if not) variables.

Exports

CopyableType

LinearStructureType T

ExpressionType

bracket: Tuple T → %

Conversion of a tuple whose length is *dim*

lastIndex: MachineInteger

The biggest index of a direct product.

generator: % → Generator T

The factors of a direct product.

map: ((T → T) → T) → (% , %) → %

Componentwise mapping.

map: (T → T) → % → %

Mapping

map!: (T → T) → % → %

Mapping that may modify its second arg

DistributedMultivariatePolynomial0

Usage

import from DistributedMultivariatePolynomial0 (R,E)

Parameter	Type	Description
R	ArithmeticType ExpressionType	The coefficient domain
E	GeneralExponentCategory V	The exponent domain

Description

DistributedMultivariatePolynomial0 (R,E) provides an implementation of the free module over R with basis E . Roughly speaking, the elements of DistributedMultivariatePolynomial0 (R,E) are polynomials that can be multiplied only by a constant from R . Each of these *polynomials* x is coded as a list of term (r, e) with $r \in R, r \neq 0$ and $e \in E$ such that x is the sum of these terms.

Exports

CopyableType
IndexedFreeModule(R,E)

Usage

import from DistributedMultivariatePolynomial1 (R,V,E)

Parameter	Type	Description
R	Ring	The coefficient ring
V	VariableType	The variable type
E	ExponentCategory V	The exponent domain

Description

DistributedMultivariatePolynomial1 (R,V,E) provides a basic domain for multivariate polynomials with coefficients in R and variables in V . The monomials are coded by means of exponents from E . Polynomials are represented in a sparse and distributed way. This means that each polynomial x is coded as a list of term (r, e) with $r \in R, r \neq 0$ and $e \in E$ such that x is the sum of these terms.

Exports

FiniteAbelianMonoidRing0(R,V,E)

ExponentCategory

Usage

ExponentCategory V: Category

Parameter	Type	Description
V	VariableType	The domain of variables

Description

ExponentCategory V provides multivariate monomials (i.e. products of variables from V) looked as an additive ordered monoid (with cancellation) by associating to every product of variables its sequence of degrees.

Exports

GeneralExponentCategory

exponent:	$V \rightarrow \%$	The exponent of a variable
exponent:	$(V, Z) \rightarrow \%$	The exponent of a power of a variable
exponent:	Generator Cross $(V, Z) \rightarrow \%$	The exponent of a power product
exponent:	$(\text{List } V, \text{List } Z) \rightarrow \%$	The exponent of a power product
terms:	$\% \rightarrow \text{Generator Cross } (V, Z)$	Inverse map of exponent
mainVariable:	$\% \rightarrow \text{Partial } V$	The biggest variable, if any
variables:	$\% \rightarrow \text{List } V$	The list of variables of a monomial
degree:	$(\%, V) \rightarrow Z$	The degree w.r.t. a variable
totalDegree:	$\% \rightarrow Z$	The sum of the degrees of an exponent
totalDegree:	$(\%, \text{List } V) \rightarrow Z$	The sum of the degrees w.r.t. a list
gcd:	$(\%, \%) \rightarrow \%$	Monomial gcd
lcm:	$(\%, \%) \rightarrow \%$	Monomial lcm
syzygy:	$(\%, \%) \rightarrow (\%, \%)$	Cofactors w.r.t. lcm

where

$Z == \text{Integer}$

Usage

FiniteAbelianMonoidRing0 (R,V,E): Category

Parameter	Type	Description
R	ArithmeticType ExpressionType	The coefficient domain
V	VariableType	The variable type
E	ExponentCategory V	The exponent domain

Description

FiniteAbelianMonoidRing0 (R,V,E) is a model for *distributed* polynomials. Such polynomials are looked as sums of terms $c_i m_i$ where the r_i are coefficients from R and the m_i are monomials from the free abelian monoid generated by V . Moreover these monomials are coded by means of exponents from E . These exponents commute with each other and with the coefficients. However the coefficients may or may not commute.

Exports

IndexedFreeAlgebra(R,E)

PolynomialRing0(R,V)

add!: (R, %, R, E, %) → % add!(c_1, x, c_2, e, y) is $c_1 x + \text{term}(c_2, e)y$ and may modify x map: (E → E) → % → % map(f)(x) maps f on the exponents of x map!: (E → E) → % → % map(f)(x) returns map(f)(x) and may modify x

FiniteVariableType

Usage

FiniteVariableType: Category

Description

FiniteVariableType is a category for multivariate polynomial variables that belong to a finite set. Hence, a domain of this category implements a finite set of ordered variables. The operations `variable` and `index` define a one-to-one map from % onto a range of (machine) integers.

Exports

VariableType

<code>variable:</code>	<code>MachineInteger → %</code>	Retuns the n -th variable of the type, if any
<code>index:</code>	<code>% → MachineInteger</code>	Retuns the associated machine integer
<code>#:</code>	<code>MachineInteger</code>	The number of elements of the type
<code>max:</code>	<code>%</code>	The greatest element of the type
<code>min:</code>	<code>%</code>	The smallest element of the type
<code>minToMax:</code>	<code>List %</code>	The elements of the type sorted in increasing order
<code>maxToMin:</code>	<code>List %</code>	The elements of the type sorted in decreasing order

Usage
variable *i*

Signature
variable: MachineInteger → %

Parameter	Type	Description
<i>i</i>	MachineInteger	an index

Returns
Returns the *i*-th variable of the type if *i* is the range of indices associated with the type. Otherwise, an error or an exception is raised.

Usage

GeneralExponentCategory: Category

Description

GeneralExponentCategory is the category of monomials looked as an additive ordered monoid with a cancellation function and endowed with an order such that the addition is consistent with this order. By consistent addition we mean that $a \geq b$ implies $a + c \geq b + c$. By a cancellation function we mean an operation f such that $f(a, b)$ is either *failed* or c such that $a = b + c$ holds. Here are two common examples of this operation. For integers $f(a, b)$ is $a - b$ if $a \geq b$ and *failed* otherwise. For multivariate monomials $f(a, b)$ is c if $a = bc$ and *failed* if no such c exists.

Exports

ExpressionType		
TotallyOrderedType		
0:	%	zero
+:	(%, %) → %	sum
add!:	(%, %) → %	in-place sum
cancel:	(%, %) → %	cancellation
cancel?:	(%, %) → Boolean	cancellation
cancelIfCan:	(%, %) → Partial %	cancellation
times:	(Integer, %) → %	product by an integer
zero?:	% → Boolean	test for 0

Remarks

GeneralExponentCategory is meant to implement efficient monomial arithmetic. In particular, for multivariate monomials with a finite set of variables it is meant to code exponents with primitive arrays of machine integers. Hence we cannot claim that every exponent domain belongs to **AbelianMonoid**. Since we cannot subtract any exponent to every other, we cannot claim that every exponent domain belongs to **AdditiveType** neither, which explains the need for this category.

Usage

`cancel(x,y)`
`cancel?(x,y)`
`cancelIfCan(x,y)`

Signatures

`cancel:` $(\%, \%) \rightarrow \%$
`cancel?:` $(\%, \%) \rightarrow \text{Boolean}$
`cancelIfCan:` $(\%, \%) \rightarrow \text{Partial } \%$

Parameter	Type	Description
x, y	$\%$	Elements of the type

Description

`cancelIfCan(x,y)` returns z such that $z + y = x$ if there exist such a z in the type viewed as a monoid only, *failed* otherwise, while `cancel?(x,y)` returns whether `cancelIfCan(x,y)` would fail, and `cancel(x,y)` returns z such that $z + y = x$, assuming that `cancelIfCan(x,y)` would not fail.

Usage

times(*n*, *x*)

Signature

times: (Integer, %) → %

Parameter	Type	Description
<i>n</i>	Integer	An integer
<i>x</i>	%	An element of the type

Returns

Returns the product *nx*.

IntegerExponentVectorCategory

Usage

IntegerExponentVectorCategory V: Category

Parameter	Type	Description
V	FiniteVariableType	The type of variables

Description

IntegerExponentVectorCategory V is a category for the exponents of the monomials (or power products) generated by the finite set of variables V and coded as direct products of non-negative integers.

Exports

CopyableType

ExponentCategory V

HashType

free!: $\% \rightarrow ()$

Asserts that the input will no longer be used

exponent: $\text{Tuple Integer} \rightarrow \%$

Creation from a tuple

exponent: $\text{PrimitiveArray Integer} \rightarrow \%$

Creation from a primitive array

Degrees start at slot 1

Slot 0 must be the total degree

parray: $\% \rightarrow \text{PrimitiveArray Integer}$

Inverse mapping of `exponent`

IntegerPolynomial

Usage

import from IntegerPolynomial

Description

IntegerPolynomial provides a basic domain for multivariate polynomials with **Integer** coefficients and variables from **OrderedSymbol**. The representation is sparse and recursive by means of the **SparseUnivariatePolynomial** univariate polynomial domain constructor.

Exports

RecursiveMultivariatePolynomialCategory0(Z,OS)

coerce: String $\rightarrow$ % Conversion.

univariate: % $\rightarrow$ SUP(%) Convert to univariate w.r.t. the greatest variable

multivariate: (SUP(%), OS) $\rightarrow$ % Convert to multivariate with given variable

where

OS == OrderedSymbol

Z == Integer

SUP == SparseUnivariatePolynomial

Usage

```
import from MachineIntegerDegreeLexicographicalExponent V
```

Parameter	Type	Description
V	<code>FiniteVariableType</code>	The type of the variables

Description

`MachineIntegerDegreeLexicographicalExponent V` implements the exponents of the monomials (or power products) generated by the finite set of variables V . Such an exponent is represented by a primitive array of machine integers with length $\dim = n + 1$ where n is the number of elements in V . Given e in % and i in $1 \cdots n$ the degree of e w.r.t. the i -th variable of V is stored in slot i . Slot 0 is used to store the total degree of e , that is the sum of the content of all the other slots. An exponent a is greater than an exponent b if a is greater than b w.r.t. the degree-lexicographical ordering induced by V (by comparing a_i and b_i for i running from 1 to n , after comparing the total degrees of a and b).

Exports

```
MachineIntegerExponentVectorCategory V
```

Usage

```
import from MachineIntegerDegreeReverseLexicographicalExponent V
```

Parameter	Type	Description
V	<code>FiniteVariableType</code>	The type of the variables

Description

`MachineIntegerDegreeReverseLexicographicalExponent V` implements the exponents of the monomials (or power products) generated by the finite set of variables V . Such an exponent is represented by a primitive array of machine integers with length $\dim = n + 1$ where n is the number of elements in V . Given e in % and i in $1 \cdots n$ the degree of e w.r.t. the i -th variable of V is stored in slot i . Slot 0 is used to store the total degree of e , that is the sum of the content of all the other slots. An exponent a is greater than an exponent b if a is greater than b w.r.t. the degree-reverse-lexicographical ordering induced by V (by comparing a_i and b_i for i running from n to 1, after comparing the total degrees of a and b).

Exports

```
MachineIntegerExponentVectorCategory V
```

MachineIntegerExponentVectorCategory

Usage

MachineIntegerExponentVectorCategory V: Category

Parameter	Type	Description
V	FiniteVariableType	The type of variables

Description

MachineIntegerExponentVectorCategory V is a category for the exponents of the monomials (or power products) generated by the finite set of variables V and coded as direct products of non-negative machine integers.

Exports

CopyableType

ExponentCategory V

HashType

free!: $\% \rightarrow ()$ Asserts that the input will no longer be used

exponent: $\text{Tuple } I \rightarrow \%$ Creation from a tuple

exponent: $\text{PrimitiveArray } I \rightarrow \%$ Creation from a primitive array

Degrees start at slot 1

Slot 0 must be the total degree

parray: $\% \rightarrow \text{PrimitiveArray } I$ Inverse mapping of **exponent**

where

$I == \text{MachineInteger}$

MachineIntegerLexicographicalExponent

Usage

import from MachineIntegerLexicographicalExponent V

Parameter	Type	Description
V	FiniteVariableType	The type of the variables

Description

MachineIntegerLexicographicalExponent V implements the exponents of the monomials (or power products) generated by the finite set of variables V . Such an exponent is represented by a primitive array of machine integers with length $\dim = n + 1$ where n is the number of elements in V . Given e in % and i in $1 \cdots n$ the degree of e w.r.t. the i -th variable of V is stored in slot i . Slot 0 is used to store the total degree of e , that is the sum of the content of all the other slots. An exponent a is greater than an exponent b if a is greater than b w.r.t. the lexicographical ordering induced by V (by comparing a_i and b_i for i running from 1 to n).

Exports

MachineIntegerExponentVectorCategory V

OrderedSymbol

Usage

import from OrderedSymbol

Description

OrderedSymbol implements symbols as a type of variables.

Exports

VariableType

orderedSymbol: `String` $\rightarrow$ % Conversion to an element of the type.

OrderedVariableList

Usage

import from OrderedVariableList t

Parameter	Type	Description
<i>t</i>	List Symbol	The symbols defining the variables of the type

Description

OrderedVariableList *t* implements the finite set of ordered variables given by *t*. This set has *n* elements numbered from 1 to *n*, where *n* is the size of the *t*. For $i = 1 \dots n$ the *i*-th variable of the type is **variable *i*** and uses the output form of the *i*-th item in *t*. For $i, j = 1 \dots n$ **variable *i*** > **variable *j*** holds iff $i < j$ holds. Elements of OrderedVariableList *t* are internally represented as machine integers. The input list *t* may contain duplicates since *t* is only used for output forms matter. Operations from **ExpressionType**, **Parsable**, **HashType** and **SerializableType** are taken from **MachineInteger**.

Exports

FiniteVariableType

OrderedVariableTuple

Usage

import from OrderedVariableTuple t

Parameter	Type	Description
<i>t</i>	Tuple Symbol	The symbols defining the variables of the type

Description

OrderedVariableTuple *t* implements the finite set of ordered variables given by *t*.
OrderedVariableTuple *t* is implemented as OrderedVariableList *l* where *l* is the list of items in *t* in the same order.

Exports

FiniteVariableType

PolynomialRing

Usage

PolynomialRing (R,V): Category

Parameter	Type	Description
R	ArithmeticType ExpressionType	The coefficient domain
V	TotallyOrderedType ExpressionType	The variable domain

Description

PolynomialRing (R,V) is a category which inherits from PolynomialRing0(R,V) and exports some additionnal operations related to differentiation, evaluation and polynomial type conversion.

Exports

PolynomialRing0(R,V)

eval: $(\%, V, R) \rightarrow \%$ eval(x, v, r) evaluates x at $v = r$

eval: $(\%, V, \%) \rightarrow \%$ eval(x, v, y) evaluates x at $v = y$

where

GEN == Generator

PZ == Cross($\%$, Integer)

if R has CommutativeRing then

differentiate: $(\%, V) \rightarrow \%$ Differentiation w.r.t. a variable

differentiate: $(\%, V, Integer) \rightarrow \%$ Iterated differentiation w.r.t. a variable

if R has CommutativeRing then

CommutativeRing

if R has IntegralDomain then

IntegralDomain

if R has GcdDomain then

GcdDomain

Usage

PolynomialRing0 (R,V): Category

Parameter	Type	Description
R	ArithmeticType ExpressionType	The coefficient domain
V	TotallyOrderedType ExpressionType	The variable domain

Description

PolynomialRing0 (R,V) is the category of the domains that implement a polynomial ring with coefficients in R and variables in V . If V is a finite set $\{v_1, \dots, v_l\}$ this polynomial ring is just $R[v_1, \dots, v_l]$. If V is finite or not, the set of monomials is the free abelian monoid E generated by V . Moreover the default total ordering endowing E is the lexicographical one induced by V . Observe that the domain V is not assumed to satisfy **VariableType**. In fact, only weaker conditions are required. Hence it is possible to use any totally ordered set as a domain of variables. For instance a set of algebraically independent numbers. Observe that PolynomialRing0 (R,V) provides essentially operations related to the structure of a free algebra. For more sophisticated operations (differentiation, evaluation, ...) see the category constructor PolynomialRing.

Exports

PolynomialTypeRing(R, V)
FreeAlgebra R

Usage

PolynomialTypeRing (R,V): Category

Parameter	Type	Description
R	ArithmeticType ExpressionType	The coefficient domain
V	ExpressionType	The variable domain

Description

PolynomialTypeRing (R,V) is the category of standard filtered rings generated over R by power products of the form $v_1^{e_1} \cdots v_n^{e_n}$ for $v_i \in V$ and $e_i \in \mathbb{N}$. No commutativity is assumed either between the variables in V, or between R and V.

Exports

StandardFilteredRing(R, V)		
coefficient:	(%, V, Z) → %	Coefficient w.r.t. a power v^n .
coefficient:	(%, L V, L Z) → %	Coefficient w.r.t. a power product
degree:	(%, V) → Z	Degree w.r.t. a variable
degrees:	% → GEN VZ	Degrees w.r.t. all variables
leadingCoefficient:	(%, V) → %	Leading coefficient w.r.t. a variable
multivariate:	(P:IFRR R) → (P, V) → %	Conversion from univariate view
	(P:IFRR P) → (P, V) → %	
reductum:	(%, V) → %	Reductum w.r.t. a variable
univariate:	(P:IFRR P) → (%, V) → P	Conversion to univariate view

If V has TotallyOrderedType then

initial:	% → %	Leading coefficient w.r.t. main variable
univariate:	(P:IFRR P) → % → P	Conversion to univariate view

where

GEN	==	Generator
IFRR	==	IndexedFreeRRing
L	==	List
Z	==	Integer
VZ	==	Cross (V, Integer)

RecursiveMultivariatePolynomial0

Usage

```
import from RecursiveMultivariatePolynomial0 (UP,R,V)
```

Parameter	Type	Description
<i>UP</i>	$(T: \text{Join}(\text{ArithmeticType}, \text{ExpressionType})) \rightarrow \text{POL } T$	Univariate functor
<i>R</i>	Ring	The coefficient ring
<i>V</i>	VariableType	The variables

where

```
POL == UnivariatePolynomialAlgebra
```

Description

RecursiveMultivariatePolynomial0 (UP,R,V) provides a basic domain for multivariate polynomials with coefficients in *R* and variables in *V*. The representation is recursive by means of *UP*.

Exports

```
RecursiveMultivariatePolynomialCategory0(R,V)
```

```
univariate:    % → UP %          Convert to univariate w.r.t. the greatest variable
```

```
multivariate:  (UP(%), V) → %    Convert to multivariate with given variable
```

Usage

RecursiveMultivariatePolynomialCategory0 (R,V): Category

Parameter	Type	Description
R	ArithmeticType ExpressionType	The coefficient domain
V	TotallyOrderedType ExpressionType	The variable domain

Description

RecursiveMultivariatePolynomialCategory0 (R,V) extends `PolynomialRing(R,V)` with some additional operations related to the recursive vision of a multivariate polynomial (as a univariate polynomial w.r.t. its main variable).

Exports

<code>PolynomialRing(R,V)</code>		
<code>variables:</code>	<code>% → SortedSetV</code>	The sorted set of variables of a polynomial
<code>mvar:</code>	<code>% → V</code>	Greatest variable of a polynomial
<code>mdeg:</code>	<code>% → Integer</code>	Degree w.r.t. greatest variable
<code>rank:</code>	<code>% → %</code>	Leading monomial as univariate w.r.t. greatest variable
<code>init:</code>	<code>% → %</code>	Leading coefficient as univariate w.r.t. greatest variable
<code>tail:</code>	<code>% → %</code>	Reductum as univariate w.r.t. greatest variable
<code>head:</code>	<code>% → %</code>	Leading term as univariate w.r.t. greatest variable

Usage

```
import from ResultantOfBivariatePolynomialsOverSmallPrimeField (Kp, Up, Vp)
```

Parameter	Type	Description
Kp	SmallPrimeFieldCategory	
Up	UnivariatePolynomialAlgebra Kp	
Vp	UnivariatePolynomialAlgebra Up	

Description

ResultantOfBivariatePolynomialsOverSmallPrimeField (Kp, Up, Vp) provides resultant computations for two bivariate polynomials over a small prime field.

Exports

evaluationResultant:	$(Vp, Vp, Z) \rightarrow Up$	resultant computed by an evaluation and interpolation scheme. Arg 3 is a degree bound.
evaluationReduction:	$(Vp, Kp) \rightarrow Up$	evaluation at a point
interpolation:	$(A\ Kp, A\ Kp) \rightarrow Up$	Lagrange interpolant where the arrays give points and values resp.
integerImage:	$Up \rightarrow DUP\ I$	conversion.

where

DUP	==	DenseUnivariatePolynomial
I	==	MachineInteger
Z	==	Integer
A	==	Array

SparseIntegerMultivariatePolynomial

Usage

```
import from SparseIntegerMultivariatePolynomial (V)
```

Parameter	Type	Description
V	<code>VariableType</code>	The variables

Description

`SparseIntegerMultivariatePolynomial (V)` provides a basic domain for multivariate polynomials with coefficients over the `Integer` ring and variables in V . The representation is sparse and recursive by means of *SUP*.

Exports

```
RecursiveMultivariatePolynomialCategory0(Z,V)
```

```
univariate:    %  $\rightarrow$  SUP(%)      Convert to univariate w.r.t. the greatest variable
```

```
multivariate:  (SUP(%), V)  $\rightarrow$  %      Convert to multivariate with given variable
```

where

```
Z      == Integer
```

```
SUP    == SparseUnivariatePolynomial
```

SparseMultivariatePolynomial

Usage

```
import from SparseMultivariatePolynomial (R,V)
```

Parameter	Type	Description
R	Ring	The coefficient ring
V	VariableType	The variables

Description

SparseMultivariatePolynomial (R,V) provides a basic domain for multivariate polynomials with coefficients in R and variables in V . The representation is sparse and recursive by means of *SUP*.

Exports

```
RecursiveMultivariatePolynomialCategory0(R,V)
```

```
univariate:    %  $\rightarrow$  SUP(%)      Convert to univariate w.r.t. the greatest variable
```

```
multivariate:  (SUP(%), V)  $\rightarrow$  %      Convert to multivariate with given variable
```

where

```
SUP == SparseUnivariatePolynomial
```


StandardFilteredRing

Usage

StandardFilteredRing (R,V): Category

Parameter	Type	Description
R	ArithmeticType ExpressionType	The coefficient domain
V	ExpressionType	The variable domain

Description

StandardFilteredRing (R,V) is the category of R-rings with a standard filtration for which R is the ring of elements of degree 0, and V generates the elements of degree 1. It is the set of finite linear combinations of the form $\sum r_i w_i$ where the r_i are in R and the w_i are words of V.

Exports

CopyableType

FreeRRing R

coerce: $V \rightarrow \%$

Conversion of a variable to a polynomial

ground?: $\% \rightarrow \text{Boolean}$

Membership test to the coefficient ring

totalDegree: $\% \rightarrow \mathbb{Z}$

Greatest total degree among all the monomials

term: $(R, V, \mathbb{Z}) \rightarrow \%$

term(r, v, n) returns $r v^n$ provided $n \geq 0$

term: $(R, \text{GEN } VZ) \rightarrow \%$

Term from a power product and a coeff.

term: $(R, \text{L } V, \text{L } \mathbb{Z}) \rightarrow \%$

Term from a power product and a coeff.

times: $(\%, R, V, \mathbb{Z}) \rightarrow \%$

times(x, r, v, n) is $x (r v^n)$

times!: $(\%, R, V, \mathbb{Z}) \rightarrow \%$

times(x, r, v, n) is $x (r v^n)$ and may modify x

univariate?: $\% \rightarrow \text{Boolean}$

Check whether elt is univariate

variable: $\% \rightarrow V$

Conversion of a polynomial to a variable

variable?: $\% \rightarrow \text{Boolean}$

Membership test to the variable domain

variableProduct: $\% \rightarrow \text{GEN } VZ$

variableProduct x is g s.t. **term**(1, g) is x

variables: $\% \rightarrow \text{GEN } V$

The variables occuring in a polynomial

where

GEN == Generator

L == List

Z == Integer

VZ == Cross (V, Integer)

if R has Parsable and V has Parsable then

Parsable

if V has TotallyOrderedType then

mainVariable: $\% \rightarrow V$ Greatest variable occuring

VariableType

Usage

VariableType: Category

Description

VariableType is a category for multivariate polynomial variables looked as symbols with additionnal properties such as a total order.

Exports

TotallyOrderedType
ExpressionType
Parsable
SerializableType
HashType
variable: Symbol $\rightarrow$ Partial % Associated variable, if any
symbol: % $\rightarrow$ Symbol Associated symbol

ExpressionTree

Usage

import from ExpressionTree

Description

ExpressionTree is a type whose elements are expression trees.

Exports

OutputType

PrimitiveType

aldor: (TEXT, %) → TEXT

Conversion to A#code

apply: (OP, List %) → %

Apply an operator to arguments

apply: (OP, Tuple %) → %

Apply an operator to arguments

arguments: % → List %

Take the arguments of the root

axiom: (TEXT, %) → TEXT

Conversion to Axiom code

C: (TEXT, %) → TEXT

Conversion to C code

extree: ExpressionTreeLeaf → %

Conversion to a tree

fortran: TEXT, % → TEXT

Conversion to FORTRAN code

infix: (TEXT, %) → TEXT

Conversion to one-dim infix output

is?: (% , OP) → Boolean

Test for a specific operator

leaf: % → ExpressionTreeLeaf

Conversion to a leaf

leaf?: % → Boolean

Test whether tree is a leaf

lisp: TEXT, % → TEXT

Conversion to Lisp code

maple: (TEXT, %) → TEXT

Conversion to Maple code

operator: % → OP

Take the root operator

tex: (TEXT, %) → TEXT

Conversion to L^AT_EX

texParen?: (MachineInteger, %) → Boolean

Check whether to parenthesize

where

TEXT == TextWriter

OP == ExpressionTreeOperator

Usage*format*(p, t)**Signature**

aldor, axiom, C, fortran, infix, lisp, maple, tex: (TextWriter, %) → TextWriter

Parameter	Type	Description
<i>p</i>	TextWriter	The port to write to
<i>t</i>	%	An expression tree

DescriptionWrites to *p* the expression corresponding to the tree *t* in the requested format.

Usage

apply(op, [t₁, ..., t_n])
op [t₁, ..., t_n]

Signature

apply: (ExpressionTreeOperator, List %) → %

Parameter	Type	Description
<i>op</i>	ExpressionTreeOperator	An operator
<i>t_i</i>	%	Expression trees

Returns

Returns the tree whose root is *op*, with arguments *t*₁, ..., *t*_{*n*}.

Usage

arguments t

Signature

arguments: % $\rightarrow$ List %

Parameter	Type	Description
<i>t</i>	%	An expression tree

Returns

Returns the list of arguments of the root operator of *t*, which must not be a leaf.

See also

operator

Usage

extree a

Signature

extree: ExpressionTreeLeaf → %

Parameter	Type	Description
<i>a</i>	ExpressionTreeLeaf	A leaf

Returns

extree a returns *a* as an expression tree.

Usage

is?(t, op)

Signature

is?: (% , ExpressionTreeOperator) → Boolean

Parameter	Type	Description
<i>t</i>	%	An expression tree
<i>op</i>	ExpressionTreeOperator	An operator

Returns

is?(t, op) returns *true* if t is of the form op(args), *false* otherwise.

Usage

leaf t
leaf? t

Signatures

leaf: % $\rightarrow$ ExpressionTreeLeaf
leaf?: % $\rightarrow$ Boolean

Parameter	Type	Description
<i>t</i>	%	An expression tree

Returns

leaf a returns *a* as a leaf is *a* is a leaf. leaf? a returns *true* if a is a leaf, *false* otherwise.

Usage

negate t

Signature

negate: % $\rightarrow$ %

Parameter	Type	Description
t	%	An expression tree

Returns

Returns the leaf $-t$ if t is a numerical leaf with $t < 0$, and returns s if t is of the form $(-s)$ for some tree s . t must be of one of the above 2 forms.

See also

negative?

Usage

negative? t

Signature

negative?: % $\rightarrow$ Boolean

Parameter	Type	Description
<i>t</i>	%	An expression tree

Returns

Returns *true* if either *t* is a numerical leaf and $t < 0$, or if *t* is of the form $(-s)$ for some tree *s*, *false* otherwise.

See also

negate

Usage

operator t

Signature

operator: % $\rightarrow$ ExpressionTreeOperator

Parameter	Type	Description
<i>t</i>	%	An expression tree

Returns

Returns the root operator of *t*, which must not be a leaf.

See also

arguments

Usage

texParen?(prec, t)

Signature

texParen?: (MachineInteger, %) → Boolean

Parameter	Type	Description
<i>prec</i>	MachineInteger	An operator precedence.
<i>t</i>	%	An expression tree

Returns

Returns *true* if *t* should be parenthetized when appearing as argument of an operator of precedence *prec*, *false* otherwise.

ExpressionTreeAnd

Usage

import from ExpressionTreeAnd

Description

ExpressionTreeAnd is the *logical and* operator for expression trees.

Exports

ExpressionTreeOperator

Remarks

The following functions are not yet implemented and produce dummy results : **C**, **fortran**.

ExpressionTreeAssign

Usage

import from ExpressionTreeAssign

Description

ExpressionTreeAssign is the assignment operator for expression trees.

Exports

ExpressionTreeOperator

ExpressionTreeBigO

Usage

import from ExpressionTreeBigO

Description

ExpressionTreeBigO is the $\mathcal{O}$ operator for expression trees.

Exports

ExpressionTreeOperator

ExpressionTreeCase

Usage

import from ExpressionTreeCase

Description

ExpressionTreeCase is the *multi conditional* operator for expression trees.

Exports

ExpressionTreeOperator

Remarks

The following functions are not yet implemented and produce dummy results : **axiom**, **C**, **fortran**, **maple**.

ExpressionTreeComplex

Usage

import from ExpressionTreeComplex

Description

ExpressionTreeComplex is the complex operator for expression trees.

Exports

ExpressionTreeOperator

ExpressionTreeEqual

Usage

import from ExpressionTreeEqual

Description

ExpressionTreeEqual is the *equal* operator for expression trees.

Exports

ExpressionTreeOperator

Remarks

The following functions are not yet implemented and produce dummy results : **C**, **fortran**.

ExpressionTreeExpt

Usage

import from ExpressionTreeExpt

Description

ExpressionTreeExpt is the exponentiation operator for expression trees.

Exports

ExpressionTreeOperator

ExpressionTreeFactorial

Usage

import from ExpressionTreeFactorial

Description

ExpressionTreeFactorial is the generalized factorial operator for expression trees.

Exports

ExpressionTreeOperator

ExpressionTreeGreaterEqual

Usage

import from ExpressionTreeGreaterEqual

Description

ExpressionTreeGreaterEqual is the *greater or equal* operator for expression trees.

Exports

ExpressionTreeOperator

Remarks

The following functions are not yet implemented and produce dummy results : **C**, **fortran**, **lisp**.

ExpressionTreeGreaterThan

Usage

import from ExpressionTreeGreaterThan

Description

ExpressionTreeGreaterThan is the *greater than* operator for expression trees.

Exports

ExpressionTreeOperator

Remarks

The following functions are not yet implemented and produce dummy results : **C**, **fortran**, **lisp**.

ExpressionTreeIf

Usage

import from ExpressionTreeIf

Description

ExpressionTreeIf is the *conditional* operator for expression trees.

Exports

ExpressionTreeOperator

Remarks

The following function is not yet implemented and produces dummy results : **fortran**.

Usage

```
import from ExpressionTreeLeaf
```

Description

ExpressionTreeLeaf is a type whose elements are the leafs (atoms) of expression trees. It provides conversions to and from the basic atomic types.

Exports

OutputType

PrimitiveType

aldor:	(TEXT, %) → TEXT	Conversion to ALDOR code
axiom:	(TEXT, %) → TEXT	Conversion to Axiom code
boolean:	% → Boolean	Conversion to a boolean
boolean?:	% → Boolean	Test for a boolean
C:	(TEXT, %) → TEXT	Conversion to C code
doubleFloat:	% → DoubleFloat	Conversion to a double precision float
doubleFloat?:	% → Boolean	Test for a double precision float
float:	% → Float	Conversion to a software big float
float?:	% → Boolean	Test for a software big float
fortran:	TEXT, %) → TEXT	Conversion to FORTRAN code
infix:	(TEXT, %) → TEXT	Conversion to one-dim infix output
integer:	% → Integer	Conversion to a software big integer
integer?:	% → Boolean	Test for a software big integer
leaf:	Boolean → %	Conversion to a leaf
leaf:	DoubleFloat → %	Conversion to a leaf
leaf:	MachineInteger → %	Conversion to a leaf
leaf:	Integer → %	Conversion to a leaf
leaf:	SingleFloat → %	Conversion to a leaf
leaf:	String → %	Conversion to a leaf
leaf:	Symbol → %	Conversion to a leaf
lisp:	(TEXT, %) → TEXT	Conversion to Lisp code
singleFloat:	% → SingleFloat	Conversion to a single precision float
singleFloat?:	% → Boolean	Test for a single precision float
machineInteger:	% → MachineInteger	Conversion to a machine integer
machineInteger?:	% → Boolean	Test for a machine integer
maple:	(TEXT, %) → TEXT	Conversion to Maple code
string:	% → String	Conversion to a string
string?:	% → Boolean	Test for a string
symbol:	% → Symbol	Conversion to a symbol
symbol?:	% → Boolean	Test for a symbol
tex:	(TEXT, %) → TEXT	Conversion to L ^A T _E X

ExpressionTreeLeaf

`texParen?: % → Boolean` Check whether to parenthesize

where

`TEXT == TextWriter`

Usage*format*(p, a)**Signature**

aldor,axiom,C,fortran,infix,lisp,maple,tex: (TextWriter, %) → TextWriter

Parameter	Type	Description
<i>p</i>	TextWriter	The port to write to
<i>a</i>	%	A leaf

DescriptionWrites to *p* the expression corresponding to the leaf *a* in the requested format.

Usage

boolean a
boolean? a

Signatures

boolean: % $\rightarrow$ Boolean
boolean?: % $\rightarrow$ Boolean

Parameter	Type	Description
<i>a</i>	%	A leaf

Returns

boolean a returns the value of *a* as a Boolean if that is the type of *a*.
boolean? a returns *true* if *a* is a Boolean, *false* otherwise.

Usage

doubleFloat a
doubleFloat? a

Signatures

doubleFloat: % → DoubleFloat
doubleFloat?: % → Boolean

Parameter	Type	Description
<i>a</i>	%	A leaf

Returns

doubleFloat a returns the value of *a* as a `DoubleFloat` if that is the type of *a*.
doubleFloat? a returns *true* if *a* is a `DoubleFloat`, *false* otherwise.

Usage

float a
float? a

Signatures

float: % → Float
float?: % → Boolean

Parameter	Type	Description
<i>a</i>	%	A leaf

Returns

float a returns the value of *a* as a **Float** if that is the type of *a*.
float? a returns *true* if a is a **Float**, *false* otherwise.

Usage

integer a
integer? a

Signatures

integer: % → Integer
integer?: % → Boolean

Parameter	Type	Description
<i>a</i>	%	A leaf

Returns

integer a returns the value of *a* as an **Integer** if that is the type of *a*.
integer? a returns *true* if *a* is an **Integer**, *false* otherwise.

Usage

leaf *a*

Signatures

leaf: Boolean $\rightarrow$ %
 leaf: DoubleFloat $\rightarrow$ %
 leaf: Integer $\rightarrow$ %
 leaf: Float $\rightarrow$ %
 leaf: MachineInteger $\rightarrow$ %
 leaf: SingleFloat $\rightarrow$ %
 leaf: String $\rightarrow$ %
 leaf: Symbol $\rightarrow$ %

Parameter	Type	Description
<i>a</i>	Boolean DoubleFloat Float Integer MachineInteger SingleFloat String Symbol	A constant

Returns

leaf *a* returns *a* as a leaf.

Remarks

A string leaf prints with quotes, and should be used for string constants, while a symbol leaf prints without quotes, and should be used for names.

Usage

negate a

Signature

negate: $\% \rightarrow \%$

Parameter	Type	Description
<i>a</i>	$\%$	A leaf

Returns

Returns the leaf $-a$ if a is a numerical leaf, a otherwise.

See also

negative?

Usage

negative? a

Signature

negative?: % $\rightarrow$ Boolean

Parameter	Type	Description
<i>a</i>	%	A leaf

Returns

Returns *true* if *a* is a numerical leaf and $a < 0$, *false* otherwise.

See also

negate

Usage

machineInteger a
machineInteger? a

Signatures

machineInteger: % → MachineInteger
machineInteger?: % → Boolean

Parameter	Type	Description
<i>a</i>	%	A leaf

Returns

machineInteger a returns the value of *a* as a MachineInteger if that is the type of *a*.
machineInteger? a returns *true* if a is a MachineInteger, *false* otherwise.

Usage

singleFloat a
singleFloat? a

Signatures

singleFloat: % → SingleFloat
singleFloat?: % → Boolean

Parameter	Type	Description
<i>a</i>	%	A leaf

Returns

singleFloat a returns the value of *a* as a **SingleFloat** if that is the type of *a*.
singleFloat? a returns *true* if *a* is a **SingleFloat**, *false* otherwise.

Usage

string a
string? a

Signatures

string: % → **String**
string?: % → **Boolean**

Parameter	Type	Description
<i>a</i>	%	A leaf

Returns

string a returns the value of *a* as a **String** if that is the type of *a*.
string? a returns *true* if *a* is a **String**, *false* otherwise.

Usage

symbol a
symbol? a

Signatures

symbol: % $\rightarrow$ Symbol
symbol?: % $\rightarrow$ Boolean

Parameter	Type	Description
<i>a</i>	%	A leaf

Returns

symbol a returns the value of *a* as a **Symbol** if that is the type of *a*.
symbol? a returns *true* if a is a **Symbol**, *false* otherwise.

Usage

texParen? a

Signature

texParen?: % → Boolean

Parameter	Type	Description
<i>a</i>	%	A leaf

Returns

Returns *true* if the leaf *a* should be parenthetized, *false* otherwise.

ExpressionTreeLessEqual

Usage

import from ExpressionTreeLessEqual

Description

ExpressionTreeLessEqual is the *less or equal* operator for expression trees.

Exports

ExpressionTreeOperator

Remarks

The following functions are not yet implemented and produce dummy results : **C**, **fortran**, **lisp**.

ExpressionTreeLessThan

Usage

import from ExpressionTreeLessThan

Description

ExpressionTreeLessThan is the *less than* operator for expression trees.

Exports

ExpressionTreeOperator

Remarks

The following functions are not yet implemented and produce dummy results : **C**, **fortran**, **lisp**.

ExpressionTreeLispList

Usage

import from ExpressionTreeLispList

Description

ExpressionTreeLispList is the lisp list operator for expression trees.

Exports

ExpressionTreeOperator

Remarks

The following function is not yet implemented and produces dummy results : **fortran**.

ExpressionTreeList

Usage

import from ExpressionTreeList

Description

ExpressionTreeList is the list operator for expression trees.

Exports

ExpressionTreeOperator

Remarks

The following function is not yet implemented and produces dummy results : **fortran**.

ExpressionTreeMatrix

Usage

import from ExpressionTreeMatrix

Description

ExpressionTreeMatrix is the matrix operator for expression trees.

Exports

ExpressionTreeOperator

Remarks

The following function is not yet implemented and produces dummy results : **fortran**.

ExpressionTreeMinus

Usage

import from ExpressionTreeMinus

Description

ExpressionTreeMinus is the unary/binary minus operator for expression trees.

Exports

ExpressionTreeOperator

ExpressionTreeNotEqual

Usage

import from ExpressionTreeNotEqual

Description

ExpressionTreeNotEqual is the *not equal* operator for expression trees.

Exports

ExpressionTreeOperator

Remarks

The following functions are not yet implemented and produce dummy results : **C**, **fortran**.

Usage

ExpressionTreeOperator: Category

Description

ExpressionTreeOperator is the category of operators for expression trees.

Exports

aldor:	(TEXT, List TREE) → TEXT	Conversion to <i>A</i> #code
arity:	MachineInteger	Number of arguments
axiom:	(TEXT, List TREE) → TEXT	Conversion to Axiom code
C:	(TEXT, List TREE) → TEXT	Conversion to C code
fortran:	(TEXT, List TREE) → TEXT	Conversion to FORTRAN code
infix:	(TEXT, List TREE) → TEXT	Conversion to one-dim infix output
lisp:	(TEXT, List TREE) → TEXT	Conversion to Lisp code
maple:	(TEXT, List TREE) → TEXT	Conversion to Maple code
name:	Symbol	Operator name
tex:	(TEXT, List TREE) → TEXT	Conversion to $\text{\LaTeX}$
texParen?:	MachineInteger → Boolean	Check whether to parenthesize
uniqueId:	MachineInteger	A unique key per operator

where

TEXT	==	TextWriter
TREE	==	ExpressionTree

Usage

format(*p*, [*t*₁, ..., *t*_{*n*}])

Signature

aldor,axiom,C,fortran,infix,lisp,maple,tex: (TextWriter, List ExpressionTree) → TextWriter

Parameter	Type	Description
<i>p</i>	TextWriter	The port to write to
<i>t</i> _{<i>i</i>}	ExpressionTree	The arguments of the operator

Description

Writes to *p* the expression corresponding to this operator applied to the arguments (*t*₁, ..., *t*_{*n*}) in the requested format.

Usage

arity

Signaturearity: $\rightarrow$ MachineInteger**Returns**

Returns -1 if this operator can be applied to any number of arguments, $n \geq 0$ if this operator can be applied to exactly n arguments.

Usage

name

Signaturename: $\rightarrow$ Symbol**Returns**

Returns the name of this operator.

Usage

texParen? prec

Signature

texParen?: MachineInteger $\rightarrow$ Boolean

Parameter	Type	Description
<i>prec</i>	MachineInteger	An operator precedence.

Returns

Returns *true* if an expression tree with this operator as root should be parenthetized when appearing as argument of an operator of precedence *prec*, *false* otherwise.

Usage

uniqueId

Signature

uniqueId: $\rightarrow$ MachineInteger

Returns

Returns a integer key which is associated to this operator only. This is used for testing whether two operators are equal.

ExpressionTreeOperatorTools

Usage

```
import from ExpressionTreeOperatorTools
```

Description

ExpressionTreeOperatorTools provides utilities that make it simpler to write new operator types.

Exports

```
infix:  (TEXT, List TREE, TREE) → Boolean,
        (TEXT, TREE) → TEXT,
        String, String, String) → TEXT      Write as infix
prefix: (TEXT, List TREE,
        (TEXT, TREE) → TEXT,
        String, String, String) → TEXT      Write as prefix
```

where

```
TEXT == TextWriter
TREE == ExpressionTree
```

Usage

`infix(p, [t1, ..., tn], paren?, farg, op, left, right)`

Signatures

`infix: (TEXT, List TREE, TREE → Boolean, (TEXT, TREE) → TEXT, String, String, String) → TEXT`

Parameter	Type	Description
<i>p</i>	TEXT	The port to write to
<i>[t₁, ..., t_n]</i>	List TREE	The arguments of the operator
<i>paren?</i>	TREE → Boolean	The parenthetization function
<i>farg</i>	(TEXT, TREE) → TEXT	The function for the arguments
<i>op</i>	String	The infix symbol
<i>left</i>	String	The left parenthesis (optional)
<i>right</i>	String	The right parenthesis (optional)

where

TEXT == TextWriter

TREE == ExpressionTree

Description

Writes $farg(t_1) \text{ op } \dots \text{ op } farg(t_n)$ to *p*, calling *farg* on each argument, and calling *paren?* to decide whether to parenthetize each argument. Uses *left* and *right*, which default to “(” and “)” when parenthetizing.

See also

`prefix`

Usage

lisp(*p*, *s*, [*t*₁, . . . , *t*_{*n*}])

Signature

lisp: (TEXT, String, List TREE) → TEXT

Parameter	Type	Description
<i>p</i>	TEXT	The port to write to
<i>s</i>	String	A Lisp operator name
[<i>t</i> ₁ , . . . , <i>t</i> _{<i>n</i>}]	List TREE	The arguments of the operator

Description

Writes (*st*₁ . . . *t*_{*n*}) to *p*, where each *t*_{*i*} is written in Lisp format.

Usage

```
prefix(p, t, paren?, farg, op, left, right)
prefix(p, [t1, ..., tn], farg, op, left, right)
```

Signatures

```
prefix: (TEXT, TREE, TREE → Boolean, (TEXT, TREE) → TEXT,
        String, String, String) → TEXT
prefix: (TEXT, List TREE, (TEXT, TREE) → TEXT,
        String, String, String) → TEXT
```

Parameter	Type	Description
<i>p</i>	TEXT	The port to write to
<i>[t₁, ..., t_n]</i>	List TREE	The arguments of the operator
<i>paren?</i>	TREE → Boolean	The parenthetization function
<i>farg</i>	(TEXT, TREE) → TEXT	The function for the arguments
<i>op</i>	String	The prefix symbol
<i>left</i>	String	The left parenthesis (optional)
<i>right</i>	String	The right parenthesis (optional)

where

```
TEXT == TextWriter
TREE == ExpressionTree
```

Description

Writes $op(farg(t_1), \dots, farg(t_n))$ to *p*, calling *farg* on each argument. Uses *left* and *right*, which default to “(” and “)” for parenthetizing. The unary version calls *paren?* to decide whether to parenthetize the argument.

See also

`infix`

ExpressionTreePlus

Usage

import from ExpressionTreePlus

Description

ExpressionTreePlus is the addition operator for expression trees.

Exports

ExpressionTreeOperator

ExpressionTreePrefix

Usage

import from ExpressionTreePrefix s

Description

ExpressionTreePrefix s is the prefix operator with name *s* for expression trees.

Exports

ExpressionTreeOperator

Remarks

All those operators share the same `uniqueId`, so they are equal as expression trees even though their names may be different.

ExpressionTreeQuotient

Usage

import from ExpressionTreeQuotient

Description

ExpressionTreeQuotient is the division operator for expression trees.

Exports

ExpressionTreeOperator

ExpressionTreeSubscript

Usage

import from ExpressionTreeSubscript

Description

ExpressionTreeSubscript is the subscript operator for expression trees.

Exports

ExpressionTreeOperator

ExpressionTreeTimes

Usage

import from ExpressionTreeTimes

Description

ExpressionTreeTimes is the multiplication operator for expression trees.

Exports

ExpressionTreeOperator

ExpressionTreeVector

Usage

import from ExpressionTreeVector

Description

ExpressionTreeVector is the vector operator for expression trees.

Exports

ExpressionTreeOperator

Remarks

The following function is not yet implemented and produces dummy results : **fortran**.

Evaluator

Usage

Evaluator R: Category

Parameter	Type	Description
R	PartialRing	Resulting type of the evaluation

Description

Evaluator R is a category of interpreters, *i.e.* types which convert expression trees into elements of R whenever possible.

Exports

eval!:	$(\text{TEXT}, \text{TREE}, \text{SYM } R) \rightarrow \overline{R}$	Interpret an arbitrary tree
evalLeaf!:	$(\text{LEAF}, \text{SYM } R) \rightarrow \overline{R}$	Interpret a leaf
evalOp!:	$(\text{TEXT}, \text{TREE}, \text{SYM } R) \rightarrow \overline{R}$	Interpret $op(args)$
evalPrefix!:	$(\text{String}, \text{Z}, \text{List } R, \text{SYM } R) \rightarrow \overline{R}$	Interpret $op(args)$, op is prefix

where

Z	==	MachineInteger
LEAF	==	ExpressionTreeLeaf
$\overline{R}$	==	Partial R
SYM	==	SymbolTable
TEXT	==	TextWriter
TREE	==	ExpressionTree

InfixExpressionParser

Usage

import from InfixExpressionParser

Description

InfixExpressionParser implements infix expression parsers.

Exports

ParserReader

precedences!: (% , MachineInteger, MachineInteger) → % Set operator precedences

Usage

precedences!(p, op, n)

Signature

precedences!: (% , MachineInteger, MachineInteger) → %

Parameter	Type	Description
<i>p</i>	%	A parser
<i>op</i>	MachineInteger	An operator code
<i>n</i>	MachineInteger	Its new precedence

Description

Sets the precedence of op to n and returns the new parser. op must be one of `PARSER__PLUS`, `PARSER__MINUS`, `PARSER__TIMES`, `PARSER__DIVIDE` or `PARSER__POWER`.

LispExpressionParser

Usage

import from LispExpressionParser

Description

LispExpressionParser implements lisp expression parsers.

Exports

ParserReader

MakePartialRing

Usage

import from MakePartialRing(R, f)

Parameter	Type	Description
R	Ring	A ring
f	$R \rightarrow \text{Partial Integer}$	A retraction to the integers

Description

MakePartialRing(R, f) is a partial ring isomorphic to R. The function f is used to retract elements of this partial ring to integers whenever possible. This type can be used in order to build an evaluator that interprets expression trees into R.

Exports

PartialRing

coerce: $R \rightarrow \%$ Convert an element of R to one of the partial ring

coerce: $\% \rightarrow R$ Convert an element of the partial ring to one of R

Maple

Usage

import from Maple

Description

Maple provides utilities that allow its clients to batch MAPLE sessions and recover the output.

Exports

input:	% → <code>TextWriter</code>	Input stream of the maple session
maple:	() → %	Create a maple session
run:	% → <code>Partial ExpressionTree</code>	Run a maple session

Usage

input m

Signature

input: % $\rightarrow$ TextWriter

Parameter	Type	Description
m	%	A maple session

Returns

Returns the input stream for the maple session.

Remarks

Use that stream to send sequences of valid maple commands, making sure that all the commands are terminated with ‘:’ in order to avoid any printing from MAPLE. Do not use any of MAPLE’s printing functions. Note that the system maple is not started until ‘run’ is called.

Usage

maple()

Signature

maple: () → %

Description

Creates a maple session, by associating unique communications channels to and from that session.

Remarks

The maple input and output files used for communication are located in the /tmp directory and are deleted after the session is run, unless the call `maple(true)` is used, in which case they remain and can be inspected. Note that the system maple is not started until 'run' is called.

Usage

run m

Signature

run: % $\rightarrow$ Partial ExpressionTree

Parameter	Type	Description
m	%	A maple session

Description

Launches the system maple and executes all the commands that were sent to the input stream of the session. Returns the expression tree corresponding to the value returned by the last maple command executed.

Example

This examples shows how to call MAPLE to compute the integral of the 5th Legendre polynomial:

```
import from Integer, Maple, ExpressionTree, Partial ExpressionTree;

n := 5;
-- create a session (maple is not launched but a unique link is created)
mapl := maple();

-- send the maple code to compute the integral of the n-th legendre poly
-- note that all the maple commands are terminated with ":"
-- so that they do not generate any output
-- here again, nothing happens, the commands are only stored
input(mapl) << "with(orthopoly): p := P(" << n << ", x): int(p, x):";

-- now launch maple and recover the result of the last command ("int")
tree := run mapl;

failed? tree => error "Unable to parse Maple's output";
retract tree;
```

Running the above code produces the following expression tree:

```
(+ (- (* (/ 21 16) (^ x 6)) (* (/ 35 16) (^ x 4))) (* (/ 15 16) (^ x 2)))
```

Parsable

Usage

Parsable: Category

Description

Parsable is the category of types that convert expression trees into themselves whenever possible.

Exports

InputType

eval: ExpressionTree → Partial % Interpret a tree

eval: ExpressionTreeLeaf → Partial % Interpret a leaf

eval: (MachineInteger, List ExpressionTree) → Partial % Interpret a node

Usage

```
eval e
eval t
eval(op,[e1,...,en])
```

Signatures

```
eval: ExpressionTree → Partial %
eval: ExpressionTreeLeaf → Partial %
eval: (MachineInteger, List ExpressionTree) → Partial %
```

Parameter	Type	Description
e, e_i	ExpressionTree	Expression trees
t	ExpressionTreeLeaf	A leaf
op	MachineInteger	Code for an operator

Returns

eval(e) and eval(t) return the result of evaluating the given tree or leaf in the type, while eval(op,[e₁,...,e_n]) returns the result of evaluating $op(e_1, \dots, e_n)$ in the type, where op is a code from `include/algebrauid.as`.

Parser

Usage

Parser: Category

Description

Parser is the category for parser objects.

Exports

<code>eof?:</code>	<code>% → Boolean</code>	Check for end of input
<code>lastError:</code>	<code>% → MachineInteger</code>	Code for last parsing error
<code>parse!:</code>	<code>% → Partial ExpressionTree</code>	Parse one expression

Usage

eof? p

Signatureeof?: % $\rightarrow$ Boolean

Parameter	Type	Description
p	%	A parser

ReturnsReturns *true* if the input is finished, *false* otherwise.

Usage

lastError p

Signature

lastError: % $\rightarrow$ MachineInteger

Parameter	Type	Description
<i>p</i>	%	A parser

Returns

Returns the code for the last parsing error.

Usage

parse! p

Signature

parse!: % $\rightarrow$ Partial ExpressionTree

Parameter	Type	Description
p	%	A parser

Returns

Returns either an expression tree for the next parsed expression, or *failed* in case of error or end of input.

See also

lastError(%)

ParserReader

Usage

ParserReader: Category

Description

ParserReader is the category for parser objects that parse text readers.

Exports

parser: TextReader $\rightarrow$ % Create a parser

Usage

parser r

Signature

parser: TextReader $\rightarrow$ %

Parameter	Type	Description
<i>r</i>	TextReader	The input stream to parse

Returns

Returns a parser that takes its input on r.

ParsingTools

Usage

import from ParsingTools R

Parameter	Type	Description
<i>R</i>	Parsable ArithmeticType	A parsable arithmetic system

Description

ParsingTools R provides tools for converting expression trees into elements of R.

Exports

evalArith: (Z, List TREE) → Partial R Interpret an arithmetic expression
evalInt: TREE → Partial Integer Interpret an integer

where

TREE == ExpressionTree
Z == MachineInteger

Usage

`evalArith(op,[ $e_1, \dots, e_n$ ])`

Signature

`evalArith: (MachineInteger, List ExpressionTree)  $\rightarrow$  Partial R`

Parameter	Type	Description
op	MachineInteger	Code for an operator
e_i	ExpressionTree	Expression trees

Returns

Returns the result of evaluating $op(e_1, \dots, e_n)$ where op is a code from `include/algebrauid.as`. Provides support for the evaluation of the operators $+$, $-$, $*$ and $\wedge$, as well as $/$ when R has `CommutativeRing` or `FloatType`.

Usage

evalInt e

Signature

evalInt: ExpressionTree $\rightarrow$ Partial Integer

Parameter	Type	Description
e	ExpressionTree	An expression tree

Returns

Returns the value of e as an integer if it is an integer-valued leaf, *failed* otherwise.

PartialRing

Usage

PartialRing: Category

Description

PartialRing is the category of rings where all the arithmetic operations are partial, *i.e.* they are allowed to fail. Typical examples are matrices of different sizes, or unions of several true rings.

Exports

ExpressionType		
0:	%	Additive identity
1:	%	Multiplicative identity
-:	% → Partial %	Negation
-:	(%, %) → Partial %	Subtraction
+:	(%, %) → Partial %	Addition
*:	(%, %) → Partial %	Multiplication
/:	(%, %) → Partial %	Exact division
^:	(%, %) → Partial %	Exponentiation
<:	(%, %) → Partial %	Comparison
>:	(%, %) → Partial %	Comparison
≤:	(%, %) → Partial %	Comparison
≥:	(%, %) → Partial %	Comparison
[]:	Tuple % → Partial %	Construct a structure
coerce:	Boolean → %	Convert a boolean to a ring element
coerce:	Integer → %	Convert an integer to a ring element
integer:	% → Partial Integer	Convert to an integer
product:	List % → Partial %	Multiplication
sum:	List % → Partial %	Addition

Scanner

Usage

`import from Scanner`

Description

Scanner provides a simple scanner for mathematical expressions.

Exports

`scan: TextReader → Token` Scan a token

Usage

scan *p*

Signature

scan: `TextReader` $\rightarrow$ `Token`

Parameter	Type	Description
<i>p</i>	<code>TextReader</code>	Text to scan

Returns

Returns the next token read from the reader *p*.

import from Shell(P, R, E)

Parameter	Type	Description
P	Parser	A parser type for the input
R	PartialRing	Resulting type of the evaluation
E	Evaluator R	An interpreter with target type R

Shell(P, R, E) provides a basic read-eval-loop that converts its input to expressions of type R.

center:	$(TW, \text{String}) \rightarrow TW$	center a line of text
shell!:	$(P, TW, TW, \text{SymbolTable } R, \text{Boolean}, \text{Boolean}, \text{String}) \rightarrow Z$	Read-eval loop

```
shell!: (P, TW, TW, SymbolTable R, Boolean, Boolean, String) → Z  Read-eval loop
```

```
TW == TextWriter
Z   == MachineInteger
```

```
Z == MachineInteger
```

Usage

center(*p*, *s*)

Signature

center: (TextWriter, String) → TextWriter

Parameter	Type	Description
<i>p</i>	TextWriter	The port to write to
<i>s</i>	String	A string to center

Description

Writes *s* centered in a 66-character line to the port *p*, followed by a newline, and returns the port after the write.

Usage

shell!(r, p₁, p₂, t, verbose?, time?, s)

Signature

shell!: (P, TW, TW, SymbolTable R, Boolean, Boolean, String) → MachineInteger

where

TW == TextWriter

Parameter	Type	Description
<i>r</i>	P	A parser
<i>p₁, p₂</i>	TextWriter	The ports to write to
<i>t</i>	SymbolTable R	The initial symbol table
<i>verbose?</i>	Boolean	Select verbose or quiet mode
<i>time?</i>	Boolean	Select whether to return times in msec
<i>s</i>	String	A string to write after processing commands

Description

Starts a read-eval loop which takes its input from *r* and writes its output to *p₁* or *p₂*. Mathematical output generated by user commands is written to *p₁*, while prompts and execution times are written to *p₂*. if *verbose?* is *false* (quiet mode), nothing is printed to *p₂* and no prompts or execution times are communicated. Regardless of *verbose?*, if *time?* is *true*, then the execution and garbage collection times are written to *p₁* in milliseconds after each command. If *s* is not empty, then it is sent to *p₁* after each command is executed. The table *t* contains the initial environment, and can be modified by the loop. Returns an integer $n = b_1b_0$ where b_0 is 1 if a syntax error occurred, 0 otherwise, and b_1 is 1 if any other error occurred, 0 otherwise.

Token

Usage

import from Token

Description

Token is a type whose elements are parser tokens.

Exports

OutputType

PrimitiveType

float:	(List Character, List Character) → %	Create a float token
integer:	List Character → %	Create an integer token
leaf:	% → ExpressionTreeLeaf	Conversion to a leaf
leaf?:	% → Boolean	Test for a leaf
name:	List Character → %	Create a constant name token
operator:	% → ExpressionTreeOperator	Conversion to an operator
operator?:	% → Boolean	Test for an operator
prefix:	List Character → %	Create a prefix function token
special:	% → MachineInteger	Conversion to a special token
special?:	% → Boolean	Test for a special token
string:	List Character → %	Create a string
token:	Character → Partial %	Create a single character token

Usage

`float( $l_1, l_2$ )`

Signature

`float: (List Character, List Character) → %`

Parameter	Type	Description
$[d_0, \dots, d_n]$	List Character	A list of digits
$[e_0, \dots, e_m]$	List Character	A list of digits

Returns

`float( $[d_0, \dots, d_n], [e_0, \dots, e_m]$ )` returns the float $d_n d_{n-1} \dots d_0 . e_m e_{m-1} \dots e_0$ as a token.

See also

`integer`

Usage

integer l

Signature

integer: List Character $\rightarrow$ %

Parameter	Type	Description
$[d_0, \dots, d_n]$	List Character	A list of digits

Returns

integer($[d_0, \dots, d_n]$) returns the integer $d_0 + 10d_1 + \dots + 10^nd_n$ as a token.

See also

float

Usage

leaf t
leaf? t

Signatures

leaf: % → ExpressionTreeLeaf
leaf?: % → Boolean

Parameter	Type	Description
<i>t</i>	%	A token

Returns

leaf a returns *a* as a leaf is *a* is a leaf. leaf? a returns *true* if a is a leaf, *false* otherwise.

See also

operator, special

Token	name
-------	------

Usage
 name l

Signature
 name: List Character $\rightarrow$ %

Parameter	Type	Description
$[c_0, \dots, c_n]$	List Character	A list of characters

Returns
 name($[c_0, \dots, c_n]$) returns the name $c_n c_{n-1} \dots c_0$ as a token representing a constant symbol.

See also
 prefix, string

Usage

operator t
operator? t

Signatures

operator: % → ExpressionTreeOperator
operator?: % → Boolean

Parameter	Type	Description
<i>t</i>	%	A token

Returns

operator a returns *a* as an operator is *a* is an operator. operator? a returns *true* if a is an operator, *false* otherwise.

See also

leaf, special

Usage
prefix l

Signature
prefix: List Character $\rightarrow$ %

Parameter	Type	Description
$[c_0, \dots, c_n]$	List Character	A list of characters

Returns
prefix($[c_0, \dots, c_n]$) returns the name “ $c_n c_{n-1} \dots c_0''$ ” as a token representing a prefix function.

See also
name

Usage

special t
special? t

Signatures

special: % $\rightarrow$ MachineInteger
special?: % $\rightarrow$ Boolean

Parameter	Type	Description
<i>t</i>	%	A token

Returns

special a returns *a* as a special token is *a* is a special token. special? a returns *true* if a is a special token, *false* otherwise.

See also

leaf, special

Usage
string l

Signature
string: List Character $\rightarrow$ %

Parameter	Type	Description
$[c_0, \dots, c_n]$	List Character	A list of characters

Returns
string($[c_0, \dots, c_n]$) returns the string “ $c_n c_{n-1} \dots c_0''$ ” as a token representing a constant.

See also
name, prefix

Usage

token a

Signatures

token: `Character` $\rightarrow$ `Partial %`

token: `ExpressionTreeOperator` $\rightarrow$ `Partial %`

token: `MachineInteger` $\rightarrow$ `Partial %`

Parameter	Type	Description
<i>a</i>	<code>Character</code>	A single character token
	<code>ExpressionTreeOperator</code>	An operator
	<code>MachineInteger</code>	A code of a special token

Returns

Returns the token corresponding to the single character *a*, *failed* if there is none.

AlgebraLibraryInformation

Usage

import from AlgebraLibraryInformation

Description

AlgebraLibraryInformation provides version information about the **Algebra** library.

Exports

VersionInformationType

IndexedVariable

Usage

```
import from IndexedVariable S
import from IndexedVariable(S, x)
```

Parameter	Type	Description
S	ExpressionType TotallyOrderedType	The type of the indices
x	Symbol	The root variable (optional)

Description

IndexedVariable provides sorted symbols of the form x_s where $s \in S$.

Exports

```
ExpressionType
TotallyOrderedType
index:    %  $\rightarrow$  S  Index of a variable
variable: S  $\rightarrow$  %    Create an indexed variable
```

Usage

index v

Signature

index: $\% \rightarrow S$

Parameter	Type	Description
v	$\%$	An indexed variable

Returns

Returns the index of v .

Usage
variable s

Signature
variable: S → %

Parameter	Type	Description
<i>s</i>	S	An index

Returns
Returns the variable x_s .

Permutation

Usage

import from Permutation n

Parameter	Type	Description
n	MachineInteger	The number of elements

Description

Permutation(n) implements the group of permutations on n elements.

Exports

CopyableType

Group

apply: $(\%, Z) \rightarrow Z$ Image of an element

mapping: $\% \rightarrow (Z \rightarrow Z)$ Action of a permutation

sign: $\% \rightarrow Z$ Sign

transpose: $(Z, Z) \rightarrow \%$ Transposition

transpose!: $(\%, Z, Z) \rightarrow \%$ Compose with a transposition

where

$Z == \text{MachineInteger}$

Usage

apply(σ , x)
 σx

Signature

apply: ($\%$, MachineInteger) $\rightarrow$ MachineInteger

Parameter	Type	Description
σ	$\%$	A permutation
x	MachineInteger	An index

Returns

Returns σx .

Usage

mapping σ

Signature

mapping: $\% \rightarrow (\text{MachineInteger} \rightarrow \text{MachineInteger})$

Parameter	Type	Description
σ	$\%$	A permutation

Returns

Returns the map corresponding to σ .

See also

apply

Usage

sign σ

Signature

sign: $\% \rightarrow \text{MachineInteger}$

Parameter	Type	Description
σ	$\%$	A permutation

Returns

Returns the sign of σ , *i.e.* $(-1)^\epsilon$ where ϵ is the number of transpositions in the factorization of σ .

Usage

```
transpose(x,y)
transpose!( $\sigma$ ,x,y)
```

Signatures

```
transpose:  (MachineInteger, MachineInteger) → %
transpose!: (% , MachineInteger, MachineInteger) → %
```

Parameter	Type	Description
σ	%	A permutation
x,y	MachineInteger	Indices

Returns

transpose(x,y) returns the transposition (xy) , while transpose!(σ ,x,y) returns the composition $(xy) \circ \sigma$.

Remarks

When using transpose!, the storage used by σ is allowed to be destroyed or reused, so do not use it unless σ has been locally allocated, and is guaranteed not to share space with other elements. Some functions are not necessarily copying their arguments and can thus create memory aliases. Since there is no guarantee of reuse, you should always use the permutation returned by transpose! rather than σ after the call.

Sequence

Usage

import from Sequence R

Parameter	Type	Description
R	ExpressionType ArithmeticType	The coefficient domain

Description

Sequence R implements infinite sequences with coefficients in R.

Exports

LinearStructureType R

UnivariateFreeLinearArithmeticType R

#:	$\% \rightarrow \mathbb{Z}$	number of computed elements
bound:	$\% \rightarrow \text{MachineInteger}$	upper bound on the support size
dot:	$\text{Vector } R \rightarrow \text{Vector } \% \rightarrow \%$	linear combination
finite?:	$\% \rightarrow \text{Boolean}$	check whether the support is finite
interlacing:	$\text{List } \% \rightarrow \%$	interlacing
sequence:	$\text{Stream } R \rightarrow \%$	create a sequence

if R has Ring then

random: $() \rightarrow \%$ random sequence

Usage

s

Signatures

#: % → MachineInteger

Parameter	Type	Description
<i>s</i>	%	a sequence

Returns

Returns the number of elements of *s* that have been computed.

Usage

bound s
 finite? s

Signatures

bound: $\% \rightarrow \text{MachineInteger}$
 finite?: $\% \rightarrow \text{Boolean}$

Parameter	Type	Description
s	$\%$	a sequence

Returns

finite?(s) returns *true* if s is known to have finite support and *false* otherwise, while bound(s) returns $n \geq 0$ if s is known to have finite support and $s.m = 0$ for $m \geq n$, -1 otherwise.

Usage

```
dot [r1, ..., rn]
dot([r1, ..., rn])([s1, ..., sn])
```

Signature

```
dot: Vector R → Vector % → %
```

Parameter	Type	Description
r_i	R	coefficients
s_i	%	sequences

Returns

dot(r)(s) returns the sequence $\sum_{i=1}^n r_i s_i$, while dot(r) returns the map $[s_1, \dots, s_n] \rightarrow \text{sum}_{i=1}^n r_i s_i$.

Remarks

Using dot is more efficient than building up the same linear combination via additions and multiplications, since the intermediate combinations are not storing their computed elements when using dot.

Usage

interlacing $[s_1, \dots, s_n]$

Signature

interlacing: List % $\rightarrow$ %

Parameter	Type	Description
s_i	%	sequences

Description

Given $s_i = r_{i1}, r_{i2}, \dots$, returns the sequence $r_{11}, r_{21}, \dots, r_{n1}, r_{12}, r_{22}, \dots, r_{n2}, \dots$

Usage

random()

Signature

random: () → %

Returns

Returns a sequence with random entries.

Usage

sequence s

Signature

sequence: Stream R → %

Parameter	Type	Description
<i>s</i>	Stream R	a stream

Returns

Returns *s* viewed as a sequence.

4 libaldor Reference Manual

AldorInteger

Usage

import from AldorInteger

Description

AldorInteger provides an interface to the software (“infinite” precision) integers provided by the ALDOR virtual machine.

Exports

IntegerType

AdditiveType

Usage

AdditiveType: Category

Description

AdditiveType is the category of types with addition/substraction operations.

Exports

PrimitiveType

0 :	%	zero
+	(%, %) → %	addition
- :	% → %	opposite
- :	(%, %) → %	substraction
add! :	(%, %) → %	In-place addition
minus! :	% → %	In-place opposite
minus! :	(%, %) → %	In-place subtraction
zero? :	% → Boolean	test for 0

Usage

0

Signature

0: %

Returns

Return the 0 constant of the type.

Usage

$x + y$
 $x - y$
 $-x$

Signatures

$-$: $\% \rightarrow \%$
 $+, -$: $(\%, \%) \rightarrow \%$

Parameter	Type	Description
x, y	$\%$	elements of the type

Returns

$x + y, x - y$ return respectively the sum and difference x with y , while $-x$ returns the opposite of x .

See also

`add!`, `minus!`

Usage

```
add!(x, y)
minus!(x, y)
minus! x
```

Signatures

```
minus!:      % → %
add!, minus!: (%, %) → %
```

Parameter	Type	Description
x, y	$\%$	Elements of the type

Returns

`add!( $x, y$ )` and `minus!( $x, y$ )` returns respectively $x + y$ and $x - y$, while `minus! x` returns the opposite of x . In all cases, the storage used by x is allowed to be destroyed or reused, so x is lost after this call.

Remarks

Those functions may cause x to be destroyed, so do not use them unless x has been locally allocated, and is guaranteed not to share space with other elements. Some functions are not necessarily copying their arguments and can thus create memory aliases.

See also

`+, -`

Usage

zero? x

Signature

zero?: %₀ → Boolean

Parameter	Type	Description
x	$\%_0$	an element of the type

Returns

Returns the result of $x = 0$ using the semantics of $=$ of the type.

ArithmeticType

Usage

ArithmeticType: Category

Description

ArithmeticType is the category of types with standard arithmetic operations.

Exports

AdditiveType

1:	%	one
*:	(%, %) → %	product
^:	(%, MachineInteger) → %	exponentiation
commutative?:	Boolean	check whether * is commutative
one?:	% → Boolean	test for 1
times!:	(%, %) → %	In-place product

Usage

1

Signature

1: %

Returns

Return the 1 constant of the type.

Usage

$x * y$
 $x \wedge n$

Signatures

$*$: $(\%, \%) \rightarrow \%$
 $\wedge$: $(\%, \text{MachineInteger}) \rightarrow \%$

Parameter	Type	Description
x, y	$\%$	elements of the type
n	MachineInteger	an exponent

Returns

$x * y$ returns the product of x with y , while $x \wedge n$ returns x to the power n .

See also

times!

Usage

commutative?

Signature

commutative?: Boolean

Returns

Returns *true* if *** is commutative, *false* otherwise.

Usage

one? x

Signature

one?: % → Boolean

Parameter	Type	Description
<i>x</i>	%	an element of the type

Returns

Returns the result of $x = 1$ using the semantics of $=$ of the type.

Usage

times!(x, y)

Signature

times!: (% , %) → %

Parameter	Type	Description
x, y	%	Elements of the type

Returns

Return xy , where the storage used by x is allowed to be destroyed or reused, so x is lost after this call.

Remarks

This function may cause x to be destroyed, so do not use it unless x has been locally allocated, and is guaranteed not to share space with other elements. Some functions are not necessarily copying their arguments and can thus create memory aliases.

See also

*

BinaryPowering

Usage

import from BinaryPowering(T , Z)

Parameter	Type	Description
T	ArithmeticType	An arithmetic system
Z	IntegerType	An integer-like type

Description

BinaryPowering provides binary exponentiation of elements of T with exponents in Z .

Exports

binaryExponentiation: $(T, Z) \rightarrow T$ Binary powering
binaryExponentiation!: $(T, Z) \rightarrow T$ In-place binary powering

Usage

binaryExponentiation(a, n)
binaryExponentiation!(a, n)

Signature

binaryExponentiation: $(T, Z) \rightarrow T$

Parameter	Type	Description
a	T	The element to exponentiate
n	Z	The exponent

Returns

Returns a^n . The exponent n must be nonnegative. When using `binaryExponentiation!(a, n)`, the storage used by `a` and `n` is allowed to be destroyed or reused, so `a` and `n` are lost after this call.

Remarks

A call to `binaryExponentiation!(a, n)` may cause `a` and `n` to be destroyed, so do not use it unless `a` and `n` have been locally allocated, and are guaranteed not to share space with other elements. Some functions are not necessarily copying their arguments and can thus create memory aliases.

Boolean

Usage

import from Boolean

Description

Boolean implements the boolean values *true* and *false*.

Exports

BooleanArithmeticType

HashType

false: % *false*

true: % *true*

Usage

true
false

Signature

true,false: %

Returns

true and false return the boolean values *true* and *false* respectively.

BooleanArithmeticType

Usage

BooleanArithmeticType: Category

Description

BooleanArithmeticType is the category of types allowing boolean arithmetic.

Exports

PrimitiveType

$\sim$: $\% \rightarrow \%$ negation

$\wedge$: $(\%, \%) \rightarrow \%$ and

$\vee$: $(\%, \%) \rightarrow \%$ or

xor: $(\%, \%) \rightarrow \%$ exclusive or

Usage

$\sim a$
 $a \wedge b$
 $a \vee b$
 $\text{xor}(a, b)$

Signatures

$\sim$: $\% \rightarrow \%$
 $\wedge, \vee, \text{xor}$: $(\%, \%) \rightarrow \%$

Parameter	Type	Description
a, b	$\%$	elements of the type

Returns

$\sim a$ returns $\text{not}(a)$, while $a \vee b$ returns $(a \text{ or } b)$, $a \wedge b$ returns $(a \text{ and } b)$, and $\text{xor}(a, b)$ returns

$$(a \wedge \sim b) \vee (\sim a \wedge b).$$

The semantics of *not*, *or* and *and* can be logical or bitwise, depending on the actual type.

Remarks

For the type `Boolean`, the difference between $a \vee b$ and `a or b` is that $a \vee b$ guarantees that both expressions a and b are evaluated while `a or b` may evaluate only a and return *true* if a evaluates to *true*. There is a similar difference between $a \wedge b$ and `a and b`.

Example

If `a` and `b` are the `MachineInteger` 5 and 7, then $\sim a = -6$, $a \wedge b = 5$, $a \vee b = 7$ and $\text{xor}(a, b) = 2$.

Complex

Usage

import from Complex R

Parameter	Type	Description
R	ArithmeticType	Type to be extended

Description

Complex R implements the algebraic extension of R generated by a root of $X^2 + 1 = 0$. The coefficient type R must satisfy 2 additional mathematical properties for the arithmetic of Complex R to be correct, namely:

- $X^2 + 1$ is irreducible over R .
- The element α such that $\alpha^2 + 1 = 0$ must commute with all the elements of R (which is obviously satisfied when the multiplication of R is commutative).

Exports

ArithmeticType

CopyableType

LinearCombinationType R

coerce: $R \rightarrow \%$ Natural embedding

complex: $(R, R) \rightarrow \%$ create a complex

conjugate: $\% \rightarrow \%$ conjugation

conjugate!: $\% \rightarrow \%$ in-place conjugation

copy!: $(\%, R, R) \rightarrow \%$ in-place copy

imag: $\% \rightarrow R$ imaginary part

real: $\% \rightarrow R$ real part

if R has FloatType then

$/:$ $(\%, \%) \rightarrow \%$ Division

if R has InputType then

InputType

if R has OutputType then

OutputType

if R has SerializableType then

SerializableType

Usage

coerce x
x::%

Signature

coerce: $R \rightarrow \%$

Parameter	Type	Description
x	R	an element of the base type

Returns

Returns the complex $x + 0\sqrt{-1}$.

Usage`complex(x,y)`**Signature**`complex: (R, R) → %`

Parameter	Type	Description
<i>x, y</i>	R	real and imaginary parts

ReturnsReturns the complex $x + y\sqrt{-1}$.

Usage

conjugate z
conjugate! z

Signature

conjugate: % $\rightarrow$ %

Parameter	Type	Description
z	%	a complex

Description

Returns $x - y\sqrt{-1}$ where $z = x + y\sqrt{-1}$. When using conjugate!, the storage used by z is allowed to be destroyed or reused, so z is lost after this call.

Usage

copy!(z,x,y)

Signature

copy!:: (% , R, R) → %

Parameter	Type	Description
z	%	a complex
x, y	R	real and imaginary parts

Description

Replaces z by $x + y\sqrt{-1}$ and return z , where the storage used by z is allowed to be destroyed or reused, so z is lost after this call.

Remarks

This call may cause z to be destroyed, so do not use it unless z has been locally allocated, and is guaranteed not to share space with other elements. Some functions are not necessarily copying their arguments and can thus create memory aliases.

See also

copy!

Usage

imag,real z

Signatureimag,real: % $\rightarrow$ R

Parameter	Type	Description
z	%	a complex

Returnsimag(z) and real(z) return respectively y and x where $z = x + y\sqrt{-1}$.

Usage

norm z

Signature

norm: % → %

Parameter	Type	Description
z	%	a complex

Returns

Returns $z\bar{z}$ where $\bar{z}$ is the conjugate of z .

Remarks

When R is commutative, the imaginary part of $z\bar{z}$ is 0, and the usual complex norm can be computed via `real norm z`. The imaginary part of $z\bar{z}$ is not necessarily 0 when R is not commutative.

DoubleFloat

Usage

import from DoubleFloat

Description

DoubleFloat implements the single-precision signed machine floats.

Exports

CopyableType

FloatType

PackableType

$\wedge$: ($\%$, $\%$) $\rightarrow$ $\%$ exponentiation

coerce: `SingleFloat` $\rightarrow$ $\%$ conversion to double precision

max: $\%$ largest double-precision machine float

min: $\%$ smallest double-precision machine float

single: $\%$ $\rightarrow$ `SingleFloat` conversion to single precision

Usage

$x \wedge y$

Signatures

$\wedge: (\%,\%) \rightarrow \%$

Parameter	Type	Description
x,y	$\%$	floats

Returns

Returns x to the power y .

Usage

`x::%`
`single y`

Signatures

`coerce: SingleFloat → %`
`single: % → SingleFloat`

Parameter	Type	Description
<i>x</i>	SingleFloat	a single float
<i>y</i>	%	a double float

Description

`coerce(x)` returns `x` converted to a double float, while `single(y)` truncates `y` to a single precision float.

Usage

max
min

Signature

max,min: %

Returns

max and min return respectively the largest and the smallest double-precision machine floats.

FloatType

Usage

FloatType: Category

Description

FloatType is the category of types representing floats.

Exports

InputType

OrderedArithmeticType

OutputType

SerializableType

$/:$ $(\%, \%) \rightarrow \%$ division

coerce: `MachineInteger` $\rightarrow \%$ conversion to a float

fraction: $\% \rightarrow \%$ fractional part

truncate: $\% \rightarrow \text{AldorInteger}$ truncation

Usage
 x/y

Signature
 $/: (\%,\%) \rightarrow \%$

Parameter	Type	Description
x,y	$\%$	floats

Returns
Returns the quotient of x by y .

Usage
n::%

Signature
coerce: MachineInteger → %

Parameter	Type	Description
<i>n</i>	MachineInteger	a machine integer

Returns
Returns n converted to a float.

Usage

fraction x
truncate x

Signatures

fraction: % $\rightarrow$ %
truncate: % $\rightarrow$ AldorInteger

Parameter	Type	Description
x	%	a float

Returns

truncate(x) returns n such that $nx \geq 0$ and $|n| \leq |x| < |n| + 1$, while fraction(x) returns $x - \text{truncate}(x)$.

GMPFloat

Usage

import from GMPFloat

Description

GMPFloat implements arbitrary precision floats with the float arithmetic provided by GMP.

Exports

CopyableType

FloatType

coerce:	SingleFloat → %	conversion
	DoubleFloat → %	
	GMPInteger → %	
defaultPrecision:	() → Z	default precision
limbs:	% → Generator Z	iteration
machine:	% → DoubleFloat	conversion to double precision
new:	() → %	initialization (for calling GMP)
precision:	% → Z	precision
setDefaultPrecision:	Z → Z	adjust default precision
setPrecision!:	(%, Z) → %	adjust precision
single:	% → SingleFloat	conversion to single precision

where

Z == MachineInteger

Usage

`x::%`

Signatures

`coerce: SingleFloat → %`
`coerce: DoubleFloat → %`
`coerce: GMPInteger → %`

Parameter	Type	Description
<i>x</i>	SingleFloat	a number
	DoubleFloat	
	GMPInteger	

Returns

Converts *x* to a GMP float.

Usage

for d in limbs x repeat { ... }

Signature

limbs: % $\rightarrow$ Generator MachineInteger

Parameter	Type	Description
x	%	a float

Description

This function allows the individual limbs of a GMP float to be iterated independently of the machine size. This generator yields the limbs from the least to the most significant.

Remarks

The limbs of x describe $|x|$, so combine it with **sign** if you need a complete description of x .

Usage

machine x

single x

Signaturesmachine: % $\rightarrow$ DoubleFloatsingle: % $\rightarrow$ SingleFloat

Parameter	Type	Description
x	%	a float

Description

machine(x) and single(x) truncate x respectively to a double precision or single precision float.

Usage`new()`**Signature**`new: () → %`**Returns**

Returns an initialized GMP float but with no value stored into it. Results from this function can be used only as parameters to explicit calls to `mpf_` functions.

Usage

```
precision x
setPrecision!(x, n)
```

Signatures

```
precision:      % → MachineInteger
setPrecision!:  (% , MachineInteger) → %
```

Parameter	Type	Description
x	%	a float
n	MachineInteger	a precision

Description

precision(x) returns the precision actually used for x , while setPrecision!(x , n) sets the precision for x to be at least n bits.

Remarks

Those functions are wrapper to the `mpf_get_prec` and `mpf_set_prec` GMP functions, so setPrecision! can involve a call to `realloc`.

See also

```
defaultPrecision, setDefaultPrecision
```

GMPIInteger

Usage

import from GMPIInteger

Description

GMPIInteger implements arbitrary precision integers with the integer arithmetic provided by GMP.

Exports

CopyableType

IntegerType

coerce: AldorInteger $\rightarrow$ % conversion

coerce: % $\rightarrow$ AldorInteger conversion

limbs: % $\rightarrow$ Generator MachineInteger iteration

new: () $\rightarrow$ % initialization (for calling GMP)

Usage`m::%``n::AldorInteger`**Signatures**`coerce: AldorInteger → %``coerce: % → AldorInteger`

Parameter	Type	Description
<i>m</i>	%	an ALDOR integer
<i>n</i>	AldorInteger	a gmp integer

Description

Those functions convert between ALDOR and GMP integers.

Remarks

The conversion to an `AldorInteger` can be quadratic in the number of bits of the integer, which is expensive.

Usage

for d in limbs n repeat { ... }

Signature

limbs: % $\rightarrow$ Generator MachineInteger

Parameter	Type	Description
n	%	an integer

Description

This function allows the individual limbs of a GMP integer to be iterated independently of the machine size. This generator yields the limbs from the least to the most significant.

Remarks

The limbs of n describe $|n|$, so combine it with **sign** if you need a complete description of n .

Signature

new: $() \rightarrow \%$

Returns

Returns an initialized GMP integer but with no value stored into it. Results from this function can be used only as parameters to explicit calls to `mpz_` functions.

IntegerSegment

Usage

import from IntegerSegment Z

Parameter	Type	Description
Z	IntegerType	an integer type

Description

IntegerSegment(Z) implements open and closed segments of Z , *i.e.* a selection of equally spaced integers in a range of the form $[a, b]$ or $[a, +\infty)$.

Exports

PrimitiveType

InputType

OutputType

SerializableType

...: $Z \rightarrow \%$ creation of a segment
 $(Z, Z) \rightarrow \%$

by: $(\%, Z) \rightarrow \%$ change the spacing

generator: $\% \rightarrow \text{Generator } Z$ iterate over a segment

high: $\% \rightarrow Z$ upper bound

low: $\% \rightarrow Z$ lower bound

open?: $\% \rightarrow \text{Boolean}$ check whether a segment is open

step: $\% \rightarrow Z$ spacing

a..
a..b

$$\begin{array}{l} \dots \quad Z \rightarrow \% \\ \dots \quad (Z, Z) \rightarrow \% \end{array}$$

Parameter	Type	Description
a, b	Z	integers

a.. returns the open range $[a, +\infty)$ while a..b returns the closed range $[a, b]$. Every integer in the range belongs to the resulting segment.

by

Usage

s by n
 step s

Signatures

by: $(\%, \mathbb{Z}) \rightarrow \%$
 step: $\% \rightarrow \mathbb{Z}$

Parameter	Type	Description
s	$\%$	a segment
n	$\mathbb{Z}$	a step

Returns

s by n changes s to become the segment consisting of every n^{th} integer in its range, *i.e.* the integers $a, a + n, a + 2n, \dots$ that are within the range $[a, b]$ or $[a, +\infty)$ of s, while step(s) returns n such that s is the segment consisting of every n^{th} integer in its range.

Remarks

The function s by n does not create a new segment but side-effects s, whose former step is lost.

Usage

for x in s repeat { ... }
 for x in generator s repeat { ... }

Signature

generator: % $\rightarrow$ Generator Z

Parameter	Type	Description
<i>s</i>	%	a segment

Description

This functions allows a segment to be iterated. This generator yields the integers of s in succession.

Example

The following code computes the sum of all the positive even machine integers that are smaller than *n*:

```
evenSum(n:MachineInteger):MachineInteger == {
  s := 0;
  for x in 2..prev(n) by 2 repeat s := s + x;
  s;
}
```

Usage

high s
low s

Signature

high,low: % $\rightarrow$ Z

Parameter	Type	Description
<i>s</i>	%	a segment

Returns

high(s) and low(s) return respectively the upper and lower bound of the range of s. The result of high(s) is undefined if s is an open segment.

Usage

open? s

Signatureopen?: % $\rightarrow$ Boolean

Parameter	Type	Description
<i>s</i>	%	a segment

ReturnsReturns *true* if the range of s is infinite, *false* otherwise.

IntegerType

Usage

IntegerType: Category

Description

IntegerType is the category of types representing integers.

Exports

BooleanArithmeticType

HashType

InputType

OrderedArithmeticType

OutputType

SerializableType

bit?: (% , MachineInteger) → Boolean

check a bit

clear: (% , MachineInteger) → %

clear a bit

coerce: MachineInteger → %

conversion from machine integer

divide: (% , %) → (% , %)

Euclidean division

even?: % → Boolean

test whether a number is even

factorial: % → %

factorial

gcd: (% , %) → %

greatest common divisor

lcm: (% , %) → %

least common multiple

length: % → MachineInteger

number of bits

machine: % → MachineInteger

conversion to a machine integer

mod: (% , %) → %

remainder

(% , MachineInteger) → MachineInteger

next: % → %

next greater integer

nthRoot: (% , %) → (Boolean, %)

n^{th} -root

odd?: % → Boolean

test whether a number is odd

prev: % → %

next smaller integer

quo: (% , %) → %

quotient

random: () → %

random integer

MachineInteger → %

rem: (% , %) → %

remainder

set: (% , MachineInteger) → %

set a bit

shift: (% , MachineInteger) → %

shift

shift!: (% , MachineInteger) → %

in-place shift

Usage

bit?(a, n)
clear(a, n)
set(a, n)

Signatures

bit?: (`%`, `MachineInteger`) → `Boolean`
clear, set: (`%`, `MachineInteger`) → `%`

Parameter	Type	Description
<i>a</i>	<code>%</code>	an integer
<i>n</i>	<code>MachineInteger</code>	a nonnegative machine integer

Returns

bit?(a, n) returns *true* if the n^{th} bit of a is 1, *false* if it is 0, while clear(a, n) and set(a, n) return copies of a where the n^{th} bit is set respectively to 0 and 1. For all 3 functions, the rightmost bit of a is the 0^{th} bit and so on.

Usage

n::%
machine a

Signatures

coerce: MachineInteger → %
machine: % → MachineInteger

Parameter	Type	Description
<i>a</i>	%	an integer
<i>n</i>	MachineInteger	a machine integer

Returns

n::% returns n converted to the current type, while machine(a) returns the low machine word of a converted to a MachineInteger. That operation can cause a loss of precision if a is greater than a machine word.

Usage

divide(a, b)
 a mod n
 a mod b
 a quo b
 a rem b

Signatures

divide: $(\%, \%) \rightarrow (\%, \%)$
 mod, quo, rem: $(\%, \%) \rightarrow \%$
 mod: $(\%, \text{MachineInteger}) \rightarrow \text{MachineInteger}$

Parameter	Type	Description
a, b	$\%$	integers, $b \neq 0$
n	MachineInteger	a nonzero machine integer

Returns

$a \bmod b$ (resp. $a \bmod n$) returns m such that $0 \leq m < |b|$ (resp. $0 \leq m < |n|$) and $a \equiv m \pmod{b}$ (resp n), while $a \bmod b$ returns r such that $-|b| < r < |b|$ and $a \equiv r \pmod{b}$, $a \text{ quo } b$ returns $(a - (a \bmod b))/b$, and $\text{divide}(a, b)$ returns the pair $(a \text{ quo } b, a \bmod b)$.

Remarks

mod returns a unique remainder modulo b, but is more expensive to compute than rem, and is not guaranteed to be compatible with the result of quo. The version whose second argument is a **MachineInteger** allows for more efficient implementations.

Usage

even? a

odd? a

Signatureeven?,odd?: % $\rightarrow$ Boolean

Parameter	Type	Description
<i>a</i>	%	an integer

Returnseven?(a) and odd?(a) return *true* when a is even, respectively odd, *false* otherwise.

Usage

factorial a

Signature

factorial: % $\rightarrow$ %

Parameter	Type	Description
a	%	a nonnegative integer

Returns

Returns $a! = \prod_{i=1}^a i$.

Usage

gcd(a, b)
lcm(a, b)

Signature

gcd,lcm: (%,%) $\rightarrow$ %

Parameter	Type	Description
<i>a, b</i>	%	integers

Returns

gcd(a, b) and lcm(a, b) return respectively a greatest common divisor and a least common multiple of a and b.

Usage

length a

Signature

length: % $\rightarrow$ MachineInteger

Parameter	Type	Description
<i>a</i>	%	an integer

Returns

Returns the number of binary bits of a, *i.e.* n such that `bit?(a, n - 1)` is *true* and `bit?(a, m)` is *false* for $m \geq n$.

Usage

next a

prev a

Signaturenext,prev: % $\rightarrow$ %

Parameter	Type	Description
<i>a</i>	%	an integer

Returnsnext(a) and prev(a) return $a + 1$ and $a - 1$ respectively.

Usage

nthRoot(a, b)

Signature

nthRoot: (%,%) $\rightarrow$ (Boolean,%)

Parameter	Type	Description
a	%	an integer
b	%	a positive integer

Returns

Returns (found?, n) such that $a = n^b$ if found? is *true*. Otherwise, found? is *false* and

$$n^b < a < (n + 1)^b.$$

Usage

random()
random *n*

Signatures

random: () → %
random: MachineInteger → %

Parameter	Type	Description
<i>n</i>	MachineInteger	a positive size

Returns

random() returns a random integer, while random(*n*) returns a random integer with *n* limbs.

Usage

shift(*a*, *n*)
shift!(*a*, *n*)

Signature

shift: (*%*, MachineInteger) → *%*

Parameter	Type	Description
<i>a</i>	<i>%</i>	an integer
<i>n</i>	MachineInteger	a machine integer

Returns

Returns *a* shifted left *n* times if $n \geq 0$, shifted right $-n$ times if $n \leq 0$.

Remarks

shift! does not make a copy of *x*, which is therefore modified after the call. It is unsafe to use the variable *x* after the call, unless it has been assigned to the result of the call, as in `x := shift!(x, n)`.

LinearCombinationType

Usage

LinearCombinationType R:Category

Parameter	Type	Description
R	AdditiveType	The coefficient domain

Description

LinearCombinationType R is the category of types containing linear combinations of their elements with coefficients in R .

Exports

AdditiveType

<code>*</code>	<code>(R, %) → %</code>	Left-multiplication by a scalar
<code>add!</code>	<code>(%, R, %) → %</code>	In-place product and sum
<code>times!</code>	<code>(R, %) → %</code>	In-place product by a scalar

Usage

`r * p`

Signature

`*`: $(R, \%) \rightarrow \%$

Parameter	Type	Description
r	R	A scalar
p	$\%$	An element of the type

Returns

Returns the product rp .

See also

`times!`

Usage

add!(p, r, q)

Signature

add!: ($\%$, R, $\%$) $\rightarrow$ $\%$

Parameter	Type	Description
p	$\%$	An element of the type (to be destroyed)
r	R	A scalar
q	$\%$	An element of the type

Returns

add!(p, r, q) returns $p + rq$.

Remarks

The storage used by p is allowed to be destroyed or reused, so p is lost after this call. This function may cause p to be destroyed, so do not use it unless p has been locally allocated, and is guaranteed not to share space with other elements. Some functions are not necessarily copying their arguments and can thus create memory aliases.

See also

times!

Usage

times!(r, p)

Signature

times!: (R, %) → %

Parameter	Type	Description
<i>r</i>	R	A scalar to be multiplied by p
<i>p</i>	%	An element of the type (to be destroyed)

Returns

Returns the product *rp*.

Remarks

The storage used by p is allowed to be destroyed or reused, so p is lost after this call. This may cause p to be destroyed, so do not use this unless p has been locally allocated, and is thus guaranteed not to share space with other elements. Some functions are not necessarily copying their arguments and can thus create memory aliases.

See also

*,add!

MachineInteger

Usage

import from MachineInteger

Description

MachineInteger implements the full-word signed machine integers.

Exports

CopyableType

IntegerType

bytes:	%	machine word-size
max:	%	largest machine integer
min:	%	smallest machine integer
mod_+:	(%, %, %) → %	modular addition
mod_-:	(%, %, %) → %	modular subtraction
mod_*:	(%, %, %) → %	modular multiplication
mod_/:	(%, %, %) → %	modular division
mod_^:	(%, %, %) → %	modular exponentiation
modInverse:	(%, %) → %	modular inverse

Usage

bytes
max
min

Signature

bytes,max,min: %

Returns

bytes, max and min return respectively the size in bytes of a machine integer, the largest and the smallest machine integers.

Usage

mod_X(a, b, n)
 modInverse(a, n)

Parameter	Type	Description
a, b, n	%	machine integers

Signatures

mod_X: ($\%$, $\%$, $\%$) $\rightarrow$ $\%$
 modInverse: ($\%$, $\%$) $\rightarrow$ $\%$

Returns

mod_X(a, b, n) returns $(aXb) \pmod n$ where X is one of $+, -, *, /, \wedge$, while modInverse(a, b) returns the inverse of a modulo n .

Remarks

Those operations require that $0 \leq a, b < n$.

OrderedArithmeticType

Usage

OrderedArithmeticType: Category

Description

OrderedArithmeticType is the category of ordered types with the standard arithmetic operations.

Exports

ArithmeticType

TotallyOrderedType

abs: % $\rightarrow$ % norm

sign: % $\rightarrow$ MachineInteger sign

Usage

abs x

Signature

abs: % $\rightarrow$ %

Parameter	Type	Description
x	%	an element of the type

Returns

Returns the norm $|x|$ of x.

Usage

sign x

Signature

sign: % $\rightarrow$ MachineInteger

Parameter	Type	Description
x	%	an element of the type

Returns

Returns 1 if $x > 0$, 0 if $x = 0$ and -1 if $x < 0$.

PartiallyOrderedType

Usage

PartiallyOrderedType: Category

Description

PartiallyOrderedType is the category of partially ordered types. Hence, a domain of this category is endowed with a transitive binary relation $>$ such that either $x > y$ or $y > x$ does not hold for every x and y . If both do not hold then x and y are called not comparable for $>$. In particular if $x = y$ holds then x and y are not comparable for $>$.

Exports

PrimitiveType

$<$: $(\%, \%) \rightarrow \text{Boolean}$ stricly less than
 $>$: $(\%, \%) \rightarrow \text{Boolean}$ stricly greater than
 $\leq$: $(\%, \%) \rightarrow \text{Boolean}$ less than or equal to
 $\geq$: $(\%, \%) \rightarrow \text{Boolean}$ greater than or equal to

Usage $a < b$ $a > b$ $a \leq b$ $a \geq b$ **Signature** $<,>: (\%,\%) \rightarrow \text{Boolean}$

Parameter	Type	Description
a, b	$\%$	elements of the type

Returns

$a < b$, $a > b$, $a \leq b$, $a \geq b$ return *true* when a is respectively stricly smaller than, stricly greater than, less than or equal to, greater than or equal to b . Observe that if neither $a > b$ nor $a = b$ hold, this does not imply that $a < b$ holds, since a and b may not be comparable, except if the type has `TotallyOrderedType`.

PackableType

Usage

PackableType: Category

Description

PackableType is the category of types providing their own primitive packed arrays over themselves. You can use `PackedPrimitiveArray(T)` as a packed alternative to `PrimitiveArray(T)` whenever *T* provides such functions.

Exports

<code>getPackedArray:</code>	<code>(Pointer, MachineInteger) → %</code>	access an array element
<code>newPackedArray:</code>	<code>MachineInteger → Pointer</code>	create an array
<code>setPackedArray!:</code>	<code>(Pointer, MachineInteger, %) → ()</code>	changes an array element

Usage

getPackedArray(*a*, *n*)
setPackedArray!(*a*, *n*, *x*)

Signatures

getPackedArray: (Pointer, MachineInteger) → %
setPackedArray!: (Pointer, MachineInteger, %) → ()

Parameter	Type	Description
<i>a</i>	Pointer	an array
<i>n</i>	MachineInteger	an index
<i>x</i>	%	a value

Description

getPackedArray(*a*, *n*) returns the element at position *n* in *a*, while setPackedArray!(*a*, *n*, *x*) sets the element at position *n* in *a*. Note that both functions are 0-indexed.

Usage

newPackedArray n

Signature

newPackedArray: MachineInteger → Pointer

Parameter	Type	Description
<i>n</i>	MachineInteger	a size

Returns

Returns an array for *n* elements

PrimitiveType

Usage

PrimitiveType: Category

Description

PrimitiveType is the category of the most basic types.

Exports

`=:` `(%, %) → Boolean` equality test
`~=:` `(%, %) → Boolean` inequality test

Usage

a = b
a ~= b

Signatures

=: (%,%)
~=: (%,%)

Parameter	Type	Description
a, b	%	elements of the type

Returns

If *a = b* returns *true*, then *a* and *b* are guaranteed to represent the same element of the type. The behavior if *a = b* returns *false* depends on the type, since a full equality test might not be available. At least, it is guaranteed that *a* and *b* do not share the same memory location in that case. The semantics of *a ~= b* is the boolean negation of *a = b*.

RandomNumberGenerator

Usage

```
import from RandomNumberGenerator
```

Description

RandomNumberGenerator provides independent pseudo-random number generators.

Exports

<code>apply:</code>	<code>% → Z</code>	generate a random number
<code>generator:</code>	<code>% → Generator Z</code>	generate random numbers
<code>max:</code>	<code>% → Z</code>	largest number that can be generated
<code>min:</code>	<code>% → Z</code>	smallest number that can be generated
<code>numberOfGenerators:</code>	<code>Z</code>	number of predefined generators
<code>randomGenerator:</code>	<code>() → %</code>	get a generator
<code>randomGenerator:</code>	<code>Z → %</code>	get a generator
<code>randomGenerator:</code>	<code>(Z, Z) → %</code>	get a bounded generator
<code>randomGenerator:</code>	<code>(Z, Z, Z) → %</code>	get a bounded generator
<code>randomInteger:</code>	<code>() → Z</code>	generate a random number
<code>seed:</code>	<code>(%, Z) → Z</code>	set the seed

where

```
Z == MachineInteger
```

Usage

apply r
r()

Signature

apply: % $\rightarrow$ MachineInteger

Parameter	Type	Description
<i>r</i>	%	a pseudo-random number generator

Returns

Returns a pseudo-random integer.

Usage

```
for n in r repeat { ... }  
for n in generator r repeat { ... }
```

Signature

generator: % $\rightarrow$ Generator MachineInteger

Parameter	Type	Description
<i>r</i>	%	a pseudo-random number generator

Description

This functions allows a for-loop that generates infinitely many pseudo-random numbers.

Example

The following code computes the number of tries it takes for a random generator to generate a multiple of 10:

```
multiple10():MachineInteger == {  
  r := randomGenerator();  
  for n in r for tries in 1.. repeat {  
    zero?(n rem 10) => return tries;  
  }  
  never;  
}
```

Usage

max r
min r

Signature

max,min: % $\rightarrow$ MachineInteger

Parameter	Type	Description
<i>r</i>	%	a pseudo-random number generator

Returns

max(r) and min(r) return respectively the largest and smallest random integers that r can generate.

Usage

numberOfGenerators

Signature

numberOfGenerators: `MachineInteger`

Returns

Returns the number of independent generators provided.

Usage

```
randomGenerator()  
randomGenerator n  
randomGenerator(a, b)  
randomGenerator(a, b, n)
```

Signatures

```
randomGenerator: () → %  
randomGenerator: MachineInteger → %  
randomGenerator: (MachineInteger, MachineInteger) → %  
randomGenerator: (MachineInteger, MachineInteger, MachineInteger) → %
```

Parameter	Type	Description
<i>a, b</i>	MachineInteger	bounds for the generator
<i>n</i>	MachineInteger	identifier for the generator (optional)

Returns

randomGenerator() returns a pseudo-random generator, while randomGenerator(a, b) returns a pseudo-random number generator that generates numbers between a and b inclusive. If the optional argument n is given, then the n^{th} independent generator is returned, which allows for several independent sources of random numbers.

See also

numberOfGenerators

Usage

randomInteger()

Signature

randomInteger: () → MachineInteger

Returns

Returns a pseudo-random integer.

Usage`seed(r, s)`**Signature**`seed: (% , MachineInteger) → %`

Parameter	Type	Description
<i>r</i>	%	a pseudo-random number generator
<i>s</i>	MachineInteger	a nonzero seed

Description

Sets the seed of *r* to *s* and returns *s*. This is useful when a reproducible pseudo-random sequence is desired. If an unseeded generator is called, then it seeds itself from the system clock the first time it is used, so the sequence is not reproducible. Note that setting the seed to 0 causes the generator to generate an infinite sequence of 0.

SingleFloat

Usage

import from SingleFloat

Description

SingleFloat implements the single-precision signed machine floats.

Exports

CopyableType

FloatType

PackableType

$\wedge$: (% , %) $\rightarrow$ % exponentiation

max: % largest single-precision machine float

min: % smallest single-precision machine float

Usage

$$x \wedge y$$

Signatures

$$\wedge: (\%,\%) \rightarrow \%$$

Parameter	Type	Description
x,y	$\%$	floats

Returns

Returns x to the power y .

Usage

max
min

Signature

max,min: %

Returns

max and min return respectively the largest and the smallest single-precision machine floats.

TotallyOrderedType

Usage

TotallyOrderedType: Category

Description

TotallyOrderedType is the category of totally ordered types, *i.e.* partially ordered types where every pair $x \neq y$ is comparable.

Exports

PartiallyOrderedType

compare:	(%,%) $\rightarrow$ MachineInteger	comparison
max:	(%,%) $\rightarrow$ %	greater element
min:	(%,%) $\rightarrow$ %	smaller element

Signature

compare: (%,%) $\rightarrow$ MachineInteger

Usage

compare(a,b)

Parameter	Type	Description
<i>a, b</i>	%	elements of the type

Returns

compare(a,b) returns $n < 0$ if $a < b$, $n = 0$ if $a = b$ and $n > 0$ if $a > b$.

Usage

max(a,b)
min(a,b)

Signature

max,min: (%,%) $\rightarrow$ %

Parameter	Type	Description
<i>a</i> , <i>b</i>	%	elements of the type

Returns

max(a,b) and min(a,b) respectively the largest and the smallest among a and b.

BinaryReader

Usage

import from BinaryReader

Description

BinaryReader provides various binary input streams.

Exports

bin:	%	standard input stream
binaryReader:	() → Byte → %	create a stream
options:	SymbolTable MachineInteger	global options
read!:	% → Byte	read from a stream

Usage

bin

Signature

bin: %

Returns

bin is the standard input stream.

Usage

binaryReader f

Signature

binaryReader: $() \rightarrow \text{Byte} \rightarrow \%$

Parameter	Type	Description
<i>f</i>	$() \rightarrow \text{Byte}$	the single-byte read function

Returns

Returns the input stream for which *f*() reads a byte.

Usage

options

Signature

options: SymbolTable MachineInteger

Returns

Returns a table of global options for binary input. Those options can be queried by your implementations of <<.

Usage

read! s

Signature

read!: % → Byte

Parameter	Type	Description
<i>s</i>	%	a stream

Returns

Reads a byte from *s* and returns it. Some streams may return the byte `eof` if the end of file is reached.

BinaryWriter

Usage

import from BinaryWriter

Description

BinaryWriter provides various binary output streams.

Exports

berr:	%	the binary standard error stream
bout:	%	the binary standard output stream
binaryWriter:	(Byte → ()) → %	create a stream
binaryWriter:	(Byte → (), () → ()) → %	create a stream
flush!:	% → %	flush a stream
options:	SymbolTable MachineInteger	global options
write!:	(Byte, %) → ()	write to a stream

Usage

berr
bout

Signature

berr,bout: %

Returns

berr and bout are the binary standard error and standard output streams respectively.

Usage

binaryWriter f
binaryWriter(f, g)

Signatures

binaryWriter: (Byte → ()) → %
binaryWriter: (Byte → (), () → ()) → %

Parameter	Type	Description
<i>f</i>	Byte → ()	the single-byte write function
<i>g</i>	() → ()	the flush function (optional)

Returns

Returns the output stream for which $f(c)$ writes the byte c and such that $g()$ flushes the stream. If g is not given, then flushing the resulting stream has no effect.

Usage

flush! s

Signature

flush!: % → %

Parameter	Type	Description
<i>s</i>	%	a stream

Description

flush!(s) causes all previous values inserted into s to be really written and returns the stream. Has no effect on unbuffered streams, such as **berr**.

Usage

options

Signature

options: SymbolTable MachineInteger

Returns

Returns a table of global options for binary output. Those options can be queried by your implementations of <<.

Usage

write!(c, s)

Signature

write!: (Byte, %) → ()

Parameter	Type	Description
<i>c</i>	Byte	byte to write
<i>s</i>	%	a stream

Returns

Writes the byte *c* to the stream *s* and returns *c*.

Byte

Usage

import from Byte

Description

Byte implements machine bytes.

Exports

HashType

InputType

OutputType

PackableType

SerializableType

coerce: % → MachineInteger conversion to an integer

coerce: % → Character conversion to a character

coerce: Character → % conversion from a character

eof: % end-of-file marker

lowByte: MachineInteger → % low-byte of an integer

Usage

```
b::MachineInteger
b::Character
c::%
```

Signatures

```
coerce: % → MachineInteger
coerce: % → Character
coerce: Character → %
```

Parameter	Type	Description
<i>b</i>	%	a byte
<i>c</i>	Character	a character

Returns

`b::MachineInteger` and `b::Character` return `b` converted to an integer and a character respectively, while `c::%` returns `c` converted to a byte.

Byte

eof

Usage

eof

Signature

eof: %

Returns

eof is the end-of-file marker.

Usage
lowByte n

Signature
lowByte: MachineInteger → %

Parameter	Type	Description
<i>n</i>	MachineInteger	an integer

Returns
Returns the low-byte of n.

Character

Usage

import from Character

Description

Character implements machine characters.

Exports

HashType

InputType

OutputType

PackableType

SerializableType

TotallyOrderedType

char:	MachineInteger → %	create a character
digit?:	% → Boolean	test for a decimal digit
eof:	%	end-of-file character
letter?:	% → Boolean	test for a letter
lower:	% → %	convert to lower case
newline:	%	newline character
null:	%	null character
ord:	% → MachineInteger	character code
space?:	% → Boolean	test for a blank space
tab:	%	tab character
upper:	% → %	convert to upper case

Usage

char *n*
ord *c*

Signatures

char: `MachineInteger` $\rightarrow$ %
ord: % $\rightarrow$ `MachineInteger`

Parameter	Type	Description
<i>n</i>	<code>MachineInteger</code>	a character code
<i>c</i>	%	a character

Returns

char(*n*) returns the character whose code is *n*, while ord(*c*) returns the code corresponding to the character *c*.

Usage

digit? c

letter? c

space? c

Signaturedigit?,letter?,space?: % $\rightarrow$ Boolean

Parameter	Type	Description
<i>c</i>	%	a character

Returns

digit?(c) returns *true* if c is in the range '0'–'9', *false* otherwise, while letter?(c) returns *true* if c is in the range 'a'–'z' or the range 'A'–'Z', *false* otherwise and space?(c) returns *true* if c is a blank space, *i.e.* a space or a tab, *false* otherwise.

Usage

eof
newline
null
tab

Signature

eof,newline,null,tab: %

Returns

eof is the end-of-file character, newline is the newline character, null is the 0-character (used to terminate strings) and tab is the tab character.

Usage

lower c
upper c

Signature

lower,upper: % $\rightarrow$ %

Parameter	Type	Description
<i>c</i>	%	a character

Returns

lower(c) and upper(c) return c converted to lower, respectively upper, case.

File

Usage

import from File

Description

File is a type whose elements are operating system files.

Exports

close!:	% → MachineInteger	close a file
coerce:	% → BinaryReader	conversion to a binary input stream
coerce:	% → BinaryWriter	conversion to a binary output stream
coerce:	% → TextReader	conversion to a text input stream
coerce:	% → TextWriter	conversion to a text output stream
fileAppend:	MachineInteger	mode for open
fileBinary:	MachineInteger	mode for open
fileRead:	MachineInteger	mode for open
fileText:	MachineInteger	mode for open
fileWrite:	MachineInteger	mode for open
open:	(String, Z) → %	open a file
position:	% → Z	current position
remove:	String → ()	removes a file
setPosition!:	(%, Z) → Z	moves to a new position
setPositionFromEnd!:	(%, Z) → Z	moves to a new position
setRelativePosition!:	(%, Z) → Z	moves to a new position
uniqueName:	String → String	get a unique filename

where

Z == MachineInteger

Usage

close! f

Signature

close!: % $\rightarrow$ MachineInteger

Parameter	Type	Description
<i>f</i>	%	a file

Description

Closes the file *f*, returning 0 if no error occurred, a nonzero error code (the value of `errno` from C) otherwise.

Usage

```
f::BinaryReader
f::BinaryWriter
f::TextReader
f::TextWriter
```

Signatures

```
coerce: % → BinaryReader
coerce: % → BinaryWriter
coerce: % → TextReader
coerce: % → TextWriter
```

Parameter	Type	Description
<i>f</i>	%	a file

Description

Converts the file *f* to to a binary or text reader or writer. This is necessary before reading from or writing to the file. The file must have been opened in an appropriate mode for reading or writing respectively.

Remarks

Coercing a file to a reader or writer allocates memory, so it is advisable to assign the resulting stream to a variable. Unlike the ones for **String**, those coercions do not reset the file to its beginning.

Usage

fileAppend
fileBinary
fileRead
fileText
fileWrite

Signature

fileAppend,fileBinary,fileRead,fileText,fileWrite: `MachineInteger`

Description

Those constants are for use in the mode parameter of the `open` function.

Usage

`open(s,m)`

Signature

`open: (String, MachineInteger) → %`

Parameter	Type	Description
<i>s</i>	String	a filename
<i>m</i>	MachineInteger	a mode (optional)

Description

Opens the file with the name `s` in the mode `m`, and returns the opened file. The mode is any combination of the constants `fileAppend`, `fileRead`, and `fileWrite`, together with one of `fileBinary` or `fileText`, grouped together with `+` or `\`. The default is `fileRead + fileText`.

Remarks

`open` returns `nil` and throws the exception `FileException` if the file cannot be opened for any reason.

Usage

position *f*

Signature

position: % $\rightarrow$ MachineInteger

Parameter	Type	Description
<i>f</i>	%	a file

Returns

Returns the file position indicator of *f* (same as `ftell` in C).

See also

`setPosition!`

Usage

remove s

Signature

remove: `String` $\rightarrow$ `()`

Parameter	Type	Description
<i>s</i>	<code>String</code>	A file name

Description

Removes the file with name *s* in the file system.

Usage

setPosition!,setPositionFromEnd!,setRelativePosition!(f,n)

Signature

setPosition!,setPositionFromEnd!,setRelativePosition!: $(\%,Z) \rightarrow Z$

where

$Z == \text{MachineInteger}$

Parameter	Type	Description
f	$\%$	a file
n	MachineInteger	a signed offset

Description

setPosition!(f, n) sets the file position indicator of f to n , while setPositionFromEnd!(f, n) sets it to n from the end of file and setRelativePosition!(f, n) adds n to it. All three return 0 if no error occurred, a nonzero error code (the value of `errno` from C) otherwise.

See also

position

Usage

uniqueName s

Signature

uniqueName: `String` $\rightarrow$ `String`

Parameter	Type	Description
<i>s</i>	<code>String</code>	A root string

Returns

Returns a unique name with prefix *s* in the file system.

FileException

Usage

```
throw FileException  
try ... catch E in { E has FileExceptionType => ... }
```

Description

FileException is an exception type thrown by file operations.

FileExceptionType

Usage

FileExceptionType: Category

Description

FileExceptionType is the category of exceptions thrown by file operations.

InputType

Usage

InputType: Category

Description

InputType is the category of types whose objects can be read in in text format.

Exports

<<: TextReader $\rightarrow$ % read using text encoding

Usage

<< s

Signature

<<: TextReader → %

Parameter	Type	Description
s	TextReader	an input stream

Returns

<< s reads an element of the current type in text format from the stream s and returns the element read.

OutputType

Usage

OutputType: Category

Description

OutputType is the category of types whose objects can be written onto text writers.

Exports

```
<<: (TextWriter, %) → TextWriter  write using text encoding
<<: % → TextWriter → TextWriter  write function for an element
```

Usage

```
s << x
<< x
```

Signatures

```
<<: (TextWriter, %) → TextWriter
<<: % → TextWriter → TextWriter
```

Parameter	Type	Description
<i>s</i>	TextWriter	an output stream
<i>x</i>	%	an object of the type

Returns

`s << x` writes `x` in text format to the stream `s` and returns `s` after the write, while `<< x` returns the function $s \rightarrow s << x$.

Example

```
import from TextWriter, MachineInteger, Character;
stdout << 65 << space;
writes "65 " to the standard output stream, which could also be obtained via:

import from TextWriter, MachineInteger, Character;
f := << 65;
f(stdout) << space;
```

SerializableType

Usage

SerializableType: Category

Description

SerializableType is the category of types whose objects can be read in and written out in binary mode.

Exports

```
<<: BinaryReader → %           read using binary encoding  
<<: (BinaryWriter, %) → BinaryWriter  write using binary encoding
```

Usage

out << x
<< in

Signatures

<<: (BinaryWriter, %) → BinaryWriter
<<: BinaryReader → %

Parameter	Type	Description
<i>in</i>	BinaryReader	an input stream
<i>out</i>	BinaryWriter	an output stream
<i>x</i>	%	an object of the type

Returns

out << x writes x in binary format to the stream out and returns the stream after the write, while << in reads an element of the type in binary format from the stream in and returns the element read.

SyntaxException

Usage

```
throw SyntaxException  
try ... catch E in { E has SyntaxExceptionType => ... }
```

Description

SyntaxException is an exception type thrown by read operations from a `TextReader`.

SyntaxExceptionType

Usage

SyntaxExceptionType: Category

Description

SyntaxExceptionType is the category of exceptions thrown by read operations from a **TextReader**.

TextReader

Usage

import from TextReader

Description

TextReader provides various input streams.

Exports

options:	SymbolTable MachineInteger	global options
push!:	(C, %) → ()	push back a character
read!:	% → C	read one character
readUntil!:	(StringBuffer, %, C) → C	read several characters
readWhile!:	(StringBuffer, %, C → Boolean) → C	read several characters
skipUntil!:	(%, C) → C	skip characters
skipWhile!:	(%, C → Boolean) → C	skip characters
stdin:	%	standard input stream
textReader:	((() → C, C → ())) → %	create a stream

where

C == Character

Usage

options

Signature

options: SymbolTable MachineInteger

Returns

Returns a table of global options for text input. Those options can be queried by your implementations of <<.

Usage

push!(c, s)

Signature

push!:: (Character,%) → ()

Parameter	Type	Description
<i>c</i>	Character	a character
<i>s</i>	%	a stream

Returns

Pushes the character *c* back onto *s*. One character of pushback is guaranteed, this function might however fail if it is called too many times on the same stream without intervening read or positionning calls.

Usage

read! s

Signature

read!: % → Character

Parameter	Type	Description
<i>s</i>	%	a stream

Returns

Reads a character from *s* and returns it. Returns `eof` if the end of file is reached.

Usage

```
readUntil!(str, s, c)
readWhile!(str, s, read?)
skipUntil!(s, c)
skipWhile!(s, read?)
```

Signatures

```
readUntil!: (StringBuffer, %, Character) → Character
readWhile!: (StringBuffer, %, Character → Boolean) → Character
skipUntil!: (%, Character) → Character
skipWhile!: (%, Character → Boolean) → Character
```

Parameter	Type	Description
<i>str</i>	StringBuffer	a buffer
<i>s</i>	%	a stream
<i>c</i>	Character	a character
<i>read?</i>	Character → Boolean	a predicate

Description

`readUntil!(str, s, c)` reads characters from *s* and stores them in *str*, until the character *c* is read. Returns *c* if it is read, `eof` otherwise. `readWhile!(str, s, read?)` reads characters from *s* and stores them in *str* as long as *read?* is *true* on the characters read. Returns either the first character read for which *read?* is *false* or `eof`. `skipUntil!` and `skipWhile!` work like `readUntil!` and `readWhile!` except that the characters read are not stored anywhere.

Usage

stdin

Signature

stdin: %

Returns

stdin is the standard input stream.

Usage

textReader(*f*, *g*)

Signature

textReader: $((\text{Character} \rightarrow \text{Character}) \rightarrow \text{Character} \rightarrow ()) \rightarrow \%$

Parameter	Type	Description
<i>f</i>	$() \rightarrow \text{Character}$	the single-character read function
<i>g</i>	$\text{Character} \rightarrow ()$	the single-character pushback function

Returns

Returns the input stream for which *f*() reads a character and *g*(*c*) pushes back the character *c*.

Example

textReader can be used to modify existing readers by “hooking” their **read!** and **push!** functions in order to customize their behavior. For example, the following function **counter** transforms any **TextReader** into one that keeps track of the number of characters and lines read from it.

```
macro COUNT == Record(chars:MachineInteger, lines:MachineInteger);

counter(rd:TextReader, ct:COUNT):TextReader == {
  ct.chars := ct.lines := 0;    -- initialize counts to 0
  textReader(readAndCount!(rd, ct), pushAndCount!(rd, ct));
}

local readAndCount!(rd:TextReader, ct:COUNT)():Character == {
  c := read! rd;
  if c ~= eof then ct.chars := next(ct.chars);
  if c = newline then ct.lines := next(ct.lines);
  c;
}

local pushAndCount!(rd:TextReader, ct:COUNT)(c:Character):() == {
  if c ~= eof then ct.chars := prev(ct.chars);
  if c = newline then ct.lines := prev(ct.lines);
  push!(c, rd);
}
```

With the above function, the statements

```
cnt:COUNT := [0, 0];
countin := counter(stdin, cnt);
```

produces the reader **countin** that reads from **stdin** while side-effecting the variable **cnt** in order to keep track of the characters and lines read.

TextWriter

Usage

import from TextWriter

Description

TextWriter provides various output streams.

Exports

flush!:	% → %	flush a stream
options:	SymbolTable MachineInteger	global options
stderr:	%	the standard error stream
stdout:	%	the standard output stream
textWriter:	(Character → ()) → %	create a stream
textWriter:	(Character → (), () → ()) → %	create a stream
write!:	(Character, %) → ()	write to a stream

Usage

flush! s

Signature

flush!: % → %

Parameter	Type	Description
<i>s</i>	%	a stream

Description

flush!(s) causes all previous values inserted into s to be really written and returns the stream. Has no effect on unbuffered streams, such as `stderr`.

Usage

options

Signature

options: SymbolTable MachineInteger

Returns

Returns a table of global options for text output. Those options can be queried by your implementations of <<.

Usage

stderr
stdout

Signature

stderr,stdout: %

Returns

stderr and stdout are the standard error and standard output streams respectively.

Usage

TextWriter f
TextWriter(f, g)

Signatures

TextWriter: (Character → ()) → %
TextWriter: (Character → (), () → ()) → %

Parameter	Type	Description
<i>f</i>	Character → ()	the single-character write function
<i>g</i>	() → ()	the flush function (optional)

Returns

Returns the output stream for which $f(c)$ writes the character c and such that $g()$ flushes the stream. If g is not given, then flushing the resulting stream has no effect.

Usage

write!(c, s)

Signature

write!: (Character, %) → ()

Parameter	Type	Description
<i>c</i>	Character	character to write
<i>s</i>	%	a stream

Returns

Writes the character *c* to the stream *s* and returns *c*.

WriterManipulator

Usage

import from WriterManipulator

Description

WriterManipulator provides manipulators for text or binary writers.

Exports

OutputType

<code><<:</code>	<code>(BinaryWriter, %) → BinaryWriter</code>	manipulate a binary writer
<code>endl:</code>	<code>%</code>	send a newline and flush the stream
<code>flush:</code>	<code>%</code>	flush the stream

Usage

out << x

Signature

<<: (BinaryWriter, %) → BinaryWriter

Parameter	Type	Description
<i>out</i>	BinaryWriter	an output stream
<i>x</i>	%	a manipulator

Returns

out << x takes the action given by x on the stream out and returns the stream after the action.

Usage

endl

Signature

endl: %

Description

Sending endl to a text or binary writer causes a **newline** to be sent to the stream and then the stream to be flushed, so `s << endl` is equivalent to `flush!(s << newline)`.

Usage

flush

Signature

flush: %

Description

Sending flush to a text or binary writer causes the stream to be flushed, so `s << flush` is equivalent to `flush!(s)`. Has no effect on unbuffered streams, such as `stderr`.

Array

Usage

import from Array T

Parameter	Type	Description
<i>T</i>	Type	the type of the array entries

Description

Array provides arrays of entries of type *T*, 0-indexed and without bound checking.

Exports

ArrayType(*T*, PrimitiveArray *T*)

ArrayException

Usage

```
throw ArrayException
```

```
try ... catch E in { E has ArrayExceptionType => ... }
```

Description

ArrayException is an exception type thrown by array access.

ArrayExceptionType

Usage

ArrayExceptionType: Category

Description

ArrayExceptionType is the category of exceptions thrown by array access.

ArrayType

Usage

ArrayType(*T*, *P*): Category

Parameter	Type	Description
<i>T</i>	Type	the type of the array entries
<i>P</i>	PrimitiveArrayType <i>T</i>	the type of the underlying data

Description

ArrayType is the category of arrays whose entries are of type *T* and whose underlying data is of type *P*.

Exports

BoundedFiniteLinearStructureType *T*

array:	(<i>P</i> , MachineInteger) → %	construction of an array
data:	% → <i>P</i>	access to raw data
new:	MachineInteger → %	creation of an array
resize!:	(%, MachineInteger) → %	resize an array
sort!:	(%, (<i>T</i> , <i>T</i>) → Boolean) → %	sort an array

if *T* has TotallyOrderedType then

TotallyOrderedType

binarySearch:	(<i>T</i> , %) → (Boolean, MachineInteger)	binary search
sort!:	% → %	sort an array

Usage

array(p, n)

Signature

array: (P, MachineInteger) → %

Parameter	Type	Description
<i>p</i>	P	a primitive array structure
<i>n</i>	MachineInteger	a number of elements

Returns

Returns an array containing the first n entries of p. No copying is made.

Usage

binarySearch(t, a)

Signature

binarySearch: (T, %) → (Boolean, MachineInteger)

Parameter	Type	Description
t	T	the value to search for
a	%	an array

Returns

Returns (found?, i) such that $0 \leq i < \#a$ and $t = a.i$ if found? is *true*. Otherwise, found? is *false* and:

- if $i < 0$ then $t < a.0$;
- if $i \geq \#a - 1$ then $t > a(\#a - 1)$;
- if $0 \leq i < \#a - 1$, then $a.i < t < a(i + 1)$.

The array a must be sorted in increasing order. If a is sorted with respect to a different order, it is still possible to use binary search, but from **BinarySearch**.

See also

linearSearch

Usage

data a

Signature

data: % $\rightarrow$ P

Parameter	Type	Description
<i>a</i>	%	an array

Returns

Returns the raw data of the array a. No copying is made. This function can be used for efficiency before accessing the elements of a inside a loop, or in order to pass the elements of a to a C function.

Usage

new n

Signature

new: MachineInteger $\rightarrow$ %

Parameter	Type	Description
n	MachineInteger	a nonnegative size

Returns

Returns an array of n uninitialized entries.

Usage

```
resize!(a, n)
```

Signature

```
resize!:  (%MachineInteger) → %
```

Parameter	Type	Description
a	%	an array
n	MachineInteger	a nonnegative size

Returns

Returns an array of n entries, whose first $\#a$ entries are the first $\#a$ entries of a and whose remaining entries are uninitialized.

Usage

```
sort! a
sort!(a, f)
```

Signature

```
sort!:  (%,(T,T) → Boolean) → %
```

Parameter	Type	Description
a	%	a primitive array
f	$(T, T) \rightarrow \text{Boolean}$	a comparison function

Description

Sorts the array a using the ordering $x < y \iff f(x, y)$. The comparison function f is optional if T has `TotallyOrderedType`, in which case the order function of T is taken.

BoundedFiniteDataStructureType

Usage

BoundedFiniteDataStructureType T: Category

Parameter	Type	Description
T	Type	the type of the entries

Description

BoundedFiniteDataStructureType is the category of finite general structures whose entries are of type T and whose size is always known.

Exports

CopyableType
DataStructureType
#: % → MachineInteger number of entries
generator: % → Generator T iteration over a structure

if T has PrimitiveType then
 findAll: (T, %) → Generator Cross(MachineInteger, T) linear search
 member?: (T, %) → Boolean look for a value

if T has HashType then
 HashType

if T has OutputType then
 OutputType

Usage
a

Signatures
#: % → MachineInteger

Parameter	Type	Description
<i>a</i>	%	a finite data structure

Returns
Returns the number of entries in the structure *a*.

Usage

for pair in findAll(*t*, *a*) repeat { (*pos*, *val*) := pair; ... }

Signature

findAll: (*T*, %) → Generator Cross(MachineInteger, *T*)

Parameter	Type	Description
<i>t</i>	<i>T</i>	the value to search for
<i>a</i>	%	a finite data structure

Description

Iterates through all pairs (i, x) such that $t = x$ (using the equality of the type T). The index i is the position of x in the iteration of t by the function **generator**: $i = 1$ means x is the first element generated, $i = 2$ means x is the second element generated, etc.

See also

member?

Usage

for x in a repeat { ... }
 for x in generator a repeat { ... }

Signature

generator: % $\rightarrow$ Generator T

Parameter	Type	Description
<i>a</i>	%	a finite data structure

Description

This function allows a structure to be iterated independently of its representation. This generator yields the elements of *a* in some order, which is determined by the actual type.

Example

The following code computes the sum of all the elements of an array of machine integers:

```
sum(a:Array MachineInteger, n:MachineInteger):MachineInteger == {
  s:MachineInteger := 0;
  for x in a repeat s := s + x;
  s;
}
```


Usage

member?(t, a)

Signature

member?: (T, %) → Boolean

Parameter	Type	Description
<i>t</i>	T	the value to search for
<i>a</i>	%	a finite data structure

Returns

Returns *true* if t is a member of a, *false* otherwise.

BoundedFiniteLinearStructureType

Usage

BoundedFiniteLinearStructureType T: Category

Parameter	Type	Description
T	Type	the type of the entries

Description

BoundedFiniteLinearStructureType is the category of finite linear structures whose entries are of type T and whose size is always known.

Exports

BoundedFiniteDataStructureType T
FiniteLinearStructureType T
map: $(T \rightarrow T) \rightarrow \% \rightarrow \%$ lift a mapping
map!: $(T \rightarrow T) \rightarrow \% \rightarrow \%$ lift a mapping

if T has PrimitiveType then
PrimitiveType
linearSearch: $(T, \%) \rightarrow (\text{Boolean}, \text{MachineInteger}, T)$ linear search
 $(T, \%, \text{MachineInteger}) \rightarrow (\text{Boolean}, \text{MachineInteger}, T)$

if T has InputType then
InputType

if T has SerializableType then
SerializableType

Usage

linearSearch(*t*, *a*)
linearSearch(*t*, *a*, *n*)

Signature

linearSearch: (T, %, MachineInteger) → Boolean, MachineInteger, T

Parameter	Type	Description
<i>t</i>	T	the value to search for
<i>a</i>	%	a finite linear structure
<i>n</i>	MachineInteger	an initial index (optional)

Returns

Returns (found?, *i*, *a.i*) such that $t = a.i$ if found? is *true*, *t* is not in *a* otherwise. If the optional argument *n* is present, then the search starts at the entry *a.n* and ignores the previous ones.

See also

findAll

Usage

```
map f
map! f
map(f)(a)
map!(f)(a)
```

Signature

```
map: (T → T) → % → %
```

Parameter	Type	Description
<i>f</i>	$T \rightarrow T$	a map
<i>a</i>	%	a finite linear structure

Returns

map(f)(a) returns the new structure [f(x) for x in a], while map(f) returns the mapping $a \rightarrow [f(x) \text{ for } x \text{ in } a]$. In both cases, map! does not make a copy of the structure a but modifies it in place.

CheckingArray

Usage

import from CheckingArray T

Parameter	Type	Description
<i>T</i>	Type	the type of the array entries

Description

CheckingArray provides arrays of entries of type *T*, 0-indexed and with bound checking.

Exports

ArrayType(*T*, PrimitiveArray *T*)

Remarks

The functions `apply` and `set!` throw the exception `ArrayException` when attempting to access an array out of its bounds.

CheckingList

Usage

import from CheckingList T

Parameter	Type	Description
T	Type	the type of the list entries

Description

CheckingList provides lists of entries of type T , 1-indexed and with bound checking.

Exports

ListType T

CheckingMemoryBlock

Usage

```
import from CheckingMemoryBlock
```

Description

CheckingMemoryBlock provides packed arrays of bytes, 0-indexed and with bound checking.

Exports

```
ArrayType(Byte, PrimitiveMemoryBlock)
```

Remarks

The functions `apply` and `set!` throw the exception `ArrayException` when attempting to access a memory block out of its bounds.

DataSetructureType

Usage

DataSetructureType: Category

Description

DataSetructureType is the category of general data structures, not necessarily finite or linear.

Exports

empty?:: % → Boolean test whether a structure is empty
free!:: % → () memory disposal

Usage

empty? a

Signature

empty?: % → Boolean

Parameter	Type	Description
<i>a</i>	%	a data structure

Returns

Returns *true* if *a* contains no element, *false* otherwise.

Usage

free! a

Signature

free!: % → ()

Parameter	Type	Description
<i>a</i>	%	a data structure

Description

Releases the memory occupied by *a*.

DynamicDataStructureType

Usage

DynamicDataStructureType T: Category

Parameter	Type	Description
T	Type	the type of the entries

Description

DynamicDataStructureType is the category of finite general structures in which entries of type T can be inserted or removed dynamically.

Exports

BoundedFiniteDataStructureType T

insert: (T, %) → % add an element

insert!: (T, %) → % add an element

if T has PrimitiveType then

remove: (T, %) → % remove an element

remove!: (T, %) → % remove an element

removeAll: (T, %) → % remove all occurrences of an element

removeAll!: (T, %) → % remove all occurrences of an element

Usage

insert(*t*, *a*)
insert!(*t*, *a*)

Signature

insert: (T, %) → %

Parameter	Type	Description
<i>t</i>	T	an element to add
<i>a</i>	%	a dynamic data structure

Description

Adds *t* to *a* and returns the new structure. insert creates a new structure while insert! modifies *a* itself.

Usage

remove(*t*, *a*)
remove!(*t*, *a*)
removeAll(*t*, *a*)
removeAll!(*t*, *a*)

Parameter	Type	Description
<i>t</i>	T	an element to remove
<i>a</i>	%	a dynamic data structure

Description

remove(*t*, *a*) and remove!(*t*, *a*) remove the first occurrence of *t* in *a* and return the new structure, while removeAll(*t*, *a*) and removeAll!(*t*, *a*) remove all the occurrences of *t* in *a*. remove and removeAll create a new structure, while remove! and removeAll! modify *a* itself.

FiniteLinearStructureType

Usage

FiniteLinearStructureType T: Category

Parameter	Type	Description
T	Type	the type of the entries

Description

FiniteLinearStructureType is the category of finite linear structures whose entries are of type T .

Exports

LinearStructureType T		
<code>[]:</code>	<code>Tuple T → %</code>	construction of a structure
<code>empty:</code>	<code>%</code>	empty structure
<code>new:</code>	<code>(MachineInteger, T) → %</code>	creation of a structure

Usage

$[t_1, \dots, t_n]$

Signature

$[]: \text{ Tuple } T \rightarrow \%$

Parameter	Type	Description
$t_1, \dots, t_n$	T	elements of T

Returns

Returns the structure $[t_1, \dots, t_n]$.

Usage

empty

Signature

empty: %

Returns

Returns an empty structure.

See also

empty?

Usage

new(*n*, *x*)

Signature

new: (MachineInteger, T) → %

Parameter	Type	Description
<i>n</i>	MachineInteger	a nonnegative size
<i>x</i>	T	an entry

Returns

Returns a structure of *n* entries, all of them set to *x*.

Flags

Usage

import from Flags

Description

Flags is a type whose elements are 1-indexed contiguous blocks of boolean flags. The functionality is similar to using arrays of `Boolean`, but the implementation is more compact, each flag using only one machine bit.

Exports

`PrimitiveArrayType Boolean`

`dimension: (% , Z) → Z` number of flags that are set

where

`Z == MachineInteger`

Signature

dimension: (`%`, `MachineInteger`) $\rightarrow$ `MachineInteger`

Usage

dimension(`b`, `n`)

Parameter	Type	Description
<i>b</i>	<code>%</code>	A block of flags
<i>n</i>	<code>MachineInteger</code>	An index

Returns

Returns the number of flags that are set whose index is between 1 and *n*.

HashTable

Usage

```
import from HashTable(K, V)
import from HashTable(K, V, h)
```

Parameter	Type	Description
K	PrimitiveType HashType	the type of the keys
V	Type	the type of the entries
h	$K \rightarrow \text{MachineInteger}$	the hash function to use

Description

HashTable provides hash tables with keys of type K , entries of type V and that uses the hash-function h . If K has HashType, then the parameter h is optional as the function `hash` is used by default in that case.

Exports

```
TableType(K, V)
forget:    (K  $\rightarrow$  V)  $\rightarrow$  %   creation of a table with default
remember: (K  $\rightarrow$  V)  $\rightarrow$  %   creation of a table with default
remove!:  (K, %)  $\rightarrow$  %         removal of a key
```

Usage

forget *f*
remember *f*

Signature

forget,remember: $(K \rightarrow V) \rightarrow \%$

Parameter	Type	Description
<i>f</i>	$K \rightarrow V$	a default function

Description

Creates and return an empty table with default function *f*. That function is used by **apply** and **find** to compute the value of the entry whenever the key is not already present in the table. In addition, the pair (k, v) is entered into the table if **remember** was used to create it. Thus the result of **remember** *f* can be viewed as a version of *f* that remembers the values of its previous calls.

See also

table

Usage

remove!(*k*, *t*)

Parameter	Type	Description
<i>k</i>	K	a key to remove
<i>t</i>	%	a hash table

Returns

Removes the occurrence of *k* in *t* if any, and returns *t* after the removal.

KeyEntry

Usage

import from KeyEntry(K, V)

Parameter	Type	Description
K	Type	the type of the keys
V	Type	the type of the entries

Description

KeyEntry(K, V) provides pairs consisting of a key from K , and a entry from V . When K is a `PrimitiveType`, then two pairs are equal if they share the same key. When K is a `TotallyOrderedType`, then the pair x is greater than the pair y if $key(x) > key(y)$. Hence key-entry pairs are useful for building tables of entries where each slot is given by a unique key.

Exports

CopyableType

<code>[]:</code>	<code>(K, V) → %</code>	construction of a key-entry pair
<code>entry:</code>	<code>% → V</code>	get the entry
<code>explode:</code>	<code>% → (K, V)</code>	get the key and the entry
<code>key:</code>	<code>% → K</code>	get the key
<code>free!:</code>	<code>% → ()</code>	memory disposal
<code>setEntry!:</code>	<code>(%, V) → V</code>	change the entry
<code>setKey!:</code>	<code>(%, K) → K</code>	change the key

if K has `PrimitiveType` then

`PrimitiveType`

if K has `TotallyOrderedType` then

`TotallyOrderedType`

Usage
[*k*,*v*]

Signature
[]: (K, V) → %

Parameter	Type	Description
<i>k</i>	K	a key
<i>v</i>	V	an entry

Returns
Returns the key-entry pair [*k*, *v*].

Usage

entry p
(k, v) := explode p
key p

Signatures

entry: % $\rightarrow$ V
explode: % $\rightarrow$ (K, V)
key: % $\rightarrow$ K

Parameter	Type	Description
<i>p</i>	%	a key-entry pair

Returns

entry(*p*) and key(*p*) return respectively the entry and key of *p*, while explode(*p*) returns the pair (key *p*, entry *p*).

Usage

setEntry!(p, v)
setKey!(p, k)

Signatures

setEntry!: ($\%$, V) $\rightarrow$ V
setKey!: ($\%$, K) $\rightarrow$ K

Parameter	Type	Description
p	$\%$	a key-entry pair
k	K	a key
v	V	an entry

Description

setEntry!(p, v) (resp. setKey!(p, k)) changes the entry (resp. key) of p to v (resp. k) and returns v (resp. k).

HashType

Usage

HashType: Category

Description

HashType is the category of types whose objects can be hashed into machine integers.

Exports

PrimitiveType

hash: % $\rightarrow$ MachineInteger hash function

Usage

hash x

Signature

hash: % → MachineInteger

Parameter	Type	Description
<i>x</i>	%	an element of the type

Returns

Returns a hash-code for x.

LinearStructureType

Usage

LinearStructureType T: Category

Parameter	Type	Description
T	Type	the type of the entries

Description

LinearStructureType is the category of linear structures whose entries are of type T.

Exports

DataStructureType

<code>[]:</code>	<code>Generator T → %</code>	construction of a structure
<code>apply:</code>	<code>(%, MachineInteger) → T</code>	extraction of an entry
<code>firstIndex:</code>	<code>MachineInteger</code>	index of first element
<code>set!:</code>	<code>(%, MachineInteger, T) → T</code>	modification of an entry

if T has PrimitiveType then

<code>equal?:</code>	<code>(%, %, MachineInteger) → Boolean</code>	compare the first n elements
----------------------	---	--------------------------------

Usage
[t for t in g]

Signature
[]: Generator T → %

Parameter	Type	Description
<i>g</i>	Generator T	an iterator producing elements of T

Returns
Returns the structure composed of all the elements generated by g in the order in which they are generated.

Usage

apply(*a*, *n*)
a.n

Signature

apply: (*%*, MachineInteger) → T

Parameter	Type	Description
<i>a</i>	<i>%</i>	a linear structure
<i>n</i>	MachineInteger	an index

Returns

Returns the element of *a* with index *n*. The position of that element depends on the indexing scheme of each actual type, for example if it is 0-indexed, then *a.n* returns the $(n + 1)^{\text{st}}$ element of *a*, while if it is 1-indexed, then *a.n* returns the n^{th} element of *a*.

Remarks

Bound checking depends on the actual type, some perform it and some do not.

Usage

firstIndex

Signature

firstIndex: MachineInteger

Returns

Returns the index of the first element of a structure.

Usage

```
set!(a, n, x)
a.n := x;
```

Signature

```
set!: (% , MachineInteger, T) → T
```

Parameter	Type	Description
a	%	a linear structure
n	MachineInteger	an index
x	T	an entry

Description

Sets the element of a with index n to x and returns x . The position of that element depends on the indexing scheme of each actual type, for example if it is 0-indexed, then $a.n := x$ sets the $(n + 1)^{\text{st}}$ element of a , while if it is 1-indexed, then $a.n := x$ sets the n^{th} element of a .

Remarks

Bound checking depends on the actual type, some perform it and some do not.

Usage

equal?(a, b, n)

Signature

equal?: (% , % , MachineInteger) → Boolean

Parameter	Type	Description
<i>a</i> , <i>b</i>	%	linear structures
<i>n</i>	MachineInteger	a nonnegative integer

Returns

Returns *true* if the first *n* elements of *a* and *b* are all equal, *false* otherwise.

Remarks

a and *b* must both have at least *n* elements.

List

Usage

import from List T

Parameter	Type	Description
T	Type	the type of the list entries

Description

List provides lists of entries of type T , 1-indexed and without bound checking.

Exports

ListType T

ListException

Usage

```
throw ListException  
try ... catch E in { E has ListExceptionType => ... }
```

Description

ListException is an exception type thrown by list access.

ListExceptionType

Usage

ListExceptionType: Category

Description

ListExceptionType is the category of exceptions thrown by list access.

ListType

Usage

ListType T: Category

Parameter	Type	Description
T	Type	the type of the list entries

Description

ListType is the category of lists of entries of type T .

Exports

BoundedFiniteLinearStructureType T

DynamicDataStructureType T

<code>+</code>	<code>(%, MachineInteger) → %</code>	translate the base
<code>append!</code>	<code>(%, T) → %</code>	adds an entry at the end
<code>append!</code>	<code>(%, %) → %</code>	adds a list at the end
<code>cons</code>	<code>(T, %) → %</code>	adds an entry at the front
<code>delete!</code>	<code>(%, MachineInteger) → %</code>	remove an entry
<code>first</code>	<code>% → T</code>	first entry
<code>merge!</code>	<code>(%, %, (T, T) → Boolean) → %</code>	merge sorted lists
<code>rest</code>	<code>% → %</code>	all entries after the first
<code>reverse</code>	<code>% → %</code>	reverse a list
<code>reverse!</code>	<code>% → %</code>	reverse a list in-place
<code>setFirst!</code>	<code>(%, T) → T</code>	changes the first element of a list
<code>setRest!</code>	<code>(%, %) → %</code>	changes the rest of a list
<code>sort!</code>	<code>(%, (T, T) → Boolean) → %</code>	sort a list
<code>split!</code>	<code>(%, T → Boolean) → (% , %)</code>	split a list

if T has PrimitiveType then

`find`: `(T, %) → (% , MachineInteger)` linear search

if T has TotallyOrderedType then

TotallyOrderedType

`merge!`: `(%, %) → %` merge sorted lists

`sort!`: `% → %` sort a list

Usage
 $l + n$
Signature
 $+: (\%, \text{MachineInteger}) \rightarrow \%$

Parameter	Type	Description
l	$\%$	a list
n	<code>MachineInteger</code>	an index

Returns

$l+n$ returns the sublist of l starting at the $(n+1)^{\text{st}}$ element of l , *i.e.* $l+0$ returns l , $l+1$ returns the sublist starting at the second element of l , etc. . . . No copy of l is made.

Example

If l is the list of `MachineInteger` `[1, 2, 3, 4, 5]`, then $l+2$ is the list `[3, 4, 5]`.

Usage

```
append!(l, x)
append!(l, s)
```

Signatures

```
append!:: (%, T) → %
append!:: (%, %) → %
```

Parameter	Type	Description
l, s	%	lists
x	T	an entry

Returns

append!(l,x) and append!(l,s) return the lists $[l, x]$ and $[l, s]$ respectively.

Remarks

append! does not make a copy of l , which is therefore modified after the call. It is unsafe to use the variable l after the call, unless it has been assigned to the result of the call, as in `l := append!(l, x)`.

See also

cons

Usage

`cons(x, l)`

Signature

`cons: (T, %) → %`

Parameter	Type	Description
x	T	an entry
l	%	a list

Returns

Returns the list $[x, l]$. Does not make a copy of l .

See also

`append!`

Usage

delete!(l, n)

Signature

delete!: ($\%$, MachineInteger) $\rightarrow$ $\%$

Parameter	Type	Description
l	$\%$	a list
n	MachineInteger	an index

Returns

Returns the list l with $l.n$ removed. Does not make a copy of l .

Usage

find(*t*, *l*)

Signature

find: (T, %) → (% , MachineInteger)

Parameter	Type	Description
<i>t</i>	T	the value to search for
<i>l</i>	%	a list

Returns

Returns (*b*, *i*) such that:

- if *b* is not **empty**, then $l.i = t$ is the first occurrence of *t* in *l*, and *b* is the sublist of *l* starting at *l.i*,
- if *b* is **empty**, then *i* is undefined and *t* does not occur in *l*.

The list *a* does not need to be sorted, and the complexity is expected to be linear in its size.

Remarks

No copy of *l* is made: if *b* is not **empty**, then its data is shared with *l*. This function allows all the occurrences of *t* to be found successively in a list, as in the example below.

Example

If *l1* is the list of MachineInteger [1,6,2,5,3,7,2,4], then (*l2*, *n*) := find(2, *l1*) sets *l2* to [2,5,3,7,2,4] and *n* to 3, the further call (*l3*, *n*) := find(2, rest *l2*) sets *l3* to [2,4] and *n* to 4, and the further call (*l4*, *n*) := find(2, rest *l3*) sets *l4* to **empty**.

Usage

first l

Signature

first: % $\rightarrow$ T

Parameter	Type	Description
<i>l</i>	%	a nonempty list

Returns

Returns the first entry of *l*.

Usage

```
merge!(l1, l2)
merge!(l1, l2, f)
```

Signature

```
merge!: (%,%,(T,T) → Boolean) → %
```

Parameter	Type	Description
l_1, l_2	%	sorted lists
f	$(T, T) \rightarrow \text{Boolean}$	a comparison function

Description

Given sorted $l_i = [t_{i1}, t_{i2}, \dots]$ where $f(t_{ij}, t_{ik})$ is *true* whenever $j < k$, returns the list containing all the elements of l_1 and l_2 , sorted using the ordering $x < y \iff f(x, y)$. The comparison function f is optional if T has `TotallyOrderedType`, in which case the order function of T is taken.

Remarks

`merge!` does not make copies of l_1 and l_2 , which are therefore modified after the call. It is unsafe to use the variable l_1 and l_2 after the call, unless the result of the call has been assigned, as in `l := merge!(l1, l2)`.

See also

```
sort!
```

Usage

rest l

Signature

rest: % $\rightarrow$ %

Parameter	Type	Description
<i>l</i>	%	a nonempty list

Returns

Returns *l* with the first element removed. Does not make a copy of *l*.

See also

setRest!

Usage

reverse l
reverse! l

Signature

reverse: % $\rightarrow$ %

Parameter	Type	Description
<i>l</i>	%	a list

Returns

reverse(l) returns a copy of *l* with the elements in reverse order, while reverse!(l) reverses *l* without copying it.

Remarks

reverse! does not make a copy of *l*, which is therefore modified after the call. It is unsafe to use the variable *l* after the call, unless it has been assigned to the result of the call, as in `l := reverse! l`.

Usage`setFirst!(l,t)`**Signature**`setFirst!:` $(\%,T) \rightarrow T$

Parameter	Type	Description
l	$\%$	a nonempty list
t	$\%$	an element

Description

Replaces the first element of l by t and returns t .

Remarks

`setFirst!(l,t)` does not make a copy of l , which is therefore modified after the call.

See also`setRest!`

Usage

setRest!(l,t)

Signature

setRest!: ($\%$, $\%$) $\rightarrow$ $\%$

Parameter	Type	Description
l	$\%$	a nonempty list
t	$\%$	a list

Description

Replaces l by the list `[first(l),t]` and returns t .

Remarks

setRest!(l,t) does not make a copy of l , which is therefore modified after the call.

See also

setFirst!

Usage

```
sort! l
sort!(l, f)
```

Signature

```
sort!: (%,(T,T) → Boolean) → %
```

Parameter	Type	Description
l	%	a list
f	$(T, T) \rightarrow \text{Boolean}$	a comparison function

Description

Sorts the list a using the ordering $x < y \iff f(x, y)$. The comparison function f is optional if T has `TotallyOrderedType`, in which case the order function of T is taken.

Remarks

`sort!` does not make a copy of l , which is therefore modified after the call. It is unsafe to use the variable l after the call, unless it has been assigned to the result of the call, as in `l := sort! l`.

See also

```
merge!
```

Usage

split!(l, f)

Signature

split!: ($\%$, $T \rightarrow \text{Boolean}$) $\rightarrow$ ($\%$, $\%$)

Parameter	Type	Description
l	$\%$	a list
f	$T \rightarrow \text{Boolean}$	a predicate

Returns

Returns sublists l_t and l_f of l such that l_t is the list of elements of l for which f is *true*, and l_f is the list of elements of l for which f is *false*.

Remarks

split! does not make a copy of l , which is therefore destroyed after the call. No new list is created by split!, but the elements of l are relinked into l_t and l_f .

MemoryBlock

Usage

```
import from MemoryBlock
```

Description

MemoryBlock provides packed arrays of bytes, 0-indexed and without bound checking (the debug version of `libaldor` provides bound-checking for memory blocks).

Exports

```
ArrayType(Byte, PrimitiveMemoryBlock)
```

PackedPrimitiveArray

Usage

import from PackedPrimitiveArray T

Parameter	Type	Description
<i>T</i>	PackableType	the type of the array entries

Description

PackedPrimitiveArray provides packed arrays of entries of type *T*, 0-indexed and without bound checking (the debug version of libaldor provides bound-checking for packed primitive arrays).

Exports

PrimitiveArrayType T

PrimitiveArray

Usage

import from PrimitiveArray T

Parameter	Type	Description
T	Type	the type of the array entries

Description

PrimitiveArray provides arrays of entries of type T , 0-indexed and without bound checking (the debug version of libaldor provides bound-checking for primitive arrays).

Exports

PrimitiveArrayType T

PrimitiveArrayType

Usage

import from PrimitiveArrayType T

Parameter	Type	Description
T	Type	the type of the array entries

Description

PrimitiveArrayType is the category of primitive arrays whose entries are of type T .

Exports

FiniteLinearStructureType T

array: (Pointer, Z) → % conversion from a C-array

new: Z → % creation of an array

pointer: % → Pointer conversion to a C-array

resize!: (% , Z, Z) → % resize an array

sort!: (% , Z, Z, (T, T) → Boolean) → % sort an array

if T has SerializableType then

read: (BinaryReader, Z) → % read using binary encoding

write: (BinaryWriter, % , Z) → BinaryWriter write using binary encoding

if T has TotallyOrderedType then

sort!: (% , Z, Z) → % sort an array

where

Z == MachineInteger

Usage

array(*p*, *n*)
 pointer *a*

Signatures

array: (Pointer, MachineInteger) → %
 pointer: % → Pointer

Parameter	Type	Description
<i>p</i>	Pointer	a C-array
<i>n</i>	MachineInteger	a nonnegative size
<i>a</i>	%	A primitive array

Description

Use those functions to safely convert between pointers returned or needed by C-functions and primitive arrays. Those function have no effect in the release version of `libaldor`, but they are necessary when using the debug version, so it is recommended to use them in all cases.

Usage

new n

Signature

new: MachineInteger → %

Parameter	Type	Description
<i>n</i>	MachineInteger	a nonnegative size

Returns

Returns a primitive array of *n* uninitialized entries.

Usage

read(*in*, *n*)
write(*out*, *a*, *n*)

Signatures

read: (BinaryReader, MachineInteger) → %
write: (BinaryWriter, %, MachineInteger) → BinaryWriter

Parameter	Type	Description
<i>in</i>	BinaryReader	an input stream
<i>out</i>	BinaryWriter	an output stream
<i>n</i>	MachineInteger	a nonnegative size
<i>a</i>	%	A primitive array

Returns

read(*in*, *n*) reads a primitive array of *n* elements in binary format from the stream *in* and returns the array read, while write(*out*, *a*, *n*) writes the first *n* elements of *a* to the stream *out* and returns the stream after the write.

Usage

```
resize!(a, n, m)
```

Signature

```
resize!:: (%,MachineInteger,MachineInteger) → %
```

Parameter	Type	Description
<i>a</i>	%	a primitive array
<i>n, m</i>	MachineInteger	nonnegative sizes

Returns

Returns a primitive array of m entries, whose first n entries are the first n entries of a and whose remaining entries are uninitialized.

Remarks

resize! may free the space previously used by a , so it is unsafe to use the variable a after the call, unless it has been assigned to the result of the call, as in `a := resize!(a, n, m)`.

Usage

```
sort!(a, n, m)
sort!(a, n, m, f)
```

Signature

```
sort!: (%MachineInteger,MachineInteger, (T,T) → Boolean) → %
```

Parameter	Type	Description
a	%	a primitive array
n, m	MachineInteger	indices
f	$(T, T) \rightarrow \text{Boolean}$	a comparison function

Description

Sorts the subarray $[a.n, \dots, a.m]$ using the ordering $x < y \iff f(x, y)$. The comparison function f is optional if T has `TotallyOrderedType`, in which case the order function of T is taken.

PrimitiveMemoryBlock

Usage

import from PrimitiveMemoryBlock

Description

PrimitiveMemoryBlock provides packed arrays of bytes, 0-indexed and without bound checking (the debug version of `libaldor` provides bound-checking for primitive memory blocks).

Exports

PrimitiveArrayType Byte

coerce: % → BinaryReader conversion to a binary input stream

coerce: % → BinaryWriter conversion to a binary output stream

Usage

a::BinaryReader
a::BinaryWriter

Signatures

coerce: % → BinaryReader
coerce: % → BinaryWriter

Parameter	Type	Description
<i>a</i>	%	a primitive memory block

Description

a::T where T is an I/O stream type converts the block s to a binary reader or writer, allowing one to read data or write data to it.

Remarks

When writing to a memory block, you must ensure that the block is large enough for all the data that will be written to it, since the block will not be extended and this function does not protect you against overwriting. When reading from or writing to a memory block, each coercion to a reader or writer resets the stream to the beginning of the block, and the block is not side-affected by the subsequent read or write operations, while the stream is side-affected. Thus, when reading several values from the same block, you must assign the reader to a variable and read the values from that variable.

See also

coerce

.

Queue

Usage

import from Queue T

Parameter	Type	Description
T	Type	the type of the entries

Description

Queue provides queues of entries of type T , 1-indexed and without bound checking.

Exports

BoundedFiniteLinearStructureType T

DynamicDataStructureType T

remove!: % $\rightarrow$ T remove an element

Usage

remove! q

Signature

remove!: % $\rightarrow$ T

Parameter	Type	Description
<i>q</i>	%	a nonempty queue

Description

Removes the oldest element of the queue *q* and returns it.

Set

Usage

import from Set T

Parameter	Type	Description
T	PrimitiveType	the type of the set entries

Description

Set provides sets of entries of type T , 1-indexed and without bound checking.

Exports

BoundedFiniteLinearStructureType T

DynamicDataStructureType T

—: (% , %) → % set difference

intersection: (% , %) → % intersect two sets

intersection!: (% , %) → % intersect two sets

minus!: (% , %) → % set difference

union: (% , T) → % add an element

(% , %) → % add a set of elements

union!: (% , T) → % add an element

(% , %) → % add a set of elements

Usage

$x - y$

Signature

$-: (\%, \%) \rightarrow \%$

Parameter	Type	Description
x, y	$\%$	sets

Returns

Return $x - y = \{a \in x \text{ such that } a \notin y\}$.

See also

`minus!`

Usage

intersection(x,y)
intersection!(x,y)

Signature

intersection: (% , %) → %

Parameter	Type	Description
<i>x,y</i>	%	sets

Returns

Return $x \cap y = \{a \text{ such that } a \in x \text{ and } a \in y\}$.

Remarks

intersection! does not make a copy of x , which is therefore modified after the call. It is unsafe to use the variable x after the call, unless it has been assigned to the result of the call, as in `x := intersection!(x, y)`.

Usage

minus!(*x*,*y*)

Signature

minus!:: (*%*, *%*) → *%*

Parameter	Type	Description
<i>x</i> , <i>y</i>	<i>%</i>	sets

Description

Return $x - y = \{a \in x \text{ such that } a \notin y\}$.

Remarks

minus! does not make a copy of *x*, which is therefore modified after the call. It is unsafe to use the variable *x* after the call, unless it has been assigned to the result of the call, as in `x := minus! (x, y)`.

See also

—

Usage

```
union(x,t)
union(x,y)
union!(x,t)
union!(x,y)
```

Signature

```
union:  (% , %) → %
```

Parameter	Type	Description
x, y	%	sets
t	T	an element

Returns

`union(x,y)` and `union!(x,y)` both return $x \cup y = \{a \text{ such that } a \in x \text{ or } a \in y\}$, while `union(x,t)` and `union!(x,t)` both return $x \cup \{t\}$.

Remarks

`union!` does not make a copy of x , which is therefore modified after the call. It is unsafe to use the variable x after the call, unless it has been assigned to the result of the call, as in `x := union!(x, y)`.

SortedAssociationSet

Usage

import from SortedAssociationSet(K , V)

Parameter	Type	Description
K	TotallyOrderedType	the type of the keys
V	Type	the type of the entries

Description

SortedAssociationSet(K , V) implements sorted sets of key-entry pairs. The keys come from K with a total ordering and the entries come from T which is any type. A sorted association set can be viewed as a hash-table with the identity as hash-function.

Exports

TableType(K , V)

SortedList

Usage

import from SortedList T

Parameter	Type	Description
<i>T</i>	PartiallyOrderedType	the type of the entries

Description

SortedList(T) provides sorted lists whose entries belong to a partially ordered type. Duplicate entries are allowed.

Exports

- BoundedFiniteLinearStructureType T
- DynamicDataStructureType T

SortedSet

Usage

import from SortedSet T

Parameter	Type	Description
<i>T</i>	PartiallyOrderedType	the type of the entries

Description

SortedSet(T) provides sorted sets whose entries belong to a partially ordered type. Duplicate entries are not allowed.

Exports

BoundedFiniteLinearStructureType T
DynamicDataStructureType T

Stream

Usage

import from Stream T

Parameter	Type	Description
T	Type	the type of the stream entries

Description

Stream provides streams of entries of type T , 0-indexed and with bound checking.

Exports

LinearStructureType T		
#:	$\% \rightarrow Z$	number of computed elements
constant:	$\% \rightarrow (Z, \text{Partial } T)$	check for an eventually constant stream
generator :	$\% \rightarrow \text{Generator } T$	iteration over a stream
interlacing:	List $\% \rightarrow \%$	interlacing
map!:	$(T \rightarrow T) \rightarrow \% \rightarrow \%$	lift a mapping
orbit:	$(T, T \rightarrow T) \rightarrow \%$	creation of a stream
stream:	$T \rightarrow \%$	creation of a stream
	$(() \rightarrow T) \rightarrow \%$	
	$((Z, T) \rightarrow T) \rightarrow \%$	
	$(Z, Z \rightarrow T) \rightarrow \%$	
	$(Z, Z \rightarrow T, Z, T) \rightarrow \%$	
	$(\text{Generator } T, T) \rightarrow \%$	
	$(\text{Generator } T, Z, T) \rightarrow \%$	

where

$Z == \text{MachineInteger}$

if T has OutputType then

OutputType

Usage

s

Signatures

#: % $\rightarrow$ MachineInteger

Parameter	Type	Description
<i>s</i>	%	a stream

Returns

Returns the number of elements of *s* that have been computed.

Usage

constant s

Signature

constant: % $\rightarrow$ (MachineInteger, Partial T)

Parameter	Type	Description
<i>s</i>	%	a stream

Returns

Returns $(-1, failed)$ if *s* is not known to be eventually constant, otherwise returns $(n \geq 0, [t])$ such that $s.k = s.n = t$ for $k \geq n$.

Usage

for x in s repeat { ... }
 for x in generator s repeat { ... }

Signature

generator: % $\rightarrow$ **Generator** T

Parameter	Type	Description
<i>s</i>	%	a stream

Description

This function allows the elements of a stream to be iterated.

Remarks

Since those generators are infinite, you should have a termination condition either inside the loop or in parallel with the generator in order to guarantee termination.

Example

The following code creates the stream of all the squares for $n \geq 0$ and prints those of them that are smaller than 1000:

```
import from MachineInteger, Stream MachineInteger;
s := stream(0, (n:MachineInteger):MachineInteger +-> n^2);
for x in s while x < 1000 repeat { stdout << x << newline; }
```

Usage

interlacing $[s_1, \dots, s_n]$

Signature

interlacing: `List %` $\rightarrow$ `%`

Parameter	Type	Description
s_i	<code>%</code>	streams

Description

Given $s_i = t_{i1}, t_{i2}, \dots$, returns the stream $t_{11}, t_{21}, \dots, t_{n1}, t_{12}, t_{22}, \dots, t_{n2}, \dots$

Usage

map! f
map!(f)(s)

Signature

map!: $(T \rightarrow T) \rightarrow \% \rightarrow \%$

Parameter	Type	Description
f	$T \rightarrow T$	a map
s	$\%$	a stream

Returns

map!(f)(s) returns the stream `[f(x) for x in s]`, while map!(f) returns the mapping $s \rightarrow [f(x) \text{ for } x \text{ in } s]$. In both cases, the stream `s` is modified in place and no copy is made.

Usage

orbit(t, f)

Signature

orbit: (T, T → T) → %

Parameter	Type	Description
t	T	an element of T
f	T → T	a function

Returns

Returns the stream $[t, f(t), f^2(t), \dots]$.

See also

stream

Usage

```

stream t
stream f
stream h
stream(n, g)
stream(n, g, m, t)
stream(gen, t)
stream(gen, m, t)

```

Signatures

```

stream:  T → %
stream:  (() → T) → %
stream:  ((MachineInteger, %) → T) → %
stream:  (MachineInteger, MachineInteger → T) → %
stream:  (MachineInteger, MachineInteger → T, MachineInteger, T) → %
stream:  (Generator T, T) → %
stream:  (Generator T, MachineInteger, T) → %

```

Parameter	Type	Description
n, m	<code>MachineInteger</code>	machine integers
t	<code>T</code>	an element
f	<code>() → T</code>	a function
g	<code>MachineInteger → T</code>	a function
h	<code>(MachineInteger, %) → T</code>	a function
gen	<code>Generator T</code>	a generator

Description

`stream(t)` returns the constant stream $[t, t, t, \dots]$, while `stream(f)` returns the stream obtained by successive calls to `f()`, `stream(n, g)` returns the stream $[g(n), g(n+1), g(n+2), \dots]$, `stream(n, g, m, t)` returns the eventually constant stream $[g(n), g(n+1), g(n+2), \dots, g(m-1), t, t, t, \dots]$ and `stream(h)` returns the stream $[s_0, s_1, s_2, \dots]$ where $s_k = h(k, s_0, \dots, s_{i-k})$ for any $k \geq 0$. When using generators, `stream(gen, t)` returns the stream $[x \text{ for } x \text{ in } gen]$ followed by $[t, t, t, \dots]$ if `gen` is finite, while `stream(gen, m, t)` returns the first m elements of the stream $[x \text{ for } x \text{ in } gen]$ followed by $[t, t, t, \dots]$.

See also

`orbit`

Remarks

The function g must be defined for all $n \leq k$ (resp. $n \leq k < m$) and h must be defined for all $n \geq 0$, even if you intend to use `set!` to set specific values of the stream later. Note that `stream(f)` does not necessarily returns a constant stream since each call to `f()` can side-effect its environment. Constant or eventually constant streams do not repeatedly store their constant value and are therefore preferable. For example, `stream(0)` stores only one value no matter how many values are requested, while `stream(() : MachineInteger + - > 0)` stores n times 0 when its n^{th} element is requested.

Example

The following code creates the stream of the Fibonacci numbers and prints the first 10 of them:

```
import from MachineInteger, AldorInteger, Stream AldorInteger;
fib(n:MachineInteger, f:Stream AldorInteger):AldorInteger == {
    n = 0 or n = 1 => 1;
    f(n-1) + f(n-2);
}
sfib := stream fib;
a := sfib.9;    -- computes sfib.0 to sfib.9
stdout << a << newline;
```

String

Usage

import from String

Description

String provides basic strings, null-terminated, 0-indexed and without bound checking.

Exports

```
ArrayType(Character, PackedPrimitiveArray Character)
+:          (% , %) → %          concatenation
char:      % → Character        first character
coerce:    Character → %        conversion to a string
coerce:    % → TextReader       conversion to a text input stream
coerce:    % → TextWriter       conversion to a text output stream
error:     % → Exit             error exit
leftTrim:  (% , Character) → %  trim leading part
rightTrim: (% , Character) → %  trim trailing part
rightTrim!: (% , Character) → % trim trailing part
new:       Z → %                buffer allocation
pointer:   % → Pointer          conversion to a null-terminated C-string
string:    Pointer → %          conversion from a null-terminated C-string
substring: (% , Z) → %          substring
           (% , Z , Z) → %
```

where

```
Z == MachineInteger
```

Usage

`s + t`

Signature

`+: (% , %) → %`

Parameter	Type	Description
<code>s, t</code>	<code>%</code>	strings

Returns

`s+t` returns the string `st`. Copies all the characters of `s` and `t`, so `s` is unchanged after the call, and `s` and `t` do not share characters with `s + t`.

Remarks

`s + empty` and `empty + s` both return a copy of `s`.

Example

If `s` is the string “abcde”, then `s + (s + 2)` is the string “abcdecde”.

Usage

char s

Signature

char: % → Character

Parameter	Type	Description
<i>s</i>	%	a nonempty string

Description

Returns the first character of s.

Usage

```
c::%
s::TextReader
s::TextWriter
```

Signatures

```
coerce: Character → %
coerce: % → TextReader
coerce: % → TextWriter
```

Parameter	Type	Description
<i>c</i>	Character	a character
<i>s</i>	%	a string

Description

`c::String` converts the character `c` to the string “`c`”, while `s::T` where `T` is an I/O stream type converts the string `s` to a text reader or writer, allowing one to read data or write data to it.

Remarks

When writing to a string, you must ensure that the string is large enough for all the data that will be written to it, since the string will not be extended and this function does not protect you against overwriting. When reading from or writing to a string, each coercion to a reader or writer resets the stream to the beginning of the string, and the string is not side-affected by the subsequent read or write operations, while the stream is side-affected. Thus, when reading several values from the same string, you must assign the reader to a variable and read the values from that variable, as in the example below.

Example

```
import from MachineInteger, String;
s := " 12 56";
a:MachineInteger := << s::TextReader;
b:MachineInteger := << s::TextReader;
assigns the value 12 to both a and b, while

import from MachineInteger, String;
s := " 12 56";
p := s::TextReader;
a:MachineInteger := << p;
b:MachineInteger := << p;
assigns 12 to a and 56 to b.
```

See also

PrimitiveMemoryBlock

coerce

String

error

Usage

error s

Signature

error: % $\rightarrow$ Exit

Parameter	Type	Description
<i>s</i>	%	a string

Description

Writes the message s to **stderr** and exits.

Usage

leftTrim(s, c)
rightTrim(s, c)
rightTrim!(s, c)

Signature

leftTrim,rightTrim: (% , Character) → %

Parameter	Type	Description
<i>s</i>	%	a string
<i>c</i>	Character	a character

Description

leftTrim(s,c) (resp. rightTrim(s,c)) returns a copy of *s* without its leading (resp. trailing) occurrences of *c*. No copy is made and *s* is returned if the leading (resp. trailing) character of *s* is not *c*. rightTrim!(s,c) removes the trailing occurrences of *c* from *s* without making a copy.

Usage

new n

Signature

new: `MachineInteger` $\rightarrow$ %

Parameter	Type	Description
n	<code>MachineInteger</code>	a nonnegative size

Returns

Returns a string of $n + 1$ null characters. This is useful when creating a memory buffer of a specified number of bytes, or when creating a string to be used as an `TextWriter`.

Usage

pointer *s*
string *p*

Signatures

pointer: $\% \rightarrow \text{Pointer}$
string: $\text{Pointer} \rightarrow \%$

Parameter	Type	Description
<i>s</i>	$\%$	A libaldor string
<i>p</i>	Pointer	A null-terminated C-string

Description

Use those functions to safely convert between null-terminated returned or needed by C-functions and libaldor strings. Those functions have no effect in the release version of libaldor, but they are necessary when using the debug version, so it is recommended to use them in all cases. The C-type `char*` should be replaced by **Pointer** in the prototypes when using C-functions in libaldor clients.

Usage

substring(*s*, *n*)
substring(*s*, *n*, *m*)

Signatures

substring: (*%*, MachineInteger) → *%*
substring: (*%*, MachineInteger, MachineInteger) → *%*

Parameter	Type	Description
<i>s</i>	<i>%</i>	a string
<i>n</i>	MachineInteger	an index
<i>m</i>	MachineInteger	a length

Description

substring(*s*,*n*) returns a copy of the substring starting at position *n* of *s*, while substring(*s*,*n*,*m*) returns a copy of the substring of length *m* starting at position *n* of *s*.

Remarks

substring(*s*,0) returns a copy of *s*.

StringBuffer

Usage

import from StringBuffer

Description

StringBuffer provides extensible character buffers that can be used as input or output, either for reading an undetermined number of characters from a **TextReader** into them, or for writing an undetermined number of characters into them.

Exports

DataStructureType

OutputType

PrimitiveType

coerce: % → **TextReader** conversion to a text input stream

coerce: % → **TextWriter** conversion to a text output stream

empty: % create an empty string

string: % → **String** conversion to a regular string

Usage`s::TextReader``s::TextWriter`**Signatures**`coerce: % → TextReader``coerce: % → TextWriter`

Parameter	Type	Description
<i>s</i>	%	a string

Description

`s::T` where `T` is an I/O stream type converts the string `s` to a text reader or writer, allowing one to read data or write data to it.

Remarks

When writing to an extensible string, the string grows to accomodate all the data that you write to it.

Signature

new: $() \rightarrow \%$

Returns

Returns a new empty buffer.

Usage

string s

Signature

string: % $\rightarrow$ String

Parameter	Type	Description
<i>s</i>	%	a string

Description

Returns the underlying string.

Remarks

This function does not make a copy of the string, so if *s* is modified after calling `str := strings`, then *str* is affected. Use `copy` on *str* if you need to prevent this.

SymbolTable

Usage

import from SymbolTable T

Parameter	Type	Description
T	PrimitiveType	Type of the symbols in the table

Description

SymbolTable T provides symbol tables where all the symbols have type T.

Exports

<code>apply:</code>	<code>(%, Symbol) → Partial T</code>	Search for a symbol
<code>set!:</code>	<code>(%, Symbol, T) → T</code>	Add a symbol
<code>table:</code>	<code>() → %</code>	Create an empty table

Usage

apply(*t*, *x*)
t *x*

Signature

apply: (% , Symbol) → Partial T

Parameter	Type	Description
<i>t</i>	%	A symbol table
<i>x</i>	Symbol	A variable name

Returns

Returns the value that *x* has in *t* if it is found, *failed* otherwise.

Usage

set!(t, x, v)
t.x := v

Signature

set!: (% , Symbol, T) → T

Parameter	Type	Description
<i>t</i>	%	A symbol table
<i>x</i>	Symbol	A variable name
<i>v</i>	T	A value

Description

Assigns the value v to x in t. If x already had a value in t, the older value is lost.

Returns

Returns v.

Usage

table()

Signature

table: () → %

Returns

Returns an empty symbol table.

See also

set!

TableException

Usage

```
throw TableException
```

```
try ... catch E in { E has TableExceptionType => ... }
```

Description

TableException is an exception type thrown by table access.

TableExceptionType

Usage

TableExceptionType: Category

Description

TableExceptionType is the category of exceptions thrown by table access.

TableType

Usage

TableType(K, V): Category

Parameter	Type	Description
K	PrimitiveType	the type of the keys
V	Type	the type of the entries

Description

TableType(K, V) is the category of tables, *i.e.* discrete many-to-one mappings from keys to entries. More precisely, every element of a domain of this category is a table whose slots contain elements from V and such that every slot is given by a unique key from K .

Exports

BoundedFiniteDataStructureType Cross(K, V)

<code>[]:</code>	<code>Tuple Cross(K, V) → %</code>	creation of a table
<code>apply:</code>	<code>(%, K) → V</code>	extraction of an entry
<code>entries:</code>	<code>% → Generator V</code>	iterate through the entries
<code>find:</code>	<code>(K, %) → Partial V</code>	search for an entry
<code>keys:</code>	<code>% → Generator K</code>	iterate through the keys
<code>numberOfEntries:</code>	<code>% → MachineInteger</code>	number of entries
<code>set!:</code>	<code>(%, K, V) → V</code>	modification of an entry
<code>table:</code>	<code>() → %</code>	creation of a table
	<code>MachineInteger → %</code>	

if K has InputType and V has InputType then
InputType

if K has OutputType and V has OutputType then
OutputType

if K has SerializableType and V has SerializableType then
SerializableType

Usage

$[(k_1, v_1), \dots, (k_n, v_n)]$

Signature

`[]`: `Tuple Cross(K, V)` $\rightarrow$ %

Parameter	Type	Description
$k_1, \dots, k_n$	K	keys
$v_1, \dots, v_n$	V	values

Returns

Returns the table $[k_1 = t_1, \dots, k_n = v_n]$.

See also

`table`

Usage

apply(*t*, *k*)
t.k

Signature

apply: (*%*, *K*) → *V*

Parameter	Type	Description
<i>t</i>	<i>%</i>	a table
<i>k</i>	<i>K</i>	a key

Returns

Returns the element of *t* with key *k*, which must be present in the table.

Remarks

Produces an error if *k* is not in *t*, use **find** if it is not known whether *k* is present in table.

Usage

for v in entries t repeat {...}
for k in keys t repeat {...}

Signatures

entries: % → Generator V
keys: % → Generator K

Parameter	Type	Description
<i>t</i>	%	a table

Description

These generators yield respectively all the entries, or keys in the table *t*.

Usage

find(k, t)

Signature

find: (K, %) → Partial V

Parameter	Type	Description
k	K	a key
t	%	a table

Returns

Returns *failed* if there is no element with key k in t , the element of t with key k otherwise.

See also

apply

Usage

numberOfEntries t

Parameter	Type	Description
<i>t</i>	%	a table

Returns

Returns the actual number of entries in the table *t*. That number can be different from the size of the table.

Usage

set!(t, k, v)
t.k := v;

Signature

set!: (% , K, V) → V

Parameter	Type	Description
<i>t</i>	%	a table
<i>k</i>	K	a key
<i>v</i>	V	an entry

Returns

Sets the element of *t* with key *k* to *v* and returns *v*.

Usage

table()
table n

Signature

table: MachineInteger → %

Parameter	Type	Description
<i>n</i>	MachineInteger	a starting size (optional)

Returns

Returns an empty table with initial space for *n* entries. That space grows when needed as elements are inserted in the table.

See also

[]

AldorLibraryInformation

Usage

import from AldorLibraryInformation

Description

AldorLibraryInformation provides `libaldor` version information.

Exports

VersionInformationType

VersionInformationType

Usage

VersionInformationType: Category

Description

VersionInformationType is the category of types providing version information.

Exports

credits:	List String	various credits
major:	MachineInteger	major version number
minor:	MachineInteger	minor version number
name:	String	library name
patch:	MachineInteger	patch version number
version:	String	version information

Signature

credits: List String

Returns

Returns a list of lines, crediting the various authors of a library.

Usage

major,minor,patch

Signature

major,minor,patch: `MachineInteger`

Returns

Returns the major and minor version of a library as well as its patchlevel.

Usage

name

Signature

name: String

Returns

Returns the name of the library.

Usage

version

Signature

version: `String`

Returns

Returns a string describing the current version of a library.

BinarySearch

Usage

import from BinarySearch(R , S)

Description

BinarySearch(R , S) provides a general version of binary search.

Parameter	Type	Description
R	IntegerType	The space being searched
S	TotallyOrderedType	The target values being searched for

Exports

binarySearch: $(S, R \rightarrow S, R, R) \rightarrow (\text{Boolean}, R)$ binary search

Usage

binarySearch(s, f, a, b)

Signature

binarySearch: $(S, R \rightarrow S, R, R) \rightarrow (\text{Boolean}, R)$

Parameter	Type	Description
s	S	The value to search for
f	$R \rightarrow S$	A monotonic increasing function
a	R	The left end of the interval to search
b	R	The right end of the interval to search

Returns

Returns (found?, r) such that $s = f(r)$ if found? is *true*. Otherwise, found? is *false* and:

- if $r < a$ then $s < f(a)$ or $b < a$;
- if $r \geq b$ then $s > f(b)$;
- if $a \leq r < b$, then $f(r) < s < f(r + 1)$;

CommandLine

Usage

import from CommandLine

Description

CommandLine provides utilities for command-line processing.

Exports

arguments:	Array String	command line arguments
command:	String	command line command
flag:	Character → List String	value of a command line flag
flag?:	Character → Boolean	test for a command line flag

Usage

arguments

Signature

arguments: Array String

Returns

Returns an array containing all the arguments of the command line, the command itself is not included.

Signature

command: String

Returns

Returns the command, *i.e.* the first word of the command line.

Usage

flag c
flag(c, s)
flag? c
flag?(c, s)

Signatures

flag: Character → List String
flag: (Character,String)→List String
flag?: Character → Boolean
flag?: (Character, String) → Boolean

Parameter	Type	Description
<i>c</i>	Character	flag code to look for
<i>s</i>	String	special flag codes (optional)

Returns

flag(c) returns all the values of the flag c each time it is present in the command line, an empty list otherwise.

flag?(c) returns *true* if the flag c is present in the command line, *false* otherwise.

In both functions, the optional argument s contains a list of flag codes which cause the rest of the argument to be skipped.

Example

If the command line to the program was `myprog -lsalli -l gmp -v`, then `flag?(char "a")` and `flag?(char "v")` both return *true*, while `flag?(char "b")` and `flag?(char "a", "l")` both return *false*. In addition, `flag(char "l")` returns the list [`'salli'`, `'gmp'`].

CopyableType

Usage

CopyableType: Category

Description

CopyableType is the category of types whose objects can be copied.

Exports

`copy:` $\% \rightarrow \%$ Make a copy
`copy!:` $(\%, \%) \rightarrow \%$ In-place copy

Usage

copy y
copy!(x, y)

Signatures

copy: % $\rightarrow$ %
copy!: (% , %) $\rightarrow$ %

Parameter	Type	Description
<i>x, y</i>	%	Element of the type

Returns

copy(y) returns a copy of y, while copy!(x, y) returns a copy of y, where the storage used by x is allowed to be destroyed or reused, so x is lost after this call.

Remarks

Use copy before making in-place operations on a parameter. The call copy!(x, y) may cause x to be destroyed, so do not use it unless x has been locally allocated, and is guaranteed not to share space with other elements. Some functions are not necessarily copying their arguments and can thus create memory aliases.

Generator

Usage

import from Generator T

Parameter	Type	Description
<i>T</i>	Type	the type of the elements generated

Description

Generator T is a type which allows values of type T to be obtained serially in a ‘repeat’ or ‘collect’ form.

Exports

concat:	List % → %	concatenation
interlacing:	List % → %	interlacing
next!:	% → T	get the next element
partialNext!:	% → Partial T	get the next element

Usage

concat $[g_1, \dots, g_n]$

Signature

concat: List % $\rightarrow$ %

Parameter	Type	Description
g_i	%	generators

Description

Given that each g_i generates $t_{i1}, t_{i2}, \dots$, returns a generator that generates

$$t_{11}, t_{12}, \dots, t_{21}, t_{22}, \dots, t_{n1}, t_{n2}, \dots$$

and that stops after g_n stops.

Usage

interlacing $[g_1, \dots, g_n]$

Signature

interlacing: List % $\rightarrow$ %

Parameter	Type	Description
g_i	%	generators

Description

Given that each g_i generates $t_{i1}, t_{i2}, \dots$, returns a generator that generates

$$t_{11}, t_{21}, \dots, t_{n1}, t_{12}, t_{22}, \dots, t_{n2}, \dots$$

and that stops whenever any of the g_i stops.

Usage

next! g
partialNext! g

Signatures

next!: % → T
partialNext!: % → **Partial** T

Parameter	Type	Description
<i>g</i>	%	a generator

Description

next!(*g*) and partialNext!(*g*) both return the next element produced by *g*, updating *g*. If *g* is empty, then next!(*g*) throws the exception **GeneratorException**, while partialNext!(*g*) returns *failed*.

GeneratorException

Usage

```
throw GeneratorException
```

```
try ... catch E in { E has GeneratorExceptionType => ... }
```

Description

GeneratorException is an exception type thrown by stepping through an empty generator.

GeneratorExceptionType

Usage

GeneratorExceptionType: Category

Description

GeneratorExceptionType is the category of exceptions thrown by generators.

Partial

Usage

import from Partial T

Parameter	Type	Description
T	Type	a type

Description

Partial(T) implements a union of T and *failed*.

Exports

`[]`: $T \rightarrow \%$ create an element
`failed`: $\%$ the element *failed*
`failed?`: $\% \rightarrow \text{Boolean}$ check for the element *failed*
`retract`: $\% \rightarrow T$ convert to an element of T

if T has `PrimitiveType` then
 `PrimitiveType`

if T has `HashType` then
 `HashType`

if T has `InputType` then
 `InputType`

if T has `OutputType` then
 `OutputType`

if T has `SerializableType` then
 `SerializableType`

Usage
[t]

Signature
[]: T → %

Parameter	Type	Description
<i>t</i>	T	an element

Returns
Returns the element *t* converted to an element of %.

Usage

failed

failed? x

Signatures

failed: %

failed?: % $\rightarrow$ Boolean

Parameter	Type	Description
<i>x</i>	%	a partial element

Returns

failed returns the special element *failed*, while failed?(x) returns *true* if x is *failed*, *false* otherwise.

Usage

retract x

Signature

retract: % $\rightarrow$ T

Parameter	Type	Description
<i>x</i>	%	a partial element

Returns

Returns the element x converted to an element of T, provided that x is not *failed*.

PartialFunction

Usage

import from PartialFunction(R, S)

Parameter	Type	Description
R	Type	Contains the domain of the function
S	Type	Contains the image of the function

Description

PartialFunction provides partial functions from R to S , *i.e.* functions from R to S which are allowed to fail on some elements of R .

Exports

apply:	$(\%, R) \rightarrow S$	Apply a partial function
inDomain?:	$(\%, R) \rightarrow \text{Boolean}$	Check if an element is in the domain
mapping:	$\% \rightarrow (R \rightarrow S)$	Action of a function
partialApply:	$(\%, R) \rightarrow \text{Partial } S$	Apply a partial function
partialMapping:	$\% \rightarrow (R \rightarrow \text{Partial } S)$	Action of a function
predicate:	$\% \rightarrow (R \rightarrow \text{Boolean})$	Domain of a partial function
partialFunction:	$(R \rightarrow \text{Partial } S) \rightarrow \%$	Create a partial function
partialFunction:	$((R \rightarrow \text{Boolean}, R \rightarrow S)) \rightarrow \%$	Create a partial function

Usage

apply(σ , x)
 σx

Signature

apply: $(\%, R) \rightarrow S$

Parameter	Type	Description
σ	$\%$	A partial function
x	R	An element of R

Returns

Returns σx .

Remarks

This map can cause an error if it is used on an element which is not in the domain of σ . Use only when x is known to be in the domain, otherwise use *partialApply*.

See also

inDomain?, predicate, partialApply

Usage

inDomain? x

Signature

inDomain?: (% , R) → Boolean

Parameter	Type	Description
σ	%	A partial function
x	R	An element of R

Returns

Returns *true* if x is in the domain of σ , *false* otherwise.

See also

predicate

Usage

mapping σ

Signature

mapping: $\% \rightarrow (R \rightarrow S)$

Parameter	Type	Description
σ	$\%$	A partial function

Returns

Returns the map corresponding to the action of σ on R .

Remarks

The map returned can cause an error if it is used on an element which is not in the domain of σ . Use only when x is known to be in the domain, otherwise use *partialMapping*.

See also

inDomain?, partialMapping, predicate

Usage

partialMapping σ

Signature

partialMapping: $\% \rightarrow (R \rightarrow \text{Partial } S)$

Parameter	Type	Description
σ	$\%$	A partial function

Returns

Returns the map corresponding to the action of σ on R .

See also

inDomain?, mapping, predicate

Usage

predicate σ

Signature

predicate: $\% \rightarrow (R \rightarrow \text{Boolean})$

Parameter	Type	Description
σ	$\%$	A partial function

Returns

Returns the predicate defining the domain of σ .

See also

inDomain?

Usage

partialFunction f
 partialFunction(p, g)

Signatures

partialFunction: $(R \rightarrow \text{Partial } S) \rightarrow \%$
 partialFunction: $(R \rightarrow \text{Boolean}, R \rightarrow S) \rightarrow \%$

Parameter	Type	Description
f	$R \rightarrow \text{Partial } S$	A partial map
p	$R \rightarrow \text{Boolean}$	A predicate
g	$R \rightarrow S$	A map

Returns

partialFunction(f) returns the partial function σ on R given by

$$\sigma x = f(x)$$

for any $x \in R$, while partialFunction(p, g) returns the partial function σ on R given by

$$\sigma x = \begin{cases} g(x) & \text{if } p(x) = \text{true} \\ \text{failed} & \text{if } p(x) = \text{false} \end{cases}$$

Usage

partialApply x

Signature

partialApply: ($\%$, R) $\rightarrow$ Partial S

Parameter	Type	Description
σ	$\%$	A partial function
x	R	An element of R

Returns

Returns σx , or *failed* if x is not in the domain of σ .

See also

apply, inDomain?, predicate

Pointer

Usage

import from Pointer

Description

Pointer implements machine pointers.

Exports

HashType

InputType

OutputType

SerializableType

coerce: % → MachineInteger conversion to an integer

coerce: MachineInteger → % conversion from an integer

nil: % the nil pointer

nil?: % → Boolean test for the nil pointer

Usage

nil

nil? p

Signatures

nil: %

nil?: % $\rightarrow$ Boolean

Parameter	Type	Description
p	%	a pointer

Returns

nil returns the nil pointer, while nil?(p) returns *true* if p is the nil pointer, *false* otherwise.

Symbol

Usage

import from Symbol

Description

Symbol provides symbols, *i.e.* read-only strings with constant-time comparison.

Exports

HashType

InputType

OutputType

SerializableType

—: String → % Create a symbol

name: % → String Name of a symbol

new: () → % Create a new symbol

Symbol

Usage

$-s$

Signature

$-: \text{String} \rightarrow \%$

Parameter	Type	Description
s	String	A string

Returns

Returns s as a symbol.

Symbol	name
--------	------

Usage

name s

Signature

name: % → String

Parameter	Type	Description
<i>s</i>	%	A symbol

Returns

Returns a new copy of the name of s.

Remarks

Modifying name(s) does not modify s, since a new copy is created at each call.

Usage

new()

Signature

new: () → %

Returns

Returns a new symbol.

Timer

Usage

import from Timer

Description

Timer is a type whose elements are stopwatch timers, which can be used to time precisely various sections of code, including garbage collection. The precision can be up to 1 millisecond but depends on the operating system. The times returned are CPU times (user + gc) used by the process that created the timer.

Exports

<code>gc:</code>	<code>% → MachineInteger</code>	read a timer
<code>read:</code>	<code>% → MachineInteger</code>	read a timer
<code>reset!:</code>	<code>% → %</code>	reset a timer to 0
<code>start!:</code>	<code>% → MachineInteger</code>	start or restart a timer
<code>stop!:</code>	<code>% → MachineInteger</code>	stop a timer
<code>timer:</code>	<code>() → %</code>	create a new timer

Signature

gc: % $\rightarrow$ MachineInteger

Parameter	Type	Description
<i>t</i>	%	The timer to read

Description

Reads the timer *t* without stopping it.

Returns

Returns the total accumulated garbage collection time in milliseconds by all the start/stop cycles since *t* was created or last reset. If *t* is running, the garbage collection time since the last start is added in, and *t* is not stopped or affected.

See also

read

Usage

read t

Signature

read: % $\rightarrow$ MachineInteger

Parameter	Type	Description
<i>t</i>	%	The timer to read

Description

Reads the timer *t* without stopping it.

Returns

Returns the total accumulated time in milliseconds by all the start/stop cycles since *t* was created or last reset. This times includes any eventual garbage collection time (see `gc` to extract this information). If *t* is running, the time since the last start is added in, and *t* is not stopped or affected.

Example

The following function takes a positive `MachineInteger` *n* as input, computes and prints a machine approximation of $\sum_{i=1}^n 1/i$, and returns the CPU time needed to compute it, but not the time needed to print it.

```
timeHarmonic(n:MachineInteger):MachineInteger == {
  import from MachineInteger, SingleFloat, Timer, Character, TextWriter;
  t := timer();
  m:SingleFloat := 1;
  start! t;
  for i in 2..n repeat m := m + 1 / (i::SingleFloat);
  stop! t;
  stdout << "H" << n << " = " << m << newline;
  read t;
}
```

See also

`gc`, `start!`, `stop!`

Usage

reset! t

Signature

reset!: % → %

Parameter	Type	Description
<i>t</i>	%	The timer to reset

Description

Resets the timer *t* to 0 and stops it if it is running.

Returns

Returns the timer *t* after it is reset.

Usage

start! t

Signature

start!: % $\rightarrow$ MachineInteger

Parameter	Type	Description
<i>t</i>	%	The timer to start

Description

Starts or restarts t, without resetting it to 0, It has no effect on t if it is already running.

Returns

Returns 0 if t was already running, the absolute time at which the start/restart was done otherwise.

See also

read, stop!

Usage

stop! t

Signature

stop!: % → MachineInteger

Parameter	Type	Description
<i>t</i>	%	The timer to stop

Description

Stops t without resetting it to 0. It has no effect on t if it is not running.

Returns

Returns the elapsed time in milliseconds since the last time t was restarted, 0 if t was not running.

See also

read, start!

Usage

timer()

Signature

timer: () → %

Description

Creates a timer, set to 0 and stopped.

Returns

Returns the timer that has been created.

See also

reset!

Trace

Usage

import from Trace

Description

Trace provides support for the `-wdebug` option of the ALDOR compiler.

Exports

<code>name:</code>	<code>Type → String</code>	Type name
<code>shortName:</code>	<code>Type → String</code>	Type name
<code>traceActivate:</code>	<code>Boolean → ()</code>	start traces
<code>traceStop:</code>	<code>() → ()</code>	stop traces

Usage
name T
shortName T

Signature
name,shortName: Type → String

Parameter	Type	Description
T	Type	A type

Returns
Returns the name of the type T . In case T is the result of a function call $F(T_1, \dots, T_n)$, then `name(T)` returns the full expanded name of the type, while `shortName(T)` returns $F(*, \dots, *)$.

Usage

```

traceActivate()
traceActivate(abbr?)
traceStop()

```

Signatures

```

traceActivate: Boolean → ()
traceStop:      () → ()

```

Parameter	Type	Description
<i>abbr?</i>	Boolean	A flag to control type names (optional)

Description

Tracing of functions from source files compiled with the `-Wdebug` option is enabled between calls to `traceActivate()` and `traceStop()`. If *abbr?* is missing or *false*, then full type names are printed, otherwise the arguments to types are not printed, see `name` for details.

Example

Suppose that `foo.as` contains the following code:

```

#include "aldor"
#include "aldorio"

local fact(n:Integer):Integer == {
    assert(n >= 0);
    zero? n or one? n => 1;
    n * fact(n-1);
}

main():() == {
    import from Boolean, Integer, Trace;
    traceActivate();
    stdout << "5! = " << fact 5 << newline;
    traceStop();
    stdout << "20! = " << fact 20 << newline;
}

main();

```

Compiling and running it with `aldor -grun -Wdebug -laldor foo.as` yields the following output:

```

{Line: 7 Entering: fact
{Line: 7 Entering: fact
{Line: 7 Entering: fact
{Line: 7 Entering: fact
{Line: 7 Entering: fact
Line: 11 Return: AldorInteger is: 1}
Line: 11 Return: AldorInteger is: 2}
Line: 11 Return: AldorInteger is: 6}
Line: 11 Return: AldorInteger is: 24}
Line: 11 Return: AldorInteger is: 120}
5! = 120
20! = 2432902008176640000

```

Note that the call to `fact 20` is not traced because it is done after the call to `traceStop()`.

Index

- *
 - AbelianMonoid, 22
 - LinearCombinationFraction, 162
 - LinearCombinationType, 654
 - MatrixCategory, 277
- +
 - ListType, 792
 - String, 836
- - Set, 819
 - Symbol, 897
- /
 - FloatType, 622
 - RittRing, 121
- 0
 - AbelianMonoid, 20
- <,>
 - PartiallyOrderedType, 664
- <<
 - InputType, 716
 - OutputType, 718
 - SerializableType, 720
 - WriterManipulator, 737
- =
 - PrimitiveType, 669
- *
 - Monoid, 108
- **
 - DoubleFloat, 618
 - LinearArithmeticType, 103
 - Monoid, 109
 - SingleFloat, 679
- *,**
 - ArithmeticType, 600
- +
 - AbelianMonoid, 21
- +,-
 - AdditiveType, 595
- ..
 - IntegerSegment, 636
- /
 - FractionFieldCategory0, 160
- Group, 77
- FiniteLinearStructureType, 768
 - KeyEntry, 777
 - LinearStructureType, 783
 - Partial, 883
 - TableType, 855
- 0
 - AdditiveType, 594
- 1
 - ArithmeticType, 599
 - Monoid, 107
- AbelianGroup, 18
- AbelianMonoid, 19
- abs
 - OrderedArithmeticType, 661
- AdditiveType, 593
- aldor,axiom,C,fortran,infix,lisp,maple,tex
 - ExpressionTree, 477
 - ExpressionTreeLeaf, 500
 - ExpressionTreeOperator, 521
- AldorInteger, 592
- AldorLibraryInformation, 862
- Algebra, 25
- AlgebraLibraryInformation, 572
- allPrimes
 - PrimeCollection, 201
- and,or,not,xor
 - BooleanArithmeticType, 609
- apply
 - Automorphism, 130
 - Derivation, 139
 - ExpressionTree, 478
 - FiniteSet, 55
 - LinearStructureType, 784
 - MatrixCategory, 279
 - PartialFunction, 887
 - Permutation, 577
 - RandomNumberGenerator, 671
 - SymbolTable, 849
 - TableType, 856

- UnivariateFreeLinearArithmeticType, 371
- UnivariateMonomial, 393
- UnivariatePolynomialAlgebra, 403
- arguments
 - CommandLine, 871
 - ExpressionTree, 479
- ArithmeticType, 598
- arity
 - ExpressionTreeOperator, 522
- Array, 740
- array
 - ArrayType, 744
- array,pointer
 - PrimitiveArrayType, 809
- ArrayException, 741
- ArrayExceptionType, 742
- ArrayType, 743
- Automorphism, 129
- Backsolve, 226
- backsolve
 - Backsolve, 227
- backsolve2
 - Backsolve, 228
- balancedRemainder
 - HeuristicGcd, 344
- berlekamp
 - PrimeFieldUnivariateFactorizer, 352
- berr,bout
 - BinaryWriter, 690
- bin
 - BinaryReader, 685
- binaryExponentiation
 - BinaryPowering, 605
- BinaryPowering, 604
- BinaryReader, 684
- binaryReader
 - BinaryReader, 686
- BinarySearch, 868
- binarySearch
 - ArrayType, 745
 - BinarySearch, 869
- BinaryWriter, 689
- binaryWriter
 - BinaryWriter, 691

- binomial
 - IntegerCategory, 92
- bit?,clear,set
 - IntegerType, 642
- BivariateUtilitiesPackage, 445
- Boolean, 606
- boolean
 - ExpressionTreeLeaf, 501
- BooleanArithmeticType, 608
- bound,finite?
 - Sequence, 583
- BoundedFiniteDataStructureType, 750
- BoundedFiniteLinearStructureType, 755
- bracket
 - MatrixCategory, 278
- by,step
 - IntegerSegment, 637
- Byte, 695
- bytes,max,min
 - MachineInteger, 658
- cancel,cancelIfCan
 - GeneralExponentCategory, 455
- CanonicalSimplification, 133
- canonicalUnitNormal?
 - CommutativeRing, 28
- cantorZassenhaus
 - PrimeFieldUnivariateFactorizer, 353
- center
 - Shell, 560
- char
 - String, 837
- char,ord
 - Character, 700
- Character, 699
- characteristic
 - Ring, 116
- CharacteristicZero, 26
- CheckingArray, 758
- CheckingList, 759
- CheckingMemoryBlock, 760
- ChineseRemaindering, 134
- coefficient
 - IndexedFreeLinearCombinationType, 84
 - UnivariateMonomial, 394

- coefficients
 - UnivariateFreeLinearArithmeticType, 372
- coerce
 - Byte, 696
 - Complex, 611
 - File, 706
 - FloatType, 623
 - GMPFloat, 626
 - GMPInteger, 632
 - LinearArithmeticType, 104
 - PrimitiveMemoryBlock, 815
 - Ring, 117
 - String, 838
 - StringBuffer, 845
- coerce,expand,trailExpand
 - UnivariateFactorialPolynomial, 368
- coerce,machine
 - IntegerType, 643
 - SmallPrimeFieldCategory0, 223
- coerce,single
 - DoubleFloat, 619
- column,row
 - MatrixCategory, 282
- columns,rows
 - MatrixCategory, 283
- combine
 - ChineseRemaindering, 135
- command
 - CommandLine, 872
- CommandLine, 870
- commutative?
 - ArithmeticType, 601
- CommutativeRing, 27
- companion
 - MatrixCategory, 284
 - UnivariateFreeRing, 378
- compare
 - TotallyOrderedType, 682
- Complex, 137, 610
- complex
 - Complex, 612
- compose
 - UnivariatePolynomialQuotient, 186
- compose,translate
 - UnivariatePolynomialRing, 427
- concat
 - Generator, 877
- conjugate
 - Complex, 613
- cons
 - ListType, 794
- constant
 - Stream, 828
- content,primitive,primitivePart
 - FreeModule, 64
- copy
 - CopyableType, 875
- CopyableType, 874
- credits
 - VersionInformationType, 864
- cutoff
 - CommutativeRing, 29
- cycle
 - LinearAlgebra, 234
- data
 - ArrayType, 746
- DataStructureType, 761
- DecomposableRing, 33
- definingPolynomial
 - UnivariatePolynomialQuotient, 187
- degree
 - UnivariateMonomial, 395
- degree,finite?
 - UnivariateTaylorSeriesType, 432
- degree,trailingDegree
 - IndexedFreeModule, 87
- degreeBound
 - UnivariatePolynomialCRTLinearAlgebra, 323
- denominator,numerator
 - FractionCategory, 156
- denominators
 - LinearEliminationCategory, 263
- DenseMatrix, 229
- DenseUnivariatePolynomial, 333
- DenseUnivariateTaylorSeries, 334
- dependence
 - LinearEliminationCategory, 264
- Derivation, 138
- derivation

- Derivation, 140
- DifferentialRing, 39
- deter
 - LinearEliminationCategory, 265
 - ModulopGaussElimination, 304
- determinant
 - LinearAlgebra, 235
 - LinearAlgebraRing, 249
 - LinearEliminationCategory, 266
 - UnivariatePolynomialCRTLinearAlgebra, 324
- diagonal
 - MatrixCategory, 285
- DifferentialExtension, 36
- DifferentialRing, 38
- differentiate
 - DifferentialRing, 40
 - UnivariateTaylorSeriesNewtonSolver, 443
- differentiate,integrate
 - UnivariateTaylorSeriesType, 433
- digit?,letter?,space?
 - Character, 701
- dimension
 - Flags, 772
- dimensions
 - MatrixCategory, 286
- diophantine
 - EuclideanDomain, 42
- DirectProduct, 446
- DirectProductCategory, 447
- discreteLogTable
 - SmallPrimeFieldCategory0, 224
- dispersion,integerDistances
 - UnivariatePolynomialAlgebra, 420
- DistributedMultivariatePolynomial0, 448
- DistributedMultivariatePolynomial1, 449
- divDiffProd
 - IntegralDomain, 96
- divide,mod,quo,rem
 - IntegerType, 644
- divide,quo,rem
 - EuclideanDomain, 43
- DivisionFreeGaussElimination, 230
- divisors
 - Product, 170
- divSumProd
 - IntegralDomain, 97
- dot
 - Sequence, 584
 - Vector, 328
- DoubleFloat, 617
- doubleFloat
 - ExpressionTreeLeaf, 502
- DynamicDataStructureType, 764
- empty
 - FiniteLinearStructureType, 769
- empty?
 - DataStructureType, 762
- endl
 - WriterManipulator, 738
- entries, keys
 - TableType, 857
- entry,explode,key
 - KeyEntry, 778
- eof
 - Byte, 697
- eof,newline,null,tab
 - Character, 702
- eof?
 - Parser, 548
- equal?
 - LinearStructureType, 787
 - UnivariatePolynomialAlgebra, 404
- error
 - String, 839
- euclid
 - EuclideanDomain, 45
- EuclideanDomain, 41
- euclideanSize
 - EuclideanDomain, 46
- eval
 - Parsable, 546
- evalArith
 - ParsingTools, 554
- evalInt
 - ParsingTools, 555
- Evaluator, 536
- even?,odd?
 - IntegerType, 645
- exactQuotient

- IntegralDomain, 98
- expand
 - Product, 171
 - UnivariateTaylorSeriesType2Poly, 436
- expandFraction
 - Product, 172
 - UnivariateTaylorSeriesType2Poly, 437
- ExponentCategory, 450
- ExpressionTree, 476
- ExpressionTreeAnd, 487
- ExpressionTreeAssign, 488
- ExpressionTreeBigO, 489
- ExpressionTreeCase, 490
- ExpressionTreeComplex, 491
- ExpressionTreeEqual, 492
- ExpressionTreeExpt, 493
- ExpressionTreeFactorial, 494
- ExpressionTreeGreaterEqual, 495
- ExpressionTreeGreaterThanOr, 496
- ExpressionTreeIf, 497
- ExpressionTreeLeaf, 498
- ExpressionTreeLessEqual, 513
- ExpressionTreeLessThan, 514
- ExpressionTreeLispList, 515
- ExpressionTreeList, 516
- ExpressionTreeMatrix, 517
- ExpressionTreeMinus, 518
- ExpressionTreeNotEqual, 519
- ExpressionTreeOperator, 520
- ExpressionTreeOperatorTools, 526
- ExpressionTreePlus, 530
- ExpressionTreePrefix, 531
- ExpressionTreeQuotient, 532
- ExpressionTreeSubscript, 533
- ExpressionTreeTimes, 534
- ExpressionTreeVector, 535
- ExpressionType, 126
- extendedEuclidean
 - EuclideanDomain, 47
- extendedLastSPRS
 - Resultant, 360
- extendedRowEchelon
 - LinearEliminationCategory, 267
- extendedRowEchelonForm
 - LinearEliminationCategory, 268

- extree
 - ExpressionTree, 480
 - ExpressionType, 127
- factor
 - FactorizationRing, 336
 - PrimeFieldUnivariateFactorizer, 355
 - UnivariateIntegralFactorizer, 390
 - UnivariatePolynomialAlgebra, 418
- factorial
 - IntegerType, 646
 - Ring, 118
- FactorizationRing, 335
- factorOfDeterminant
 - LinearAlgebra, 236
 - LinearAlgebraRing, 250
- factors
 - PrimitiveRoots, 215
- failed
 - Partial, 884
- fft
 - FFTRing, 339
- fftCutoff
 - FFTRing, 341
- FFTRing, 338
- Field, 50
- File, 704
- fileAppend,fileBinary,fileRead,fileText,fileWrite
 - File, 707
- FileException, 713
- FileExceptionType, 714
- find
 - ListType, 796
 - TableType, 858
- findAll
 - BoundedFiniteDataStructureType, 752
- FiniteAbelianMonoidRing0, 451
- FiniteCharacteristic, 51
- FiniteLinearStructureType, 767
- FiniteSet, 53
- FiniteVariableType, 452
- first
 - ListType, 797
- firstDependence
 - LinearAlgebra, 237

- firstIndex
 - LinearStructureType, 785
- flag
 - CommandLine, 873
- Flags, 771
- float
 - ExpressionTreeLeaf, 503
 - Token, 563
- FloatType, 621
- flush
 - WriterManipulator, 739
- forget,remember
 - HashTable, 774
- fourierPrime
 - PrimeCollection, 202
- Fraction, 142
- fraction,truncate
 - FloatType, 624
- FractionalRoot, 143
- fractionalRoot,integralRoot
 - FractionalRoot, 145
- fractionalRoots,roots
 - FactorizationRing, 337
 - UnivariatePolynomialAlgebra, 419
- FractionBy, 150
- FractionByCategory, 151
- FractionByCategory0, 152
- FractionCategory, 155
- FractionFieldCategory, 158
- FractionFieldCategory0, 159
- FractionFreeGaussElimination, 231
- FreeAlgebra, 59
- FreeLinearArithmeticType, 60
- FreeLinearCombinationType, 61
- FreeModule, 63
- FreeRRing, 71
- function
 - Automorphism, 131
 - Derivation, 141
- gc
 - Timer, 901
- gcd,lcm
 - GcdDomain, 74
 - IntegerType, 647
- GcdDomain, 73
- gcdquo
 - GcdDomain, 75
- gcdquoUP
 - UnivariateGcdRing, 388
- gcdUP
 - UnivariateGcdRing, 387
- GeneralExponentCategory, 454
- Generator, 876
- generator
 - BoundedFiniteDataStructureType, 753
 - IntegerSegment, 638
 - Product, 173
 - RandomNumberGenerator, 672
 - Stream, 829
- generator,terms
 - IndexedFreeModule, 88
- GeneratorException, 880
- GeneratorExceptionType, 881
- GenericModularPolynomialGcdPackage, 342
- GMPFloat, 625
- GMPInteger, 631
- ground?
 - FreeRRing, 72
- Group, 76
- HalfWordSizePrimes, 198
- hash
 - HashType, 781
- HashTable, 773
- HashType, 780
- height
 - UnivariatePolynomialAlgebra, 405
- hermite
 - UnivariatePolynomialPopovLinearAlgebra, 326
- HermiteGaussElimination, 232
- HeuristicGcd, 343
- heuristicGcd
 - HeuristicGcd, 345
- high,low
 - IntegerSegment, 639
- Horner
 - UnivariatePolynomialAlgebra, 406
- imag,real

- Complex, 615
- index
 - FiniteSet, 56
 - IndexedVariable, 574
- IndexedFreeAlgebra, 79
- IndexedFreeLinearArithmeticType, 80
- IndexedFreeLinearCombinationType, 82
- IndexedFreeModule, 86
- IndexedFreeRRing, 90
- IndexedVariable, 573
- inDomain?
 - PartialFunction, 888
- infix
 - ExpressionTreeOperatorTools, 527
- InfixExpressionParser, 537
- input
 - Maple, 542
- InputType, 715
- insert
 - DynamicDataStructureType, 765
- integer
 - ExpressionTreeLeaf, 504
 - IntegerCategory, 93
 - Token, 564
- IntegerCategory, 91
- IntegerExponentVectorCategory, 457
- IntegerPolynomial, 458
- integerRoots,rationalRoots
 - RationalRootRing, 357
 - UnivariateFactorialPolynomial, 369
 - UnivariateIntegralFactorizer, 391
 - UnivariatePolynomialAlgebra, 421
- IntegerSegment, 635
- IntegerType, 641
- integral?
 - FractionalRoot, 144
- IntegralDomain, 95
- integralValue
 - FractionalRoot, 146
- integrate
 - UnivariatePolynomialAlgebra, 407
- interlacing
 - Generator, 878
 - Sequence, 585
 - Stream, 830

- interpolate
 - ChineseRemaindering, 136
- intersection
 - Set, 820
- inv
 - Group, 78
 - RittRing, 122
- inverse
 - LinearAlgebra, 238
 - LinearAlgebraRing, 251
- invertibleSubmatrix
 - LinearAlgebra, 239
 - LinearAlgebraRing, 252
- is?
 - ExpressionTree, 481
- kernel
 - LinearAlgebra, 240
 - LinearAlgebraRing, 253
 - ModulopGaussElimination, 309
- KeyEntry, 776
- knownIrreducible?
 - UnivariatePolynomialQuotient, 188
- lastError
 - Parser, 549
- lastSPRS
 - Resultant, 359
- LazyHalfWordSizePrimes, 197
- leadingCoefficient,trailingCoefficient
 - FreeModule, 66
- leadingMonomial,leadingTerm,trailingMonomial,trailingTerm
 - IndexedFreeModule, 89
- leaf
 - ExpressionTree, 482
 - ExpressionTreeLeaf, 505
 - Token, 565
- leftExactQuotient
 - NonCommutativeIntegralDomain, 113
- leftTrim,rightTrim
 - String, 840
- length
 - IntegerType, 648
- lift
 - DifferentialExtension, 37

- PrimeFieldCategory0, 213
- UnivariatePolynomialAlgebra, 408
- UnivariatePolynomialQuotient, 189
- limbs
 - GMPFloat, 627
 - GMPInteger, 633
- LinearAlgebra, 233
- LinearAlgebraRing, 248
- LinearArithmeticType, 102
- LinearCombinationFraction, 161
- LinearCombinationType, 653
- linearDependence
 - LinearAlgebraRing, 254
- LinearEliminationCategory, 262
- linearSearch
 - BoundedFiniteLinearStructureType, 756
- LinearStructureType, 782
- lisp
 - ExpressionTreeOperatorTools, 528
- LispExpressionParser, 539
- List, 788
- ListException, 789
- ListExceptionType, 790
- ListType, 791
- log
 - Product, 174
- lookup
 - FiniteSet, 57
- lowByte
 - Byte, 698
- lower,upper
 - Character, 703
- machine,single
 - GMPFloat, 628
- MachineInteger, 657
- machineInteger
 - ExpressionTreeLeaf, 508
- MachineIntegerDegreeLexicographicalExponent, 459
- MachineIntegerDegreeReverseLexicographicalExponent, 460
- MachineIntegerExponentVectorCategory, 461
- MachineIntegerLexicographicalExponent, 462
- major,min,patch
 - VersionInformationType, 865
- makeCollIntegral
 - MatrixCategoryOverFraction, 300
- makeIntegral
 - LinearCombinationFraction, 163
- makeIntegralBy
 - LinearCombinationFraction, 164
- MakePartialRing, 540
- makeRational
 - LinearCombinationFraction, 165
- makeRowIntegral
 - MatrixCategoryOverFraction, 301
- makeRowIntegralBy
 - MatrixCategoryOverFraction, 302
- map
 - BoundedFiniteLinearStructureType, 757
 - FreeLinearCombinationType, 62
 - MatrixCategory, 287
 - MatrixCategory2, 298
 - UnivariateFreeRing2, 384
 - UnivariateMonomial, 396
- Maple, 541
- maple
 - Maple, 543
- mapping
 - PartialFunction, 889
 - Permutation, 578
- MatrixCategory, 275
- MatrixCategory2, 297
- MatrixCategoryOverFraction, 299
- max,min
 - DoubleFloat, 620
 - RandomNumberGenerator, 673
 - SingleFloat, 680
 - TotallyOrderedType, 683
- maxInvertibleSubmatrix
 - LinearAlgebra, 241
 - LinearAlgebraRing, 255
 - LinearEliminationCategory, 269
- maxPrime
 - PrimeCollection, 203
- member?
 - BoundedFiniteDataStructureType, 754
- MemoryBlock, 805
- minIntegerRoot,maxIntegerRoot

- UnivariatePolynomialAlgebra, 422
- ModularComputation, 167
- modularGcd
 - ModularUnivariateGcd, 348
- ModularUnivariateGcd, 347
- Module, 105
- ModulopGaussElimination, 303
- ModulopUnivariateGcd, 349
- modX,modInverse
 - MachineInteger, 659
- monic
 - FreeModule, 67
- monicDivide,monicQuotient, monicRemainder
 - UnivariatePolynomialAlgebra, 409
- monicNewtonSeries
 - UnivariateTaylorSeriesType2Poly, 439
- Monoid, 106
- monom
 - UnivariateFreeLinearArithmeticType, 373
 - UnivariatePolynomialQuotient, 190
- monomial
 - UnivariateMonomial, 397
- monomial,term
 - IndexedFreeLinearCombinationType, 85
- morphism
 - Automorphism, 132
- multiplicity
 - FractionalRoot, 147
- musser
 - UnivariatePolynomialSquareFree, 429
- name
 - ExpressionTreeOperator, 523
 - Symbol, 898
 - Token, 566
 - VersionInformationType, 866
- name,shortName
 - Trace, 908
- negate
 - ExpressionTree, 483
 - ExpressionTreeLeaf, 506
- negative?
 - ExpressionTree, 484
 - ExpressionTreeLeaf, 507
- new
 - ArrayType, 747
 - FiniteLinearStructureType, 770
 - GMPFloat, 629
 - GMPInteger, 634
 - PrimitiveArrayType, 810
 - String, 841
 - StringBuffer, 846
 - Symbol, 899
- newPackedArray
 - PackableType, 667
- newtonSeries
 - UnivariatePolynomialQuotientSqfr, 194
- next,prev
 - IntegerType, 649
- nextPrime
 - PrimeCollection, 204
- nil
 - Pointer, 895
- NonCommutativeIntegralDomain, 112
- nonZeroCoefficients
 - FreeModule, 68
- norm
 - Complex, 616
- norm,trace
 - UnivariatePolynomialQuotientSqfr, 195
- normalize
 - FractionCategory, 157
 - LinearCombinationFraction, 166
- nthRoot
 - IntegerType, 650
- numberOfColumns,numberOfRows
 - MatrixCategory, 288
- numberOfEntries
 - TableType, 859
- numberOfGenerators
 - RandomNumberGenerator, 674
- one,one?
 - MatrixCategory, 289
- one?
 - ArithmeticType, 602
 - Monoid, 110
- open
 - File, 708
- open?

- IntegerSegment, 640
- operator
 - ExpressionTree, 485
 - Token, 567
- options
 - BinaryReader, 687
 - BinaryWriter, 693
 - TextReader, 724
 - TextWriter, 732
- orbit
 - Stream, 832
- order
 - FractionByCategory0, 153
 - IntegralDomain, 99
- OrderedArithmeticType, 660
- OrderedSymbol, 463
- OrderedVariableList, 464
- OrderedVariableTuple, 465
- orderquo
 - IntegralDomain, 100
- OrdinaryGaussElimination, 316
- ordinaryPoint
 - UnivariatePolynomialAlgebra, 410
- OutputType, 717
- OverdeterminedLinearSystemSolver, 317
- PackableType, 665
- PackedPrimitiveArray, 806
- Parsable, 545
- Parser, 547
- parser
 - ParserReader, 552
- ParserReader, 551
- ParsingTools, 553
- Partial, 882
- partialApply
 - PartialFunction, 893
- PartialFunction, 886
- partialFunction
 - PartialFunction, 892
- PartiallyOrderedType, 663
- partialMapping
 - PartialFunction, 890
- PartialRing, 556
- particularSolution
 - LinearAlgebra, 242
 - LinearAlgebraRing, 256
- Permutation, 576
- pExponentiation
 - PthPowering, 218
- pivot
 - LinearEliminationCategory, 270
- Pointer, 894
- pointer,string
 - String, 842
- PolynomialRing, 466
- PolynomialRing0, 467
- polynomials
 - UnivariateTaylorSeriesType2Poly, 438
- PolynomialTypeRing, 468
- position
 - File, 709
- predicate
 - PartialFunction, 891
- prefix
 - ExpressionTreeOperatorTools, 529
 - Token, 568
- previousPrime
 - PrimeCollection, 205
- PrimeCollection, 200
- PrimeField2, 209
- PrimeFieldCategory, 210
- PrimeFieldCategory0, 212
- PrimeFieldUnivariateFactorizer, 351
- primeInCollection?
 - PrimeCollection, 206
- PrimitiveArray, 807
- PrimitiveArrayType, 808
- PrimitiveMemoryBlock, 814
- primitiveRoot
 - PrimitiveRoots, 216
- PrimitiveRoots, 214
- PrimitiveType, 668
- primRoot
 - PrimeCollection, 207
- Product, 168
- provablyIrreducible?
 - DecomposableRing, 34
- pseudoDivide
 - UnivariatePolynomialAlgebra, 411

- pseudoRemainder
 - UnivariatePolynomialAlgebra, 412
- pthPower
 - FiniteCharacteristic, 52
 - UnivariateMonomial, 398
- PthPowering, 217
- Queue, 816
- quotient,quotientBy
 - IntegralDomain, 101
- radixInterpolate
 - HeuristicGcd, 346
- random
 - FiniteSet, 58
 - IntegerType, 651
 - MatrixCategory, 290
 - Ring, 119
 - Sequence, 586
 - UnivariateFreeRing, 380
 - Vector, 329
- randomGenerator
 - RandomNumberGenerator, 675
- randomInteger
 - RandomNumberGenerator, 676
- RandomNumberGenerator, 670
- randomPrime
 - PrimeCollection, 208
- rank
 - LinearAlgebra, 243
 - LinearAlgebraRing, 257
 - LinearEliminationCategory, 271
- rankLowerBound
 - LinearAlgebra, 244
 - LinearAlgebraRing, 258
 - SpecializationLinearAlgebra, 320
- rationalReconstruction
 - EuclideanDomain, 48
 - IntegerCategory, 94
 - UnivariatePolynomialAlgebra, 413
- RationalRootRing, 356
- read
 - Timer, 902
- read,write
 - PrimitiveArrayType, 811
- reciprocal
 - CommutativeRing, 30
- RecursiveMultivariatePolynomial0, 469
- RecursiveMultivariatePolynomialCategory0, 470
- reduce
 - UnivariatePolynomialQuotient, 191
- ReducibleModulusException, 178
- ReducibleModulusExceptionType, 179
- reductum
 - FreeModule, 69
- relativeSize
 - ExpressionType, 128
- remove
 - File, 710
- remove,removeAll
 - DynamicDataStructureType, 766
- ResidueClassRing, 177
- rest
 - ListType, 799
- Resultant, 358
- resultant
 - Resultant, 361
 - UnivariatePolynomialAlgebra, 414
- ResultantOfBivariatePolynomialsOverSmallPrimeField, 471
- retract
 - Partial, 885
- reverse
 - ListType, 800
- revert
 - UnivariateFreeRing, 381
- rightExactQuotient
 - NonCommutativeIntegralDomain, 114
- Ring, 115
- RittRing, 120
- root
 - UnivariateTaylorSeriesNewtonSolver, 444
- roots
 - PrimeFieldUnivariateFactorizer, 354
- rootsSqfr
 - PrimeFieldCategory, 211
- rowEchelon
 - LinearEliminationCategory, 272
- rowEchelonForm
 - LinearEliminationCategory, 273

- RRing, 123
- run
 - Maple, 544
- scan
 - Scanner, 558
- Scanner, 557
- seed
 - RandomNumberGenerator, 677
- Sequence, 581
- sequence
 - Sequence, 587
- SerializableType, 719
- series
 - UnivariateTaylorSeriesType, 434
- Set, 818
- Shell, 559
- shift
 - FractionByCategory0, 154
 - IntegerType, 652
 - UnivariateFreeLinearArithmeticType, 374
- sign
 - OrderedArithmeticType, 662
 - Permutation, 579
- SimpleAlgebraicExtension, 180
- SimpleAlgebraicExtensionCategory, 181
- SingleFloat, 678
- singleFloat
 - ExpressionTreeLeaf, 509
- size
 - BoundedFiniteDataStructureType, 751
 - FiniteSet, 54
 - Product, 169
 - Sequence, 582
 - Stream, 827
- SmallPrimeField, 219
- SmallPrimeField0, 220
- SmallPrimeFieldCategory, 221
- SmallPrimeFieldCategory0, 222
- SmallPrimes, 196
- solve
 - LinearAlgebra, 246
 - LinearAlgebraRing, 259
- someFactors
 - DecomposableRing, 35
- SortedAssociationSet, 823
- SortedList, 824
- SortedSet, 825
- SourceOfPrimes, 182
- span
 - LinearAlgebra, 245
 - LinearAlgebraRing, 260
 - LinearEliminationCategory, 274
- SparseIntegerMultivariatePolynomial, 472
- sparseMultiple
 - UnivariatePolynomialAlgebra, 415
- SparseMultivariatePolynomial, 473
- SparseUnivariatePolynomial, 366
- SparseUnivariatePolynomial0, 364
- SparseUnivariatePolynomial1, 365
- special
 - Token, 569
- Specializable, 124
- specialization
 - Specializable, 125
- SpecializationLinearAlgebra, 319
- SPRS
 - Resultant, 362
- squareFree,squareFreePart
 - UnivariatePolynomialAlgebra, 416
- StandardFilteredRing, 474
- stderr,stdout
 - TextWriter, 733
- stdin
 - TextReader, 728
- Stream, 826
- stream
 - Stream, 833
- String, 835
- string
 - ExpressionTreeLeaf, 510
 - StringBuffer, 847
 - Token, 570
- StringBuffer, 844
- subKernel
 - LinearAlgebra, 247
 - LinearAlgebraRing, 261
- subResultantGcd
 - Resultant, 363
- substring

- String, 843
- support
 - FreeModule, 70
- Symbol, 896
- symbol
 - ExpressionTreeLeaf, 511
- SymbolTable, 848
- SyntaxException, 721
- SyntaxExceptionType, 722
- table
 - SymbolTable, 851
 - TableType, 861
- TableException, 852
- TableExceptionType, 853
- TableType, 854
- tensor
 - MatrixCategory, 293
 - Vector, 330
- term
 - Product, 175
- term?
 - FreeModule, 65
- texParen?
 - ExpressionTree, 486
 - ExpressionTreeLeaf, 512
 - ExpressionTreeOperator, 524
- TextReader, 723
- textReader
 - TextReader, 729
- TextWriter, 730
- textWriter
 - TextWriter, 734
- Timer, 900
- timer
 - Timer, 906
- times
 - GeneralExponentCategory, 456
 - UnivariateFreeRing, 382
- Token, 562
- token
 - Token, 571
- TotallyOrderedType, 681
- Trace, 907
- traceActivate, traceStop

- Trace, 909
- transpose
 - MatrixCategory, 294
 - Permutation, 580
- true,false
 - Boolean, 607
- truncate
 - UnivariateFreeLinearArithmeticType, 375
 - UnivariateTaylorSeriesType2Poly, 441
- tryExpandFraction
 - UnivariateTaylorSeriesType2Poly, 440
- TwoStepFractionFreeGaussElimination, 321
- union
 - Set, 822
- uniqueId
 - ExpressionTreeOperator, 525
- uniqueName
 - File, 712
- unit?
 - CommutativeRing, 32
- unitNormal
 - CommutativeRing, 31
- UnivariateFactorialPolynomial, 367
- UnivariateFreeLinearArithmeticType, 370
- UnivariateFreeRing, 376
- UnivariateFreeRing2, 383
- UnivariateFreeRingOverFraction, 385
- UnivariateGcdRing, 386
- UnivariateIntegralFactorizer, 389
- UnivariateMonomial, 392
- UnivariatePolynomialAlgebra, 401
- UnivariatePolynomialCRTLinearAlgebra, 322
- UnivariatePolynomialKaratsuba, 424
- UnivariatePolynomialMod, 183
- UnivariatePolynomialModSqfr, 184
- UnivariatePolynomialPopovLinearAlgebra, 325
- UnivariatePolynomialQuotient, 185
- UnivariatePolynomialQuotientSqfr, 193
- UnivariatePolynomialRing, 425
- UnivariatePolynomialSquareFree, 428
- UnivariateTaylorSeriesNewtonSolver, 442
- UnivariateTaylorSeriesType, 431
- UnivariateTaylorSeriesType2Poly, 435
- universalBound

- UnivariatePolynomialAlgebra, 423
- value
 - FractionalRoot, 149
 - UnivariatePolynomialQuotient, 192
- values
 - UnivariatePolynomialAlgebra, 417
- variable
 - FiniteVariableType, 453
 - IndexedVariable, 575
- VariableType, 475
- Vector, 327
- VectorOverFraction, 332
- version
 - VersionInformationType, 867
- VersionInformationType, 863
- WordSizePrimes, 199
- WriterManipulator, 736
- wronskian
 - MatrixCategory, 295
- yun
 - UnivariatePolynomialSquareFree, 430
- ZechPrimeField, 225
- zero
 - MatrixCategory, 296
 - Vector, 331
- zero?
 - AbelianMonoid, 24
 - AdditiveType, 597